



**INSTITUTO POLITÉCNICO
NACIONAL**
ESCUELA SUPERIOR DE
CÓMPUTO



Trabajo Terminal

2026-B136

**«Prototipo de un sistema web para la gestión
de expedientes digitales y actividad procesal de
un despacho legal, con servicios de integridad
y no repudio.»**

Presentan

Alonso Sánchez Eduardo
Vitales Herrera Hatziry

Directora

Sandra Díaz Santiago

Resumen

El presente Trabajo Terminal desarrolla un prototipo de sistema web para la gestión de expedientes digitales y la actividad procesal de un despacho legal. El prototipo implementa los servicios criptográficos de *integridad*, orientado a detectar cualquier modificación no autorizada en los expedientes, y de *no repudio*, que impide que las partes involucradas nieguen su participación en la firma de contratos mediante evidencia criptográfica verificable. La fase de implementación del núcleo criptográfico está concluida y demostrada: diez servicios criptográficos construidos en Rust sobre una arquitectura hexagonal, verificables de forma independiente con OpenSSL y respaldados por una suite de más de trescientas pruebas automatizadas que valida el correcto funcionamiento de los módulos y flujos principales.

Palabras clave: expedientes digitales, integridad, no repudio, firma digital, criptografía aplicada, gestión legal, CNPP.

Índice general

Resumen	I
1 Introducción	1
1.1 Problemática	1
1.2 Propuesta de solución	3
1.3 Objetivo	3
1.4 Justificación	5
1.5 Revisión de Plataformas de Gestión Legal	6
1.6 Alcances y Delimitaciones	8
1.7 Supuestos del Proyecto	9
2 Marco teórico	10
2.1 Seguridad de la Información en el Ámbito Jurídico	10
2.2 Sistema Legal Mexicano	11
2.3 Primitivas y Servicios Criptográficos	23
2.4 Gestión de Datos Híbrida	28
3 Análisis y diseño	29
3.1 Metodología	29
3.2 Historias de Usuario	31
3.3 Casos de Uso	35
3.4 Requerimientos del Sistema	67
3.5 Análisis de Costos	84
3.6 Análisis de Riesgos	88
3.7 Diseño de Arquitectura del Sistema	102
3.8 Modelado de Base de Datos Híbrida	119
3.9 Diagrama de Despliegue y Stack Tecnológico	121
3.10 Modelo de Datos Jurídico-Técnico	123
3.11 Diseño Criptográfico del Prototipo	123
4 Implementación del prototipo	130
4.1 Estructura del workspace y andamiaje	130
4.2 Servicios criptográficos	133
4.3 La interfaz de línea de comandos	143
4.4 Aplicación HTTP y persistencia local	146
4.5 Resultados de la exploración del sellado de tiempo	147
4.6 Divergencias respecto al diseño y decisiones registradas	148

5	Pruebas y resultados	151
5.1	Matriz de pruebas	152
5.2	Resultados criptográficos	155
5.3	Verificación independiente	158
5.4	Cobertura y criterios de éxito	161
6	Conclusiones	165
6.1	Resultados obtenidos	165
6.2	Contribuciones	166
6.3	Limitaciones	167
6.4	Trabajo futuro	168
	Referencias	169
	Glosario de Términos	177
A	Diagramas detallados de arquitectura	183
A.1	Vista detallada de Fronteras de Confianza	183
A.2	Arquitectura Hexagonal completa con puertos y adaptadores nombrados	184
B	Manual de la interfaz de línea de comandos	185
B.1	Convenciones generales	185
B.2	Comando serve: aplicación HTTP local	186
B.3	Grupo crypto: integridad de contenido	187
B.4	Grupo vault: cifrado en reposo	187
B.5	Grupo pki: autoridad certificadora interna	189
B.6	Comando sign: firma digital	190
B.7	Comando timestamp: sellado de tiempo	190
B.8	Comando verify: verificación integral	191
B.9	Grupo package: paquete de evidencia	192
B.10	Grupo auth: credenciales y segundo factor	193
B.11	Grupo audit: bitácora encadenada	194
C	Guía de verificación independiente con OpenSSL	196
C.1	Requisitos y versión de referencia	196
C.2	Estructura del paquete	197
C.3	Paso 1: integridad del documento	198
C.4	Paso 2: firma digital	198
C.5	Paso 3: certificado del firmante y revocación	199
C.6	Paso 4: sello de tiempo	199
C.7	Resumen: qué demuestra cada comprobación	200
C.8	Comportamiento ante evidencia alterada	200
C.9	Autonomía respecto del sistema	201

D	Scripts de la autoridad certificadora interna	202
D.1	Inventario y parámetros criptográficos comunes	203
D.2	init-ca.sh: creación de la CA raíz	204
D.3	issue-cert.sh: emisión de certificados de entidad final	204
D.4	revoke.sh: revocación	205
D.5	gen-crl.sh: generación de la CRL	205
D.6	issue-tsa-cert.sh: certificado de la autoridad de sellado	206
D.7	openssl.cnf: configuración de la CA	206
D.8	tsa.cnf: configuración de la autoridad de sellado	207
D.9	Flujo típico de operación	208
E	Matriz de pruebas y evidencia de ejecución	209
E.1	Nota metodológica	209
E.2	Matriz de pruebas	210
E.3	Evidencia de ejecución	217

Índice de figuras

3.1	Ciclo de la metodología XP (Extreme Programming) aplicada al proyecto.	30
3.2	Diagrama CU-01. Registro de despacho y selección de plan.	37
3.3	Diagrama CU-02. Gestión del ciclo de vida de miembros del equipo.	39
3.4	Diagrama CU-03. Inicio de sesión y gestión de sesiones.	41
3.5	Diagrama CU-04. Control de acceso por perfil.	43
3.6	Diagrama CU-05. Registro y administración de expediente penal.	45
3.7	Diagrama CU-06. Gestión del directorio de participantes.	47
3.8	Diagrama CU-07. Transición de etapa procesal.	49
3.9	Diagrama CU-08. Programación de audiencia y activación de plazos.	51
3.10	Diagrama CU-09. Configuración del calendario judicial.	52
3.11	Diagrama CU-10. Monitoreo de plazos y emisión de alertas.	54
3.12	Diagrama CU-11. Carga y clasificación de documentos.	56
3.13	Diagrama CU-12. Consulta de documentos con verificación de integridad.	58
3.14	Diagrama CU-13. Firma digital de contrato con sellado de tiempo.	60
3.15	Diagrama CU-14. Verificación de firma digital y sello de tiempo.	62
3.16	Diagrama CU-15. Consulta del tablero de control.	63
3.17	Diagrama CU-16. Generación de informes.	65
3.18	Diagrama CU-17. Consulta de bitácora de actividad.	66
3.19	Arquitectura Hexagonal: capas conceptuales y regla de dependencias. Las dependen- cias siempre apuntan hacia el dominio.	107
3.20	Vista de componentes: siete contextos limitados del sistema y sus dependencias principales.	108
3.21	Estructura del workspace de Cargo: cinco crates miembros con dependencias permiti- das (flechas negras) y prohibidas (flechas rojas punteadas).	110
3.22	Modelo de datos híbrido: PostgreSQL para datos persistentes con valor probatorio, Redis para datos volátiles.	121
3.23	Diagrama de despliegue: Astro+Svelte (cliente) → Axum/Rust (backend) → Post- greSQL + Redis + servicios externos.	122
A.1	Modelo de defensa en profundidad por capas y controles aplicados a cada cruce de frontera.	183
A.2	Arquitectura hexagonal completa: cada puerto es un trait de Rust en el crate domain; cada adaptador es una struct en el crate infrastructure.	184

Índice de tablas

1.1	Criterios de éxito por objetivo específico.	5
1.2	Resumen ampliado de productos similares (función, servicios y criptografía).	7
2.1	Comparación entre el sistema inquisitivo y el sistema acusatorio oral.	11
2.2	Sujetos procesales del CNPP y sus implicaciones en la gestión digital del expediente.	14
2.3	Términos fatales del proceso penal y su implicación para el módulo de alertas (RF-09, RF-10).	16
2.4	Principales PSC acreditados por la Secretaría de Economía relevantes para el prototipo.	21
3.1	Caso de uso CU-01: Registro de despacho y selección de plan.	36
3.2	Caso de uso CU-02: Gestión del ciclo de vida de miembros del equipo.	38
3.3	Caso de uso CU-03: Inicio de sesión y gestión de sesiones.	40
3.4	Caso de uso CU-04: Control de acceso por perfil.	42
3.5	Caso de uso CU-05: Registro y administración de expediente penal.	44
3.6	Caso de uso CU-06: Gestión del directorio de participantes.	46
3.7	Caso de uso CU-07: Transición de etapa procesal.	48
3.8	Caso de uso CU-08: Programación de audiencia y activación de plazos.	50
3.9	Caso de uso CU-09: Configuración del calendario judicial.	52
3.10	Caso de uso CU-10: Monitoreo de plazos y emisión de alertas.	53
3.11	Caso de uso CU-11: Carga y clasificación de documentos.	55
3.12	Caso de uso CU-12: Consulta de documentos con verificación de integridad.	57
3.13	Caso de uso CU-13: Firma digital de contrato con sellado de tiempo.	59
3.14	Caso de uso CU-14: Verificación de firma digital y sello de tiempo.	61
3.15	Caso de uso CU-15: Consulta del tablero de control.	63
3.16	Caso de uso CU-16: Generación de informes.	64
3.17	Caso de uso CU-17: Consulta de bitácora de actividad.	66
3.18	RF-01: Autenticación y gestión de sesiones	67
3.19	RF-02: Control de acceso basado en roles (RBAC)	68
3.20	RF-03: Gestión de equipos colaborativos	68
3.21	RF-04: Expediente penal digital	69
3.22	RF-05: Control de etapas procesales (CNPP)	69
3.23	RF-06: Directorio de participantes	70
3.24	RF-07: Gestión de audiencias especializadas	70
3.25	RF-08: Blindaje de plazos procesales	71
3.26	RF-09: Sistema de alertas preventivas	71
3.27	RF-10: Calendario jurídico integrado	72
3.28	RF-11: Cadena de custodia digital	72
3.29	RF-12: Repositorio documental	73
3.30	RF-13: Verificación de integridad de documentos	73

3.31	RF-14: Cifrado autenticado de expedientes y documentos	74
3.32	RF-15: Firma digital de contratos	74
3.33	RF-16: Sellado de tiempo (NOM-151)	75
3.34	RF-17: Verificación de firmas y sellos de tiempo	75
3.35	RF-18: Dashboard ejecutivo	76
3.36	RF-19: Generación de reportes	76
3.37	RF-20: Bitácora de actividad (Audit Trail)	77
3.38	RNF-01: Aislamiento lógico de la información del despacho	78
3.39	RNF-02: Gestión segura de credenciales	78
3.40	RNF-03: Gestión de material criptográfico	79
3.41	RNF-04: Inmutabilidad de registros críticos	79
3.42	RNF-05: Privacidad de la información del despacho	80
3.43	RNF-06: Procesamiento asíncrono	80
3.44	RNF-07: Tiempo de respuesta de interfaz	81
3.45	RNF-08: Diseño responsivo	81
3.46	RNF-09: Soporte de localización	82
3.47	RNF-10: Conformidad con el CNPP	82
3.48	RNF-11: Conformidad con la NOM-151	83
3.49	RNF-12: Cumplimiento de la LFPDPPP	83
3.50	RNF-13: Arquitectura mantenible	84
3.51	RNF-14: Aseguramiento de calidad	84
3.52	Fuentes salariales consultadas para la estimación del costo de recurso humano.	85
3.53	Escenarios salariales y tarifa horaria derivada.	85
3.54	Factores de ajuste sobre la capacidad de desarrollo.	86
3.55	Resumen de capacidad del equipo y presupuesto base.	86
3.56	Desglose de costos del proyecto en fase de prototipo.	87
3.57	Resumen financiero del proyecto.	88
3.58	Escala de probabilidad de ocurrencia.	89
3.59	Escala de impacto en tiempo.	89
3.60	Matriz de severidad (Probabilidad × Impacto).	89
3.61	Ficha del riesgo RTC-01: Inestabilidad en la integración con el PSC (RFC 3161).	90
3.62	Ficha del riesgo RTC-02: Implementación insegura de AES-256-GCM.	91
3.63	Ficha del riesgo RTC-03: Mala configuración de infraestructura.	91
3.64	Ficha del riesgo RTC-04: Pérdida de clave maestra de cifrado.	92
3.65	Ficha del riesgo RTC-05: Expiración de certificado X.509 en producción.	92
3.66	Ficha del riesgo RLC-01: Discrepancia de hashes SHA-256 (NOM-151).	93
3.67	Ficha del riesgo RLC-02: Mutabilidad en registros de auditoría (CNPP).	94
3.68	Ficha del riesgo RLC-03: Fuga de datos personales por fallas de autorización (LFPDPPP).	95
3.69	Ficha del riesgo RLC-04: Indisponibilidad o degradación del PSC (Cincel).	96
3.70	Ficha del riesgo ROP-01: Curva de aprendizaje TDD en módulos criptográficos.	97
3.71	Ficha del riesgo ROP-02: Scope creep en interfaces gráficas.	98
3.72	Ficha del riesgo ROP-03: Ruptura de cadencia de Pair Programming.	99
3.73	Resumen cuantitativo de los riesgos identificados con su severidad y VME.	100
3.74	Trazabilidad entre riesgos identificados y objetivos específicos.	102
3.75	Priorización de atributos de calidad y su mapeo a requerimientos no funcionales.	103

3.76	Comparativa de estilos arquitectónicos contra atributos de calidad priorizados.	105
3.77	Mapeo entre contextos delimitados, casos de uso y módulos del workspace de Cargo.	109
3.78	Inventario de crates externos utilizados por el proyecto, agrupados por capa arquitectónica.	113
3.79	Mapeo de alto nivel entre componentes arquitectónicos y los requerimientos que satisfacen.	118
4.1	Tamaño de los crates del workspace en líneas de código. Los tests unitarios viven embebidos en <code>src</code> ; los de integración, en el directorio <code>tests/</code> de cada crate.	131
4.2	Variables de entorno del prototipo (plantilla <code>.env.example</code>).	132
4.3	Jobs del workflow de integración continua.	132
4.4	Fitness functions implementadas y su ubicación en el repositorio.	133
4.5	Servicios criptográficos implementados: puertos del dominio y adaptadores de infraestructura.	134
4.6	Formato del archivo vault DVLTL1	136
4.7	Catálogo de subcomandos de <code>despacho-cli</code> . La bandera global <code>--json</code> aplica a todos.	144
4.8	Correspondencia entre comandos, casos de uso de <code>application</code> y adaptadores inyectados por la raíz de composición.	145
4.9	Contrato de la aplicación HTTP local.	147
4.10	Architecture Decision Records registrados en <code>docs/adr/</code> durante la implementación.	149
5.1	Distribución de las funciones de prueba por crate del workspace.	152
5.2	Matriz de pruebas por servicio criptográfico: número de funciones de prueba adscritas y tipos presentes.	153
5.3	Caso positivo y caso negativo representativos por servicio criptográfico.	153
5.4	Vectores oficiales reproducidos por la suite de pruebas.	155
5.5	Comprobaciones de interoperabilidad ejecutadas por la suite con herramientas independientes.	159
5.6	Cobertura de líneas por crate medida con <code>cargo-llvm-cov</code> y umbral aplicado en integración continua.	161
5.7	Resultados contra los criterios de éxito de los objetivos OE-3 a OE-6.	162
B.1	Variables de entorno leídas por <code>despacho-cli</code> . En las variables del proveedor remoto de sellado, un valor en blanco cuenta como ausente.	186
B.2	Opciones de los subcomandos del grupo <code>vault</code>	188
B.3	Opciones de los subcomandos del grupo <code>pki</code>	189
B.4	Opciones del comando <code>verify</code>	191
B.5	Opciones de los subcomandos del grupo <code>auth</code>	193
C.1	Marcadores de la plantilla del instructivo y valor con que se rellenan al exportar.	196
C.2	Contenido del paquete de evidencia. El documento de la ejecución de referencia se llama <code>document.txt</code>	197
C.3	Correspondencia entre cada paso de la guía, su salida esperada y la propiedad que demuestra.	200
D.1	Archivos del directorio <code>pki/</code> y su propósito.	203

D.2	Parámetros criptográficos y de vigencia de la autoridad interna.	203
E.1	Taxonomía de tipos de prueba empleada en la matriz y abreviaturas usadas.	210
E.2	Distribución de las 383 funciones de prueba por crate. El conteo de integración de infrastructure incluye la prueba ignorada.	211
E.3	Matriz de pruebas del prototipo, una fila por archivo o módulo de prueba	211
E.4	Cobertura de líneas por crate medida con cargo-llvm-cov y umbral aplicado en integración continua.	218
E.5	Jobs del flujo de integración continua que re-ejecutan la evidencia en cada cambio. .	219

CAPÍTULO 1

Introducción

1.1 Problemática

La reforma constitucional del 18 de junio de 2008 transformó el sistema de justicia penal en México, sustituyendo el modelo inquisitivo por un sistema acusatorio y oral. El Código Nacional de Procedimientos Penales (CNPP), publicado en marzo de 2014 [1], unificó los procedimientos penales bajo un marco normativo único para todas las entidades federativas y la Federación, entrando en operación gradual hasta quedar implementado en todos los estados de la República en 2016 [2]. Este modelo, sustentado en los principios de publicidad, contradicción, concentración, continuidad e inmediación (Art. 4, CNPP) [1], genera un volumen significativo de actuaciones documentales: carpetas de investigación, carpetas administrativas, registros de audiencia, evidencias y resoluciones judiciales que conforman el expediente de cada caso. Un aspecto central del CNPP es la cadena de custodia, definida en su artículo 227 como el sistema de control y registro que se aplica a la evidencia desde su localización hasta que la autoridad competente ordena su conclusión, considerando factores como identidad, estado original, condiciones de recolección, preservación y el registro de todas las personas que hayan tenido contacto con los elementos [1]. El artículo 228 establece que cuando durante el procedimiento de cadena de custodia los indicios se alteren, no perderán su valor probatorio a menos que se verifique que han sido modificados de tal forma que hayan perdido su eficacia probatoria [1]. Garantizar la integridad y trazabilidad de estos elementos es, por tanto, un requisito legal para la validez del proceso penal.

Con la modernización del sistema judicial —y de manera acelerada por la contingencia sanitaria de 2020— las instituciones del Poder Judicial iniciaron una transición hacia el expediente electrónico. El Consejo de la Judicatura Federal reformó sus disposiciones para incorporar medios electrónicos y soluciones digitales como ejes rectores del trabajo jurisdiccional, definiendo el expediente electrónico como el conjunto de documentos electrónicos y digitalizados que integran la totalidad de actuaciones judiciales [3]. A nivel local, el Poder Judicial de la Ciudad de México reporta contar con 15 millones de expedientes digitalizados con valor legal, resultado de un proceso iniciado hace más de una década y respaldado por la reforma de 2021 [4], mientras que entidades como el Estado de México han adoptado modelos de “cero papel” orientados a que escritos, acuerdos, resoluciones y notificaciones se manejen de manera completamente electrónica [5]. Esta transición responde a problemas estructurales del modelo en papel: rezago judicial, extravío de documentos y dificultades de acceso. Como señala la literatura sobre justicia digital en México, el rezago existente en los tribunales ha constituido una violación sistemática al derecho de acceso a la justicia y a la tutela judicial efectiva [6], y la digitalización ofrece beneficios concretos en reducción de tiempos procesales, acceso remoto y menor riesgo de deterioro o pérdida física.

No obstante, la digitalización de expedientes y la coordinación de procesos jurídicos mediante plataformas digitales introduce riesgos que no existían en el entorno físico. Cuando la migración al entorno digital se realiza sin mecanismos de seguridad adecuados, la información no solo queda

expuesta a accesos no autorizados, sino que pierde las propiedades que le otorgan valor probatorio ante un tribunal. El panorama de ciberseguridad en América Latina evidencia la magnitud de este riesgo: según el ESET Security Report 2025, basado en encuestas a más de 3,000 profesionales de TI en más de 15 países de la región, el 27 % de las organizaciones latinoamericanas sufrió al menos un ciberataque durante 2024, mientras que un 32 % adicional reconoció no contar con herramientas para confirmar si fueron atacadas o no [7]. El ransomware ocupa un lugar central en este panorama, con el 95 % de los encuestados ubicándolo entre sus principales amenazas y un 22 % que sufrió un incidente de este tipo en los últimos dos años [7]. En cuanto a la distribución geográfica, la telemetría de ESET para el primer semestre de 2024 posiciona a México como el segundo país con mayor número de amenazas detectadas en Latinoamérica [8].

El sector jurídico es particularmente vulnerable a estas amenazas. Los despachos legales gestionan información confidencial de múltiples clientes en repositorios centralizados, lo que los convierte en objetivos de alto valor para ciberdelincuentes que pueden obtener información de diversas fuentes en un solo ataque [9]. El ABA Legal Technology Survey Report 2023, publicado por la American Bar Association, reporta que el 29 % de los despachos de abogados ha experimentado alguna forma de brecha de seguridad [10], y el costo promedio de una brecha de datos para organizaciones de servicios profesionales —incluyendo firmas legales— alcanzó los 5.08 millones de dólares en 2024, un incremento del 10 % respecto al año anterior según el IBM Cost of a Data Breach Report 2024 [11]. Estas cifras no se limitan a grandes corporaciones: los despachos pequeños y medianos son a menudo atacados precisamente porque sus defensas tienden a ser más débiles [12].

Las consecuencias de esta vulnerabilidad trascienden lo económico y operativo, pues en el ámbito jurídico el problema central es que la evidencia digital almacenada sin garantías de integridad criptográfica resulta legalmente impugnabile. La legislación mexicana establece requisitos específicos para que un documento electrónico tenga valor probatorio pleno: el Código de Comercio, en su artículo 89, reconoce la equivalencia funcional de los documentos electrónicos con los físicos siempre que los datos de creación de la firma correspondan exclusivamente al firmante, estuvieran bajo su control exclusivo al momento de la firma, y sea posible detectar cualquier alteración posterior [13]; la NOM-151-SCFI-2016 complementa este marco al establecer que la integridad de un mensaje de datos se presume cuando ha permanecido íntegro e inalterado desde su generación, lo cual se acredita mediante una constancia de conservación emitida por un Prestador de Servicios de Certificación autorizado [14]. Cuando estos requisitos se cumplen, los documentos electrónicos gozan de presunción legal de atribución y garantía de integridad, con una fuerza probatoria que incluso supera a la del documento en papel [15]; en contraste, cuando un despacho almacena sus expedientes y contratos en plataformas que no implementan estos mecanismos, sus documentos carecen de dichas presunciones y quedan expuestos a impugnación por falta de integridad y autenticidad. Un juez puede restar valor probatorio o rechazar evidencia digital cuando la cadena de custodia presenta inconsistencias, como discrepancias en los valores hash [16], y la propia guía del Poder Judicial de la Federación para la valoración de prueba pericial en materia digital reconoce el hash como pieza clave en la cadena de custodia del contenido digital [17]. Adicionalmente, la gestión de datos personales de clientes por parte de los despachos está sujeta a la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP), cuyo artículo 19 exige implementar medidas de seguridad técnicas para proteger los datos contra daño, pérdida, alteración o acceso no autorizado [18], con sanciones administrativas significativas en caso de incumplimiento.

En síntesis, la digitalización del entorno jurídico es una transición necesaria e irreversible, pero cuando se implementa sin protocolos criptográficos que provean los servicios de integridad y no

repudio, se carece de los mecanismos técnicos para validar la inalterabilidad de los documentos en operaciones críticas. Esta ausencia de controles específicos compromete la certeza sobre la información y el valor probatorio de la evidencia digital en la gestión de expedientes de un despacho de la CDMX. Esta brecha entre la digitalización operativa y la disponibilidad de servicios de seguridad criptográfica constituye el problema que motiva el presente trabajo terminal.

1.2 Propuesta de solución

Se propone un prototipo de sistema web para la gestión de expedientes digitales y la actividad procesal de un despacho legal, que incorpora servicios criptográficos orientados a cerrar las brechas identificadas en el manejo digital de información jurídica.

Para garantizar que cualquier modificación no autorizada sobre un expediente sea detectable, el sistema implementa integridad criptográfica sobre cada documento almacenado, de modo que cualquier alteración —por mínima que sea— quede evidenciada en la siguiente consulta.

Para asegurar que el contenido de los expedientes sea accesible únicamente por las partes autorizadas, el sistema aplica confidencialidad sobre el material almacenado, de forma que un acceso indebido al repositorio no sea suficiente para comprometer la información.

Para impedir que un firmante niegue haber suscrito un contrato, el sistema genera evidencia criptográfica verificable de forma independiente sobre la autoría y el momento exacto de cada firma, proveyendo así no repudio en el flujo contractual del despacho.

Finalmente, para que los tres servicios anteriores tengan validez, el sistema controla que cada operación sea ejecutada únicamente por la identidad que le corresponde, mediante mecanismos de autenticación diferenciados por perfil de usuario.

1.3 Objetivo

Desarrollar un prototipo de sistema web para la gestión de expedientes digitales y la actividad procesal de un despacho legal, que brinde los servicios criptográficos de integridad y no repudio para el manejo de contratos; así como mecanismos de autenticación para accesos y gestión de credenciales.

Objetivos Específicos

- Realizar el análisis de requerimientos, el diseño funcional y técnico del sistema para la gestión de expedientes, contratos, casos y documentos legales. Para tener el flujo de funcionalidades con los que el usuario pueda interactuar y gestionar sus procedimientos y expedientes.
- Implementar los módulos de gestión de expedientes penales, etapas procesales, audiencias, plazos y directorio de participantes conforme a la estructura y terminología del CNPP, incluyendo operaciones CRUD y un calendario jurídico integrado que permita al despacho administrar su actividad procesal desde el prototipo.
- Implementar el servicio de no repudio para el flujo de firma de contratos mediante firma digital (véase §2.3.3) con certificados X.509 (véase §2.3.3.3) y sellado de tiempo (véase §2.3.3.4) conforme al protocolo RFC (Request for Comments) 3161, detrás de un adaptador de autoridad intercambiable. La demostración actual utiliza una TSA local; la integración con un Prestador de Servicios de Certificación autorizado conforme a la NOM-151-SCFI-2016 (véase §2.2.7) queda como extensión posterior.

- Implementar el servicio de integridad mediante dos mecanismos complementarios: verificación por hash SHA-256 (Secure Hash Algorithm de 256 bits; véase §2.3.1) para detectar modificaciones no autorizadas en los documentos almacenados, y cifrado autenticado AES-256-GCM (Estándar de Cifrado Avanzado en modo Galois/Counter; véase §2.3.2) para proveer simultáneamente confidencialidad y detección de alteraciones en los expedientes.
- Implementar mecanismos de autenticación diferenciados por rol (basada en certificados digitales para los socios del despacho y por múltiples factores (MFA) para clientes) con control de acceso basado en roles (RBAC) y gestión segura de credenciales mediante funciones de hashing resistentes a ataques de fuerza bruta.
- Validar el correcto funcionamiento de los servicios criptográficos y los módulos de gestión procesal mediante pruebas funcionales que verifiquen la detección de alteraciones, la validez de las firmas digitales y los sellos de tiempo, y la integridad del flujo de extremo a extremo; complementadas con pruebas de usabilidad con usuarios del despacho.

La siguiente tabla sintetiza, para cada objetivo específico, el criterio de éxito verificable que permitirá declarar cumplido el objetivo al concluir el desarrollo del prototipo.

Tabla 1.1: Criterios de éxito por objetivo específico.

OE	Objetivo específico (resumen)	Criterio de éxito medible
OE-1	Análisis y diseño funcional/técnico	Catálogo completo de 17 casos de uso derivados de las 7 agrupaciones de historias de usuario; arquitectura hexagonal documentada con al menos tres puertos (persistencia, criptografía, TSA); modelo de datos híbrido (relacional y volátil) definido con esquemas normalizados.
OE-2	Módulos de gestión procesal penal (CNPP)	CRUD completo para expedientes, audiencias y plazos; calendario jurídico operando sobre las cuatro etapas del CNPP (investigación, intermedia, juicio oral y recursos); cómputo automático de términos conforme al artículo 94 del CNPP.
OE-3	No repudio con sello RFC 3161	Firma digital RSA-3072 con SHA-256 verificable de forma independiente sobre el 100 % de los contratos firmados; sello de tiempo RFC 3161 local vinculado al hash del documento y verificable con OpenSSL. La tasa sobre un PSC externo queda fuera del alcance actual por la inestabilidad y el costo no transparente del sandbox evaluado.
OE-4	Integridad y confidencialidad	Detección del 100 % de alteraciones de un byte en documentos firmados mediante pruebas de mutación; cifrado AES-256-GCM con nonce único por operación y cero reutilizaciones en una suite de 10 000 cifrados; envelope encryption con DEK por documento.
OE-5	Autenticación diferenciada por rol	Argon2id calibrado a 0.5–1.0 s por intento en el hardware de referencia (m=256 MiB, t=4, p=1); MFA basado en TOTP (RFC 6238 [19]) con ventana de tolerancia ± 1 período; control de acceso RBAC con cuatro roles diferenciados (Owner, Litigante, Paralegal, Cliente).
OE-6	Validación del prototipo	Cobertura de pruebas ≥ 80 % en los módulos criptográficos; al menos 15 pruebas end-to-end que cubran los flujos de firma y verificación; matriz de pruebas funcionales con, por lo menos, un caso positivo y uno negativo por caso de uso.

1.4 Justificación

Los despachos jurídicos requieren infraestructura que provea evidencia técnica con valor probatorio, integridad verificable y mecanismos de no repudio en contratos que aseguren la trazabilidad de la autoría. La legislación mexicana establece requisitos específicos para que un documento electrónico goce de presunción legal de integridad y atribución: el Código de Comercio (art. 89) exige que la firma sea fiable y que cualquier alteración posterior sea detectable [13]; la NOM-151-SCFI-2016 requiere constancias de conservación emitidas por un Prestador de Servicios de Certificación autorizado para acreditar la inalterabilidad del documento [14]; y la LFPDPPP (art. 19) obliga a implementar medidas

de seguridad técnicas para proteger los datos personales contra alteración y acceso no autorizado [18]. Estos requisitos no son opcionales: su incumplimiento expone los expedientes a impugnación por falta de integridad y autenticidad, y al despacho a sanciones administrativas.

Las soluciones comerciales analizadas en el estado del arte priorizan la operatividad sobre estos requisitos normativos, presentando brechas técnicas que comprometen el valor probatorio de la información gestionada:

Dependencia de terceros sin verificación independiente. Las plataformas obligan a los despachos a confiar en certificaciones externas (Service Organization Control 2 (SOC 2) Tipo II e ISO 27001 [20]) sin proporcionar mecanismos para que el propio despacho verifique, de forma independiente y criptográfica, la integridad de sus archivos.

Ausencia de sellado de tiempo conforme a la normativa mexicana. Las plataformas delegan la firma en servicios externos (DocuSign, Adobe Sign) que no implementan sellado de tiempo mediante un PSC autorizado conforme a la NOM-151-SCFI-2016, lo que impide acreditar la existencia del documento en una forma determinada en un momento específico ante la legislación mexicana.

Controles de acceso sin vinculación criptográfica. Si bien existen mecanismos de autenticación, su implementación suele ser inconsistente, careciendo de una vinculación criptográfica entre la identidad del usuario y las operaciones realizadas sobre los documentos.

El prototipo propuesto aborda estas brechas al proveer servicios criptográficos orientados a la generación de evidencia técnica verificable de forma independiente:

No repudio con evidencia temporal. Implementación de firma digital mediante certificados X.509 y sellado de tiempo conforme al protocolo RFC 3161, vinculando criptográficamente la autoría del firmante con el contenido del contrato y una marca temporal emitida por la TSA configurada. En la demostración, la autoridad es local; un PSC autorizado queda reservado para una integración posterior con garantías contractuales verificables.

Integridad y confidencialidad mediante cifrado autenticado. Uso de AES-256-GCM en el almacenamiento de expedientes y documentos, proveyendo simultáneamente protección contra accesos no autorizados y detección de modificaciones, de modo que la seguridad de la información dependa de protocolos criptográficos y no únicamente de políticas de acceso administrativo.

Autenticación robusta y gestión segura de credenciales. Control de acceso mediante certificados digitales para socios y autenticación por múltiples factores (MFA) para clientes, complementado con almacenamiento de contraseñas bajo funciones hash resistentes (Argon2 o bcrypt), mitigando riesgos de suplantación de identidad.

A diferencia de las soluciones del mercado, este prototipo otorga al despacho soberanía sobre su cadena de custodia digital, proveyendo la evidencia criptográfica necesaria para sustentar el valor probatorio de sus documentos y la trazabilidad de su actividad procesal ante cualquier instancia judicial. El alcance del prototipo se acota a la gestión procesal penal conforme al CNPP, validándose con un despacho de la Ciudad de México como caso de estudio.

1.5 Revisión de Plataformas de Gestión Legal

El mercado de software de gestión para despachos legales (Legal Practice Management Software) cuenta con plataformas maduras que cubren funcionalidades operativas como administración de casos, gestión documental y comunicación con clientes. Con el fin de identificar las capacidades y limitaciones de las soluciones existentes en materia de seguridad criptográfica, se analizaron tres plataformas representativas del segmento: Clio [21], MyCase [22] y PracticePanther [23]. Los criterios

de selección fueron su presencia consolidada en el mercado hispanohablante, la disponibilidad pública de documentación técnica sobre sus mecanismos de seguridad, y su orientación a despachos de tamaño pequeño y mediano, perfil equivalente al caso de estudio de este trabajo. La [tabla 1.2](#) sintetiza la comparación en términos de funcionalidad, servicios de seguridad y mecanismos criptográficos reportados.

Tabla 1.2: Resumen ampliado de productos similares (función, servicios y criptografía).

Software	Funcionalidad principal	Cifrado a nivel documento	Firma digital	Sellado de tiempo (TSA)	Cumplimiento normativo MX
Clio	Gestión de casos, facturación, portal de clientes	No (solo en reposo a nivel infraestructura)	No	No	No reportado
MyCase	Comunicación cliente–abogado, cobros, apps móviles	No (solo en reposo a nivel infraestructura)	No	No	No reportado
PracticePanther	Plantillas, automatizaciones, registro de tiempo	No (solo en reposo a nivel infraestructura)	No	No	No reportado
Propuesta	Expedientes contratos y documentos legales	AES-256-GCM (cifrado autenticado)	X.509 (ECD-SA/RSA)	NOM-151 vía PSC	NOM-151, Código de Comercio Art. 89 bis

1.5.1 Mecanismos Criptográficos y Brechas Identificadas

Las tres plataformas convergen en un conjunto reducido de controles criptográficos: cifrado simétrico AES-256 en reposo para confidencialidad de los datos almacenados [24], Seguridad de la Capa de Transporte (TLS, Transport Layer Security) para la protección del canal de comunicación y mecanismos de autenticación basados en Autenticación por Múltiples Factores (MFA) y Control de Acceso Basado en Roles (RBAC). Clio se distingue al ser la única con certificaciones independientes (Service Organization Control 2 (SOC 2) Tipo II e ISO 27001) [20] y transparencia operativa verificable, lo que le otorga un perfil de riesgo bajo. MyCase [22] y PracticePanther [23] presentan un perfil de riesgo moderado, derivado de la ausencia de auditorías verificables y políticas de retención de datos ambiguas que dificultan la portabilidad de información.

Sin embargo, estos mecanismos son controles de prevención de acceso y confidencialidad, no de evidencia. Controlan quién puede acceder a los datos, pero no generan prueba criptográfica de quién realizó una operación ni cuándo la realizó. Del análisis se identifican tres mecanismos ausentes que son relevantes para escenarios con valor probatorio:

Firma digital con certificados X.509. Las plataformas delegan la firma en servicios externos (DocuSign, Adobe Sign), pero no implementan un flujo nativo de firma digital basada en criptografía

asimétrica que vincule criptográficamente la identidad del firmante con el documento de forma verificable de manera independiente.

Sellado de tiempo conforme a la NOM-151-SCFI-2016. Ninguna plataforma implementa sellado de tiempo mediante un Prestador de Servicios de Certificación autorizado [14], mecanismo necesario para acreditar que un documento existía en una forma determinada en un momento específico ante la legislación mexicana.

Cifrado autenticado. Si bien todas reportan AES-256, no se documenta el uso de modos de cifrado autenticado (como AES-GCM) que provean simultáneamente confidencialidad e integridad en un solo mecanismo, permitiendo detectar cualquier modificación no autorizada del contenido cifrado.

En síntesis, las plataformas existentes han alcanzado madurez en funcionalidades operativas y controles de seguridad estándar, pero presentan una brecha consistente en servicios criptográficos orientados a evidencia. La propuesta de este trabajo terminal se posiciona en esa brecha: no busca competir en amplitud de funcionalidades, sino proveer los servicios de no repudio, integridad y confidencialidad que estas plataformas no ofrecen, orientados a escenarios donde el valor probatorio de la información digital es un requisito.

1.6 Alcances y Delimitaciones

Para acotar el esfuerzo de investigación y desarrollo del prototipo, se establecen a priori los siguientes alcances y delimitaciones. Aquellos aspectos no listados expresamente como alcance del prototipo se consideran fuera del objeto del Trabajo Terminal.

- El prototipo opera bajo una arquitectura de inquilino único (single-tenant): cada instancia atiende a un solo despacho, simplificando el modelo de aislamiento y los mecanismos de cifrado por organización.
- La gestión procesal se limita a la materia penal conforme al CNPP; otras materias (civil, mercantil, laboral o administrativa) quedan fuera del alcance por requerir esquemas procesales y de valoración probatoria distintos.
- El caso de estudio corresponde a un despacho con sede en la Ciudad de México. La adaptación a particularidades procesales de otras entidades federativas no forma parte del prototipo.
- El sistema se ofrece como aplicación web accesible desde navegadores modernos; no se desarrolla una aplicación móvil nativa ni cliente de escritorio.
- El ámbito jurídico es estrictamente privado (Código de Comercio); no se contempla la integración con la Ley de Firma Electrónica Avanzada (LFEA), cuyo ámbito es la Administración Pública Federal.
- La firma digital emitida por el prototipo es firma electrónica simple en los términos del artículo 89 del Código de Comercio. Al no provenir de un Prestador de Servicios de Certificación autorizado, no alcanza la presunción de firma avanzada establecida en el artículo 97 del mismo ordenamiento. La TSA local aporta evidencia técnica RFC 3161, pero no una constancia NOM-151 ni la independencia de un tercero autorizado.
- La integración con el PSC Cincel fue evaluada como adaptador opcional, pero la falta de respuesta y estabilidad de la API, junto con el costo no transparente del sandbox, impide usarla

como evidencia de esta entrega. El prototipo demuestra el sellado con una TSA local; un despliegue productivo requeriría seleccionar y contratar conscientemente un PSC autorizado.

1.7 Supuestos del Proyecto

El desarrollo del prototipo parte de los siguientes supuestos. La falsación de cualquiera de ellos durante la ejecución del Trabajo Terminal motivaría la activación de los planes de mitigación descritos en el análisis de riesgos (§3.6).

- La TSA local de OpenSSL y sus scripts de preparación se mantienen disponibles durante el ciclo de desarrollo y pruebas. La disponibilidad de un PSC externo no se asume.
- El hardware de referencia dispone de memoria suficiente para ejecutar Argon2id con 256 MiB por proceso de autenticación sin degradar el tiempo de respuesta del inicio de sesión. Si el hardware final difiere, la calibración debe repetirse antes del despliegue.
- Existe conectividad estable a internet para consultar listas de revocación (CRL) y servicios OCSP cuando el despliegue los requiera. El flujo de demostración no depende de la conectividad con Cincel.
- Los usuarios acceden al sistema mediante navegadores modernos con soporte para ECMAScript 2020, Web Crypto API y TLS 1.2 o superior.
- El equipo de dos integrantes mantiene disponibilidad continua durante las diez semanas del cronograma XP, con cuatro jornadas efectivas por semana dedicadas al proyecto.
- La directora del Trabajo Terminal provee retroalimentación al final de cada iteración semanal, totalizando cinco entregas intermedias en la Fase I y cinco en la Fase II.
- Los socios del despacho caso de estudio están dispuestos a participar en pruebas de usabilidad de los flujos de firma y verificación.

CAPÍTULO 2

Marco teórico

El presente capítulo expone los fundamentos teóricos que sustentan el trabajo. Los temas abordados fueron seleccionados en función de los dos dominios que convergen en la problemática: el jurídico-procesal, que define qué se gestiona y bajo qué restricciones legales; y el criptográfico-técnico, que determina cómo se garantiza la validez probatoria de la información gestionada. El capítulo se organiza en cuatro bloques. El primero aborda el contexto procesal: el sistema acusatorio oral establecido por la reforma constitucional de 2008, la estructura del proceso penal conforme al CNPP —etapas, sujetos procesales, audiencias y plazos fatales— y los conceptos de expediente digital y cadena de custodia que determinan los requisitos del modelo de datos. El segundo describe el marco legal mexicano aplicable: la NOM-151-SCFI-2016, el Código de Comercio y la LFPDPPP, cuyo cumplimiento condiciona la validez probatoria de los documentos gestionados. El tercer bloque desarrolla los fundamentos criptográficos pertinentes: integridad mediante funciones hash (SHA-256, Estándar Federal de Procesamiento de Información (FIPS) 180-4 [25]) y cifrado autenticado (AES-256-GCM, FIPS 197 [26]); no repudio mediante firma digital con certificados X.509 y sellado de tiempo conforme al protocolo RFC 3161 [27]; y autenticación y control de acceso mediante certificados digitales, MFA y RBAC. El cuarto bloque cubre la arquitectura de almacenamiento híbrida y la metodología XP. Las referencias de cada sección se indican en el texto para quien desee profundizar en los temas tratados.

2.1 Seguridad de la Información en el Ámbito Jurídico

La seguridad de la información constituye la arquitectura sobre la cual se construye la confianza en el entorno digital. En la práctica jurídica, la transición del expediente físico al electrónico exige que las propiedades del documento en papel —dificultad de alteración, firma autógrafa, sello notarial— sean replicadas y superadas mediante herramientas criptográficas.

La tríada clásica de la seguridad (Confidencialidad, Integridad y Disponibilidad) [28] resulta insuficiente en el contexto del derecho, donde el concepto central es el valor probatorio: la capacidad de un documento electrónico de ser presentado como prueba legítima en un procedimiento legal. Stallings [28] señala que la seguridad no es un componente que se añade a un sistema, sino un proceso continuo orientado a garantizar que los activos de información estén protegidos contra amenazas que comprometan su veracidad.

En el ámbito jurídico digital, la seguridad se materializa en tres dimensiones verificables: que el contenido de los expedientes no haya sido alterado (integridad), que el autor de una operación —particularmente la firma de un contrato— no pueda negar haberla realizado (no repudio) y que solo las partes autorizadas accedan al contenido de los documentos almacenados (confidencialidad).

2.2 Sistema Legal Mexicano

2.2.1 El Sistema Acusatorio Oral en México

La reforma constitucional del 18 de junio de 2008 al artículo 20 de la Constitución Política de los Estados Unidos Mexicanos transformó estructuralmente el sistema de justicia penal, sustituyendo el modelo mixto-inquisitivo por un sistema de corte acusatorio y oral. Este cambio de paradigma no es meramente procedimental; representa una transformación en la ontología del derecho penal mexicano: el proceso abandona la secrecía de las oficinas judiciales para trasladarse a la audiencia pública, y el expediente de papel deja de ser el receptáculo de la verdad jurídica para convertirse en un registro auxiliar de lo que las partes producen de manera contradictoria ante el juez [1], [29].

El modelo inquisitivo anterior se sustentaba en la desconfianza: el juez delegaba sus funciones en secretarios, la “verdad” era aquello que quedaba plasmado en el papel, frecuentemente sin que el juzgador hubiera tenido contacto directo con las partes o las pruebas, y la prisión preventiva operaba como regla general. La doctrina procesal contemporánea, impulsada por figuras como Miguel Carbonell y Alberto Binder, sostiene que el sistema acusatorio busca equilibrar la eficacia en la persecución del delito con el respeto irrestricto a las garantías individuales [30], situando al imputado como sujeto de derechos y no como objeto de persecución penal. La **tabla 2.1** sintetiza las diferencias estructurales entre ambos modelos.

Tabla 2.1: Comparación entre el sistema inquisitivo y el sistema acusatorio oral.

Característica	Sistema Inquisitivo (tradicional)	Sistema Acusatorio (vigente)
Forma del proceso	Predominantemente escrito y secreto [29].	Oral, público y transparente [29].
Rol del juez	Investiga, acusa y juzga; delega funciones en secretarios [31].	Tercero imparcial; presencia obligatoria en todas las audiencias [31].
Valor de la confesión	Prueba reina; frecuentemente obtenida bajo coacción [29].	Solo tiene valor si se rinde voluntariamente ante el juez con defensa técnica [1].
Prisión preventiva	Regla general para la mayoría de los delitos [29].	Medida cautelar excepcional y proporcional; opera la presunción de inocencia [1].
El expediente	“Lo que no está en autos no está en el mundo”; centro de la verdad jurídica [31].	Registro auxiliar; la prueba se produce en la audiencia viva ante el juez [31].

Los Principios Rectores del Artículo 4 del CNPP como Base del Modelado Digital

El artículo 4 del CNPP no es un catálogo de aspiraciones, sino la estructura normativa que define la validez de todo acto procesal [1]. Cada principio impone exigencias técnicas concretas que cualquier herramienta de gestión procesal digital debe satisfacer [32].

- **Publicidad.** Las audiencias son abiertas al público salvo excepciones taxativas. En la abstracción digital este principio implica un control de acceso diferenciado por rol: el estado del proceso debe ser consultable por las partes legitimadas, mientras los documentos reservados permanecen cifrados para terceros no autorizados [1].
- **Contradicción.** Las partes tienen derecho a conocer, controvertir y confrontar los medios de prueba de la contraparte. En el ámbito digital, esto se traduce en descubrimiento probatorio: todas las partes deben tener acceso oportuno y completo a las evidencias digitales, con trazabilidad de quién las consultó y cuándo [1].
- **Concentración.** El mayor número de actos procesales posible debe realizarse en una sola audiencia o en el menor número de días consecutivos. En la práctica, este principio exige una gestión eficiente de la agenda que evite fragmentaciones innecesarias y anticipe riesgos a la continuidad de las audiencias [1].
- **Continuidad.** Las audiencias deben desarrollarse de forma sucesiva y secuencial hasta su conclusión. Este principio exige una bitácora de actividad inmutable (append-only) que registre el flujo cronológico del proceso, impidiendo saltos lógicos o alteraciones en el orden de las actuaciones [1].
- **Inmediación.** El juez debe estar presente físicamente en todas las audiencias y percibir de manera directa las pruebas y alegatos. En contextos de videoconferencia, los certificados digitales permiten verificar que quien firma una resolución es efectivamente el juzgador que presidió la sesión [1].
- **Igualdad entre las partes.** Tanto la defensa como el Ministerio Público deben tener las mismas oportunidades de acceso a la información procesal. Un esquema de control de acceso basado en roles debe reflejar esta paridad, sin conferir ventajas informacionales a ninguna de las partes [1].

2.2.2 Estructura del Proceso Penal conforme al CNPP

El artículo 211 del CNPP estructura el procedimiento ordinario en etapas secuenciales con finalidades, actuaciones, plazos y sujetos diferenciados [1]. Cada etapa habilita actuaciones específicas y activa plazos que deben monitorearse. La transición de una etapa a la siguiente solo es posible mediante el documento procesal habilitante correspondiente, lo que impide saltos lógicos que comprometerían la validez del expediente.

Etapas de Investigación

La investigación se subdivide en dos fases. La **investigación inicial** comienza con la noticia criminal —denuncia, querrela o detención en flagrancia— y concluye cuando el imputado queda a disposición del juez de control. Su instrumento documental es la **Carpeta de Investigación**, que es el registro administrativo del Ministerio Público y contiene indicios y datos de prueba aún no validados judicialmente [1], [33].

La **investigación complementaria** inicia con la formulación de cargos en la audiencia inicial y concluye con el escrito de acusación. El artículo 321 del CNPP fija plazos máximos de dos meses —cuando la pena máxima no supera los dos años— y seis meses para los demás casos, salvo prórroga

autorizada judicialmente. El cumplimiento de estos plazos es crítico, y la fase incluye actos de investigación complejos como peritajes, entrevistas a testigos e inspecciones de lugar [1], [33].

Etapas Intermedia

La etapa intermedia, también denominada de preparación del juicio oral, tiene por objeto la depuración de los hechos controvertidos y la admisión de los medios de prueba [1]. Comprende una **fase escrita**—activada con la presentación del escrito de acusación— y una **fase oral** desarrollada en la audiencia intermedia, donde las partes debaten la pertinencia y licitud de las pruebas ofrecidas. Esta etapa culmina con el **auto de apertura a juicio oral**, que determina los hechos acusados, las pruebas admitidas, los acuerdos probatorios y el tribunal competente [1]. El auto de apertura constituye el documento de transferencia de estado que habilita la etapa de juicio oral.

Etapas de Juicio Oral

El juicio oral es la fase de decisión de las cuestiones esenciales del proceso y se rige por la aplicación estricta de los seis principios del artículo 4 del CNPP [1]. En esta etapa la prueba se desahoga ante el tribunal de enjuiciamiento con plena contradicción; los registros audiovisuales de las audiencias constituyen el “*acta*” del juicio. La integridad de estos registros es prioritaria, y el fallo debe documentarse con firma digital del tribunal inmediatamente tras su pronunciamiento para garantizar su autenticidad [32].

Recursos, Impugnación y Ejecución

El proceso no finaliza con la sentencia de primer grado. El CNPP prevé **recursos de apelación** ante un Tribunal de Alzada dentro de los plazos del artículo 471 del CNPP, y posteriormente el seguimiento de la ejecución de la sentencia firme ante el Juez de Ejecución de Sanciones. En esta fase los registros del expediente original adquieren especial relevancia probatoria, dado que los tribunales revisores basan su análisis exclusivamente en las constancias documentadas [1].

2.2.3 Sujetos Procesales

El artículo 105 del CNPP define el catálogo de sujetos procesales con roles y atribuciones específicas que condicionan su interacción con el expediente digital [1]. La correcta representación de cada sujeto no es solo una cuestión de nomenclatura: determina la lógica del control de acceso basado en roles (RBAC). Un mismo individuo puede desempeñar roles distintos en procedimientos diferentes, por lo que la identidad del usuario debe vincularse con su rol procesal a nivel de caso, no de manera global [34], [35].

Tabla 2.2: Sujetos procesales del CNPP y sus implicaciones en la gestión digital del expediente.

Sujeto procesal	Atributos para el directorio (RF-07)	Implicaciones en la gestión digital
Ministerio Público	Identificador de Fiscalía, Unidad de Adscripción, Cédula Profesional [34].	Carga la acusación, solicita medidas cautelares y registra actos de investigación en la Carpeta.
Imputado	CURP, datos generales, estatus de libertad o detención [1].	Acceso restringido a la carpeta digital para consulta de pruebas y recepción de notificaciones.
Defensor	Cédula Profesional validada ante el PJ, firma electrónica, modalidad (particular o público) [34].	Presenta promociones, contesta la acusación, ofrece pruebas e interpone recursos.
Víctima u ofendido	Datos de contacto seguro, medidas de protección activas [1].	Consulta el estado del proceso y solicita reparación del daño mediante el portal restringido.
Asesor jurídico de la víctima	Datos de institución o despacho, cédula profesional [34].	Coadyuva con el Ministerio Público en la oferta de pruebas y en la presentación de alegatos.
Juez de control	Órgano jurisdiccional, Firma Electrónica del Poder Judicial de la Federación (FIREL) [3].	Dicta resoluciones, autoriza actos de investigación y preside la audiencia intermedia [1].
Tribunal de enjuiciamiento	Composición (unitario o colegiado), distrito judicial [34].	Emite el fallo y la sentencia basándose exclusivamente en los registros del juicio oral [1].

2.2.4 Actuaciones Procesales y el Expediente Digital

El proceso penal acusatorio genera dos conjuntos documentales estructuralmente distintos: la Carpeta de Investigación y el Expediente Judicial. Ambos difieren en origen, custodia y nivel de validación probatoria, lo que condiciona su modelado en cualquier herramienta digital.

Carpeta de Investigación versus Expediente Judicial

La **Carpeta de Investigación** es el registro del Ministerio Público. Contiene indicios, datos de prueba e información de gestión que aún no han sido validados judicialmente ni sometidos al contradictorio. Digitalmente, constituye un repositorio dinámico que se alimenta de reportes policiales, dictámenes periciales y actas ministeriales. Su acceso está restringido a los actores de la investigación y, en los términos del artículo 218 del CNPP, a la defensa en la medida necesaria para el ejercicio de sus derechos [1].

El **Expediente Judicial** reside en el Poder Judicial y contiene exclusivamente las actuaciones presentadas ante el juez y los registros de audiencias celebradas. El Acuerdo General 12/2020 del

Consejo de la Judicatura Federal establece que el expediente electrónico se integra cronológicamente con documentos digitalizados y generados electrónicamente, los cuales tienen el mismo valor probatorio que los físicos siempre que cuenten con firma electrónica y sello de tiempo (marca temporal criptográfica emitida por un tercero de confianza; véase §2.3.3.4) que acrediten su autenticidad e integridad [3].

Identificación Única: el NUC y la Carpeta Judicial

La trazabilidad entre instancias es el fundamento técnico de la cadena de custodia digital y se articula sobre dos identificadores complementarios:

- **Número Único de Caso (NUC).** Identificador nacional que permite rastrear un asunto desde que se denuncia hasta sentencia firme. Constituye la clave que garantiza la interoperabilidad entre fiscalías y tribunales [1].
- **Carpeta Judicial.** Número interno asignado por el juzgado al iniciar el proceso en sede jurisdiccional. La vinculación bidireccional NUC–Carpeta Judicial es necesaria para que el abogado pueda rastrear sus casos con independencia del identificador que utilice la autoridad en cada comunicación oficial [4].

La Cadena de Custodia Digital y su Valor Probatorio

La cadena de custodia, definida en el artículo 227 del CNPP como el sistema de control y registro que se aplica a los indicios desde su localización hasta que la autoridad ordena su conclusión [1], es el mecanismo jurídico que dota de valor probatorio a la evidencia digital. Su traslado al entorno digital exige tratar cada actuación procesal como una evidencia sujeta a tres garantías técnicas:

- **Integridad.** Hash SHA-256 calculado al momento de la carga y verificado en cada consulta posterior. Cualquier discrepancia evidencia modificación no autorizada.
- **Trazabilidad.** Registro de auditoría inmutable (append-only) que documenta por cada operación: el identificador del usuario, el tipo de operación, la marca de tiempo y el estado del documento afectado.
- **No repudio.** Firma digital con certificados X.509 y sellado de tiempo de un PSC autorizado conforme a la NOM-151-SCFI-2016, generando evidencia criptográfica verificable de forma independiente sobre la autoría y el momento de cada operación [1], [14].

2.2.5 Plazos y Términos Constitucionales

El proceso penal acusatorio está estructurado sobre plazos fatales cuyo vencimiento genera consecuencias jurídicas irreversibles, incluyendo la libertad del imputado o la preclusión de derechos procesales. El error en el cómputo de un plazo constitucional es una contingencia que ningún mecanismo criptográfico puede subsanar a posteriori, lo que convierte la gestión de plazos en una función de alta criticidad operativa.

Reglas de Cómputo según el Artículo 94 del CNPP

El artículo 94 del CNPP establece las siguientes reglas de cómputo [1]:

- **Plazos en horas:** Corren de momento a momento, sin interrupción por días inhábiles. Aplican a los plazos constitucionales más críticos, como las 72 horas para la vinculación a proceso.
- **Plazos en días:** Comienzan a contarse a partir de que surte efectos la notificación. No se contabilizan sábados, domingos ni días feriados del calendario del Poder Judicial, salvo tratándose de detención y medidas precautorias, donde todos los días y horas son hábiles [1].
- **Calendario configurable:** La correcta aplicación de estas reglas requiere un calendario de días inhábiles diferenciado por entidad federativa y por fuero, dado que los periodos de receso judicial varían entre jurisdicciones.

Términos Fatales del Proceso Penal

Tabla 2.3: Términos fatales del proceso penal y su implicación para el módulo de alertas (RF-09, RF-10).

Actuación / plazo	Término legal	Fundamento	Implicación para la gestión digital
Retención por el Ministerio Público	48 horas	Art. 16 Constitucional	Alerta roja al vencer sin judicialización del detenido.
Vinculación a proceso	72 / 144 horas	Art. 19 Constitucional; Art. 308 CNPP [1]	Agendar audiencia de continuación con prioridad máxima; alerta a las 48 h.
Investigación complementaria	2 o 6 meses	Art. 321 CNPP [1]	Recordatorio para solicitar prórroga o presentar cierre de investigación.
Presentación de acusación	15 días	Art. 324 CNPP [1]	Plazo tras el cierre; su vencimiento activa el sobreseimiento.
Recurso de apelación de sentencia	10 días	Art. 471 CNPP [1]	Plazo fatal desde la notificación; su pérdida cierra la vía impugnativa ordinaria.

2.2.6 Audiencias Tipificadas conforme al CNPP

El proceso acusatorio oral se articula en torno a audiencias formales cuya tipificación no puede reducirse a una etiqueta descriptiva: cada tipo activa validaciones procesales distintas y desencadena el cómputo de plazos derivados. Un catálogo de tipos de audiencia con semántica procesal explícita es necesario para garantizar la coherencia del expediente [1], [36].

- **Audiencia inicial (Arts. 307–313 CNPP).** Primera comparecencia del imputado ante el juez de control. En ella se realizan, en orden procesal definido: control de detención, formulación de cargos, oportunidad de declarar del imputado, resolución sobre medidas cautelares y vinculación o no vinculación a proceso con apertura del plazo de investigación complementaria. Cada hito

debe registrarse de forma independiente, ya que cada uno puede generar plazos y recursos propios [1].

- **Audiencia de medidas cautelares.** Puede solicitarse en cualquier momento de la investigación, de manera autónoma o como parte de la audiencia inicial. Las medidas impuestas —prisión preventiva, brazalete electrónico, firma periódica— requieren un historial con fechas de inicio, vigencia y condiciones de cumplimiento [1].
- **Audiencia intermedia (Arts. 334–347 CNPP).** Tiene por objeto depurar los hechos controvertidos y resolver sobre la admisión de medios de prueba. La gestión probatoria implica distinguir qué pruebas fueron admitidas, cuáles excluidas y los motivos de exclusión; esta etapa culmina con el auto de apertura a juicio oral como documento de transición de estado [1].
- **Audiencias de juicio oral (Arts. 348–395 CNPP).** Comprenden los alegatos de apertura, el desahogo de pruebas con interrogatorio y contrainterrogatorio, los alegatos de clausura y la deliberación del tribunal. Las incidencias de cada sesión deben vincularse a la marca de tiempo del video de sala, y el fallo debe documentarse con firma digital del tribunal inmediatamente tras su pronunciamiento [1].
- **Audiencia de individualización de sanciones y reparación del daño (Arts. 399–403 CNPP).** Realizada tras un fallo condenatorio para debatir la pena y la reparación del daño. La sentencia exacta y los montos de reparación deben vincularse al expediente con mecanismos de no repudio, dado que este documento constituye el título ejecutivo de la condena [1].

2.2.7 Normatividad Mexicana de Firma Electrónica y Datos

Para que los servicios criptográficos tengan validez en el contexto jurídico mexicano, deben alinearse con la legislación que otorga reconocimiento legal a las operaciones electrónicas. A continuación, se describen los ordenamientos aplicables.

NOM-151-SCFI-2016

La NOM-151-SCFI-2016 es la norma técnica que establece los requisitos para la conservación de mensajes de datos y la digitalización de documentos. Su disposición central establece que la integridad de un mensaje de datos se presume si este ha permanecido íntegro y sin alteraciones desde el momento en que se generó en su forma definitiva, lo cual se acredita mediante la constancia de conservación emitida por un Prestador de Servicios de Certificación autorizado. En la práctica, esto se traduce en la obtención de sellos de tiempo de un PSC autorizado para cada operación que requiera preservación probatoria.

Código de Comercio: Equivalencia Funcional

El Título Segundo del Código de Comercio (artículos 89 a 114) [13] establece el marco jurídico que otorga reconocimiento legal a las transacciones electrónicas en México. El artículo 89 define la firma electrónica como los datos en forma electrónica consignados en un mensaje de datos, o adjuntados o lógicamente asociados al mismo, que pueden ser utilizados para identificar al firmante en relación con el mensaje de datos e indicar que el firmante aprueba la información contenida en el mismo. Esta definición —denominada firma electrónica simple— no exige la intervención de un Prestador

de Servicios de Certificación y produce los mismos efectos jurídicos que la firma autógrafa, siendo admisible como prueba en juicio. Complementariamente, el artículo 89 bis consagra el principio de no discriminación: no se negarán efectos jurídicos, validez o fuerza obligatoria a cualquier tipo de información por la sola razón de que esté contenida en un mensaje de datos.

El artículo 93 establece que cuando la ley exija forma escrita para un acto jurídico, dicho requisito se cumple con un mensaje de datos siempre que la información en él contenida se mantenga íntegra y sea accesible para su ulterior consulta. A su vez, el artículo 93 bis dispone que cuando se requiera que un documento se conserve y presente en su forma original, tal requisito se satisface si el mensaje de datos se ha mantenido íntegro e inalterado a partir del momento en que se generó por primera vez en su forma definitiva, conforme a la NOM-151-SCFI-2016. Este artículo es la base legal que dota al sello de tiempo emitido por un PSC autorizado de su eficacia como constancia de conservación.

El artículo 97 establece los requisitos para que una firma electrónica sea considerada avanzada o fiable: (1) que los datos de creación de la firma correspondan exclusivamente al firmante, (2) que dichos datos estuvieran bajo su exclusivo control al momento de firmar, (3) que sea posible detectar cualquier alteración de la firma posterior a su creación, (4) que sea posible detectar cualquier alteración del mensaje de datos firmado y (5) que los datos de creación de la firma estén vinculados a un certificado emitido por un PSC autorizado. La ausencia de este último requisito es lo que distingue a la firma simple de la avanzada. No obstante, el artículo 100 introduce una cláusula de equivalencia funcional: una firma electrónica se considerará avanzada o fiable si, por cualquier otro medio, se demuestra que el método utilizado es confiable y apropiado para los fines para los cuales se utilizó el mensaje de datos [13]. Esta disposición permite argumentar la fiabilidad de una firma basada en algoritmos reconocidos internacionalmente, como RSA (Rivest-Shamir-Adleman) con SHA-256 conforme a NIST (National Institute of Standards and Technology) SP 800-57 [37], aunque el certificado no provenga de un PSC autorizado.

La distinción fundamental entre ambos tipos de firma no radica en la admisibilidad —ambas son admisibles como prueba conforme al artículo 89—, sino en la carga de la prueba. La firma electrónica avanzada goza de una presunción legal de fiabilidad: quien la impugna debe demostrar que no es confiable. La firma electrónica simple, en cambio, no goza de tal presunción: quien la invoca como prueba debe acreditar la fiabilidad del método empleado. Esta diferencia procesal es relevante para cualquier herramienta de gestión jurídica digital que pretenda sustentar el valor probatorio de sus documentos ante una instancia judicial. Adicionalmente, el artículo 33 del mismo Código obliga a los comerciantes a mantener registros mercantiles por un periodo mínimo de diez años, lo cual fundamenta los requisitos de retención y conservación de datos [13].

El valor probatorio de los documentos electrónicos en sede judicial se sustenta en disposiciones complementarias. El artículo 210-A del Código Federal de Procedimientos Civiles [38] reconoce como prueba la información generada o comunicada que conste en medios electrónicos, ópticos o en cualquier otra tecnología; el juzgador valora dicha información atendiendo a tres criterios: (1) la fiabilidad del método en que haya sido generada, comunicada, recibida o archivada, (2) la posibilidad de atribuir a las personas obligadas el contenido de la información, y (3) la posibilidad de determinar el momento en que fue generada, comunicada o archivada. Una firma digital basada en RSA con SHA-256 satisface el criterio de atribución, la función hash garantiza la integridad exigida por el criterio de fiabilidad, y un sello de tiempo emitido por un PSC autorizado provee la certeza temporal.

En materia penal, el artículo 356 del CNPP establece que el Tribunal de enjuiciamiento valora la prueba de manera libre y lógica (sistema de sana crítica), sin reglas tasadas de valor probatorio [1]. Esto significa que un documento firmado electrónicamente con criptografía robusta y respaldado por un

sello NOM-151 puede alcanzar valor probatorio pleno si se acreditan integridad, autenticidad y cadena de custodia ante el juzgador. Complementariamente, el artículo 380 del CNPP dispone que podrá incorporarse al juicio todo documento o instrumento con contenido probatorio, incluyendo registros electrónicos [1]. Finalmente, es pertinente señalar que la Ley de Firma Electrónica Avanzada (LFEA) [39], en sus artículos 1 y 2, circunscribe su ámbito de aplicación a las dependencias y entidades de la Administración Pública Federal y a los particulares cuando se relacionen con dichas dependencias; en consecuencia, las transacciones entre particulares —como es el caso de un despacho jurídico privado— se regulan exclusivamente por el Código de Comercio.

Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP)

Dado que un despacho jurídico gestiona información personal de sus clientes, debe cumplir con la LFPDPPP. El artículo 19 de esta ley exige que el responsable del tratamiento de datos implemente medidas de seguridad administrativas, técnicas y físicas para proteger los datos personales contra daño, pérdida, alteración, destrucción, o acceso y tratamiento no autorizados. Servicios como el cifrado autenticado, el control de acceso basado en roles y la gestión segura de credenciales constituyen medidas técnicas orientadas al cumplimiento de esta obligación.

Prestadores de Servicios de Certificación en México

La NOM-151-SCFI-2016 y el Código de Comercio no otorgan a cualquier entidad la facultad de emitir constancias de conservación con valor legal; esta capacidad está reservada exclusivamente a los **Prestadores de Servicios de Certificación (PSC)**, entidades acreditadas y auditadas por la Secretaría de Economía a través de la Dirección General de Normatividad Mercantil, conforme al artículo 95 bis 6 del Código de Comercio [40]. La obtención de esta acreditación representa una barrera de entrada formidable: las instituciones privadas, notarías públicas o dependencias gubernamentales deben demostrar capacidades tecnológicas, procedimentales y financieras excepcionales para operar como terceros de confianza legalmente reconocidos [40].

En términos técnicos, el PSC nunca almacena el documento original del cliente —lo que vulneraría principios elementales de privacidad—, sino que opera sobre el principio de la abstracción criptográfica unidireccional [41]: el sistema cliente calcula el hash SHA-256 del documento y transmite únicamente esa huella a la infraestructura del PSC, la cual activa su Autoridad de Sellado de Tiempo (TSA). La TSA concatena el hash con una marca de tiempo sincronizada con el Centro Nacional de Metrología (CENAM) y aplica su propia firma electrónica avanzada sobre ese conjunto de datos, generando el Sello Digital de Tiempo (SDT) conforme al protocolo RFC 3161 [41]. El documento resultante —comúnmente en formato .asn1 o .tsr— constituye la evidencia matemática irrefutable de la existencia e integridad del documento en ese momento específico [41].

La infraestructura de clave pública (PKI) mexicana se organiza jerárquicamente bajo la Autoridad Certificadora Raíz Segunda de la Secretaría de Economía (ACR2-SE). Conforme a su Política de Certificados [42], los certificados digitales emitidos bajo esta jerarquía deben contener claves públicas RSA con longitud mínima de 4096 bits para la raíz y Autoridades Certificadoras (ACs) subordinadas, y de 2048 bits para los certificados de usuario final. El algoritmo de firma obligatorio es sha256WithRSAEncryption (RSA + SHA-256); la política no contempla curvas elípticas (ECDSA). Los módulos criptográficos de las ACs deben cumplir al menos FIPS 140-2 nivel 3. Esta restricción algorítmica condiciona cualquier integración con certificados legalmente reconocidos en México, ya que la interoperabilidad con la jerarquía ACR2-SE exige el uso exclusivo de RSA.

Cada PSC es responsable de mantener mecanismos de revocación de certificados conforme al numeral 75, fracción V, de las Reglas Generales a las que deberán sujetarse los PSC [43]. En la práctica, los principales PSC mexicanos implementan Listas de Certificados Revocados (CRL, conforme a RFC 5280 [44]) y, en la mayoría de los casos, responders del Protocolo de Estado de Certificado en Línea (OCSP). SeguriData [45] publica CRLs cada 24 horas y expone un servicio OCSP público. CECOBAN [46] mantiene CRLs y un servicio OCSP con latencia de publicación de 15 minutos. La e.firma del SAT (Servicio de Administración Tributaria) opera un responder OCSP semi-público, aunque no publica CRLs en formato X.509 estándar. La propia ACR2-SE publica su CRL mensualmente o al revocar un certificado, y mantiene un servicio OCSP v1. Las URLs específicas de cada PSC se encuentran embebidas en los campos Authority Information Access (AIA) y CRL Distribution Points de los certificados que emiten.

Ecosistema de PSC acreditados por la Secretaría de Economía La Secretaría de Economía mantiene un directorio público de entidades que han superado las evaluaciones de cumplimiento para operar como PSC [40]. Es fundamental distinguir que las acreditaciones son específicas por servicio: una entidad puede estar autorizada para emitir certificados digitales, pero no necesariamente para emitir constancias de conservación NOM-151 [40]. La [tabla 2.4](#) sintetiza los principales PSC acreditados relevantes para el contexto del prototipo:

Tabla 2.4: Principales PSC acreditados por la Secretaría de Economía relevantes para el prototipo.

PSC Acreditado	Servicios acreditados relevantes	Perfil tecnológico	Acreditación (Diario Oficial de la Federación, DOF)
CECOBAN, S.A. de C.V.	NOM-151, sellos de tiempo, digitalización.	Infraestructura bancaria sistémica; rigor metrológico con sincronización CENAM. Orientado a transacciones financieras de alta frecuencia [40].	2008, 2010, 2021
EDICOM (Edicomunicaciones México)	NOM-151, sellos de tiempo, digitalización, certificados digitales.	Plataforma global de intercambio electrónico de datos (EDI); integración con eIDAS europeo y Dropbox Sign. Orientado a corporativos multinacionales [40].	2009, 2010, 2018
SeguriData Privada, S.A. de C.V.	NOM-151, sellos de tiempo, digitalización, certificados digitales.	Legado en seguridad informática corporativa; infraestructura de respaldo para plataformas de firma como Mifiel mediante API RESTful [40].	2011, 2019
Legalex GS, S.A. de C.V.	NOM-151, sellos de tiempo.	Plataforma propietaria “Covenant” para gestión de contratos digitales con NOM-151 integrada [40].	2019, 2020
Interfactura, S.A.P.I. de C.V.	NOM-151, sellos de tiempo.	Origen en certificación de facturas fiscales (PAC); herramientas de verificación pública de constancias .tsr mediante interfaz web [40].	2021
PSC World, S.A. de C.V.	NOM-151, sellos de tiempo, certificados digitales.	Integración nativa con Zoho Sign; entrega sellos en formato ASN.1 descontados como créditos de la plataforma [40].	2005, 2020
INCODE PSC, S. de R.L. de C.V.	NOM-151, sellos de tiempo.	Convergencia de verificación biométrica de identidad (IDV/KYC) con emisión criptográfica; API RESTful con validación contra INE [40].	2024
Cincel Certificadora Digital S.A.P.I. de C.V.	NOM-151, sellos de tiempo.	Primer PSC nativo de nube en México y América Latina; infraestructura AWS con modelo Zero Trust; integración orientada a API y entorno de sandbox sujeto a contratación. La información pública disponible no fue suficiente para presupuestar una campaña reproducible [40], [47].	2023

Criterios de selección del PSC para el prototipo La selección del PSC para el prototipo no es una decisión operativa sino una decisión de diseño con implicaciones jurídicas, técnicas y de viabilidad de implementación. Los criterios evaluados son cuatro:

- **Acreditación formal ante la Secretaría de Economía.** Requisito excluyente: únicamente los PSC del directorio oficial pueden emitir constancias con valor legal conforme a la NOM-151 [40].
- **Accesibilidad para integración mediante API.** El prototipo requiere un flujo de sellado automatizado en el backend (capa lógica del servidor); un PSC sin Interfaz de Programación de Aplicaciones (API, Application Programming Interface) REST (Representational State Transfer) documentada o con acceso condicionado a contratos corporativos prolongados no es viable en el contexto de un prototipo académico.
- **Disponibilidad y costo del entorno de pruebas (sandbox).** El entorno debe permitir validar el flujo criptográfico durante el desarrollo con condiciones operativas y costos conocidos; si cada solicitud tiene costo, la campaña debe presupuestarse y separarse de la suite automatizada.
- **Compatibilidad con el protocolo RFC 3161.** El sello debe adherirse al estándar internacional de sellado de tiempo para garantizar la verificabilidad independiente de las constancias generadas [27].

El análisis identificó a **Cincel** como candidato inicial, pero la evidencia de integración modificó la decisión de entrega. La selección de un PSC externo queda fuera del alcance actual; el diseño conserva un adaptador intercambiable y utiliza una TSA local para la demostración técnica:

- **Candidato técnicamente compatible.** La documentación y el adaptador construido permiten modelar una integración RFC 3161 detrás del puerto TimestampService; esto no equivale a haber validado la disponibilidad operativa del servicio.
- **Restricción observada en la integración.** El registro pudo completarse, pero la API no mostró respuesta y estabilidad suficientes para una campaña automatizada y repetible. El sandbox requiere pago y no ofrece una cotización pública verificable para definir el costo de una muestra.
- **Evidencia externa no contratada.** Sin respuestas reproducibles ni condiciones comerciales verificables, no es técnicamente responsable usar Cincel como criterio de aceptación, campaña de medición o dependencia de la suite.
- **Separación de protocolo y confianza.** RFC 3161 define el formato e intercambio técnico, pero la independencia de tercero y los efectos de una constancia NOM-151 requieren un PSC contratado y verificable. La TSA local solo demuestra el flujo técnico.
- **Ruta futura.** La contratación de Cincel, la evaluación de otro PSC o la exclusión explícita de evidencia externa son decisiones de alcance y presupuesto; ninguna debe resolverse por una conmutación implícita en el software.

La decisión vigente es conservar la TSA local basada en OpenSSL como autoridad de demostración y dejar la evidencia externa fuera del alcance de esta entrega. El módulo de sellado abstrae al emisor detrás de una interfaz interna, de modo que el despacho pueda sustituirlo por Cincel, SeguriData, CECOBAN u otro PSC del directorio de la Secretaría de Economía sin modificar la lógica central, siempre que el proveedor alternativo implemente RFC 3161 y se verifiquen por separado su contrato,

disponibilidad, cadena de confianza y precios. Esta separación preserva la soberanía tecnológica sin atribuir a la TSA local efectos jurídicos que no tiene.

2.3 Primitivas y Servicios Criptográficos

2.3.1 Integridad

La integridad es el servicio de seguridad que garantiza que la información se mantiene exacta, completa y sin modificaciones no autorizadas desde el momento de su creación. En un expediente legal, donde la alteración de una fecha, un monto o una cláusula puede modificar el curso de un procedimiento judicial, la integridad es un requisito fundamental [28].

Funciones Hash Criptográficas

El mecanismo técnico para verificar la integridad de un documento es la función hash criptográfica. Una función hash H toma una entrada de longitud arbitraria y produce una salida de longitud fija, denominada digest o huella digital. Para ser criptográficamente segura, debe cumplir tres propiedades fundamentales: resistencia a la preimagen (dado un hash, es computacionalmente inviable encontrar el mensaje original), resistencia a la segunda preimagen (dado un mensaje, es inviable encontrar otro mensaje con el mismo hash) y resistencia a colisiones (es inviable encontrar dos mensajes distintos que produzcan el mismo hash) [24], [25].

SHA-256

SHA-256 (Secure Hash Algorithm) pertenece a la familia SHA-2 estandarizada por el Instituto Nacional de Estándares y Tecnología (NIST, National Institute of Standards and Technology) en FIPS 180-4 [25]. Produce un digest de 256 bits (32 bytes) y es el estándar de facto para aplicaciones que requieren verificación de integridad, incluyendo firmas digitales, certificados X.509 y sellos de tiempo. Su resistencia a colisiones se considera computacionalmente segura con la capacidad de cómputo actual [48].

En un flujo típico de verificación de integridad, al almacenar un documento se calcula su hash SHA-256 y se conserva como referencia. En cada consulta posterior se recalcula el hash del documento almacenado y se compara con el valor de referencia. Cualquier discrepancia indica que el documento ha sido modificado.

La elección de SHA-256 se enmarca en la recomendación de niveles de fortaleza de seguridad del NIST SP 800-57 Part 1 Rev 5 [37]. Este documento define la fortaleza de seguridad (security strength) como el número de operaciones necesarias para romper un algoritmo criptográfico. SHA-256 proporciona 128 bits de fortaleza contra colisiones, lo que significa que se requieren aproximadamente 2^{128} operaciones para encontrar una colisión —un umbral considerado seguro al menos hasta 2030 para aplicaciones de uso general. El nivel de 128 bits sirve como guía para la selección coherente de todos los primitivos criptográficos de una arquitectura: AES-256 proporciona 256 bits de fortaleza, y RSA con claves de 3072 bits proporciona aproximadamente 128 bits, garantizando que ningún eslabón de la cadena criptográfica sea más débil que el nivel objetivo.

2.3.2 Confidencialidad e Integridad mediante Cifrado Autenticado

Mientras que el hash permite detectar modificaciones, no protege el contenido contra accesos no autorizados. Para expedientes y documentos almacenados se requiere un mecanismo que provea simultáneamente confidencialidad (que solo las partes autorizadas accedan al contenido) e integridad (que cualquier modificación no autorizada sea detectada). Este doble servicio lo provee el cifrado autenticado [28].

Cifrado Simétrico y el Estándar AES

El cifrado simétrico utiliza una misma clave para cifrar y descifrar. El Estándar de Cifrado Avanzado (AES, Advanced Encryption Standard) es un algoritmo de cifrado por bloques adoptado por el NIST (FIPS 197) [26] que opera sobre bloques de 128 bits con claves de 128, 192 o 256 bits. AES-256 es resistente a ataques de fuerza bruta con la capacidad computacional actual y es el estándar empleado tanto por las plataformas analizadas en el estado del arte como por agencias gubernamentales para la protección de información clasificada [24].

Modos de Operación y Cifrado Autenticado (AES-GCM)

Un algoritmo de cifrado por bloques como AES no se aplica directamente a mensajes de longitud arbitraria; requiere un modo de operación que defina cómo se procesan los bloques sucesivos. Los modos clásicos como el Encadenamiento de Bloques de Cifrado (CBC, Cipher Block Chaining) proveen confidencialidad, pero no integridad: un atacante puede modificar el texto cifrado sin que el destinatario detecte la alteración al momento del descifrado [49].

El modo GCM (Galois/Counter Mode) resuelve esta limitación al combinar el cifrado en modo Contador (CTR, Counter) con una función de autenticación basada en aritmética de campos de Galois. AES-GCM produce, además del texto cifrado, una etiqueta de autenticación (authentication tag) que permite verificar tanto la integridad del texto cifrado como la de datos adicionales no cifrados (AAD, Additional Authenticated Data, datos adicionales autenticados), tales como metadatos del expediente. Si un atacante modifica cualquier bit del texto cifrado o de los datos autenticados, la verificación de la etiqueta falla y el descifrado es rechazado [28], [50].

AES-256-GCM es el modo recomendado para el cifrado de expedientes y documentos almacenados. Cada operación de cifrado requiere un Vector de Inicialización (IV, Initialization Vector) único generado aleatoriamente, lo cual es un requisito crítico de GCM: la reutilización de un IV con la misma clave compromete tanto la confidencialidad como la autenticidad del esquema [50]. El modo GCM está especificado formalmente en NIST SP 800-38D y constituye una instancia de Cifrado Autenticado con Datos Asociados (AEAD, Authenticated Encryption with Associated Data), concepto formalizado en RFC 5116 [51].

En escenarios donde se gestionan múltiples documentos, el uso de una única clave para todo el repositorio concentra el riesgo: su compromiso expone la totalidad de la información. La arquitectura de cifrado por envoltorio (envelope encryption) mitiga este riesgo mediante dos niveles de claves. Una Clave de Cifrado de Clave (KEK, Key Encryption Key), almacenada de forma segura, se utiliza exclusivamente para envolver (cifrar) las Claves de Cifrado de Datos (DEK, Data Encryption Key) individuales, cada una generada aleatoriamente para un documento específico. El mecanismo de envolvimiento de claves está formalizado en NIST SP 800-38F [52] (Key Wrapping). Esta separación permite rotar la KEK re-cifrando únicamente las DEKs envueltas, sin necesidad de re-cifrar todos los documentos del repositorio.

2.3.3 No Repudio

El no repudio es el servicio de seguridad que impide que una entidad niegue haber realizado una operación, mediante la generación de evidencia criptográfica verificable sobre la autoría y el momento de su ejecución. A diferencia de la integridad (que protege al dato), el no repudio protege la autoría y se sustenta en dos pilares técnicos: la criptografía asimétrica y el sellado de tiempo [28].

Criptografía Asimétrica

La criptografía asimétrica emplea un par de claves matemáticamente vinculadas: una clave privada (conocida exclusivamente por su titular) y una clave pública (distribuida abiertamente). La propiedad fundamental es que lo cifrado con una clave solo puede descifrarse con la otra, y a partir de la clave pública es computacionalmente inviable derivar la clave privada. Los algoritmos de criptografía asimétrica más utilizados en el contexto de firma digital incluyen RSA (Rivest-Shamir-Adleman), basado en la dificultad de factorizar números enteros grandes, y ECDSA (Elliptic Curve Digital Signature Algorithm), basado en la dificultad del problema del logaritmo discreto sobre curvas elípticas [48], [53]. RSA define dos esquemas de firma principales en el estándar PKCS#1: la versión 1.5, ampliamente desplegada e interoperable con la infraestructura PKI existente, y PSS (Probabilistic Signature Scheme), que introduce aleatorización en el proceso de firma y ofrece una prueba de seguridad más robusta. Conforme a NIST SP 800-57 [37], una clave RSA de 3072 bits provee aproximadamente 128 bits de fortaleza de seguridad, equivalente a la de SHA-256 y AES-128.

Firma Digital

La firma digital es un esquema criptográfico que vincula la identidad del firmante con el contenido de un documento. El proceso opera en dos fases: en la generación, se calcula el hash del documento y se cifra con la clave privada del firmante, produciendo la firma; en la verificación, cualquier tercero puede descifrar la firma con la clave pública del firmante y comparar el resultado con el hash del documento. Si coinciden, se confirma que el documento no ha sido alterado y que fue firmado por el titular de la clave privada.

Este mecanismo provee simultáneamente integridad (cualquier modificación del documento invalida la firma) y no repudio de origen (solo el titular de la clave privada pudo haber generado la firma). La firma digital se distingue de la firma electrónica simple en que su validez no depende de la confianza en un proveedor de servicios, sino de propiedades matemáticas verificables de forma independiente [28], [53].

Certificados Digitales X.509

Para que una firma digital tenga valor probatorio, es necesario vincular la clave pública del firmante con su identidad legal. El estándar del Sector de Normalización de las Telecomunicaciones de la Unión Internacional de Telecomunicaciones (ITU-T) X.509 define el formato de los certificados digitales que establecen esta vinculación. Un certificado X.509 es una estructura de datos firmada por una Autoridad de Certificación (CA, Certificate Authority) que asocia una clave pública con un nombre distintivo (Distinguished Name), el cual puede incluir atributos como nombre legal, Registro Federal de Contribuyentes (RFC) o Clave Única de Registro de Población (CURP) en el contexto mexicano [54].

La cadena de confianza opera jerárquicamente: la CA raíz firma certificados de CAs intermedias, que a su vez emiten certificados a los usuarios finales. La verificación de una firma digital implica validar toda la cadena hasta una CA raíz de confianza, verificar que el certificado no haya sido revocado y confirmar su vigencia temporal [54]. El estándar RFC 5280 [44] (Internet X.509 PKI Certificate and CRL Profile) define el perfil de certificados y los dos mecanismos principales de revocación. Las Listas de Certificados Revocados (CRL, Certificate Revocation List) son documentos firmados por la CA que enumeran los certificados revocados antes de su fecha de expiración; se publican periódicamente y los verificadores las descargan para consulta local. El Protocolo de Estado de Certificado en Línea (OCSP, Online Certificate Status Protocol) permite consultar en tiempo real el estado de un certificado individual ante un responder autorizado. La elección entre ambos mecanismos depende de factores como la latencia aceptable, el volumen de certificados emitidos y la infraestructura disponible.

Sellado de Tiempo (RFC 3161)

La firma digital demuestra quién firmó un documento, pero no cuándo lo hizo. El sellado de tiempo complementa la firma al proveer evidencia criptográfica del momento en que se realizó la operación. El protocolo estándar para este servicio es RFC 3161 [27] (Internet X.509 PKI Time-Stamp Protocol), que define la interacción entre un cliente y una Autoridad de Sellado de Tiempo (TSA, Time-Stamp Authority).

El flujo opera de la siguiente manera: el cliente calcula el hash del documento y lo envía a la TSA en una solicitud de sello de tiempo (TimeStampReq). La TSA genera un token de sello de tiempo (TimeStampToken) que contiene el hash recibido, la fecha y hora precisa del sello, y la firma digital de la propia TSA sobre estos datos. El token resultante constituye prueba de que el documento existía en esa forma exacta en ese momento específico, sin que la TSA necesite conocer el contenido del documento.

En el contexto mexicano, la NOM-151-SCFI-2016 establece que la constancia de conservación debe ser emitida por un Prestador de Servicios de Certificación (PSC) autorizado, que en la práctica opera como TSA conforme al protocolo RFC 3161. La integración de este servicio en un flujo de firma digital provee evidencia temporal con validez legal.

2.3.4 Autenticación y Control de Acceso

La autenticación es el proceso de verificar la identidad de un usuario antes de permitirle el acceso a un recurso. El control de acceso determina, una vez autenticado, qué operaciones puede realizar. Ambos servicios son complementarios y su implementación adecuada es requisito para la validez de los servicios criptográficos: una firma digital carece de valor probatorio si el acceso a la clave privada del firmante no está debidamente protegido [28].

Autenticación Basada en Certificados Digitales

La autenticación mediante certificados digitales X.509 se basa en un esquema en que el usuario demuestra posesión de su clave privada firmando un desafío criptográfico generado por el servidor (challenge-response). Este mecanismo provee un nivel de aseguramiento superior a las contraseñas, ya que la clave privada nunca se transmite por la red y su compromiso requeriría acceso físico al dispositivo del usuario o la vulneración de su almacén de claves [55].

Autenticación por Múltiples Factores (MFA)

La Autenticación por Múltiples Factores (MFA) requiere la presentación de al menos dos categorías de evidencia de identidad: algo que el usuario sabe (contraseña), algo que posee (dispositivo que genera códigos temporales) o algo que es (biometría). La combinación de factores reduce significativamente el riesgo de acceso no autorizado, ya que el compromiso de un solo factor resulta insuficiente para suplantar la identidad del usuario [55]. El NIST SP 800-63B Rev 4 [56] (Digital Identity Guidelines: Authentication and Lifecycle Management) define tres Niveles de Aseguramiento de Autenticación (AAL): AAL1 (factor único), AAL2 (dos factores, uno de los cuales puede ser software) y AAL3 (dos factores con al menos uno basado en hardware criptográfico). El mecanismo TOTP (Time-Based One-Time Password), especificado en RFC 6238 [19], genera códigos temporales a partir de un secreto compartido y la hora actual, utilizando HMAC-SHA1 (Código de Autenticación de Mensajes basado en Hash con SHA-1, cuya seguridad para esta aplicación está avalada por NIST SP 800-107 Rev 1 [57]) como función base. TOTP se construye sobre HOTP (HMAC-Based One-Time Password, RFC 4226 [58]), reemplazando el contador por una marca temporal truncada. Ambos protocolos son interoperables con aplicaciones autenticadoras ampliamente adoptadas como Google Authenticator y Microsoft Authenticator.

Hashing de Contraseñas

El almacenamiento seguro de contraseñas requiere funciones de hashing especializadas, distintas de las funciones hash criptográficas de propósito general como SHA-256. Mientras que SHA-256 está diseñada para ser rápida (lo cual es deseable para verificar integridad de documentos), una función de hashing de contraseñas debe ser deliberadamente lenta y consumir recursos significativos de memoria, de modo que un atacante que obtenga la base de datos no pueda probar combinaciones masivamente [28].

Argon2, ganador de la Password Hashing Competition en 2015 y estandarizado en RFC 9106 [59], [60], fue diseñado específicamente para ser resistente a ataques con hardware especializado (GPUs y ASICs). Su propiedad distintiva es la dureza de memoria (memory-hardness): el algoritmo requiere una cantidad configurable de memoria durante su ejecución, lo que encarece proporcionalmente la construcción de hardware de ataque masivo. Argon2 define tres variantes: Argon2d (resistente a ataques de compromiso tiempo-memoria), Argon2i (resistente a ataques de canal lateral) y Argon2id (híbrido que combina las ventajas de ambas, recomendado por la OWASP (Open Worldwide Application Security Project, Proyecto Abierto de Seguridad de Aplicaciones Web) [61] para almacenamiento de contraseñas). Los tres parámetros de costo configurables son: memoria requerida (m), número de iteraciones (t) y grado de paralelismo (p). El NIST SP 800-63B Rev 4 [56] recomienda el uso de funciones de hashing de contraseñas con dureza de memoria, citando explícitamente a Argon2id como opción preferente para nuevas implementaciones. Una alternativa ampliamente utilizada es bcrypt, que incorpora un factor de costo ajustable y un salt aleatorio por contraseña, aunque carece de dureza de memoria configurable [48].

Control de Acceso Basado en Roles (RBAC)

El Control de Acceso Basado en Roles (RBAC, Role-Based Access Control) es un modelo de autorización en el que los permisos no se asignan directamente a los usuarios, sino a roles que representan funciones dentro de la organización. Cada usuario hereda los permisos del rol o roles

que tiene asignados. En el contexto del despacho, los roles típicos incluyen socio (acceso completo y capacidad de firma), abogado asociado (gestión de expedientes asignados), asistente (consulta y carga de documentos) y cliente (acceso limitado a su propio expediente mediante el portal de comunicación). Este modelo simplifica la administración de permisos y facilita el cumplimiento del principio de mínimo privilegio [62].

2.4 Gestión de Datos Híbrida

Una arquitectura de almacenamiento híbrida que combine almacenamiento relacional (SQL) y no relacional (NoSQL) se justifica por la diversidad de los datos manejados en el ámbito jurídico y las obligaciones legales de retención.

2.4.1 Almacenamiento Relacional (SQL)

El modelo relacional se utiliza para la gestión de datos estructurados con relaciones definidas: catálogo de clientes, asignación de casos, roles de acceso y registros de contratos. La integridad referencial que provee el modelo relacional —mediante restricciones de clave primaria, clave foránea y restricciones de dominio— garantiza la coherencia de los registros administrativos y procesales. Además, el modelo relacional facilita la implementación de RBAC a nivel de base de datos, asegurando que las políticas de acceso se apliquen de forma consistente.

2.4.2 Almacenamiento No Relacional (NoSQL)

Los expedientes legales contienen información de estructura variable: documentos PDF (Portable Document Format), evidencias multimedia, transcripciones de audiencias y metadatos que difieren entre tipos de caso. Un esquema NoSQL (Not Only SQL) orientado a documentos permite almacenar esta información sin la rigidez de un esquema tabular fijo, adaptando la estructura de cada registro a la naturaleza del contenido.

Desde la perspectiva de trazabilidad, el almacenamiento NoSQL se emplea para construir un registro de auditoría: cada operación sobre un documento (carga, modificación, firma, consulta) genera un registro que incluye el hash del estado del documento, el identificador del usuario, la marca de tiempo y, en su caso, el sello de tiempo de la TSA. Este registro, diseñado como estructura de solo inserción (append-only), conforma una cadena de custodia digital que permite reconstruir el historial completo de un expediente ante una auditoría o procedimiento judicial.

CAPÍTULO 3

Análisis y diseño

3.1 Metodología

Extreme Programming (XP) es una metodología ágil de desarrollo de software propuesta por Beck y Andres [63], orientada a equipos pequeños que trabajan con requisitos técnicos complejos o inciertos. Sus prácticas centrales incluyen programación en pares, desarrollo guiado por pruebas (TDD), spikes arquitectónicos para resolver incertidumbre técnica, e integración continua con propiedad colectiva del código.

Se eligió XP por tres razones vinculadas a las características del proyecto: primero, el equipo está compuesto por dos integrantes, lo que favorece la programación en pares como práctica central en lugar de ceremonias de coordinación más pesadas; segundo, la integración de mecanismos criptográficos (firma digital, sellado de tiempo, cifrado autenticado) y la vinculación con servicios externos como la Autoridad de Sellado de Tiempo introducen alta incertidumbre técnica, que XP aborda mediante *Spikes* (pruebas de concepto) y ciclos cortos de retroalimentación; tercero, el Desarrollo Guiado por Pruebas (TDD) resulta especialmente adecuado para validar la corrección de los servicios criptográficos, donde un error silencioso puede invalidar toda la evidencia generada.

Para abordar la complejidad del sistema, el proyecto estará segmentado en dos fases de iteraciones con objetivos específicos. El bloque inicial se enfocará en la planificación y la resolución de riesgos técnicos mediante *Spikes*, dejando el desarrollo iterativo continuo para el segundo bloque.

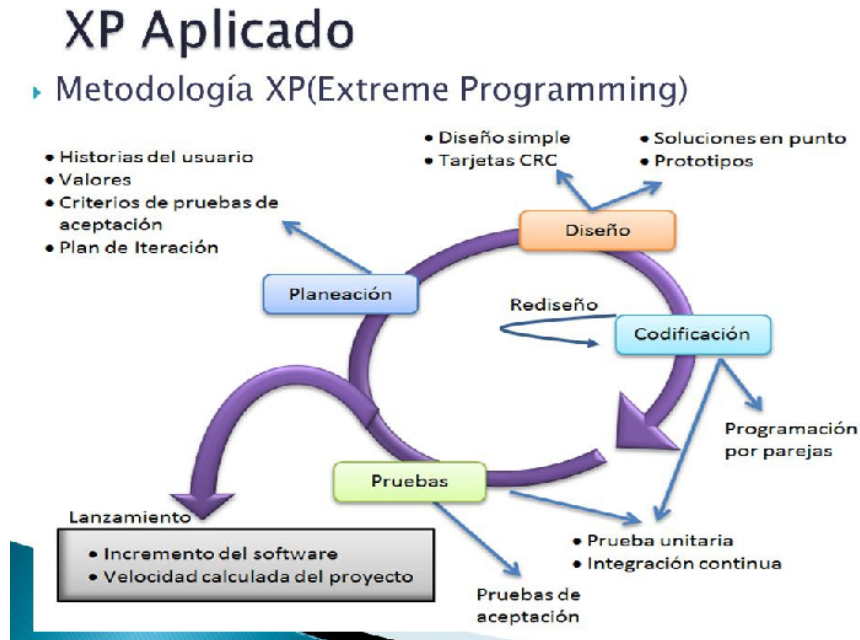


Figura 3.1: Ciclo de la metodología XP (Extreme Programming) aplicada al proyecto.

Plan de Iteraciones (10 semanas). El proyecto se estructura en dos bloques principales de 5 semanas cada uno. El primer bloque corresponde a la fase de planeación, investigación y diseño, enfocado en mitigar riesgos y modelar la solución. El segundo bloque abarca el ciclo continuo de codificación, integración y pruebas.

Fase I: Exploración, Diseño y Spikes Arquitectónicos. Durante estas 5 semanas, las iteraciones se centran en el levantamiento ágil, el análisis normativo y el diseño técnico del sistema, preparando el terreno para la implementación.

- **Iteración 1 (Semana 1) - Requerimientos y Fundamentos Criptográficos:** Investigación de la normativa mexicana referente al manejo de firmas digitales y comprobación de tiempo, y recolección de normativas sobre la Ley de Protección de Datos Personales. Investigación teórica de criptografía (Hash, RSA, Firma Digital) y levantamiento ágil de requerimientos funcionales y no funcionales.
- **Iteración 2 (Semana 2) - Análisis de Entorno y Sistemas Similares:** Investigación del Código de Procedimientos Penales Mexicanos aplicable al contexto digital. Análisis de sistemas similares en el mercado para la detección de brechas de seguridad y operativas.
- **Iteración 3 (Semana 3) - Análisis de Riesgos y Modelado Ágil:** Elaboración del análisis y matriz de riesgos. Traducción de los requerimientos mediante la redacción de Historias de Usuario priorizadas y diseño de los Casos de Uso del sistema.
- **Iteración 4 (Semana 4) - Diseño Arquitectónico y Modelo de Datos:** Diseño detallado de la arquitectura del sistema (Backend, Frontend y flujos de seguridad). Modelado de la base de datos híbrida, integrando bases de datos SQL para relaciones procesales y bases de datos Clave-Valor.
- **Iteración 5 (Semana 5) - Estrategia de Calidad y Cierre Documental:** Creación de la matriz

de pruebas (funcionales, de integración y criptográficas). Cierre y validación documental del diseño técnico para la conclusión de la primera fase.

Fase II: Construcción, Pruebas y Refactorización. Una vez validado el diseño, el desarrollo entra en un ciclo estricto de codificación iterativa mediante XP, utilizando programación en pares, Desarrollo Guiado por Pruebas (TDD) y entregas continuas. El orden de las iteraciones antepone el núcleo criptográfico a la construcción de vistas: el análisis de riesgos (sección 3.6) prescribe resolver primero los riesgos criptográficos graves y recomienda no avanzar hacia el desarrollo de vistas web hasta haber comprobado técnicamente la validez del sellado de tiempo.

- **Iteración 6 (Semana 6) - Andamiaje del Workspace y Núcleo Criptográfico Base:** Configuración del workspace de Cargo con sus cinco crates y del pipeline de integración continua (formato, análisis estático, pruebas y política de dependencias). Implementación del servicio de hash SHA-256 y del cifrado autenticado AES-256-GCM con envelope encryption, proporcionando integridad y confidencialidad en reposo. Primer contacto con el sellado de tiempo mediante el Spike de consumo y parseo del token RFC 3161.
- **Iteración 7 (Semana 7) - PKI Interna y Firma Digital:** Implementación de la infraestructura de llave pública interna: CA raíz, emisión de certificados X.509 y lista de certificados revocados (CRL). Integración de la firma digital RSA-3072 con SHA-256 sobre los certificados emitidos. Inicio del flujo de sellado de tiempo conforme a RFC 3161.
- **Iteración 8 (Semana 8) - Sellado de Tiempo, Autenticación y Auditoría:** Conclusión del flujo de sellado de tiempo RFC 3161. Implementación de la autenticación (hashing de contraseñas con Argon2id y segundo factor TOTP) y de la cadena de auditoría con encadenamiento de hashes. Verificación integral de la evidencia y demostración por línea de comandos, hito de criptografía demostrada de extremo a extremo.
- **Iteración 9 (Semana 9) - Persistencia, Control de Acceso y Gestión Procesal:** Implementación de la persistencia híbrida PostgreSQL/Redis y del control de acceso basado en roles (RBAC) con JWT. Desarrollo de la gestión procesal y documental —módulos CRUD para expedientes, casos, contratos y documentos— integrando los servicios criptográficos ya probados. Desarrollo de una interfaz web mínima.
- **Iteración 10 (Semana 10) - Pruebas Finales y Cierre:** Ejecución de la matriz de pruebas diseñada en la Fase I. Pruebas de usabilidad con usuarios representativos. Corrección de errores, refactorización final de código y elaboración de los manuales técnicos y de usuario.

3.2 Historias de Usuario

Las historias de usuario constituyen el instrumento de captura de requerimientos empleado por la metodología XP. Cada historia se redacta desde la perspectiva del actor que interactúa con el sistema, en lenguaje del dominio jurídico, estableciendo el rol, la necesidad y el beneficio esperado. Las historias se organizan en siete secciones funcionales que cubren la totalidad de los flujos del prototipo, desde la gestión de identidad hasta la operación y monitoreo del despacho. La formalización técnica de cada historia se realiza posteriormente en los casos de uso (sección 3.3), donde se especifican los flujos, precondiciones, postcondiciones y los mecanismos concretos de implementación.

3.2.1 Autenticación, Acceso y Gestión de Identidad

- **HU-01 — Registro de despacho.** Como socio fundador, quiero registrar mi despacho en el sistema proporcionando los datos de la firma, para disponer de un espacio propio y aislado donde gestionar los expedientes y al equipo de trabajo.
- **HU-02 — Selección de plan de servicio.** Como socio fundador, quiero elegir un plan de servicio (Starter o Professional) al momento del registro, para que el sistema habilite las funcionalidades y los límites de usuarios correspondientes al nivel contratado.
- **HU-03 — Invitación de miembros al equipo.** Como socio del despacho, quiero invitar nuevos colaboradores mediante correo electrónico asignándoles un perfil (Litigante o Paralegal), para que puedan acceder al sistema con los permisos apropiados a su función.
- **HU-04 — Aceptación de invitación.** Como colaborador invitado, quiero aceptar la invitación recibida por correo y completar mi registro, para incorporarme al despacho con el perfil que me fue asignado.
- **HU-05 — Inicio de sesión con credenciales.** Como usuario del sistema, quiero autenticarme con mi correo y contraseña, para acceder a las funciones habilitadas según mi perfil.
- **HU-06 — Inicio de sesión con certificado digital.** Como socio del despacho, quiero iniciar sesión presentando mi certificado digital, para acceder al sistema con un nivel de seguridad superior que vincule mi identidad de forma verificable con la sesión.
- **HU-07 — Verificación en dos pasos.** Como cliente del despacho, quiero confirmar mi identidad con un código temporal generado en mi dispositivo después de ingresar mi contraseña, para que mi expediente esté protegido ante el robo o filtración de la contraseña.
- **HU-08 — Recuperación de contraseña.** Como usuario del sistema, quiero solicitar la recuperación de mi contraseña mediante correo electrónico, para restablecer el acceso a mi cuenta sin necesidad de contactar al administrador.
- **HU-09 — Cierre de sesión por inactividad.** Como usuario del sistema, quiero que mi sesión se cierre automáticamente tras un periodo de inactividad, para prevenir accesos no autorizados si abandono mi equipo de trabajo sin cerrar sesión.
- **HU-10 — Restricción de acceso por perfil.** Como socio del despacho, quiero que cada miembro del equipo solo pueda acceder a los módulos y operaciones que corresponden a su perfil, para garantizar que únicamente las personas autorizadas realicen operaciones sensibles.
- **HU-11 — Baja de miembros del equipo.** Como socio del despacho, quiero dar de baja a un colaborador, para revocar de inmediato su acceso al sistema y a los expedientes del despacho.

3.2.2 Gestión Procesal Penal (Ciclo de Vida del Expediente)

- **HU-12 — Creación de expediente penal.** Como Litigante, quiero registrar un nuevo expediente ingresando el Número Único de Caso, la Carpeta Judicial, el tipo de delito y los datos generales del asunto, para que quede disponible en el sistema con estado inicial “Investigación”.
- **HU-13 — Registro de participantes del caso.** Como Litigante, quiero vincular a un expediente a las personas y autoridades que participan en el proceso (imputado, víctima, defensor, juez,

Ministerio Público, entre otros) con sus datos identificadores y su rol dentro del proceso, para conformar el directorio de participantes del caso.

- **HU-14 — Avance de etapa procesal.** Como Litigante, quiero avanzar un expediente a la siguiente etapa procesal cargando el documento que lo habilita (escrito de acusación para pasar a Intermedia, auto de apertura para pasar a Juicio Oral), para que el sistema registre el cambio y habilite las funciones correspondientes a la nueva etapa.
- **HU-15 — Bloqueo de avance sin documento habilitante.** Como Litigante, quiero que el sistema no permita avanzar de etapa si no se ha cargado el documento procesal requerido, para evitar inconsistencias en la secuencia del expediente que comprometan su validez.
- **HU-16 — Historial de transiciones del expediente.** Como Litigante, quiero consultar la línea de tiempo completa de las transiciones de etapa de un expediente, con la fecha, el responsable y el documento asociado a cada cambio, para verificar que la secuencia procesal esté íntegra.
- **HU-17 — Actualización de datos del expediente.** Como Litigante, quiero actualizar los datos generales de un expediente (delitos, identificadores complementarios, información general), para mantener la información vigente sin alterar el historial de transiciones.
- **HU-18 — Vista de expedientes asignados.** Como Paralegal, quiero visualizar únicamente los expedientes que me han sido asignados, para concentrarme en los casos relevantes a mi trabajo sin exposición a información de otros asuntos.

3.2.3 Calendario Jurídico, Plazos y Alertas

- **HU-19 — Programación de audiencia.** Como Litigante, quiero agendar una audiencia seleccionando su tipo (inicial, intermedia, juicio oral, individualización de sanciones), fecha, hora, participantes y modalidad, para que quede vinculada al expediente correspondiente.
- **HU-20 — Activación automática de plazos tras audiencia.** Como Litigante, quiero que al registrar el resultado de una audiencia (por ejemplo, vinculación a proceso), el sistema calcule y active automáticamente los plazos derivados (como el de investigación complementaria), para no depender del cálculo manual de los términos.
- **HU-21 — Cómputo de plazos con días inhábiles.** Como Litigante, quiero que el sistema calcule las fechas de vencimiento descontando correctamente sábados, domingos y días inhábiles del calendario judicial de la entidad federativa, para obtener fechas precisas conforme a las reglas de cómputo del CNPP.
- **HU-22 — Configuración del calendario judicial.** Como socio del despacho, quiero configurar los días inhábiles por entidad federativa y fuero, para que los cálculos de plazos reflejen los periodos de receso de cada jurisdicción donde litiga el despacho.
- **HU-23 — Alertas de vencimiento próximo.** Como Litigante, quiero recibir notificaciones automáticas por correo y en el sistema cuando un plazo esté próximo a vencer, para tomar acciones oportunas y evitar consecuencias procesales irreversibles.
- **HU-24 — Calendario unificado del despacho.** Como Litigante, quiero visualizar en un calendario interactivo todas las audiencias programadas y los plazos límite de mis casos, con

colores diferenciados por tipo de evento y acceso directo al expediente, para gestionar mi agenda procesal desde una sola vista.

3.2.4 Gestión Documental y Cadena de Custodia

- **HU-25 — Carga de documento al expediente.** Como Litigante, quiero cargar un documento procesal asociándolo a un expediente, para que quede almacenado de forma protegida y disponible para consulta por las personas autorizadas.
- **HU-26 — Clasificación de documentos.** Como Litigante, quiero asignar tipo documental, clasificación y etiquetas a cada documento cargado, para facilitar su organización, búsqueda y recuperación dentro del expediente.
- **HU-27 — Consulta de documentos según asignación.** Como Paralegal, quiero consultar y descargar los documentos de los expedientes que tengo asignados, para apoyar la gestión del caso sin acceder a información de asuntos no relacionados.
- **HU-28 — Registro automático de movimientos sobre documentos.** Como Litigante, quiero que cada operación sobre un documento o evidencia (carga, consulta, descarga, movimiento) quede registrada automáticamente con la fecha, la persona responsable y el tipo de operación, para contar con un historial completo que respalde la cadena de custodia del expediente.
- **HU-29 — Protección del historial de movimientos.** Como Litigante, quiero que ningún usuario, incluido el socio del despacho, pueda modificar ni eliminar los registros del historial de movimientos de documentos, para garantizar que la trazabilidad sea confiable ante cualquier requerimiento judicial.

3.2.5 Servicios de Protección de la Información

- **HU-30 — Huella digital del documento al momento de carga.** Como Litigante, quiero que al cargar un documento al sistema, se genere automáticamente una huella digital única del archivo, para contar con una referencia que permita comprobar en el futuro si el documento fue alterado.
- **HU-31 — Alerta de alteración en consulta posterior.** Como Litigante, quiero que cada vez que consulte un documento, el sistema verifique automáticamente que no haya sido modificado desde su carga y me alerte de inmediato si detecta alguna discrepancia, para tener certeza de la inalterabilidad de mis expedientes.
- **HU-32 — Protección del contenido almacenado.** Como Litigante, quiero que los documentos de mis expedientes se almacenen protegidos de forma que solo las partes autorizadas puedan acceder a su contenido, para que una intrusión al sistema no exponga la información de mis clientes.
- **HU-33 — Detección de manipulación en el almacenamiento.** Como Litigante, quiero que si alguien manipula directamente los archivos almacenados en el servidor, el sistema lo detecte y rechace la entrega del documento afectado notificándome la anomalía, para saber que cualquier intento de alteración será evidenciado.

3.2.6 Firma de Contratos y Constancia de Fecha

- **HU-34 — Firma de contrato con identidad verificable.** Como Litigante, quiero firmar un contrato de manera que mi identidad quede vinculada de forma verificable con el contenido del documento, para que ninguna de las partes pueda negar posteriormente haber suscrito el acuerdo.
- **HU-35 — Constancia de fecha y hora de la firma.** Como Litigante, quiero que al firmar un contrato, el sistema obtenga automáticamente una constancia emitida por una autoridad certificadora que acredite la fecha y hora exacta de la firma, para que el documento tenga valor probatorio conforme a la normatividad mexicana.
- **HU-36 — Verificación de firma y constancia de un contrato.** Como usuario con acceso al contrato, quiero poder ejecutar una verificación que me indique si la firma es válida, si el certificado del firmante está vigente y si la constancia de fecha es auténtica, para evaluar la confiabilidad del documento sin depender de un tercero.
- **HU-37 — Detección de alteración en contrato firmado.** Como usuario con acceso al contrato, quiero que si el documento fue modificado después de firmado, la verificación me indique claramente que la firma ya no es válida y me explique la causa, para disponer de evidencia de que el contrato fue alterado.

3.2.7 Operación y Monitoreo

- **HU-38 — Tablero de control del despacho.** Como socio del despacho, quiero consultar un tablero con indicadores clave (casos activos, plazos por vencer próximamente, contratos pendientes de firma, carga de trabajo por abogado), para tener visibilidad del estado general de la operación.
- **HU-39 — Generación de informes.** Como socio del despacho, quiero generar informes filtrados por periodo, abogado responsable o estado de los casos, exportables en formato descargable, para analizar el desempeño y las métricas operativas de la firma.
- **HU-40 — Registro general de actividad.** Como socio del despacho, quiero consultar un registro completo e inalterable de todas las acciones realizadas en el sistema (quién realizó la acción, qué hizo, sobre qué recurso y cuándo), para disponer de trazabilidad ante auditorías internas o requerimientos judiciales.

3.3 Casos de Uso

Los casos de uso formalizan las historias de usuario definidas en la [sección 3.2](#), especificando los flujos de interacción entre los actores y el sistema, las condiciones que deben cumplirse antes y después de cada operación, y los escenarios alternos que el prototipo debe manejar. Cada caso de uso agrupa una o más historias de usuario en un flujo funcional cohesivo e incluye la trazabilidad hacia los requerimientos funcionales y no funcionales que lo respaldan. La especificación técnica de los mecanismos criptográficos (algoritmos, estándares y protocolos) se detalla en los requerimientos de la [sección 3.4](#); los casos de uso se limitan a describir el comportamiento observable del sistema desde la perspectiva del actor.

3.3.1 Autenticación, Acceso y Gestión de Identidad

CU-01 — Registro de despacho y selección de plan

Tabla 3.1: Caso de uso CU-01: Registro de despacho y selección de plan.

ID	CU-01
Nombre	Registro de despacho y selección de plan
Actor principal	Socio fundador
Actores secundarios	Sistema de correo transaccional
Precondiciones	El usuario no posee una cuenta activa en el sistema.
Flujo principal	1. El usuario accede al formulario de registro. 2. Ingresa los datos del despacho (nombre de la firma, domicilio fiscal, datos de contacto). 3. Selecciona un plan de servicio (Starter o Professional). 4. Establece sus credenciales de acceso (correo y contraseña). 5. El sistema valida los datos ingresados y aprovisiona el espacio de trabajo del despacho con el plan seleccionado. 6. El sistema envía un correo de confirmación al socio fundador. 7. El socio fundador confirma su correo electrónico mediante el enlace recibido. 8. El sistema activa la cuenta y redirige al tablero de control del despacho.
Flujos alternos	4a. La contraseña no cumple las políticas de complejidad → el sistema indica los requisitos faltantes. 5a. El correo electrónico ya está registrado → el sistema informa que la cuenta existe y ofrece la opción de recuperación. 7a. El enlace de confirmación ha expirado → el sistema permite reenviar el correo de confirmación.
Postcondiciones	El despacho queda registrado con su espacio de datos inicializado, el plan activo y el socio fundador autenticado con rol Owner. Las credenciales se almacenan mediante funciones de hashing resistentes.
Derivado de	HU-01, HU-02
RF relacionados	RF-01, RF-02
RNF relacionados	RNF-01, RNF-02

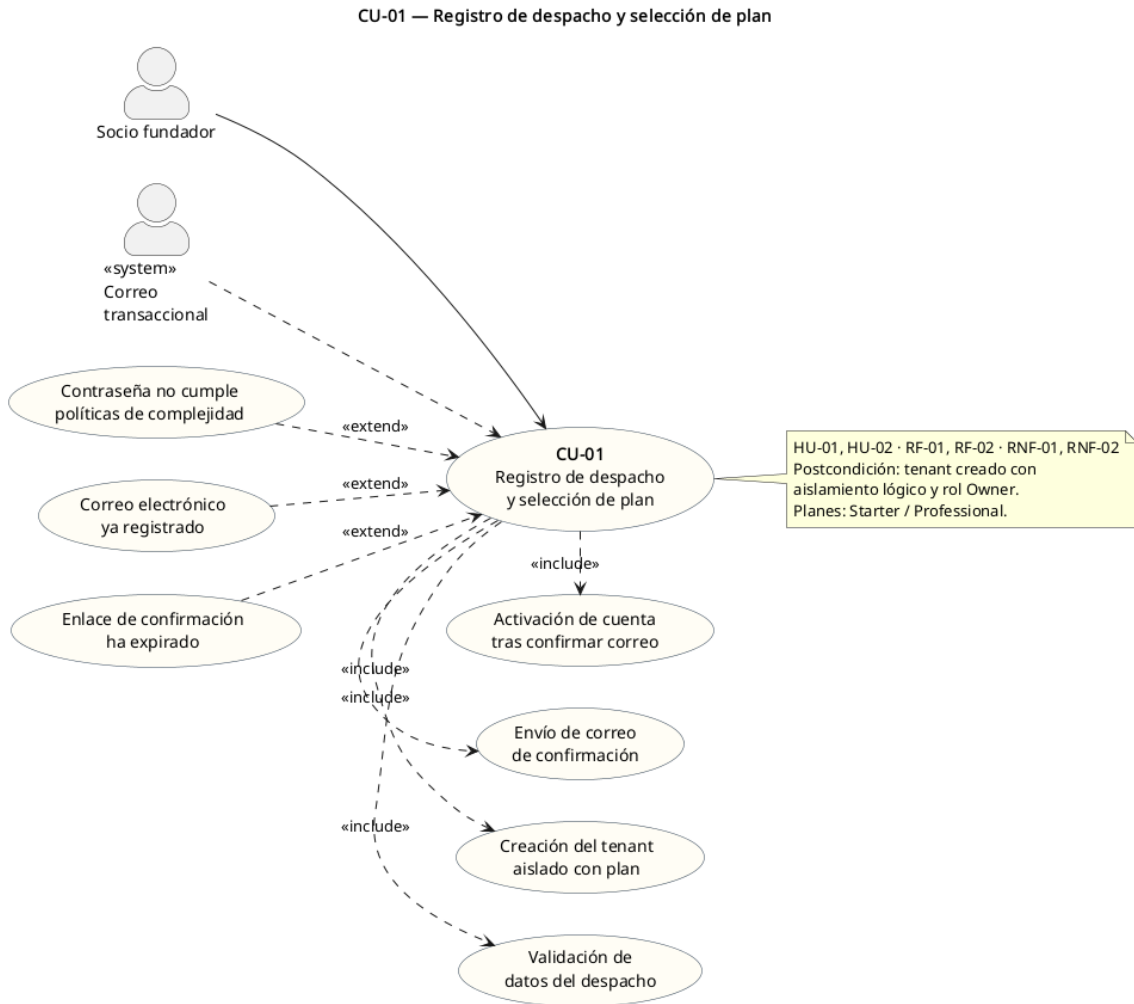


Figura 3.2: Diagrama CU-01. Registro de despacho y selección de plan.

CU-02 — Gestión del ciclo de vida de miembros del equipo**Tabla 3.2:** Caso de uso CU-02: Gestión del ciclo de vida de miembros del equipo.

ID	CU-02
Nombre	Gestión del ciclo de vida de miembros del equipo
Actor principal	Owner
Actores secundarios	Colaborador invitado, sistema de correo transaccional
Precondiciones	El Owner tiene sesión activa.
Flujo principal (alta)	1. El Owner accede al módulo de gestión de equipo. 2. Ingresa el correo electrónico del nuevo miembro y selecciona el perfil a asignar (Litigante o Paralegal). 3. El sistema verifica que el despacho no ha alcanzado el límite de usuarios del plan contratado. 4. El sistema genera un enlace de invitación con vigencia limitada y lo envía por correo. 5. El colaborador invitado accede al enlace, completa su registro con sus datos personales y establece su contraseña. 6. El sistema valida los datos, crea la cuenta asociada al despacho con el perfil indicado y registra la operación en la bitácora. 7. El nuevo miembro accede al sistema con las funciones habilitadas según su perfil.
Flujo principal (baja)	1. El Owner accede al módulo de gestión de equipo. 2. Selecciona al miembro que desea dar de baja. 3. El sistema solicita confirmación de la acción. 4. El Owner confirma la baja. 5. El sistema revoca de inmediato el acceso del colaborador, invalida sus sesiones activas y registra la operación en la bitácora. 6. Los expedientes y documentos asociados al colaborador permanecen en el sistema sin modificación; únicamente se revoca el acceso.
Flujos alternos	2a. (Alta) El correo ya pertenece a un miembro activo del despacho → el sistema rechaza la invitación e informa el motivo. 3a. (Alta) El despacho ha alcanzado el límite de usuarios → el sistema bloquea la invitación e indica la necesidad de actualizar el plan. 5a. (Alta) El enlace de invitación ha expirado → el sistema muestra un mensaje informativo; el Owner debe reenviar la invitación. 2b. (Baja) El Owner intenta darse de baja a sí mismo → el sistema rechaza la operación, ya que el despacho requiere al menos un Owner activo.
Postcondiciones	(Alta) El colaborador queda registrado en el despacho con el perfil asignado. (Baja) El acceso del colaborador queda revocado de inmediato. En ambos casos, la operación se registra en la bitácora de actividad.
Derivado de	HU-03, HU-04, HU-11
RF relacionados	RF-03
RNF relacionados	RNF-01, RNF-02

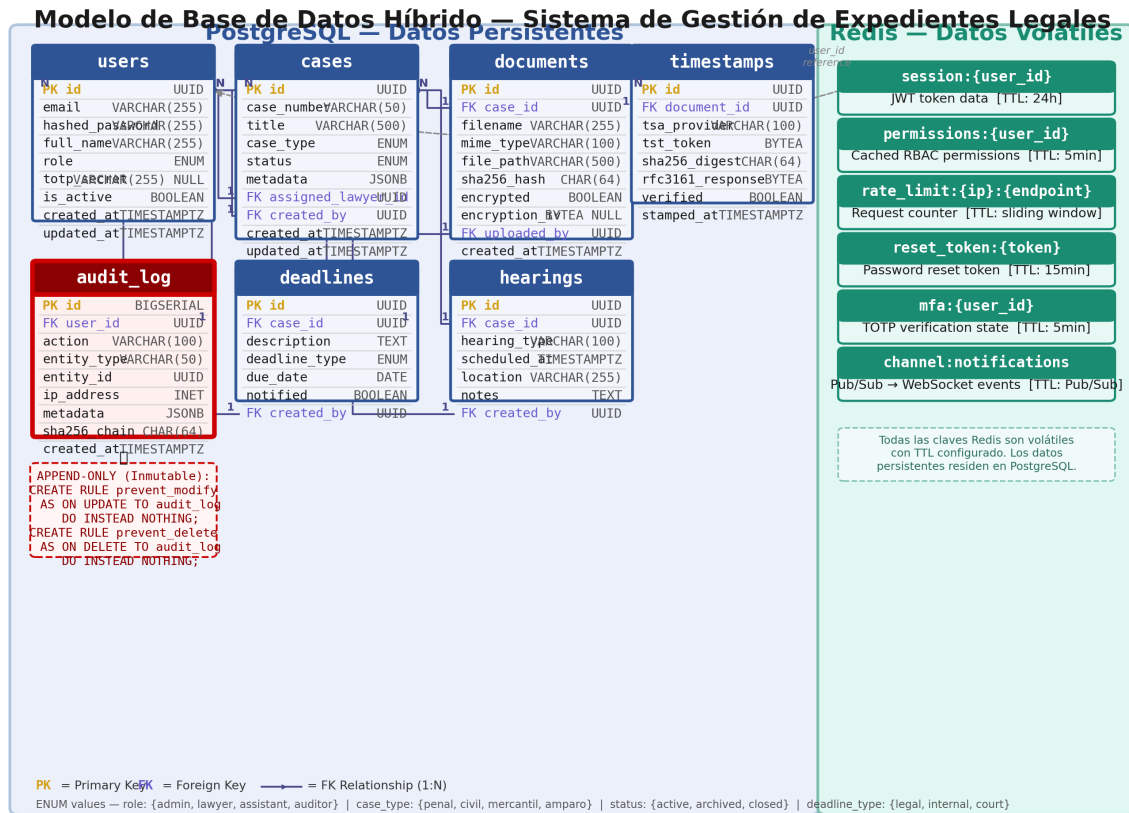


Figura 3.3: Diagrama CU-02. Gestión del ciclo de vida de miembros del equipo.

CU-03 — Inicio de sesión y gestión de sesiones**Tabla 3.3:** Caso de uso CU-03: Inicio de sesión y gestión de sesiones.

ID	CU-03
Nombre	Inicio de sesión y gestión de sesiones
Actor principal	Usuario del sistema (Owner, Litigante, Paralegal, Cliente)
Actores secundarios	—
Precondiciones	El usuario posee una cuenta activa en el sistema.
Flujo principal	1. El usuario accede a la pantalla de inicio de sesión. 2. Ingresa su correo electrónico y contraseña. 3. El sistema verifica las credenciales contra el hash almacenado. 4. Si el usuario tiene habilitada la verificación en dos pasos, el sistema solicita el código temporal generado en su dispositivo. 5. El usuario ingresa el código y el sistema lo valida. 6. El sistema crea una sesión autenticada, registra el inicio de sesión en la bitácora y redirige al usuario al módulo correspondiente a su perfil.
Flujos alternos	3a. Las credenciales son incorrectas → el sistema informa el error sin revelar cuál campo es inválido. 3b. El usuario es Owner y elige autenticarse con certificado digital → el sistema genera un desafío criptográfico; el usuario firma con su clave privada; el sistema verifica la firma contra el certificado registrado. 1a. El usuario selecciona la opción de recuperación de contraseña → el sistema solicita el correo electrónico asociado a la cuenta → envía un enlace de restablecimiento con vigencia limitada → el usuario accede al enlace y establece una nueva contraseña → el sistema la almacena mediante funciones de hashing resistentes, invalida las sesiones activas previas y confirma el restablecimiento. 5a. El código temporal es inválido o ha expirado → el sistema rechaza el acceso y permite reintentar. 6a. La sesión permanece inactiva por el tiempo configurado → el sistema la invalida y requiere nueva autenticación, preservando los datos no guardados.
Postcondiciones	El usuario queda autenticado con una sesión activa. El inicio de sesión, incluyendo método de autenticación utilizado y dirección IP, se registra en la bitácora.
Derivado de	HU-05, HU-06, HU-07, HU-08, HU-09
RF relacionados	RF-01, RF-02
RNF relacionados	RNF-02, RNF-05

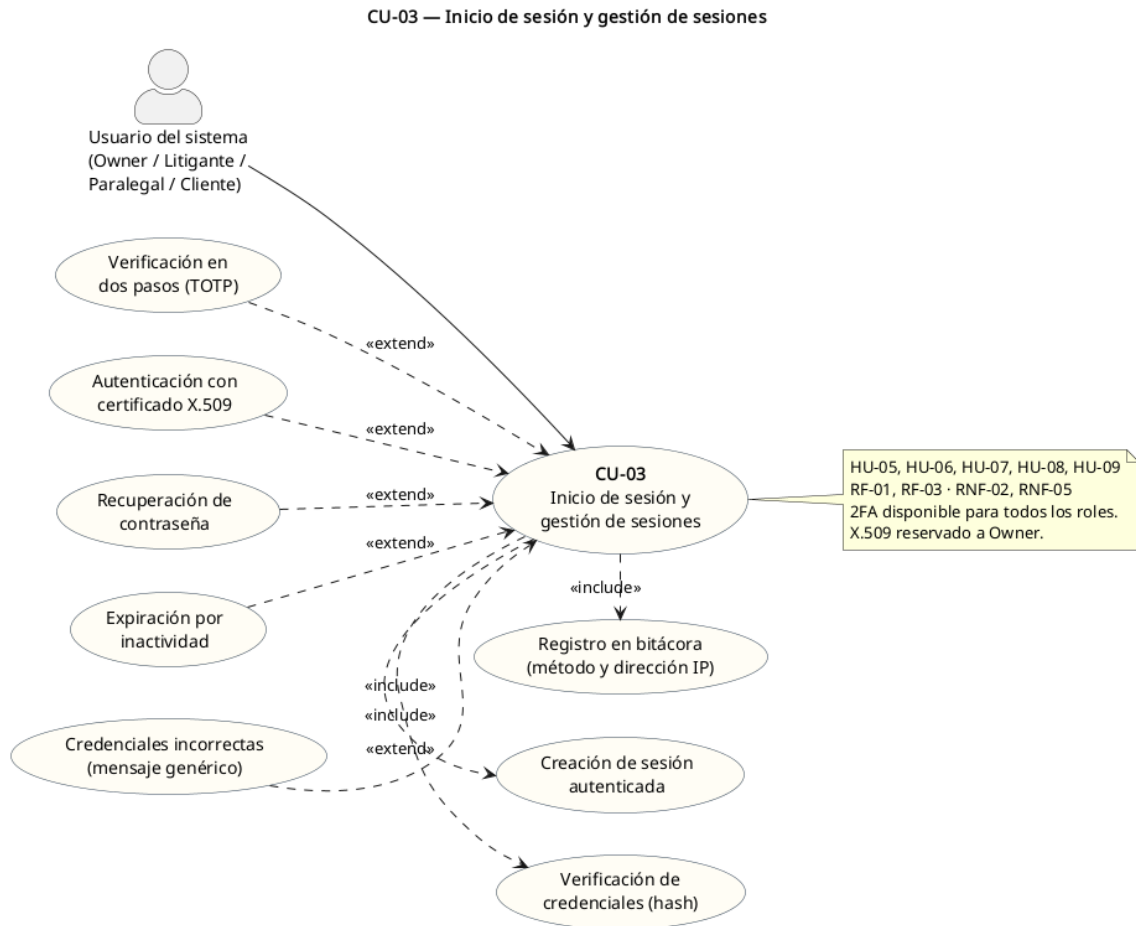


Figura 3.4: Diagrama CU-03. Inicio de sesión y gestión de sesiones.

CU-04 — Control de acceso por perfil**Tabla 3.4:** Caso de uso CU-04: Control de acceso por perfil.

ID	CU-04
Nombre	Control de acceso por perfil
Actor principal	Usuario autenticado (cualquier rol)
Actores secundarios	—
Precondiciones	El usuario tiene sesión activa.
Flujo principal	1. El usuario solicita acceso a un módulo o recurso del sistema. 2. El sistema evalúa el perfil del usuario contra la matriz de permisos del módulo solicitado. 3. Si el perfil tiene permiso, el sistema concede el acceso y presenta la interfaz correspondiente.
Flujos alternos	3a. El perfil no tiene permiso sobre el módulo solicitado → el sistema deniega el acceso con un mensaje explicativo y registra el intento en la bitácora. 3b. El usuario intenta acceder a un recurso fuera de su ámbito de autorización (por ejemplo, manipulando identificadores de expedientes no asignados) → el sistema devuelve un error de autorización sin revelar la existencia del recurso.
Postcondiciones	El acceso queda concedido o denegado. Cada intento de acceso, exitoso o fallido, se registra en la bitácora.
Derivado de	HU-10
RF relacionados	RF-02
RNF relacionados	RNF-01, RNF-05

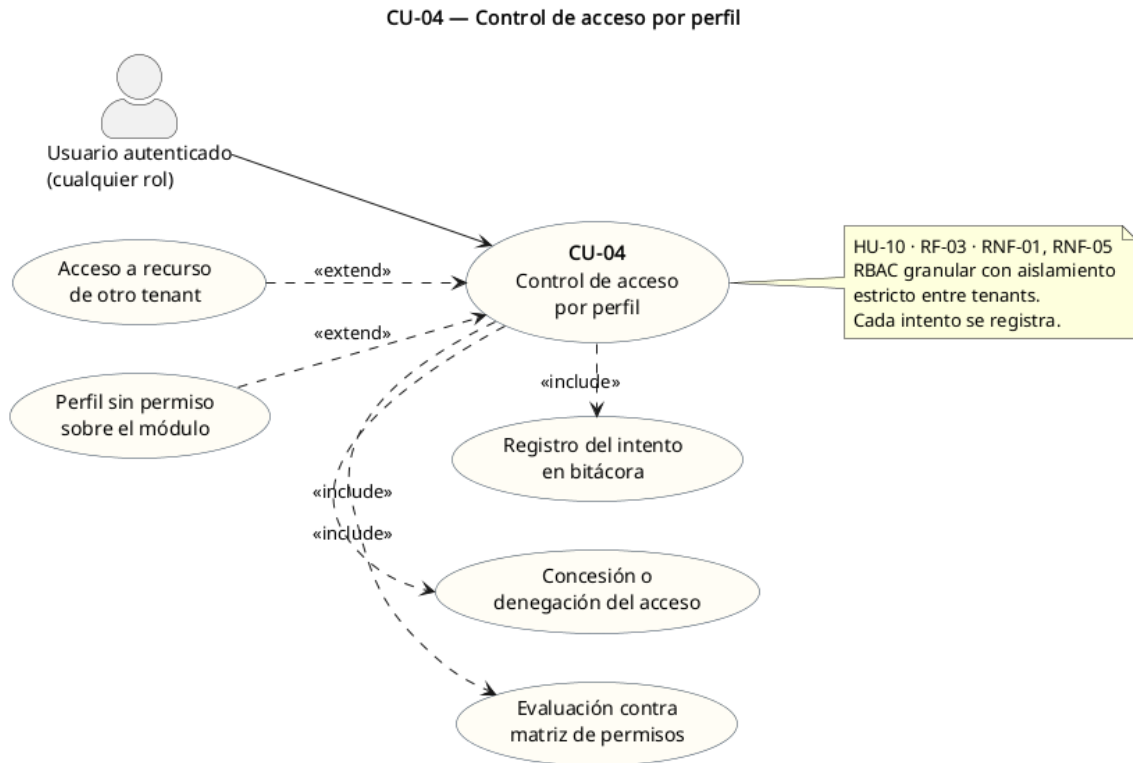


Figura 3.5: Diagrama CU-04. Control de acceso por perfil.

3.3.2 Gestión Procesal Penal (Ciclo de Vida del Expediente)

CU-05 — Registro y administración de expediente penal

Tabla 3.5: Caso de uso CU-05: Registro y administración de expediente penal.

ID	CU-05
Nombre	Registro y administración de expediente penal
Actor principal	Litigante
Actores secundarios	Paralegal (consulta)
Precondiciones	El Litigante tiene sesión activa y pertenece al despacho.
Flujo principal	1. El Litigante accede al módulo de gestión procesal y selecciona la opción de nuevo expediente. 2. Ingresa los identificadores oficiales: Número Único de Caso (NUC) y Carpeta Judicial. 3. Registra el tipo de delito, los datos generales del asunto y los metadatos asociados. 4. El sistema valida que los campos obligatorios estén completos y que los identificadores no estén duplicados dentro del despacho. 5. El sistema persiste el expediente con estado inicial “Investigación”, genera la marca de tiempo de creación y registra la operación en la bitácora.
Flujos alternos	4a. Faltan campos obligatorios (NUC o Carpeta Judicial) → el sistema rechaza el registro e indica los campos faltantes. 4b. El NUC ya existe en el despacho → el sistema informa la duplicidad.
Postcondiciones	El expediente queda registrado con estado “Investigación”, disponible para los usuarios autorizados del despacho.
Derivado de	HU-12, HU-17, HU-18
RF relacionados	RF-04
RNF relacionados	RNF-04, RNF-10

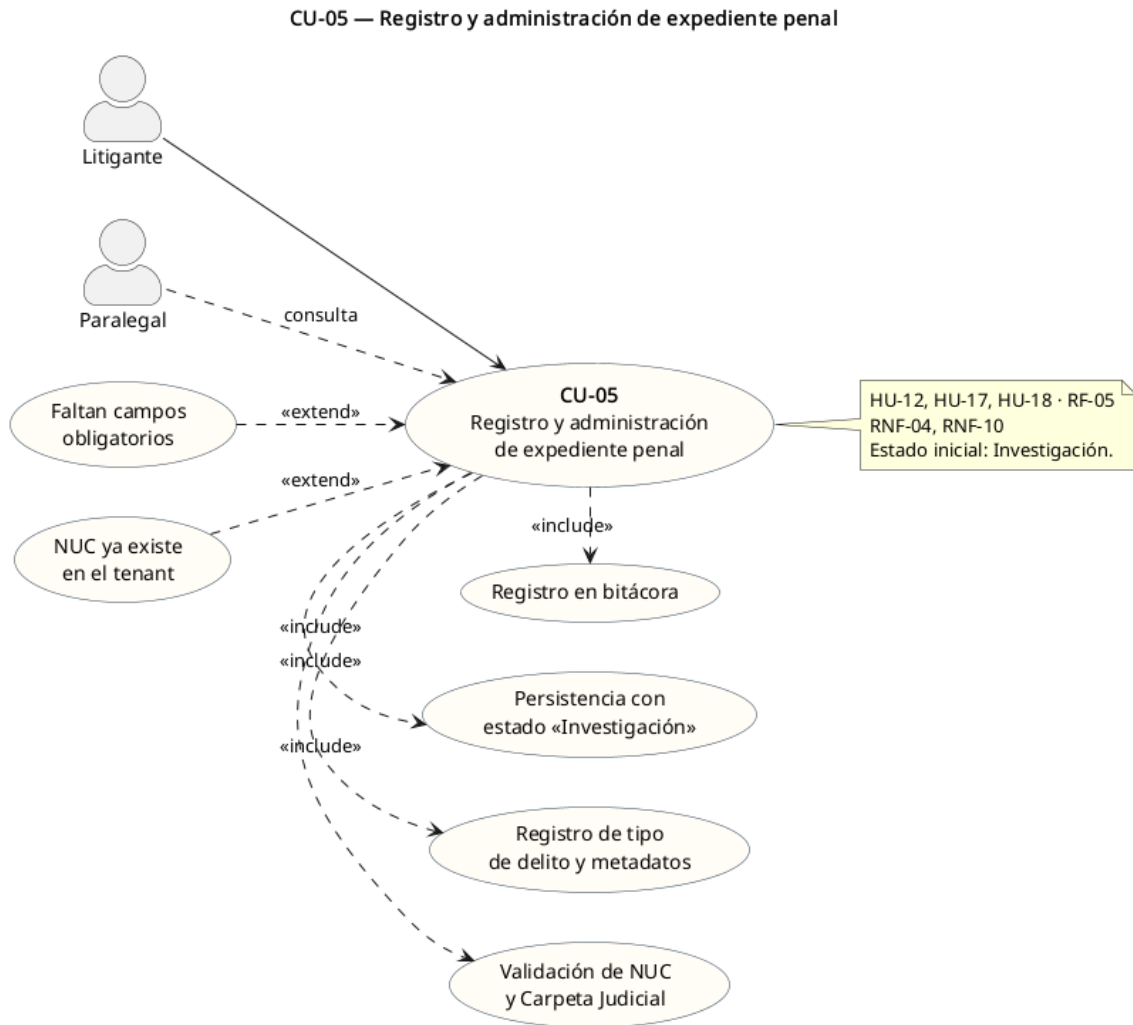


Figura 3.6: Diagrama CU-05. Registro y administración de expediente penal.

CU-06 — Gestión del directorio de participantes**Tabla 3.6:** Caso de uso CU-06: Gestión del directorio de participantes.

ID	CU-06
Nombre	Gestión del directorio de participantes
Actor principal	Litigante
Actores secundarios	—
Precondiciones	Existe un expediente activo al cual vincular participantes.
Flujo principal	1. El Litigante accede al expediente y selecciona el módulo de directorio de participantes. 2. Selecciona el tipo de sujeto procesal (imputado, víctima, defensor, juez de control, Ministerio Público, entre otros). 3. Ingresa los datos identificadores del participante según su tipo (nombre, CURP, cédula profesional, órgano jurisdiccional, según aplique). 4. El sistema valida los datos requeridos según el tipo de sujeto procesal y registra al participante vinculado al expediente.
Flujos alternos	3a. Para un juez de control, no se proporciona el órgano jurisdiccional ni la FIREL → el sistema rechaza el registro e indica los campos faltantes. 4a. El participante ya existe en el directorio del caso con el mismo rol → el sistema informa la duplicidad.
Postcondiciones	El participante queda vinculado al expediente con su rol procesal. Un mismo individuo puede tener roles distintos en expedientes diferentes.
Derivado de	HU-13
RF relacionados	RF-06
RNF relacionados	RNF-10

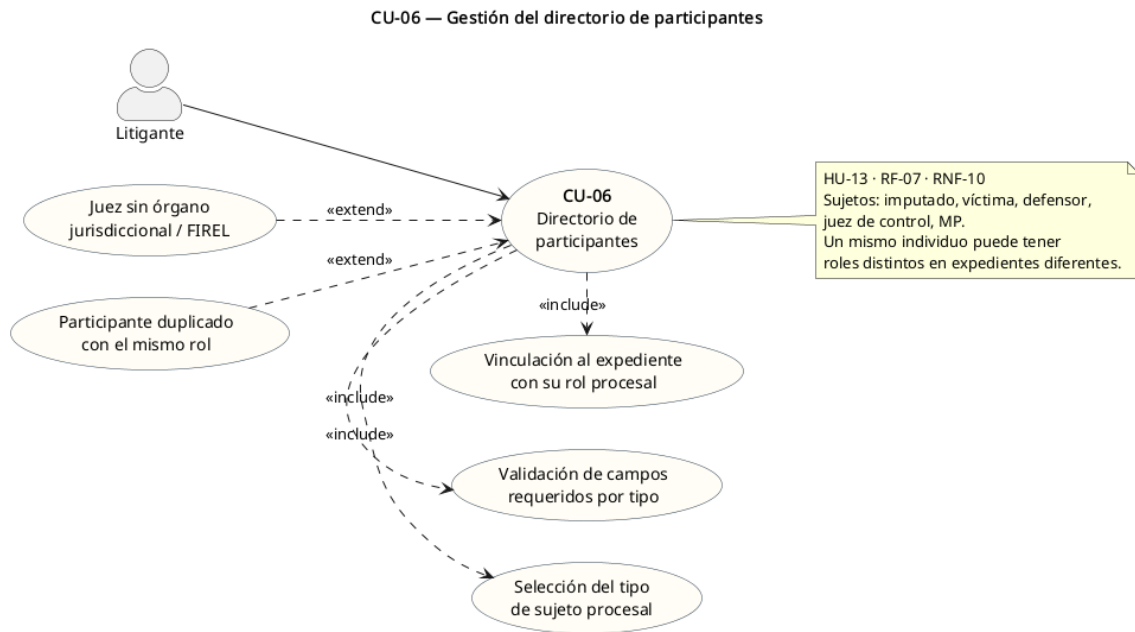


Figura 3.7: Diagrama CU-06. Gestión del directorio de participantes.

CU-07 — Transición de etapa procesal**Tabla 3.7:** Caso de uso CU-07: Transición de etapa procesal.

ID	CU-07
Nombre	Transición de etapa procesal
Actor principal	Litigante
Actores secundarios	—
Precondiciones	El expediente se encuentra en una etapa procesal que admite transición (Investigación, Intermedia o Juicio Oral).
Flujo principal	1. El Litigante accede al expediente y solicita avanzar a la siguiente etapa procesal. 2. El sistema solicita la carga del documento habilitante correspondiente (escrito de acusación para Investigación → Intermedia; auto de apertura para Intermedia → Juicio Oral). 3. El Litigante carga el documento procesal. 4. El sistema almacena el documento con las protecciones de integridad y confidencialidad correspondientes. 5. El sistema registra la transición de forma inmutable: marca de tiempo, usuario responsable y documento asociado. 6. El sistema habilita los módulos correspondientes a la nueva etapa.
Flujos alternos	2a. El Litigante intenta avanzar de etapa sin cargar el documento habilitante → el sistema rechaza la transición e indica el documento requerido. 3a. El documento cargado no corresponde al formato admitido → el sistema rechaza la carga.
Postcondiciones	El expediente se encuentra en la nueva etapa procesal. La transición queda registrada de forma inmutable en el historial del expediente y no puede revertirse.
Derivado de	HU-14, HU-15, HU-16
RF relacionados	RF-05
RNF relacionados	RNF-04, RNF-10

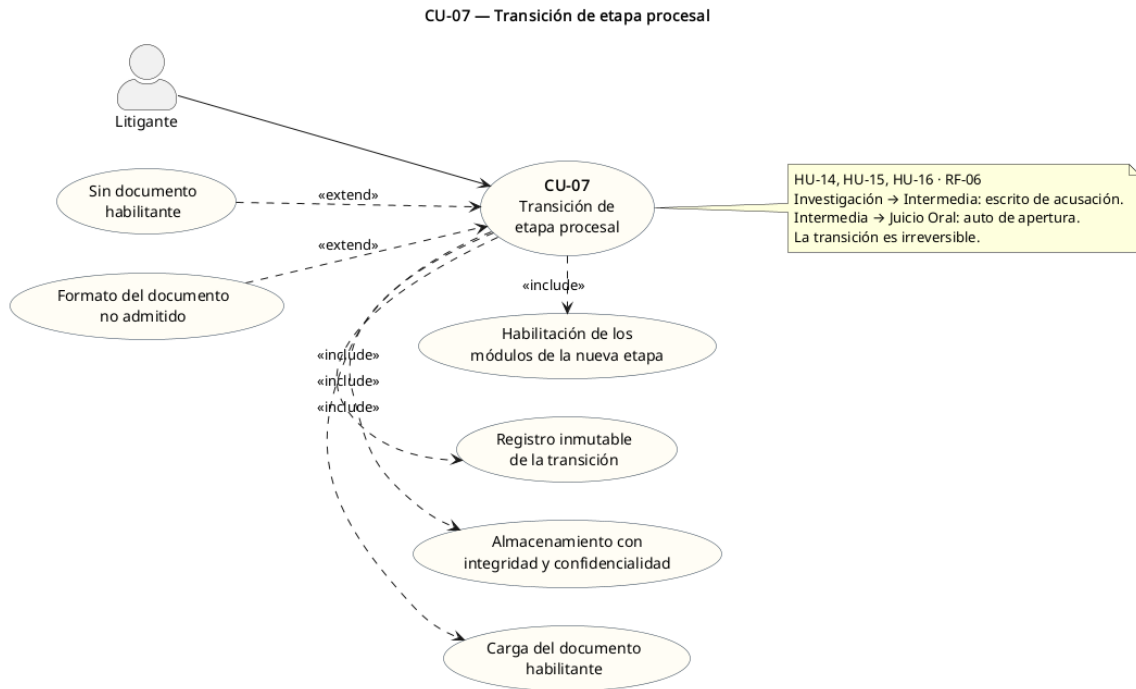


Figura 3.8: Diagrama CU-07. Transición de etapa procesal.

3.3.3 Calendario Jurídico, Plazos y Alertas

CU-08 — Programación de audiencia y activación de plazos

Tabla 3.8: Caso de uso CU-08: Programación de audiencia y activación de plazos.

ID	CU-08
Nombre	Programación de audiencia y activación de plazos
Actor principal	Litigante
Actores secundarios	Sistema de cómputo de plazos
Precondiciones	Existe un expediente activo con una etapa procesal que admite la audiencia a programar.
Flujo principal	1. El Litigante accede al expediente y selecciona la opción de programar audiencia. 2. Selecciona el tipo de audiencia del catálogo tipificado conforme al CNPP (inicial, intermedia, juicio oral, individualización de sanciones). 3. Ingresa fecha, hora, participantes y modalidad (presencial o videoconferencia). 4. El sistema valida la coherencia procesal (el tipo de audiencia corresponde a la etapa vigente del expediente). 5. El sistema registra la audiencia vinculada al expediente. 6. Al registrar el resultado de la audiencia (por ejemplo, vinculación a proceso), el sistema activa automáticamente los plazos derivados, calculando las fechas de vencimiento conforme a las reglas del artículo 94 del CNPP (véase §2.2.5) y al calendario de días inhábiles configurado.
Flujos alternos	4a. El tipo de audiencia no corresponde a la etapa procesal vigente → el sistema rechaza la programación. 6a. El resultado de la audiencia no genera plazos derivados → el sistema registra el resultado sin activar cálculos adicionales.
Postcondiciones	La audiencia queda registrada y visible en el calendario del despacho. Los plazos derivados, si los hay, quedan activos con sus fechas de vencimiento calculadas y las alertas preventivas programadas.
Derivado de	HU-19, HU-20, HU-21
RF relacionados	RF-07, RF-08
RNF relacionados	RNF-10

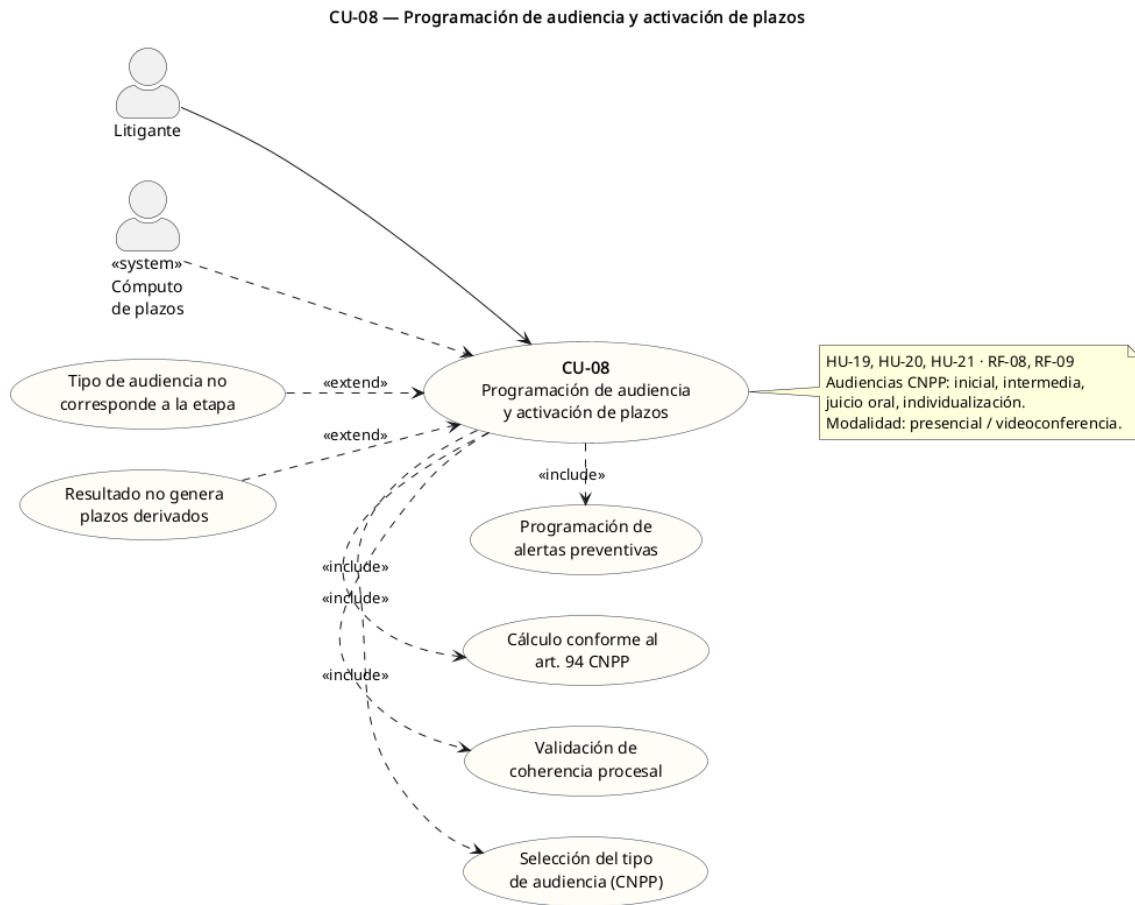
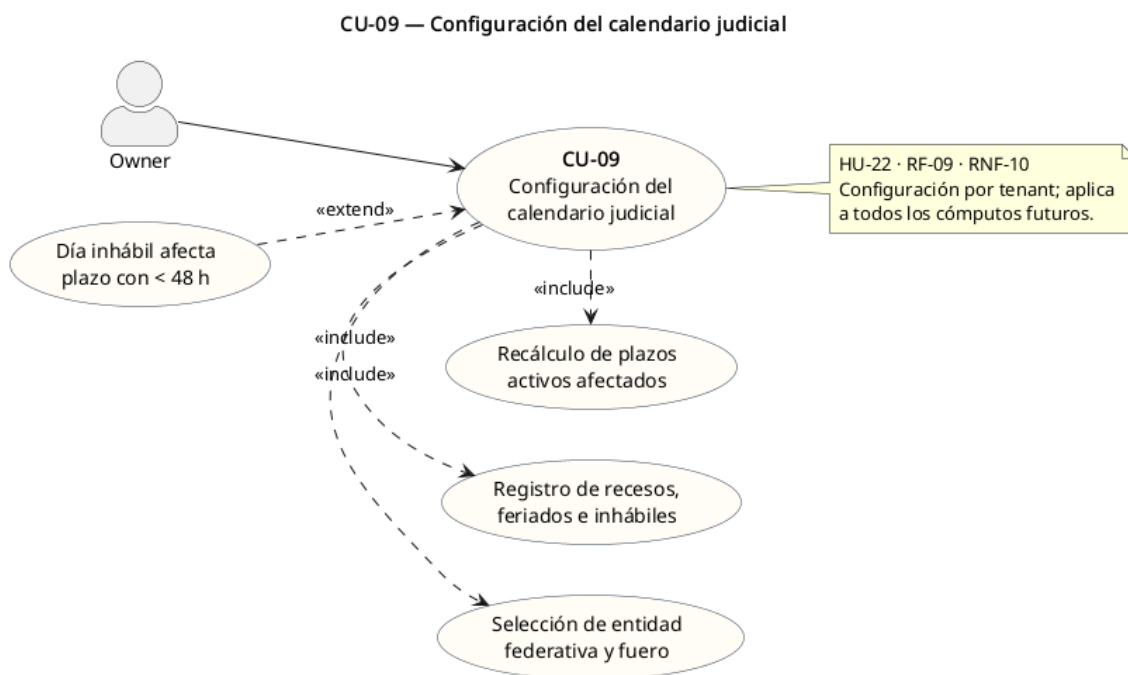


Figura 3.9: Diagrama CU-08. Programación de audiencia y activación de plazos.

CU-09 — Configuración del calendario judicial**Tabla 3.9:** Caso de uso CU-09: Configuración del calendario judicial.

ID	CU-09
Nombre	Configuración del calendario judicial
Actor principal	Owner
Actores secundarios	—
Precondiciones	El Owner tiene sesión activa.
Flujo principal	1. El Owner accede al módulo de configuración del despacho. 2. Selecciona la entidad federativa y el fuero para los cuales configurará los días inhábiles. 3. Registra los periodos de receso judicial, días feriados y días inhábiles específicos de la jurisdicción. 4. El sistema almacena la configuración y recalcula los plazos activos que dependan de ese calendario.
Flujos alternos	3a. Se registra un día inhábil que afecta un plazo con vencimiento inminente (menos de 48 horas) → el sistema recalcula la fecha y emite una alerta al Litigante responsable.
Postcondiciones	El calendario judicial del despacho queda actualizado. Todos los cálculos de plazos futuros utilizan la configuración vigente.
Derivado de	HU-22
RF relacionados	RF-08
RNF relacionados	RNF-10

**Figura 3.10:** Diagrama CU-09. Configuración del calendario judicial.

CU-10 — Monitoreo de plazos y emisión de alertas**Tabla 3.10:** Caso de uso CU-10: Monitoreo de plazos y emisión de alertas.

ID	CU-10
Nombre	Monitoreo de plazos y emisión de alertas
Actor principal	Sistema (proceso automático)
Actores secundarios	Litigante (receptor de alertas), sistema de correo transaccional
Precondiciones	Existen plazos activos en el sistema.
Flujo principal	1. El sistema ejecuta periódicamente la verificación de plazos activos. 2. Para cada plazo próximo a vencer (dentro del umbral configurado: 48 h y 24 h por defecto), el sistema genera una notificación. 3. La notificación se envía al Litigante responsable por correo electrónico y se muestra como alerta en el tablero de control. 4. El envío de la notificación se registra en la bitácora de actividad.
Flujos alternos	2a. El plazo ya venció sin que se registrara la actuación correspondiente → el sistema emite una alerta de nivel crítico.
Postcondiciones	El Litigante ha sido notificado de la proximidad del vencimiento. El envío queda registrado en la bitácora.
Derivado de	HU-23, HU-24
RF relacionados	RF-09, RF-10
RNF relacionados	RNF-06

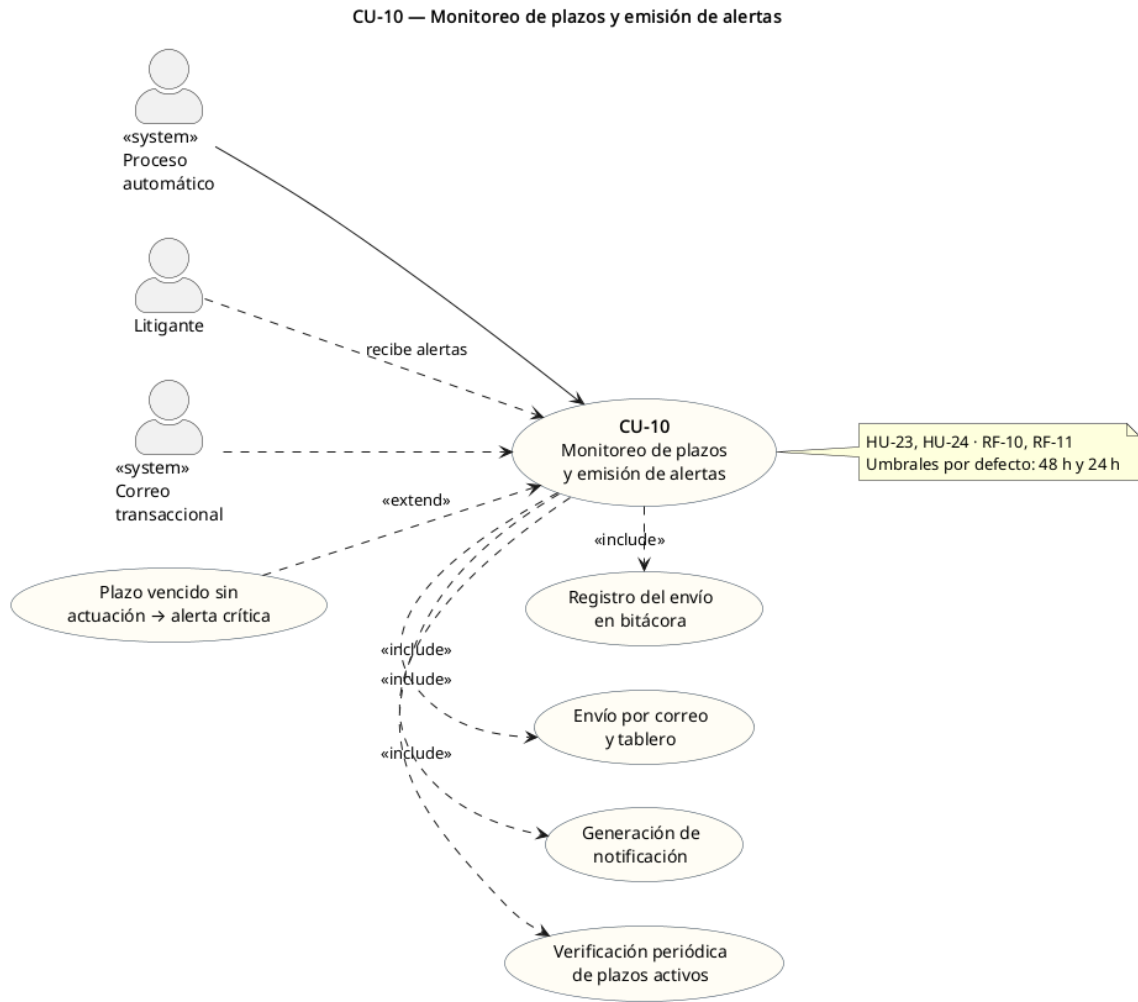


Figura 3.11: Diagrama CU-10. Monitoreo de plazos y emisión de alertas.

3.3.4 Gestión Documental y Cadena de Custodia

CU-11 — Carga y clasificación de documentos

Tabla 3.11: Caso de uso CU-11: Carga y clasificación de documentos.

ID	CU-11
Nombre	Carga y clasificación de documentos
Actor principal	Litigante
Actores secundarios	Paralegal (consulta posterior)
Precondiciones	Existe un expediente al cual asociar el documento. El usuario tiene perfil con permisos de carga.
Flujo principal	1. El Litigante accede al repositorio documental del expediente. 2. Selecciona el archivo a cargar (PDF, DOCX u otro formato admitido). 3. Asigna tipo documental, clasificación y etiquetas al documento. 4. El sistema calcula la huella digital del archivo al momento de la carga. 5. El sistema cifra el documento y lo almacena en el repositorio. 6. El sistema registra en la cadena de custodia: tipo de operación (carga), usuario responsable, marca de tiempo, huella digital del documento y estado del documento.
Flujos alternos	2a. El formato del archivo no está admitido → el sistema rechaza la carga e informa los formatos permitidos. 2b. El archivo excede el tamaño máximo admitido → el sistema rechaza la carga.
Postcondiciones	El documento queda almacenado cifrado, indexado con su huella digital y disponible para consulta por los perfiles autorizados. La operación queda registrada en la cadena de custodia.
Derivado de	HU-25, HU-26, HU-28, HU-30, HU-32
RF relacionados	RF-12, RF-13, RF-14
RNF relacionados	RNF-03, RNF-04

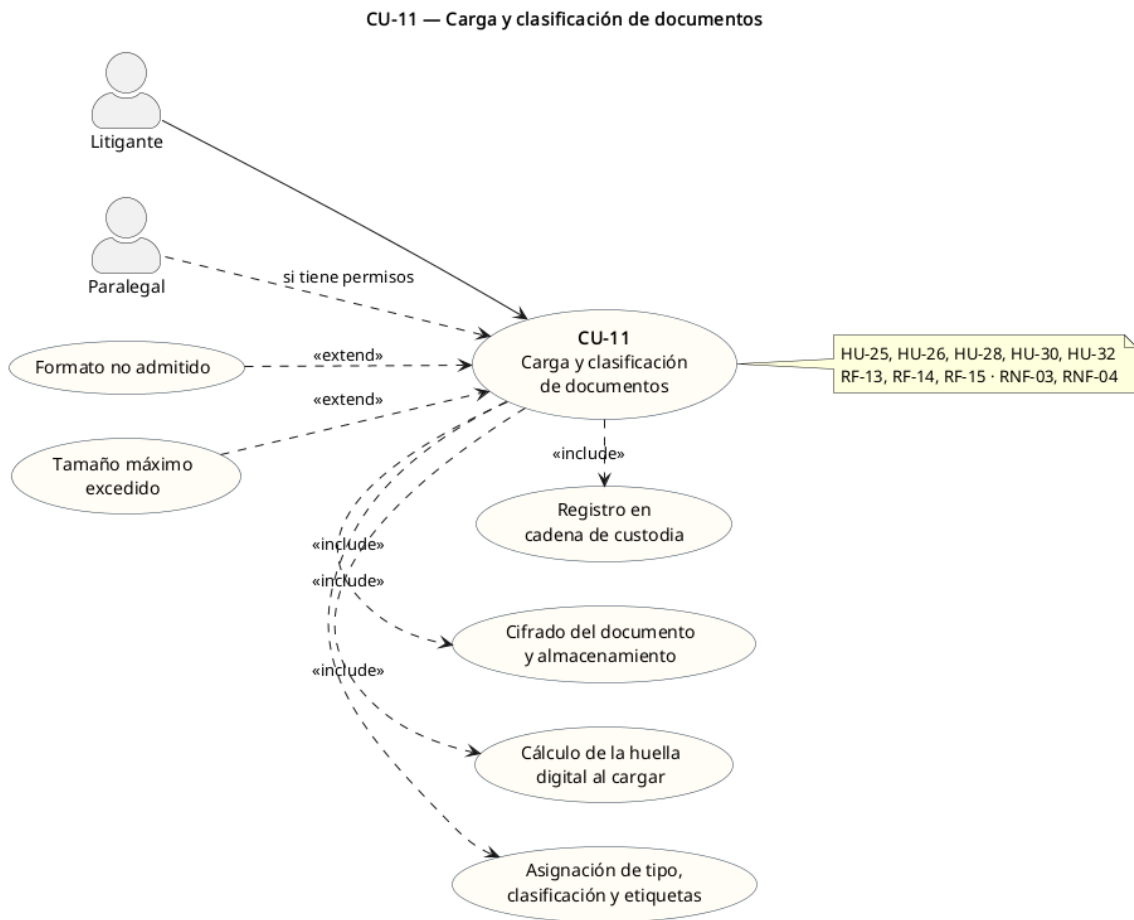


Figura 3.12: Diagrama CU-11. Carga y clasificación de documentos.

CU-12 — Consulta de documentos con verificación de integridad**Tabla 3.12:** Caso de uso CU-12: Consulta de documentos con verificación de integridad.

ID	CU-12
Nombre	Consulta de documentos con verificación de integridad
Actor principal	Litigante o Paralegal
Actores secundarios	—
Precondiciones	El usuario tiene perfil con acceso al expediente que contiene el documento.
Flujo principal	1. El usuario accede al repositorio documental del expediente. 2. Selecciona el documento a consultar. 3. El sistema verifica que el perfil del usuario tiene permiso de acceso al expediente. 4. El sistema descifra el documento. 5. El sistema recalcula la huella digital del documento descifrado y la compara con el valor de referencia almacenado al momento de la carga. 6. Si las huellas coinciden, el sistema entrega el documento al usuario. 7. La operación de consulta se registra en la cadena de custodia.
Flujos alternos	3a. El usuario no tiene permiso sobre el expediente → el sistema deniega el acceso. 4a. La verificación de la etiqueta de autenticación del cifrado falla (el contenido cifrado fue manipulado en el almacenamiento) → el sistema rechaza el descifrado, notifica al usuario y al Owner, y registra el incidente como alerta de seguridad. 5a. La huella digital recalculada no coincide con la referencia → el sistema alerta al usuario de que el documento fue modificado, bloquea la entrega y registra la discrepancia.
Postcondiciones	El usuario recibe el documento íntegro, o recibe una alerta explícita de alteración. La consulta y su resultado quedan registrados en la cadena de custodia.
Derivado de	HU-27, HU-29, HU-31, HU-33
RF relacionados	RF-11, RF-12, RF-13, RF-14
RNF relacionados	RNF-03, RNF-04

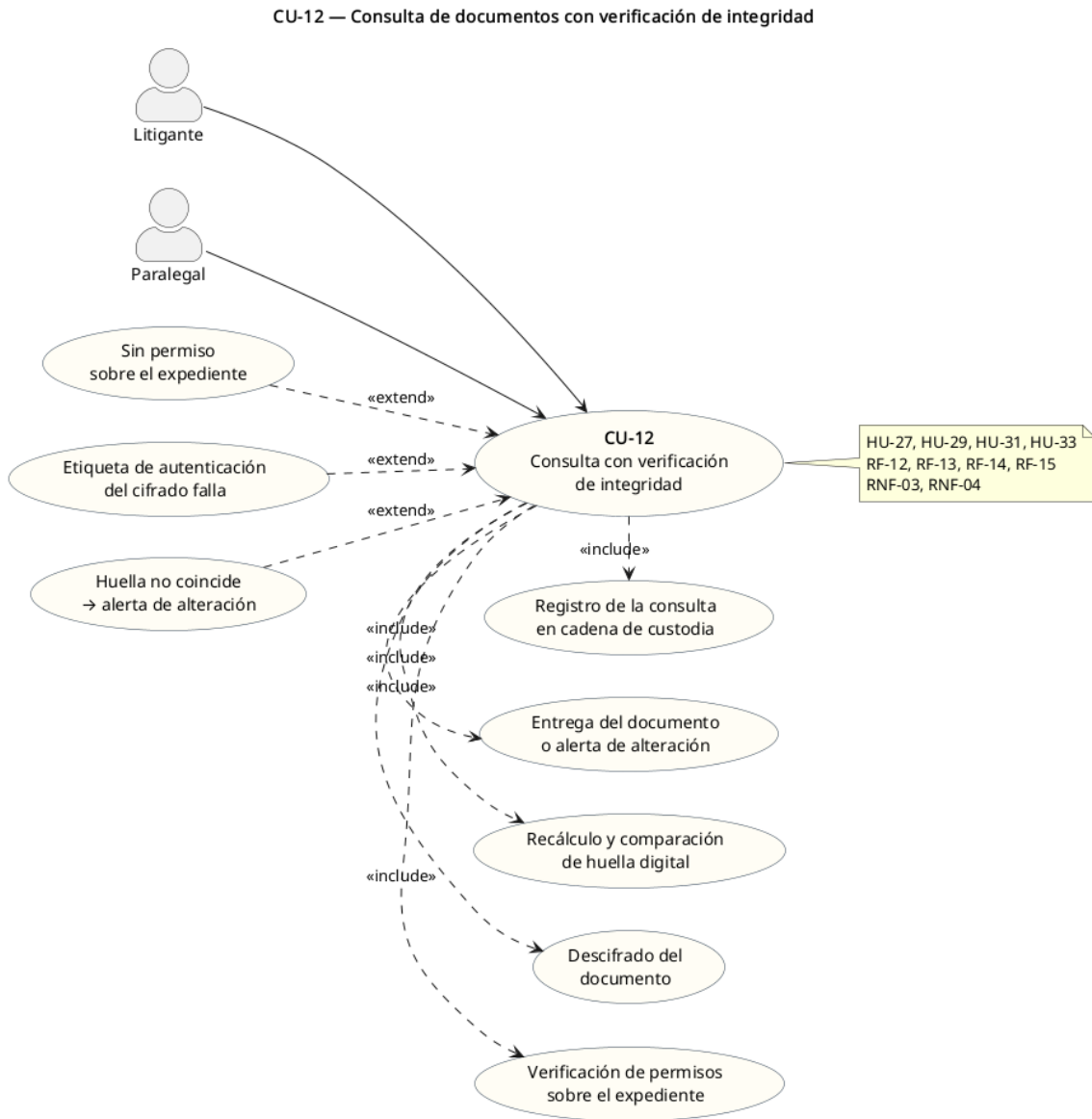


Figura 3.13: Diagrama CU-12. Consulta de documentos con verificación de integridad.

3.3.5 Firma de Contratos y Constancia de Fecha

CU-13 — Firma digital de contrato con sellado de tiempo

Tabla 3.13: Caso de uso CU-13: Firma digital de contrato con sellado de tiempo.

ID	CU-13
Nombre	Firma digital de contrato con sellado de tiempo
Actor principal	Litigante (firmante)
Actores secundarios	TSA configurada (autoridad local en la demostración; PSC autorizado solo en una integración futura)
Precondiciones	El contrato existe en el repositorio en formato PDF y está listo para firma. El firmante tiene un certificado digital X.509 vigente registrado en el sistema.
Flujo principal	1. El Litigante accede al contrato y selecciona la opción de firma digital. 2. El sistema presenta el contenido del contrato y solicita confirmación explícita de la acción de firma. 3. El Litigante confirma la firma. 4. El sistema calcula la huella digital del contrato. 5. El sistema firma la huella digital con la clave privada del certificado X.509 del firmante, vinculando criptográficamente su identidad con el contenido del documento. 6. El sistema envía la huella digital a la TSA configurada mediante el protocolo de sellado de tiempo. 7. La TSA retorna el token RFC 3161 que acredita criptográficamente el digest y su tiempo de generación. 8. El sistema almacena el contrato firmado junto con la firma digital, los metadatos del certificado y el token de sello de tiempo. 9. La operación se registra en la bitácora de actividad y en la cadena de custodia del expediente.
Flujos alternos	2a. El certificado del firmante ha expirado o está revocado → el sistema informa la situación e impide la firma. 6a. La TSA no responde dentro del tiempo límite → el sistema informa o encola la solicitud según la política explícita del emisor; no conmuta silenciosamente a otra autoridad. 7a. El token recibido no supera la validación de formato → el sistema rechaza el sello y registra el error.
Postcondiciones	El contrato queda firmado con evidencia criptográfica verificable de la identidad del firmante y del momento de la firma. Si la autoridad es la TSA local, el resultado acredita el flujo técnico RFC 3161, pero no constituye por sí mismo una constancia NOM-151 de un PSC autorizado.
Derivado de	HU-34, HU-35
RF relacionados	RF-15, RF-16
RNF relacionados	RNF-03, RNF-11

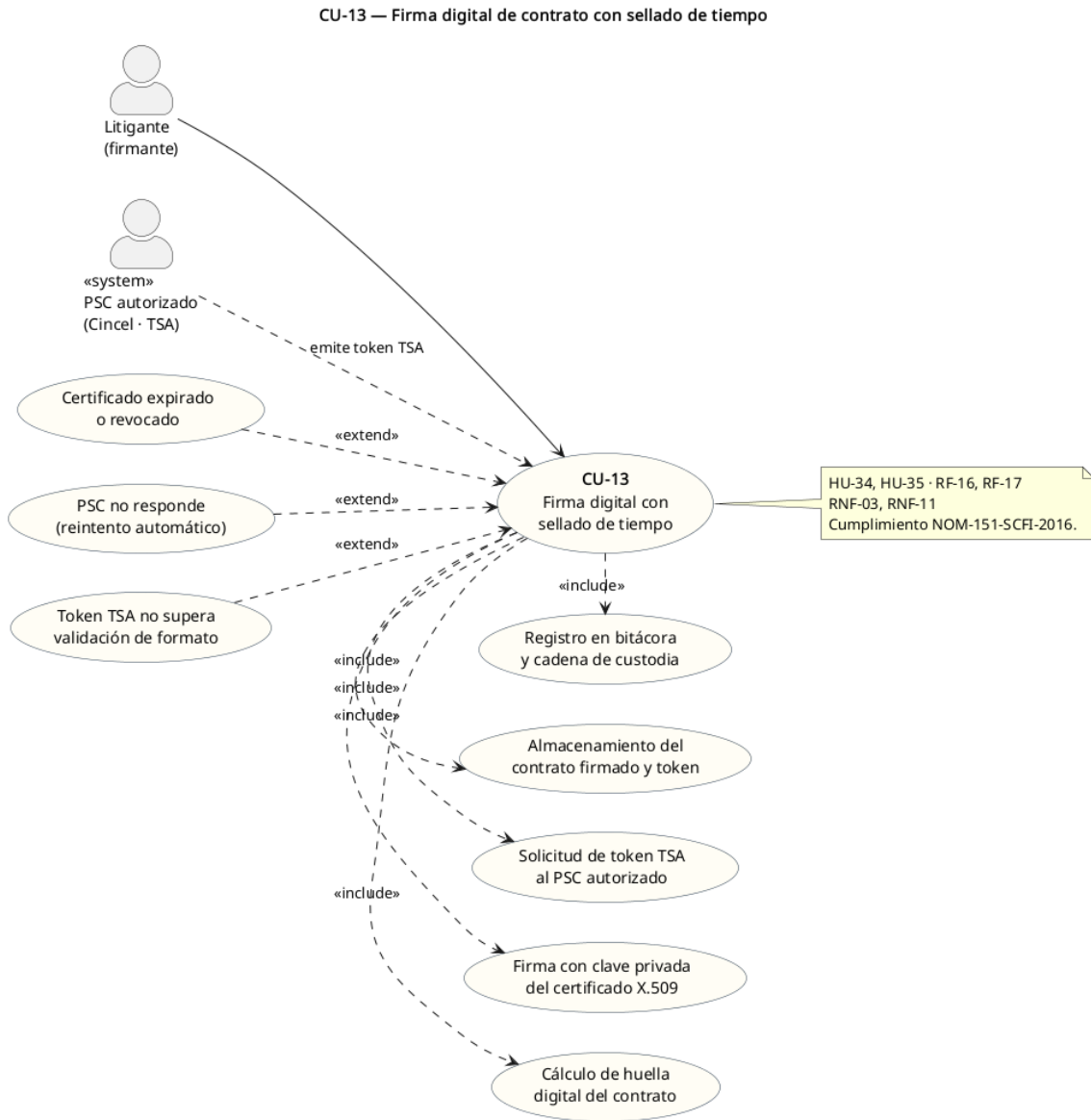


Figura 3.14: Diagrama CU-13. Firma digital de contrato con sellado de tiempo.

CU-14 — Verificación de firma digital y sello de tiempo**Tabla 3.14:** Caso de uso CU-14: Verificación de firma digital y sello de tiempo.

ID	CU-14
Nombre	Verificación de firma digital y sello de tiempo
Actor principal	Usuario autenticado con acceso al contrato
Actores secundarios	—
Precondiciones	El contrato ha sido firmado digitalmente y posee un token de sello de tiempo asociado.
Flujo principal	1. El usuario accede al contrato firmado y selecciona la opción de verificación. 2. El sistema recalcula la huella digital del contrato actual. 3. El sistema verifica la firma digital: descifra la firma con la clave pública del certificado X.509 del firmante y compara el resultado con la huella digital recalculada. 4. El sistema valida el estado del certificado del firmante (vigente, revocado o expirado) consultando la cadena de certificación. 5. El sistema verifica la autenticidad del token de sello de tiempo validando la firma del PSC. 6. El sistema presenta al usuario un reporte de verificación con el resultado de cada componente.
Flujos alternos	3a. La huella digital recalculada no coincide con la contenida en la firma → el sistema reporta que el documento fue modificado después de firmado e indica la causa de la invalidación. 4a. El certificado del firmante ha expirado o fue revocado → el sistema lo indica en el reporte sin invalidar necesariamente la firma (la firma era válida al momento del sellado). 5a. El token de sello de tiempo no supera la validación → el sistema reporta que la constancia de fecha no es verificable.
Postcondiciones	El usuario dispone de un reporte de verificación con el estado de cada componente: validez de la firma, estado del certificado, validez del sello de tiempo e integridad del documento.
Derivado de	HU-36, HU-37
RF relacionados	RF-17
RNF relacionados	RNF-11

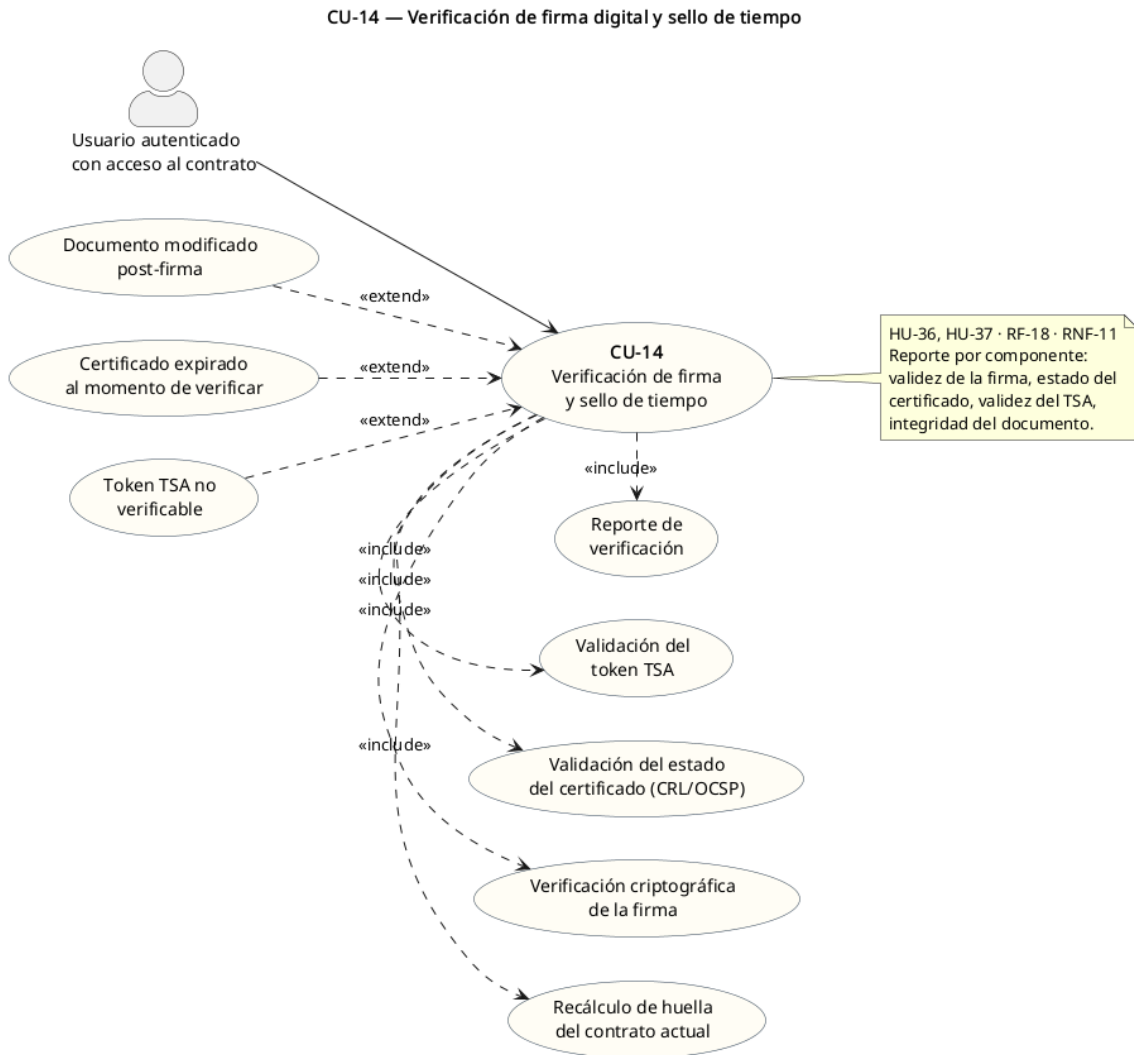


Figura 3.15: Diagrama CU-14. Verificación de firma digital y sello de tiempo.

3.3.6 Operación y Monitoreo

CU-15 — Consulta del tablero de control

Tabla 3.15: Caso de uso CU-15: Consulta del tablero de control.

ID	CU-15
Nombre	Consulta del tablero de control
Actor principal	Owner o Litigante
Actores secundarios	—
Precondiciones	El usuario tiene sesión activa.
Flujo principal	1. El usuario accede al tablero de control del despacho. 2. El sistema consulta los indicadores operativos del despacho: casos activos, plazos por vencer en 48 h y 7 días, contratos pendientes de firma y carga de trabajo por abogado. 3. El sistema presenta los indicadores con diferenciación visual por nivel de urgencia.
Flujos alternos	2a. Existen plazos fatales por vencer en menos de 48 horas → el sistema los destaca con indicador de alerta de nivel crítico, diferenciado visualmente del resto de elementos.
Postcondiciones	El usuario visualiza el estado actual del despacho.
Derivado de	HU-38
RF relacionados	RF-18
RNF relacionados	RNF-07

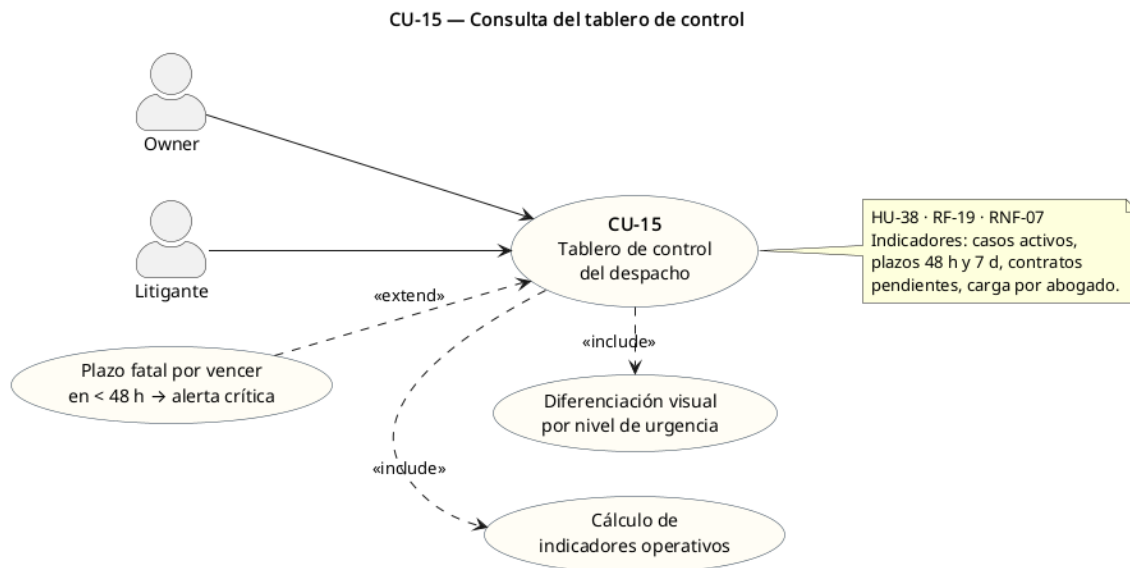


Figura 3.16: Diagrama CU-15. Consulta del tablero de control.

CU-16 — Generación de informes**Tabla 3.16:** Caso de uso CU-16: Generación de informes.

ID	CU-16
Nombre	Generación de informes
Actor principal	Owner
Actores secundarios	Sistema de colas de trabajo
Precondiciones	El Owner tiene sesión activa.
Flujo principal	1. El Owner accede al módulo de reportes. 2. Selecciona el tipo de informe y los filtros aplicables (rango de fechas, abogado responsable, estado de los casos). 3. El sistema encola la solicitud de generación como tarea en segundo plano. 4. El sistema confirma la recepción de la solicitud y muestra un indicador de progreso. 5. Al completar la generación, el sistema notifica al Owner y pone el informe disponible para descarga en formato PDF o CSV.
Flujos alternos	3a. El volumen de datos supera el umbral de procesamiento inmediato → el sistema informa el tiempo estimado y envía notificación por correo cuando el reporte esté listo.
Postcondiciones	El informe queda disponible para descarga. La interfaz permanece responsiva durante el procesamiento.
Derivado de	HU-39
RF relacionados	RF-19
RNF relacionados	RNF-06, RNF-07

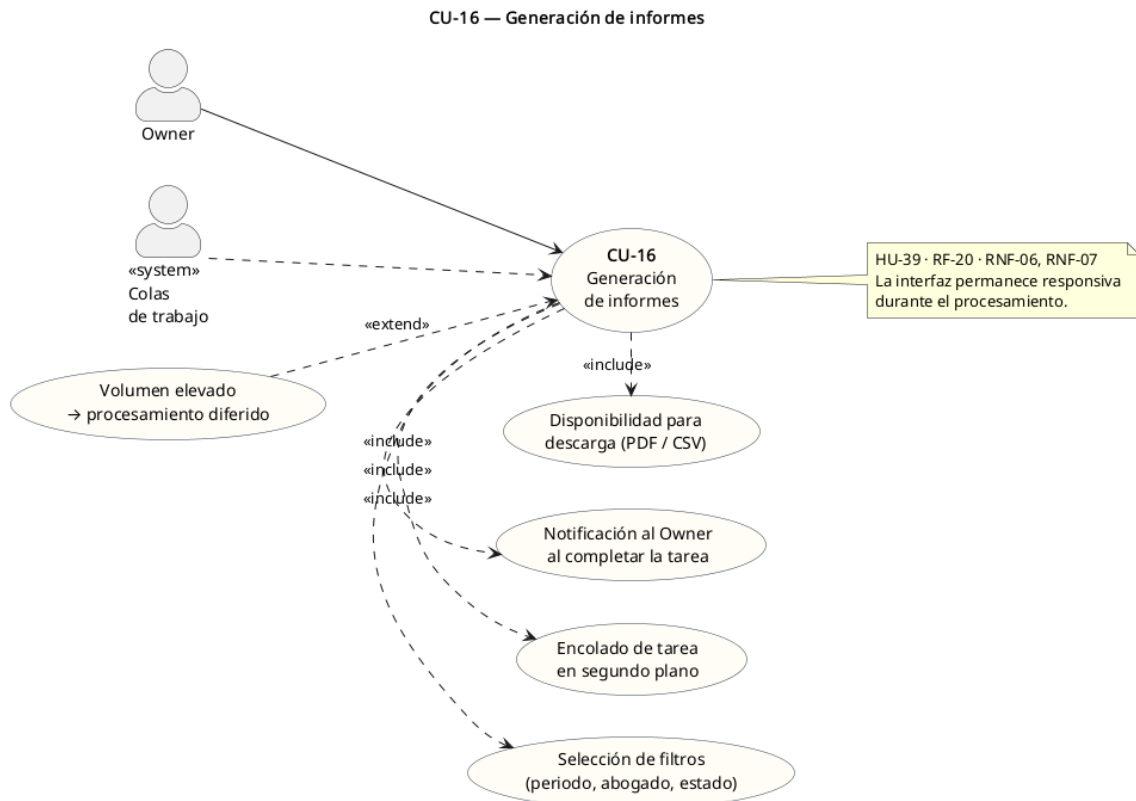
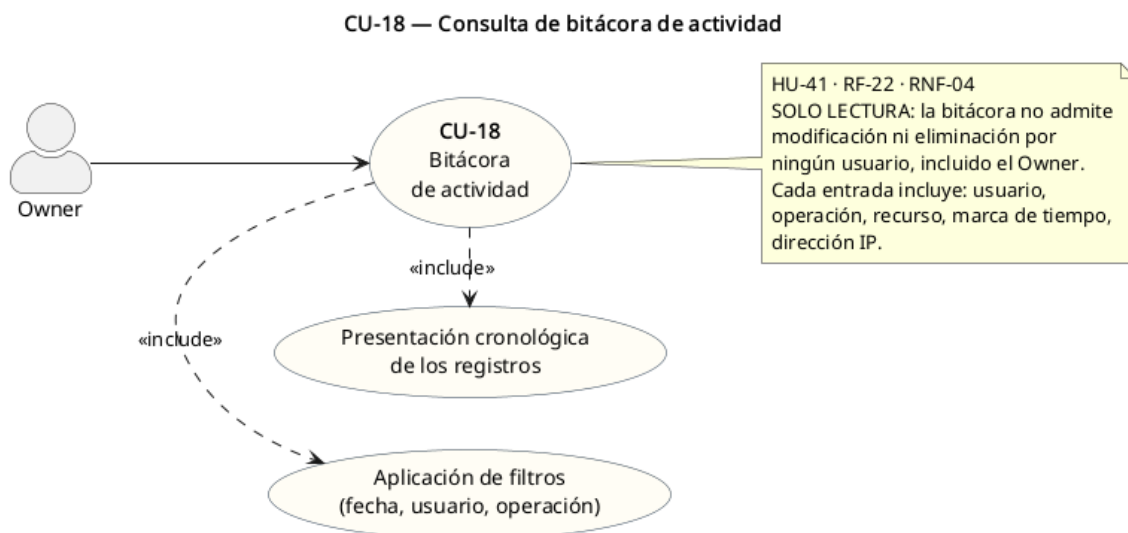


Figura 3.17: Diagrama CU-16. Generación de informes.

CU-17 — Consulta de bitácora de actividad**Tabla 3.17:** Caso de uso CU-17: Consulta de bitácora de actividad.

ID	CU-17
Nombre	Consulta de bitácora de actividad
Actor principal	Owner
Actores secundarios	—
Precondiciones	El Owner tiene sesión activa.
Flujo principal	1. El Owner accede al módulo de bitácora de actividad. 2. Aplica filtros de búsqueda (rango de fechas, usuario, tipo de operación, recurso afectado). 3. El sistema presenta los registros cronológicamente, mostrando para cada entrada: identificador del usuario, tipo de operación, recurso afectado, marca de tiempo y dirección IP.
Flujos alternos	— (la bitácora es de solo lectura; no existen operaciones de modificación o eliminación).
Postcondiciones	El Owner visualiza los registros solicitados. Los registros de la bitácora no pueden modificarse ni eliminarse por ningún usuario del sistema, incluido el Owner.
Derivado de	HU-40
RF relacionados	RF-20
RNF relacionados	RNF-04

**Figura 3.18:** Diagrama CU-17. Consulta de bitácora de actividad.

3.4 Requerimientos del Sistema

Los requerimientos se derivan de los objetivos definidos anteriormente y se organizan en funcionales y no funcionales. Los requerimientos funcionales definen los servicios que el sistema debe proporcionar y cómo debe reaccionar ante entradas particulares. Los requerimientos no funcionales describen restricciones y atributos de calidad que limitan las decisiones de diseño e implementación.

3.4.1 Requerimientos Funcionales

Los requerimientos funcionales especifican los servicios que el sistema debe proporcionar, las entradas que acepta, las salidas que produce y el comportamiento esperado ante condiciones particulares. Cada requerimiento se deriva de las historias de usuario definidas en la [sección 3.2](#) y se formaliza mediante una estructura que incluye descripción, entradas, salidas, actores involucrados y criterio de aceptación verificable. Los requerimientos se agrupan en cinco categorías funcionales alineadas con los módulos del prototipo.

Autenticación y Control de Acceso

Tabla 3.18: RF-01: Autenticación y gestión de sesiones

ID	RF-01
Nombre	Autenticación y gestión de sesiones
Descripción	El sistema debe proveer mecanismos seguros de inicio de sesión, recuperación de contraseñas, expiración de sesiones por inactividad y protección contra fijación de sesión.
Entrada	Credenciales del usuario (usuario/contraseña o certificado digital); solicitud de recuperación de contraseña.
Salida	Sesión autenticada con token de acceso; notificación de expiración; redireccionamiento al inicio de sesión tras inactividad sin pérdida de datos no guardados.
Actores involucrados	Todos los roles (Owner, Litigante, Paralegal, Cliente); sistema de gestión de sesiones.
Criterio de aceptación	Dado un usuario inactivo por el tiempo configurado, cuando vence la sesión, entonces el sistema la invalida automáticamente y requiere nueva autenticación sin pérdida de datos no guardados.

Tabla 3.19: RF-02: Control de acceso basado en roles (RBAC)

ID	RF-02
Nombre	Control de acceso basado en roles (RBAC)
Descripción	El sistema debe asignar permisos de forma granular a través de roles predefinidos (Owner, Litigante, Paralegal, Cliente) para restringir el acceso a módulos y operaciones sensibles.
Entrada	Rol asignado al usuario; recurso o módulo al que se intenta acceder.
Salida	Permiso concedido o denegado; registro en bitácora del intento de acceso.
Actores involucrados	Owner (configuración de roles); todos los usuarios como sujetos de control; sistema de autorización.
Criterio de aceptación	Dado un usuario con rol Paralegal, cuando intenta acceder a un módulo restringido a Owner o Litigante, entonces el sistema deniega el acceso con mensaje explicativo y registra el intento en la bitácora.

Tabla 3.20: RF-03: Gestión de equipos colaborativos

ID	RF-03
Nombre	Gestión de equipos colaborativos
Descripción	El sistema debe permitir invitar nuevos miembros al despacho mediante correo electrónico, gestionar la aceptación de invitaciones y administrar el alta y baja de usuarios en el equipo.
Entrada	Correo electrónico del nuevo miembro; rol a asignar; acción de aceptación del invitado.
Salida	Invitación enviada; usuario creado y asociado al despacho; acceso habilitado según el rol asignado.
Actores involucrados	Owner (invita y da de baja); usuario invitado; sistema de correo transaccional.
Criterio de aceptación	Dado un Owner que envía una invitación, cuando el invitado la acepta dentro del periodo de validez del enlace, entonces queda registrado con el rol indicado; si el enlace vence, la invitación queda inactiva automáticamente.

Gestión Procesal Penal

Tabla 3.21: RF-04: Expediente penal digital

ID	RF-04
Nombre	Expediente penal digital
Descripción	El sistema debe permitir el registro y administración integral de casos penales, incluyendo identificadores oficiales (NUC, Carpeta Judicial), delitos y metadatos asociados.
Entrada	NUC, Carpeta Judicial, tipo de delito, datos del caso y metadatos ingresados por el Litigante.
Salida	Expediente registrado con identificadores únicos, estado inicial ‘Investigación’ y timestamp de creación.
Actores involucrados	Litigante, Paralegal; base de datos relacional.
Criterio de aceptación	Dado un Litigante que registra un nuevo caso, cuando omite el NUC o la Carpeta Judicial, entonces el sistema rechaza el registro con indicación del campo faltante; cuando los datos son completos, el expediente queda persistido y disponible en el módulo de gestión procesal.

Tabla 3.22: RF-05: Control de etapas procesales (CNPP)

ID	RF-05
Nombre	Control de etapas procesales (CNPP)
Descripción	El sistema debe gestionar la transición entre etapas procesales (Investigación, Intermedia, Juicio Oral) conforme al CNPP, manteniendo un historial inmutable de cambios y validaciones procesales.
Entrada	Documento procesal habilitante cargado por el usuario (ej. escrito de acusación para transitar de Investigación a Intermedia).
Salida	Registro inmutable de la transición con marca de tiempo, identificador de usuario y documento asociado; módulo de la siguiente etapa habilitado.
Actores involucrados	Litigante (Owner); sistema de gestión procesal; bitácora de auditoría.
Criterio de aceptación	Dado un expediente en etapa de Investigación, cuando se intenta avanzar sin cargar el escrito de acusación, entonces el sistema rechaza la transición; cuando se carga el documento habilitante, la transición se registra de forma inmutable y no puede revertirse.

Tabla 3.23: RF-06: Directorio de participantes

ID	RF-06
Nombre	Directorio de participantes
Descripción	El sistema debe permitir la gestión de sujetos procesales vinculados a cada caso (imputados, víctimas, jueces, peritos), definiendo su rol procesal y estatus jurídico.
Entrada	Datos del sujeto procesal (nombre, rol, cédula, CURP u otros identificadores según tipo); caso al que se vincula.
Salida	Participante registrado y vinculado al expediente con su rol procesal; historial de cambios de rol disponible.
Actores involucrados	Litigante; sistema de gestión de directorio.
Criterio de aceptación	Dado un caso activo, cuando se registra un participante con rol ‘Juez de control’, entonces el sistema valida que se proporcione el órgano jurisdiccional y la firma electrónica (FIREL); un mismo usuario puede tener roles distintos en casos diferentes sin conflicto.

Tabla 3.24: RF-07: Gestión de audiencias especializadas

ID	RF-07
Nombre	Gestión de audiencias especializadas
Descripción	El sistema debe permitir la programación y seguimiento de audiencias tipificadas según el CNPP, registrando resultados, asistentes y acuerdos derivados.
Entrada	Tipo de audiencia (del catálogo CNPP), fecha, hora, participantes y modalidad (presencial o videoconferencia).
Salida	Audiencia agendada vinculada al expediente; activación automática del cómputo de plazos derivados; registro de resultados y acuerdos.
Actores involucrados	Litigante; sistema de calendario; módulo de plazos.
Criterio de aceptación	Dado el registro de una ‘Audiencia inicial’ con vinculación a proceso, cuando se confirma, entonces el sistema activa automáticamente el plazo de investigación complementaria (2 o 6 meses) y genera la alerta preventiva correspondiente.

Tabla 3.25: RF-08: Blindaje de plazos procesales

ID	RF-08
Nombre	Blindaje de plazos procesales
Descripción	El sistema debe permitir el registro y cálculo de términos constitucionales y plazos fatales, con derivación automática a partir de actuaciones o audiencias registradas.
Entrada	Actuación o audiencia registrada que activa un plazo constitucional; calendario de días inhábiles configurado por entidad federativa.
Salida	Plazo calculado con fecha y hora de vencimiento; alerta programada; registro en el módulo de plazos del expediente.
Actores involucrados	Litigante; sistema de cómputo de plazos; calendario judicial configurable.
Criterio de aceptación	Dado el registro de vinculación a proceso con plazo de 6 meses de investigación complementaria, entonces el sistema calcula la fecha de vencimiento excluyendo correctamente los días inhábiles del calendario configurado para la entidad federativa correspondiente.

Tabla 3.26: RF-09: Sistema de alertas preventivas

ID	RF-09
Nombre	Sistema de alertas preventivas
Descripción	El sistema debe generar notificaciones automáticas ante la proximidad de vencimientos de plazos o audiencias programadas.
Entrada	Plazos activos registrados; reglas de anticipación configurables (ej. alerta a 48 h y a 24 h del vencimiento).
Salida	Notificación enviada al Litigante responsable por correo y visible en el dashboard del sistema.
Actores involucrados	Litigante; sistema de notificaciones; módulo de colas de trabajo.
Criterio de aceptación	Dado un plazo de vinculación a proceso con vencimiento en 48 horas, cuando el sistema ejecuta la verificación periódica de plazos, entonces envía una notificación de alerta roja al Litigante responsable; el envío se registra en la bitácora.

Tabla 3.27: RF-10: Calendario jurídico integrado

ID	RF-10
Nombre	Calendario jurídico integrado
Descripción	El sistema debe proveer una visualización unificada de la agenda del despacho, integrando audiencias y plazos límite en un calendario interactivo.
Entrada	Audiencias programadas; plazos activos; configuración de la vista seleccionada (día, semana o mes).
Salida	Calendario unificado con codificación de color por tipo de evento y acceso directo al expediente relacionado desde cada entrada.
Actores involucrados	Litigante, Paralegal, Owner.
Criterio de aceptación	Dado un despacho con 5 o más casos activos, cuando el Litigante accede al calendario, entonces visualiza todos los vencimientos y audiencias de sus casos en una sola vista sin necesidad de navegar por cada expediente individualmente.

Gestión Documental y Evidencias

Tabla 3.28: RF-11: Cadena de custodia digital

ID	RF-11
Nombre	Cadena de custodia digital
Descripción	El sistema debe proveer un módulo para el registro de evidencias físicas y digitales, garantizando la trazabilidad de sus movimientos y resguardos mediante un registro cronológico inmutable.
Entrada	Evidencia física o digital ingresada al sistema; operación de movimiento o consulta de la evidencia.
Salida	Registro cronológico append-only con: tipo de operación, identificador de la evidencia, usuario responsable, timestamp y estado de la evidencia.
Actores involucrados	Litigante, Paralegal; sistema de almacenamiento NoSQL (registro de auditoría).
Criterio de aceptación	Dado un registro de evidencia existente, cuando cualquier usuario intenta modificar o eliminar una entrada de la cadena de custodia, entonces el sistema rechaza la operación; cada nuevo movimiento genera únicamente un registro de inserción nuevo.

Tabla 3.29: RF-12: Repositorio documental

ID	RF-12
Nombre	Repositorio documental
Descripción	El sistema debe permitir cargar, clasificar, consultar y compartir documentos procesales asociados a expedientes, asegurando su organización, disponibilidad y control de versiones.
Entrada	Archivo digital (PDF, DOCX, multimedia); clasificación y metadatos ingresados por el usuario; expediente al que se asocia.
Salida	Documento almacenado cifrado (AES-256-GCM), indexado con su hash SHA-256 y disponible para consulta por los roles autorizados.
Actores involucrados	Litigante, Paralegal; sistema de almacenamiento cifrado.
Criterio de aceptación	Dado un documento cargado al repositorio, cuando un Paralegal intenta acceder a un documento de un expediente no asignado, entonces el sistema deniega el acceso; el documento debe ser recuperable en menos de 3 segundos para archivos de hasta 10 MB.

Servicios Criptográficos

Tabla 3.30: RF-13: Verificación de integridad de documentos

ID	RF-13
Nombre	Verificación de integridad de documentos
Descripción	El sistema debe calcular y almacenar el hash SHA-256 (véase §2.3.1) de cada documento al momento de su carga. En cada consulta posterior, debe recalcular el hash y compararlo con el valor de referencia, notificando al usuario si se detecta una discrepancia.
Entrada	Archivo digital cargado al sistema (PDF, DOCX u otro formato admitido).
Salida	Hash SHA-256 almacenado en base de datos; alerta visible al usuario en caso de discrepancia detectada en consulta posterior.
Actores involucrados	Litigante, Paralegal; sistema de almacenamiento (backend).
Criterio de aceptación	Dado un documento cargado, cuando se modifica 1 byte del archivo almacenado, entonces el sistema detecta la discrepancia y muestra una alerta en la siguiente consulta sin requerir intervención manual.

Tabla 3.31: RF-14: Cifrado autenticado de expedientes y documentos

ID	RF-14
Nombre	Cifrado autenticado de expedientes y documentos
Descripción	El sistema debe cifrar los documentos y expedientes almacenados mediante AES-256-GCM (véase §2.3.2), proveyendo simultáneamente confidencialidad e integridad. Cada operación de cifrado debe emplear un vector de inicialización (IV) único generado aleatoriamente.
Entrada	Documento o expediente a cifrar; clave maestra del despacho; IV generado aleatoriamente por operación.
Salida	Contenido cifrado con AES-256-GCM almacenado en el repositorio; IV almacenado de forma separada de los datos cifrados.
Actores involucrados	Sistema de cifrado (backend); módulo de gestión de claves.
Criterio de aceptación	Dado un documento cifrado almacenado, cuando se modifica 1 byte del texto cifrado directamente en el almacenamiento, entonces la verificación de la etiqueta de autenticación GCM falla y el sistema rechaza el descifrado; cada operación debe utilizar un IV único, verificable mediante pruebas TDD.

Tabla 3.32: RF-15: Firma digital de contratos

ID	RF-15
Nombre	Firma digital de contratos
Descripción	El sistema debe permitir la firma digital de contratos mediante certificados X.509 (véase §2.3.3), vinculando criptográficamente la identidad del firmante con el contenido del documento.
Entrada	Contrato en formato PDF listo para firmar; certificado X.509 del firmante; confirmación de la acción de firma por el usuario.
Salida	Documento firmado con hash SHA-256 cifrado con la clave privada del firmante; metadatos de firma (timestamp local, identificador del certificado) almacenados junto al documento.
Actores involucrados	Litigante (firmante); sistema de firma digital; módulo de certificados X.509.
Criterio de aceptación	Dado un contrato firmado digitalmente, cuando se modifica cualquier byte del documento posterior a la firma, entonces la verificación retorna inválida; la firma solo puede verificarse exitosamente con la clave pública del certificado X.509 del firmante.

Tabla 3.33: RF-16: Sellado de tiempo (NOM-151)

ID	RF-16
Nombre	Sellado de tiempo (NOM-151)
Descripción	El sistema debe obtener un token RFC 3161 para cada contrato firmado mediante la TSA configurada, acreditando criptográficamente la existencia del digest en un momento determinado. La constancia NOM-151 de un PSC autorizado no se afirma para la TSA local.
Entrada	Hash SHA-256 del contrato firmado y solicitud al adaptador de TSA configurado.
Salida	Token de sello de tiempo (TimeStampToken) en formato ASN.1/.tsr almacenado y vinculado al contrato; la naturaleza técnica o regulada del emisor queda identificada en el paquete.
Actores involucrados	Sistema backend (cliente TSA); TSA local de demostración o PSC autorizado futuro; módulo de firma digital.
Criterio de aceptación	Dado un contrato firmado, cuando se solicita el sello a la TSA configurada, el sistema almacena un token RFC 3161 que cubre el digest y puede verificarse de forma independiente con OpenSSL sin requerir acceso al sistema. La campaña de estabilidad de un PSC externo queda fuera del alcance actual.

Tabla 3.34: RF-17: Verificación de firmas y sellos de tiempo

ID	RF-17
Nombre	Verificación de firmas y sellos de tiempo
Descripción	El sistema debe permitir verificar la validez de las firmas digitales y los sellos de tiempo asociados a un contrato, incluyendo la cadena de certificados X.509, vigencia del certificado e integridad del documento.
Entrada	Contrato firmado con su token de sello de tiempo; cadena de certificados X.509 del firmante.
Salida	Resultado de verificación con detalle de: validez de la firma, estado del certificado (vigente/revocado/expirado), validez del sello de tiempo e integridad del documento.
Actores involucrados	Cualquier usuario autenticado con acceso al contrato; sistema de verificación criptográfica.
Criterio de aceptación	Dado un contrato con firma válida e integridad verificada, cuando se ejecuta la verificación, entonces todos los componentes se reportan como válidos; dado un contrato modificado, cuando se verifica, entonces el sistema reporta la firma como inválida con indicación explícita de la causa.

Operación y Monitoreo

Tabla 3.35: RF-18: Dashboard ejecutivo

ID	RF-18
Nombre	Dashboard ejecutivo
Descripción	El sistema debe proveer un tablero de control que visualice indicadores operativos clave: casos activos, plazos próximos y carga de trabajo por abogado.
Entrada	Datos de casos activos, plazos próximos y asignación de carga del despacho.
Salida	Tablero visual con indicadores actualizados: número de casos activos, plazos por vencer en 48 h/7 días, contratos pendientes de firma.
Actores involucrados	Owner, Litigante; sistema de consulta de datos agregados.
Criterio de aceptación	Dado un Owner que accede al dashboard, cuando existen plazos fatales por vencer en menos de 48 horas, entonces el sistema los destaca visualmente con indicador de alerta roja diferenciado del resto de elementos del tablero.

Tabla 3.36: RF-19: Generación de reportes

ID	RF-19
Nombre	Generación de reportes
Descripción	El sistema debe permitir generar informes sobre el estado de los casos, desempeño de abogados y métricas operativas del despacho.
Entrada	Parámetros de filtrado seleccionados por el usuario (rango de fechas, abogado, estado del caso, tipo de reporte).
Salida	Reporte generado en formato PDF o exportable a CSV con los datos solicitados.
Actores involucrados	Owner, Litigante; sistema de generación de reportes asíncrono (cola de trabajo).
Criterio de aceptación	Dado un Owner que solicita un reporte mensual, cuando el procesamiento supera 3 segundos, entonces la interfaz permanece responsiva, muestra indicador de progreso y entrega el reporte mediante notificación sin bloquear la sesión.

Tabla 3.37: RF-20: Bitácora de actividad (Audit Trail)

ID	RF-20
Nombre	Bitácora de actividad (Audit Trail)
Descripción	El sistema debe mantener un registro cronológico inmutable de todas las acciones realizadas, incluyendo el identificador del usuario, la operación realizada, la marca de tiempo y el recurso afectado.
Entrada	Cualquier operación realizada por un usuario autenticado (consulta, carga, firma, modificación de rol, etc.).
Salida	Registro append-only en base de datos NoSQL con los campos: <code>user_id</code> , <code>action_type</code> , <code>resource_id</code> , <code>timestamp</code> , <code>ip_address</code> .
Actores involucrados	Todos los roles del sistema; motor NoSQL de auditoría.
Criterio de aceptación	Dado cualquier usuario autenticado, cuando ejecuta cualquier operación en el sistema, entonces se genera un registro en la bitácora en menos de 500 ms; el registro no puede eliminarse ni modificarse por ningún rol, incluido Owner.

3.4.2 Requerimientos No Funcionales

Los requerimientos no funcionales describen las restricciones y atributos de calidad que el sistema debe satisfacer de forma transversal a todas sus funcionalidades. Se formulan en términos del atributo de calidad, no de la tecnología específica que lo implementa, y se agrupan en cuatro categorías: seguridad y protección de datos, rendimiento y operación, usabilidad y experiencia de usuario, y cumplimiento normativo y mantenibilidad.

Seguridad y Protección de Datos

Tabla 3.38: RNF-01: Aislamiento lógico de la información del despacho

ID	RNF-01
Nombre	Aislamiento lógico de la información del despacho
Descripción	El sistema debe garantizar la segregación estricta de la información del despacho: cada instancia (single-tenant) atiende a un solo despacho con su propia base de datos y material criptográfico, y dentro de la instancia ninguna consulta, operación o error de lógica debe exponer información a usuarios sin autorización sobre el recurso.
Entrada	Consulta o operación de cualquier usuario autenticado sobre cualquier recurso del sistema.
Salida	Respuesta que contiene únicamente datos que el rol y las asignaciones del usuario autenticado le permiten consultar.
Actores involucrados	Todos los usuarios; capa de middleware de autorización; base de datos relacional.
Criterio de aceptación	Dado un usuario autenticado, cuando realiza cualquier consulta (incluso manipulando parámetros e identificadores de recursos), entonces el sistema nunca devuelve datos fuera de su ámbito de autorización; verificable mediante pruebas automatizadas que simulen manipulación de identificadores de recursos ajenos.

Tabla 3.39: RNF-02: Gestión segura de credenciales

ID	RNF-02
Nombre	Gestión segura de credenciales
Descripción	Las contraseñas de los usuarios deben almacenarse mediante funciones de hashing resistentes (Argon2 o bcrypt) con salt (sal) aleatorio por contraseña. El sistema no debe almacenar contraseñas en texto plano ni usar SHA-256/MD5 para este fin.
Entrada	Contraseña en texto plano proporcionada durante el registro o cambio de contraseña.
Salida	Hash de la contraseña almacenado con Argon2 o bcrypt con salt aleatorio; contraseña original descartada inmediatamente.
Actores involucrados	Sistema de autenticación; base de datos de credenciales.
Criterio de aceptación	Dado el acceso directo a la base de datos, cuando se inspeccionan las credenciales almacenadas, entonces ninguna contraseña aparece en texto plano ni como hash SHA-256 o MD5; el tiempo de verificación debe superar los 100 ms para dificultar ataques de fuerza bruta.

Tabla 3.40: RNF-03: Gestión de material criptográfico

ID	RNF-03
Nombre	Gestión de material criptográfico
Descripción	El sistema debe garantizar que las claves de cifrado se almacenen de forma segura y separada de los datos cifrados, que los IV para AES-GCM sean únicos por operación (véase §3.11.4) y que las claves privadas nunca se transmitan por la red ni se almacenen en texto plano.
Entrada	Operación de cifrado o descifrado; solicitud de uso de clave privada por el firmante.
Salida	Operación ejecutada sin exposición de material criptográfico sensible en logs, respuestas API o almacenamiento temporal.
Actores involucrados	Sistema de cifrado (backend); módulo de gestión de claves; desarrolladores (auditores).
Criterio de aceptación	Dado el análisis de los logs del sistema en cualquier nivel de verbosidad, cuando se buscan patrones de claves o IV en texto plano, entonces no debe encontrarse ninguna clave maestra, clave privada ni IV almacenado junto al contenido cifrado en el mismo registro.

Tabla 3.41: RNF-04: Inmutabilidad de registros críticos

ID	RNF-04
Nombre	Inmutabilidad de registros críticos
Descripción	Los registros de historial procesal, cadena de custodia y bitácora de actividad deben seguir un modelo de solo inserción (append-only), impidiendo la modificación o eliminación de trazas históricas.
Entrada	Intento de operación UPDATE o DELETE sobre tablas de historial procesal, cadena de custodia o bitácora de actividad.
Salida	Operación rechazada con código de error; registro del intento en la bitácora de sistema.
Actores involucrados	Cualquier usuario o proceso del sistema; motor de base de datos NoSQL.
Criterio de aceptación	Dado un registro existente en la bitácora de auditoría, cuando cualquier proceso—incluidos scripts administrativos— intenta modificarlo o eliminarlo, entonces el motor rechaza la operación; verificable mediante pruebas de integración automatizadas.

Tabla 3.42: RNF-05: Privacidad de la información del despacho

ID	RNF-05
Nombre	Privacidad de la información del despacho
Descripción	El sistema debe prohibir cualquier fuga de información hacia usuarios sin autorización o hacia el exterior de la instancia del despacho, verificable mediante pruebas automatizadas de aislamiento.
Entrada	Consultas con parámetros manipulados que incluyan identificadores de recursos fuera del ámbito del usuario.
Salida	Respuesta vacía o error de autorización; nunca datos fuera del ámbito autorizado.
Actores involucrados	Middleware de autorización; capa de acceso a datos; equipo de QA.
Criterio de aceptación	Dado un conjunto de pruebas automatizadas de aislamiento ejecutadas contra el entorno de staging, entonces el 100 % de los intentos de acceso a datos fuera del ámbito de autorización deben resultar en error de autorización.

Rendimiento y Operación

Tabla 3.43: RNF-06: Procesamiento asíncrono

ID	RNF-06
Nombre	Procesamiento asíncrono
Descripción	Las tareas intensivas (envío de notificaciones, generación de reportes, operaciones criptográficas de larga duración) deben procesarse en segundo plano mediante colas de trabajo, sin bloquear la interfaz de usuario.
Entrada	Solicitud de tarea intensiva por parte del usuario.
Salida	Confirmación inmediata de recepción; entrega del resultado mediante notificación cuando el procesamiento concluye.
Actores involucrados	Usuario solicitante; sistema de colas de trabajo (backend); módulo de notificaciones.
Criterio de aceptación	Dado un usuario que solicita la generación de un reporte, cuando el procesamiento supera 2 segundos, entonces la interfaz permanece responsiva con indicador de progreso; el usuario no necesita mantener la pantalla activa para recibir el resultado.

Tabla 3.44: RNF-07: Tiempo de respuesta de interfaz

ID	RNF-07
Nombre	Tiempo de respuesta de interfaz
Descripción	Las operaciones interactivas del sistema deben ofrecer tiempos de respuesta perceptiblemente inmediatos, con actualizaciones reactivas que no requieran recargas completas de página.
Entrada	Interacción del usuario con cualquier elemento de la interfaz (clic, búsqueda, navegación).
Salida	Actualización reactiva de la interfaz sin recarga completa de página.
Actores involucrados	Usuario final; frontend del sistema; API REST del backend.
Criterio de aceptación	Dado un usuario en condiciones de red estándar (>10 Mbps), cuando realiza cualquier operación interactiva que no involucre procesamiento asíncrono, entonces la interfaz responde con retroalimentación visual en menos de 300 ms.

Usabilidad y Experiencia de Usuario

Tabla 3.45: RNF-08: Diseño responsivo

ID	RNF-08
Nombre	Diseño responsivo
Descripción	La interfaz debe adaptarse fluidamente a dispositivos de escritorio y tabletas, respetando los lineamientos de legibilidad y jerarquía visual del sistema de diseño.
Entrada	Acceso al sistema desde dispositivos con resoluciones entre 768 px (tablet) y 1920 px (escritorio).
Salida	Interfaz adaptada correctamente al ancho de pantalla del dispositivo sin desbordamientos ni pérdida de funcionalidad.
Actores involucrados	Litigante, Paralegal, Owner; sistema de diseño (componentes UI).
Criterio de aceptación	Dado un dispositivo tablet con resolución de 768 px de ancho, cuando el usuario accede a cualquier módulo, entonces todos los elementos interactivos son accesibles y legibles sin requerir desplazamiento horizontal.

Tabla 3.46: RNF-09: Soporte de localización

ID	RNF-09
Nombre	Soporte de localización
Descripción	La plataforma debe manejar formatos de fecha, moneda y textos en español de México (es-MX) como configuración predeterminada.
Entrada	Configuración regional del sistema establecida como es-MX.
Salida	Fechas en formato DD/MM/AAAA, montos en MXN con separador de miles y dos decimales, textos de interfaz en español de México.
Actores involucrados	Sistema de internacionalización (i18n); todos los usuarios.
Criterio de aceptación	Dado cualquier módulo del sistema, cuando se muestran fechas o montos, entonces el formato corresponde a es-MX sin que el usuario deba configurar manualmente su preferencia regional.

Cumplimiento Normativo y Mantenibilidad

Tabla 3.47: RNF-10: Conformidad con el CNPP

ID	RNF-10
Nombre	Conformidad con el CNPP
Descripción	El modelado de datos y los flujos de trabajo deben adherirse estrictamente a la terminología y plazos del Código Nacional de Procedimientos Penales.
Entrada	Flujo de trabajo relacionado con etapas procesales, plazos o tipos de audiencia.
Salida	Terminología, plazos y secuencias de estados alineados al CNPP vigente.
Actores involucrados	Litigante; equipo legal del despacho (validador); desarrolladores.
Criterio de aceptación	Dado el catálogo de tipos de audiencia implementado, cuando se compara contra los artículos 307-403 del CNPP, entonces cada tipo debe tener correspondencia exacta con el artículo aplicable y activar los plazos correctos.

Tabla 3.48: RNF-11: Conformidad con la NOM-151

ID	RNF-11
Nombre	Conformidad con la NOM-151
Descripción	Los mecanismos de sellado de tiempo y conservación de mensajes de datos deben cumplir con los requisitos de la NOM-151-SCFI-2016 y el protocolo RFC 3161.
Entrada	Solicitud de sellado de tiempo de un contrato firmado.
Salida	Token RFC 3161 válido y verificable de forma independiente; la constancia de un PSC autorizado es una condición de despliegue regulado, no de la TSA local de demostración.
Actores involucrados	Sistema backend; TSA configurada; equipo legal.
Criterio de aceptación	Dado un token emitido por el sistema, cuando se verifica con OpenSSL usando el ancla de confianza correspondiente, la verificación resulta exitosa. Si el emisor es local, el resultado se reporta como evidencia técnica sin atribuir una fuente externa sincronizada con CENAM.

Tabla 3.49: RNF-12: Cumplimiento de la LFPDPPP

ID	RNF-12
Nombre	Cumplimiento de la LFPDPPP
Descripción	El tratamiento de datos personales debe cumplir con las obligaciones de la LFPDPPP, particularmente las medidas de seguridad exigidas por su artículo 19.
Entrada	Dato personal de un cliente del despacho ingresado al sistema.
Salida	Dato almacenado cifrado, accesible únicamente por roles autorizados, con registro de acceso en la bitácora.
Actores involucrados	Todos los usuarios; sistema de cifrado; Oficial de Privacidad del despacho.
Criterio de aceptación	Dado el conjunto de medidas técnicas implementadas, cuando se evalúan contra el artículo 19 de la LFPDPPP, entonces el sistema debe evidenciar: cifrado AES-256-GCM (confidencialidad), hash SHA-256 (integridad), RBAC (control de acceso) y bitácora append-only (trazabilidad).

Tabla 3.50: RNF-13: Arquitectura mantenible

ID	RNF-13
Nombre	Arquitectura mantenible
Descripción	El código debe seguir principios de diseño que faciliten la escalabilidad y el mantenimiento futuro, incluyendo separación de responsabilidades, bajo acoplamiento y alta cohesión.
Entrada	Solicitud de cambio o nueva funcionalidad que afecte un módulo existente.
Salida	Módulo modificado sin introducir regresiones; pruebas existentes continúan pasando.
Actores involucrados	Equipo de desarrollo; sistema de integración continua.
Criterio de aceptación	Dado el código fuente del sistema, cuando se analiza con métricas de código, entonces la complejidad ciclomática promedio por función no debe superar 10 y la cobertura de pruebas de los módulos criptográficos debe ser igual o mayor al 90 %.

Tabla 3.51: RNF-14: Aseguramiento de calidad

ID	RNF-14
Nombre	Aseguramiento de calidad
Descripción	El sistema debe someterse a pruebas de aceptación de usuario (UAT) al finalizar cada ciclo de desarrollo, complementadas con pruebas funcionales automatizadas y pruebas específicas de los servicios criptográficos.
Entrada	Entregable de cada iteración XP; matriz de pruebas definida en la Fase I.
Salida	Reporte de resultados de pruebas funcionales, de integración y criptográficas; lista de defectos identificados y su estado de resolución.
Actores involucrados	Equipo de desarrollo; usuarios representativos del despacho (UAT); directora del proyecto.
Criterio de aceptación	Dado el cierre de cada iteración, cuando se ejecuta la suite de pruebas automatizadas, entonces el 100 % de las pruebas de módulos criptográficos (hash, cifrado, firma, sello de tiempo) deben pasar sin errores antes de considerar la iteración completada.

3.5 Análisis de Costos

El presente análisis estima el costo total del proyecto en su fase de prototipo, considerando todos los recursos necesarios para el desarrollo, la infraestructura y los servicios externos requeridos. La estimación se basa en el costo de oportunidad del equipo de desarrollo y en la identificación exhaustiva de cada rubro de gasto, independientemente de que su costo sea cero en la fase actual.

3.5.1 Metodología de Estimación

Se utiliza un enfoque de costo de oportunidad para valorar el recurso humano: el salario de mercado que cada miembro del equipo percibiría como Desarrollador Full-Stack Junior/Mid en la Zona Metropolitana del Valle de México, convertido a tarifa horaria. Los rangos salariales se triangularon a partir de seis fuentes de datos actualizadas al periodo 2025–2026.

Tabla 3.52: Fuentes salariales consultadas para la estimación del costo de recurso humano.

Fuente	Rango Mensual (MXN)	Fecha de Datos
Glassdoor MX (Jr Full-Stack)	\$11,854 – \$20,114	Marzo 2026
Jobted MX (Jr Full-Stack)	\$11,650 promedio Jr	2026
Indeed MX (Programador Jr)	\$12,784 promedio	Febrero 2026
Coderhouse (Jr Full-Stack)	\$22,000 – \$28,000	2025
Talently (Jr Full-Stack MX)	≈\$29,000 (≈\$1,436 USD)	Mayo 2024
Data México (ENOE Q1-2025)	\$8,410 promedio nacional*	Q1 2025

* Data México reporta el promedio nacional incluyendo todas las seniorities y regiones, lo cual sesga el dato hacia abajo. Se utiliza como límite inferior de referencia.

A partir de estas fuentes, se definen tres escenarios salariales. El cálculo asume una jornada mensual estándar de 160 horas (40 hrs/semana × 4 semanas) bajo esquema de honorarios/freelance, lo cual incorpora el sobre costo por ausencia de prestaciones de ley (≈30–35 % sobre nómina tradicional).

Tabla 3.53: Escenarios salariales y tarifa horaria derivada.

Escenario	Salario Mensual Equivalente	Tarifa / Hora	Justificación
Optimista	\$18,000 MXN	\$112.50 MXN	Piso del rango Jr según Indeed/Glassdoor; escenario donde el riesgo tiene bajo costo.
Más Probable	\$25,000 MXN	\$156.25 MXN	Mediana del rango Jr-Mid; consistente con Coderhouse y Glassdoor p50.
Pesimista	\$32,000 MXN	\$200.00 MXN	Techo del rango Mid; incluye prima por conocimiento criptográfico especializado.

Valor adoptado para los cálculos del proyecto: **\$156.25 MXN/hora** (escenario Más Probable). Los escenarios Optimista y Pesimista se reservan para el análisis de sensibilidad en la sección de riesgos.

3.5.2 Capacidad del Equipo

La capacidad efectiva de programación se reduce respecto a una jornada laboral típica por factores inherentes al contexto del proyecto. Los integrantes del equipo son estudiantes de último semestre con carga académica concurrente, y la metodología XP impone overhead por prácticas como Pair Programming, TDD y ceremonias iterativas.

Tabla 3.54: Factores de ajuste sobre la capacidad de desarrollo.

Factor	Reducción Estimada	Razón
Carga académica concurrente	−30 %	Otras materias, exámenes y prácticas reducen la disponibilidad a 30 hrs/semana por desarrollador.
Pair Programming (XP)	Factor $\times 2$ en costo	Ambos desarrolladores trabajan simultáneamente en la misma tarea; se contabiliza el esfuerzo de ambos.
TDD (Test-Driven Development)	−15 %	Overhead por escritura de pruebas antes de código funcional; compensado por menor deuda técnica.
Ceremonias XP (standups, planning, retros)	−5 %	Iteraciones cortas (1–2 semanas) requieren reuniones frecuentes de coordinación.

Partiendo de una disponibilidad base de 30 horas semanales por desarrollador (jornada reducida por carga académica), y aplicando el overhead combinado de TDD y ceremonias XP (−20 %), se obtienen 24 horas efectivas de programación por desarrollador por semana. Con un equipo de 2 desarrolladores y 5 semanas de codificación intensiva (Fase II del plan de iteraciones), la capacidad total es de 240 horas-persona.

Tabla 3.55: Resumen de capacidad del equipo y presupuesto base.

Métrica	Valor
Horas efectivas por desarrollador / semana	24 horas
Horas efectivas del equipo / semana	48 horas-persona
Semanas de codificación (Fase II)	5 semanas
Capacidad total	240 horas-persona
Presupuesto base (recurso humano)	\$37,500.00 MXN

Nota: En sesiones de Pair Programming, ambos desarrolladores trabajan en la misma tarea. Esto se refleja contabilizando las horas de ambos (48 hrs-equipo/semana), pero la producción de funcionalidades corresponde a una línea de trabajo, no dos paralelas.

3.5.3 Desglose de Costos por Rubro

La siguiente tabla presenta el desglose completo de costos del proyecto en su fase de prototipo. Los rubros con costo cero se documentan explícitamente para transparentar que fueron evaluados y que su gratuidad corresponde a decisiones técnicas deliberadas, no a omisiones en la estimación.

Tabla 3.56: Desglose de costos del proyecto en fase de prototipo.

Rubro	Proveedor / Herramienta	Costo en Fase Prototipo	Justificación
Recurso humano (2 desarrolladores)	Costo de oportunidad	\$37,500.00 MXN	240 hrs-persona × \$156.25/hr. Escenario Más Probable.
Sellado de tiempo externo (PSC)	Cinzel PSC – Sandbox	No contratado	La API no ofreció garantía operativa suficiente y el sandbox requiere pago con precios no transparentes. La evidencia de esta entrega utiliza la TSA local; no se presupone un costo externo.
Infraestructura de servidor	VPS preexistente (aportación del equipo)	\$0.00 MXN	Infraestructura personal aportada por el equipo. No genera costo incremental para el proyecto.
Correo transaccional	Resend – Plan gratuito	\$0.00 MXN	Plan gratuito suficiente para el volumen de notificaciones del prototipo.
Framework y herramientas de desarrollo	Rust, Axum, Astro, Svelte, Git (open source)	\$0.00 MXN	Stack completamente open source. Sin licencias comerciales requeridas.
Base de datos	MySQL + almacenamiento NoSQL (open source)	\$0.00 MXN	Motores de base de datos open source desplegados en el VPS del equipo.
Certificados SSL/TLS	Let's Encrypt	\$0.00 MXN	Autoridad de certificación gratuita y automatizada.
Reserva de contingencia	Derivada del análisis de riesgos (sección 3.6)	\$45,578.13 MXN	VME total de línea base. Representa el costo esperado ponderado antes de actualizar el estado de materialización.

3.5.4 Resumen Financiero

Tabla 3.57: Resumen financiero del proyecto.

Concepto	Monto
Costo base del proyecto (recurso humano)	\$37,500.00 MXN
Costos conocidos de infraestructura y servicios	\$0.00 MXN
Sandbox del PSC	No incorporado
Costo total conocido sin contingencia	\$37,500.00 MXN
Reserva de contingencia (VME total de línea base)	\$45,578.13 MXN
Presupuesto conocido, sin sandbox del PSC	\$83,078.13 MXN

La reserva de contingencia de línea base representa el 121.5 % del costo base, lo cual refleja el alto riesgo técnico inherente a la integración de servicios criptográficos con tolerancia cero a fallos en un cronograma de 5 semanas de codificación. Este porcentaje es consistente con las recomendaciones del PMI para proyectos con incertidumbre técnica elevada. El presupuesto es una cota inferior: el costo del sandbox del PSC permanece separado hasta definir el tamaño de la campaña y contar con una tarifa verificable.

Es importante señalar que los costos conocidos de infraestructura y servicios son cero exclusivamente en la fase de prototipo y no incluyen el sandbox de pago del PSC. Un despliegue en producción requeriría contratar un servicio de sellado con volumen y estabilidad verificables, infraestructura cloud dedicada con respaldos, y licencias de dominio y certificados de producción, rubros que quedan fuera del alcance de la presente estimación.

3.6 Análisis de Riesgos

El presente análisis identifica, evalúa y prioriza los riesgos que pueden afectar el desarrollo del prototipo, considerando tanto su probabilidad de ocurrencia como su impacto en tiempo y costo. La evaluación combina un enfoque cualitativo (matriz de severidad) con uno cuantitativo (Valor Monetario Esperado) para proporcionar una base objetiva en la toma de decisiones sobre mitigación y asignación de recursos.

3.6.1 Modelo de Evaluación

El impacto financiero de cada riesgo materializado se calcula mediante la fórmula: $IF = H(\text{retraso}) \times N(\text{devs}) \times T(\text{hora})$, donde $H(\text{retraso})$ es el número de horas de retraso estimadas, $N(\text{devs})$ es el número de desarrolladores afectados (1 o 2) y $T(\text{hora})$ es la tarifa horaria del escenario Más Probable (\$156.25 MXN). El Valor Monetario Esperado (VME) pondera el impacto financiero por la probabilidad de ocurrencia: $VME = P \times IF$.

Tabla 3.58: Escala de probabilidad de ocurrencia.

Nivel	Valor	Rango	Criterio Orientativo
Bajo	1	< 20 %	El riesgo es poco probable; el equipo tiene experiencia previa en el área o existen mitigaciones naturales.
Medio	2	20 – 60 %	El riesgo tiene posibilidades razonables de materializarse; depende de factores parcialmente controlables.
Alto	3	> 60 %	El riesgo es altamente probable; involucra tecnología no dominada, dependencias externas críticas o restricciones de tiempo severas.

Tabla 3.59: Escala de impacto en tiempo.

Nivel	Valor	Retraso	Criterio Orientativo
Bajo	1	1 – 2 días (≤ 16 hrs)	Impacto menor; se puede absorber dentro de la iteración XP sin reprogramar.
Medio	2	3 – 5 días (17–40 hrs)	Impacto significativo; requiere repriorizar historias de usuario y posiblemente reducir alcance de la iteración.
Alto	3	> 5 días (> 40 hrs)	Impacto crítico; compromete el hito de entrega de la iteración o del proyecto completo; requiere escalamiento.

La multiplicación de ambas escalas produce un índice de severidad que permite priorizar la atención a riesgos:

Tabla 3.60: Matriz de severidad (Probabilidad $\times$ Impacto).

Prob. \ Impacto	Bajo (1)	Medio (2)	Alto (3)
Bajo (1)	1 – Trivial	2 – Menor	3 – Moderado
Medio (2)	2 – Menor	4 – Significativo	6 – Grave
Alto (3)	3 – Moderado	6 – Grave	9 – Crítico

Clasificación por rango de severidad: 1–2 (Verde): Aceptar y monitorear sin acción inmediata. 3–4 (Amarillo): Mitigar, definir plan de contingencia y asignar responsable. 6 (Naranja): Priorizar, implementar mitigación activa y considerar reducción de alcance. 9 (Rojo): Escalar, riesgo crítico que amenaza la viabilidad del proyecto y requiere acción inmediata.

3.6.2 Riesgos Técnicos y Criptográficos

Tabla 3.61: Ficha del riesgo RTC-01: Inestabilidad en la integración con el PSC (RFC 3161).

ID	RTC-01
Nombre	Inestabilidad en la integración con el PSC (RFC 3161)
Cat.	Técnico
Sev.	6 – Grave
Declaración	<i>Existe el riesgo de que la integración con la API del PSC (Cincel) no sea suficientemente estable durante el flujo de sellado de tiempo, debido a respuestas externas no deterministas, disponibilidad fuera del control del equipo y acceso de pago al sandbox, lo que podría resultar en pruebas no reproducibles, costo variable de reintentos y bloqueo temporal de la emisión de constancias externas.</i>
Mitigación	Validar el contrato remoto mediante un Spike, aislarlo detrás del puerto TimestampService, cubrir sus estados con un stub HTTP y mantener una TSA RFC 3161 local para pruebas y demostraciones deterministas.
Impacto	2 – Medio (30 h)
VME	\$6,562.50 MXN
Estado al corte	Materializado. El registro se completó, pero la API observada no ofreció estabilidad suficiente para una campaña repetible y el sandbox requiere pago.
Contingencia	Emitir tokens RFC 3161 con una TSA local cuyo certificado deriva de la CA interna. Esta vía conserva el flujo técnico, pero se etiqueta como evidencia interna y no sustituye la confianza independiente de un PSC autorizado.

Tabla 3.62: Ficha del riesgo RTC-02: Implementación insegura de AES-256-GCM.

ID	RTC-02
Nombre	Implementación insegura de AES-256-GCM
Cat.	Técnico
Sev.	6 – Grave
Declaración	<i>Existe el riesgo de que el esquema de cifrado autenticado quede criptográficamente comprometido, debido a una generación predecible o reutilización del Vector de Inicialización (IV) en AES-256-GCM, lo que podría resultar en la pérdida simultánea de confidencialidad e integridad de todos los expedientes cifrados almacenados en el sistema.</i>
Mitigación	Aplicar Pair Programming obligatorio en este módulo. Desarrollar pruebas unitarias TDD que validen la aleatoriedad y no reutilización del IV mediante random_bytes.
Impacto	3 – Alto (45 h)
VME	\$5,625.00 MXN
Contingencia	Aislar y revertir el código vulnerado, purgar la base de datos de pruebas, rotar las llaves maestras de cifrado y refactorizar la lógica criptográfica.

Tabla 3.63: Ficha del riesgo RTC-03: Mala configuración de infraestructura.

ID	RTC-03
Nombre	Mala configuración de infraestructura
Cat.	Técnico
Sev.	2 – Menor
Declaración	<i>Existe el riesgo de que el módulo de almacenamiento de objetos quede inaccesible o con permisos incorrectos, debido a una configuración errónea de políticas de acceso o del servicio de almacenamiento en el servidor de desarrollo, lo que podría resultar en el bloqueo de las operaciones de carga y descarga de documentos, impidiendo avanzar con las iteraciones de gestión documental.</i>
Mitigación	Diseñar la configuración de infraestructura con políticas de mínimo privilegio desde la Iteración 4 y documentarla en el repositorio compartido.
Impacto	1 – Bajo (15 h)
VME	\$703.13 MXN
Contingencia	Un desarrollador asume la resolución de infraestructura mientras el otro migra temporalmente el almacenamiento al disco local del servidor para no detener el avance.

Tabla 3.64: Ficha del riesgo RTC-04: Pérdida de clave maestra de cifrado.

ID	RTC-04
Nombre	Pérdida de clave maestra de cifrado
Cat.	Técnico
Sev.	3 – Moderado
Declaración	<i>Existe el riesgo de que los expedientes cifrados se vuelvan irrecuperables de forma permanente, debido a la ausencia de un procedimiento formal de resguardo, rotación y recuperación de la clave maestra AES del despacho, lo que podría resultar en la pérdida total e irreversible de la información cifrada de todos los clientes del despacho, sin posibilidad de restauración.</i>
Mitigación	Definir desde la Iteración 4 el procedimiento de resguardo de claves con almacenamiento separado de los datos cifrados (ej. variable de entorno o servicio de gestión de secretos).
Impacto	3 – Alto (>40 h)
VME	\$7,031.25 MXN
Contingencia	Restaurar desde la copia de seguridad del almacén de claves; si no existe respaldo, declarar incidente crítico de pérdida de datos y notificar a los interesados.

Tabla 3.65: Ficha del riesgo RTC-05: Expiración de certificado X.509 en producción.

ID	RTC-05
Nombre	Expiración de certificado X.509 en producción
Cat.	Técnico
Sev.	4 – Significativo
Declaración	<i>Existe el riesgo de que el flujo de firma digital quede inoperante de forma inesperada en producción, debido a la falta de monitoreo automatizado de la fecha de vencimiento del certificado X.509 utilizado por el despacho, lo que podría resultar en la interrupción del servicio de no repudio y el potencial incumplimiento de compromisos contractuales con clientes activos del despacho.</i>
Mitigación	Implementar en el módulo de administración una alerta preventiva configurable (ej. 30 y 7 días antes del vencimiento) vinculada al atributo de expiración del certificado X.509.
Impacto	2 – Medio (20 h)
VME	\$3,125.00 MXN
Contingencia	Emitir un certificado de reemplazo de forma urgente ante el PSC correspondiente; notificar a los firmantes afectados y reprogramar las operaciones de firma pendientes.

3.6.3 Riesgos Legales y de Cumplimiento

Tabla 3.66: Ficha del riesgo RLC-01: Discrepancia de hashes SHA-256 (NOM-151).

ID	RLC-01
Nombre	Discrepancia de hashes SHA-256 (NOM-151)
Cat.	Legal
Sev.	6 – Grave
Declaración	<i>Existe el riesgo de que las constancias de conservación emitidas por el PSC sean declaradas inválidas ante una instancia judicial, debido a una discrepancia entre el hash SHA-256 calculado por el sistema y el hash recibido por la TSA, originada por inconsistencias en la serialización del documento previo al sellado, lo que podría resultar en la pérdida del valor probatorio de todos los contratos firmados y el incumplimiento formal de la NOM-151-SCFI-2016, exponiendo al despacho a impugnaciones.</i>
Mitigación	Estudiar exhaustivamente el anexo técnico de la NOM-151. Auditar los digests generados en Rust cruzando resultados con herramientas certificadas (OpenSSL CLI) en cada Spike.
Impacto	3 – Alto (48 h)
VME	\$7,500.00 MXN
Contingencia	Detener el flujo de firmas, validar técnicamente la estructura del hash y refactorizar la lógica de serialización antes de cualquier nuevo envío al PSC.

Tabla 3.67: Ficha del riesgo RLC-02: Mutabilidad en registros de auditoría (CNPP).

ID	RLC-02
Nombre	Mutabilidad en registros de auditoría (CNPP)
Cat.	Legal
Sev.	2 – Menor
Declaración	<i>Existe el riesgo de que la cadena de custodia digital pierda su valor probatorio ante un tribunal revisor, debido a la ausencia de controles de inmutabilidad en la capa de persistencia que permitan operaciones UPDATE o DELETE sobre los registros de auditoría, lo que podría resultar en la ruptura de la cadena de custodia conforme a los artículos 227 y 228 del CNPP, invalidando la evidencia digital presentada en juicio.</i>
Mitigación	Diseñar la capa de persistencia y el modelo NoSQL para prohibir operaciones UPDATE o DELETE en las bitácoras, forzando un esquema estrictamente append-only desde el inicio.
Impacto	2 – Medio (24 h)
VME	\$1,125.00 MXN
Contingencia	Restaurar la base de datos desde respaldo previo a la mutación, auditar los registros afectados y aplicar un parche urgente a nivel de base de datos bloqueando la mutabilidad.

Tabla 3.68: Ficha del riesgo RLC-03: Fuga de datos personales por fallas de autorización (LFPDPPP).

ID	RLC-03
Nombre	Fuga de datos personales por fallas de autorización (LFPDPPP)
Cat.	Legal
Sev.	3 – Moderado
Declaración	<i>Existe el riesgo de que datos confidenciales del despacho sean accesibles por usuarios sin autorización, debido a una falla en el middleware de autorización que permita consultas sin restricción de rol o de ámbito del usuario, lo que podría resultar en una brecha de datos personales que viola el artículo 19 de la LFPDPPP, con exposición a sanciones administrativas y daño reputacional irreversible para el despacho.</i>
Mitigación	Inyectar restricción obligatoria de autorización (rol y ámbito del usuario) en toda consulta relacional y verificar el aislamiento mediante pruebas automatizadas en cada iteración.
Impacto	3 – Alto (50 h)
VME	\$1,562.50 MXN
Contingencia	Activar modo de mantenimiento, purgar registros expuestos, rotar credenciales comprometidas y refactorizar los middlewares de aislamiento antes de restaurar el servicio.

Tabla 3.69: Ficha del riesgo RLC-04: Indisponibilidad o degradación del PSC (Cincel).

ID	RLC-04
Nombre	Indisponibilidad o degradación del PSC (Cincel)
Cat.	Legal
Sev.	4 – Significativo
Declaración	<i>Existe el riesgo de que el módulo de sellado externo quede indisponible o opere de forma degradada durante ventanas críticas de firma, debido a interrupciones o inestabilidad en la infraestructura del Prestador de Servicios de Certificación seleccionado, lo que podría resultar en retrasos en la obtención de la constancia NOM-151 y afectación al flujo operativo.</i>
Mitigación	Mantener una interfaz abstracta que permita configurar otro PSC sin modificar la lógica central y separar las pruebas deterministas de la disponibilidad del proveedor comercial.
Impacto	2 – Medio (20 h)
VME	\$3,125.00 MXN
Estado al corte	Materializado parcialmente como degradación e inestabilidad de la API, no como una interrupción total del registro o del servicio.
Contingencia	Encolar las solicitudes externas y reintentar con límites explícitos; para continuidad técnica usar la TSA interna, sin conmutación silenciosa y sin presentar sus tokens como constancias emitidas por un tercero.

3.6.4 Riesgos Operativos y de Metodología XP

Tabla 3.70: Ficha del riesgo ROP-01: Curva de aprendizaje TDD en módulos criptográficos.

ID	ROP-01
Nombre	Curva de aprendizaje TDD en módulos criptográficos
Cat.	Operativo
Sev.	4 – Significativo
Declaración	<i>Existe el riesgo de que el desarrollo de los módulos criptográficos supere el tiempo planificado de forma significativa, debido a la dificultad adicional de formular pruebas unitarias para operaciones criptográficas cuyo comportamiento correcto no siempre es intuitivo (verificación de IV, validación de cadenas de certificados), lo que podría resultar en el consumo del margen de contingencia del cronograma de la Fase II, poniendo en riesgo la entrega del módulo de no repudio en las Iteraciones 7 y 8.</i>
Mitigación	Aplicar TDD estricto solo en los flujos criptográficos core (hash, cifrado AES, validación X.509) y relajarlo para CRUDs básicos que no comprometan la seguridad.
Impacto	2 – Medio (35 h)
VME	\$5,468.75 MXN
Contingencia	Transicionar a pruebas de integración (test-after) en componentes secundarios si un test unitario excede 4 horas de bloqueo productivo sin resolución.

Tabla 3.71: Ficha del riesgo ROP-02: Scope creep en interfaces gráficas.

ID	ROP-02
Nombre	Scope creep en interfaces gráficas
Cat.	Operativo
Sev.	4 – Significativo
Declaración	<i>Existe el riesgo de que el esfuerzo dedicado a interfaces gráficas consuma tiempo crítico de las semanas 9 y 10, debido a la tendencia a invertir iteraciones en refinamiento visual más allá de lo planificado, en detrimento de las historias de gestión procesal y documental que comparten la Iteración 9, lo que podría resultar en el desplazamiento de dichas historias y de las pruebas finales de la Iteración 10 hacia fuera del cronograma, comprometiendo el objetivo de gestión procesal del prototipo.</i>
Mitigación	Congelar las historias de usuario visuales y utilizar bibliotecas de componentes preconstruidas para la interfaz, prohibiendo el diseño a medida en la Fase II. La mitigación se refuerza estructuralmente porque el desarrollo de interfaz se traslada a la Iteración 9, posterior al hito criptográfico de la Iteración 8; el riesgo residual se concentra en el objetivo de gestión procesal (OE-2) por la compresión de esas historias en una sola iteración.
Impacto	2 – Medio (20 h)
VME	\$3,125.00 MXN
Contingencia	Recortar características no críticas (RF-19 Dashboard Ejecutivo o reportes visuales) para canalizar el esfuerzo íntegramente a las historias de gestión procesal y documental de la Iteración 9.

Tabla 3.72: Ficha del riesgo ROP-03: Ruptura de cadencia de Pair Programming.

ID	ROP-03
Nombre	Ruptura de cadencia de Pair Programming
Cat.	Operativo
Sev.	2 – Menor
Declaración	<i>Existe el riesgo de que la práctica de Pair Programming quede interrumpida por ausencia temporal de un desarrollador, debido a indisposición o compromisos académicos imprevistos de uno de los integrantes del equipo durante la Fase II de codificación, lo que podría resultar en la generación de silos de conocimiento en módulos críticos de seguridad y la pérdida de velocidad de desarrollo durante el periodo de ausencia.</i>
Mitigación	Mantener política de propiedad colectiva del código e integración continua diaria para garantizar que ambos desarrolladores conozcan todos los módulos.
Impacto	1 – Bajo (16 h)
VME	\$625.00 MXN
Contingencia	El desarrollador activo pivota a tareas de menor riesgo técnico (maquetación, diccionarios de datos) hasta que la célula de pares se restablezca.

3.6.5 Resumen Cuantitativo de Riesgos

Tabla 3.73: Resumen cuantitativo de los riesgos identificados con su severidad y VME.

ID	Nombre	Prob.	Impacto	Severidad	VME (MXN)
RTC-01	Inestabilidad en integración con PSC (RFC 3161)	3 Alto (70 %)	2 Medio (30 h)	6 – Grave	\$6,562.50 MXN
RTC-02	Implementación insegura de AES-256-GCM	2 Medio (40 %)	3 Alto (45 h)	6 – Grave	\$5,625.00 MXN
RTC-03	Mala configuración de infraestructura	2 Medio (30 %)	1 Bajo (15 h)	2 – Menor	\$703.13 MXN
RTC-04	Pérdida de clave maestra de cifrado	1 Bajo (15 %)	3 Alto (>40 h)	3 – Moderado	\$7,031.25 MXN
RTC-05	Expiración de certificado X.509 en producción	2 Medio (40 %)	2 Medio (20 h)	4 – Significativo	\$3,125.00 MXN
RLC-01	Discrepancia de hashes SHA-256 (NOM-151)	2 Medio (50 %)	3 Alto (48 h)	6 – Grave	\$7,500.00 MXN
RLC-02	Mutabilidad en registros de auditoría (CNPP)	1 Bajo (15 %)	2 Medio (24 h)	2 – Menor	\$1,125.00 MXN
RLC-03	Fuga de datos personales por fallas de autorización (LFPDPPP)	1 Bajo (10 %)	3 Alto (50 h)	3 – Moderado	\$1,562.50 MXN
RLC-04	Indisponibilidad o degradación del PSC (Cincel)	2 Medio (30 %)	2 Medio (20 h)	4 – Significativo	\$3,125.00 MXN
ROP-01	Curva de aprendizaje TDD en módulos criptográficos	2 Medio (50 %)	2 Medio (35 h)	4 – Significativo	\$5,468.75 MXN
ROP-02	Scope creep en interfaces gráficas	2 Medio (50 %)	2 Medio (20 h)	4 – Significativo	\$3,125.00 MXN
ROP-03	Ruptura de cadencia de Pair Programming	2 Medio (25 %)	1 Bajo (16 h)	2 – Menor	\$625.00 MXN

VME total del proyecto: \$45,578.13 MXN. Si llegaran a materializarse simultáneamente los riesgos de mayor severidad (RTC-01, RLC-01 y ROP-01), el retraso combinado sería de 113 horas de reloj, equivalente a 2.35 semanas ($113 \div 48$ horas-equipos/semana), lo que consumiría el 47 % del cronograma de codificación de 5 semanas. En términos de presupuesto, el impacto conjunto asciende a 226 horas-persona (113×2 desarrolladores), equivalente al 94 % de la capacidad total disponible.

3.6.6 Actualización del riesgo del proveedor al cierre del núcleo criptográfico

El registro en el entorno de Cincel pudo completarse; por tanto, el problema observado no fue una imposibilidad de alta. El riesgo se materializó durante el consumo del servicio: la API no ofreció respuesta y estabilidad suficientes para convertirla en una dependencia determinista de la suite, de la

integración continua o de la demostración, y el sandbox requiere pago con precios no transparentes. La combinación introduce fallos ajenos al código propio, costo variable al repetir solicitudes y una dependencia operacional que el equipo no controla. En consecuencia, la prueba de humo se conserva como referencia técnica ignorada y la evidencia externa de Cincel queda fuera del alcance de esta entrega.

En términos de gestión, RTC-01 cambia de amenaza a incidencia materializada. Su probabilidad de 70 % y su VME de \$6,562.50 MXN se conservan como valores de la línea base previa para mantener trazabilidad, no como medición del costo ya incurrido. El impacto efectivo permanece sin cuantificar porque no se autoriza una campaña de pago; el riesgo residual se traslada a cualquier despliegue futuro que requiera confianza externa y deberá resolverse mediante un PSC con contrato y disponibilidad verificables.

La respuesta aplicada supera la contingencia de simulación prevista originalmente. La CA interna emite un certificado exclusivo de TSA y `LocalOpenSSLTsa` produce tokens RFC 3161 reales, verificables con `openssl ts -verify`, detrás del mismo puerto que utiliza el adaptador de Cincel. Esto permite reproducir el flujo criptográfico completo sin acoplar las pruebas a la disponibilidad o al costo del proveedor. La procedencia del token debe conservarse explícita: la TSA interna aporta continuidad y evidencia técnica, pero no independencia de tercero ni una constancia NOM-151 emitida por un PSC autorizado. En producción no debe existir una conmutación silenciosa entre ambos emisores; una falla externa debe encolarse o exponerse al usuario, y la selección de otra TSA regulada debe realizarse como sustitución consciente del adaptador y de su ancla de confianza.

Recomendación: El éxito del prototipo radica en su validez como evidencia técnica, no en su amplitud estética ni de reportes. El flujo RFC 3161 local ya fue comprobado de extremo a extremo y permite avanzar hacia la API, la persistencia y la interfaz web sin depender de Cincel. La integración con un PSC externo queda como sustitución futura y consciente del adaptador, no como condición para continuar el desarrollo.

3.6.7 Trazabilidad Riesgo–Objetivo

La siguiente tabla relaciona cada riesgo identificado en las secciones 3.6.2 a 3.6.4 con los objetivos específicos que se verían comprometidos si el riesgo se materializa. Este mapeo permite priorizar las mitigaciones cuya ausencia afectaría de forma más directa la entrega del prototipo.

Tabla 3.74: Trazabilidad entre riesgos identificados y objetivos específicos.

ID	Descripción	Objetivo(s) afectado(s)
RTC-01	Inestabilidad en integración con PSC (RFC 3161)	OE-3
RTC-02	Implementación insegura de AES-256-GCM	OE-4
RTC-03	Mala configuración de infraestructura	OE-1, OE-6
RTC-04	Pérdida de clave maestra de cifrado	OE-4
RTC-05	Expiración de certificado X.509 en producción	OE-3, OE-5
RLC-01	Discrepancia de hashes SHA-256 (NOM-151)	OE-3, OE-4
RLC-02	Mutabilidad en registros de auditoría (CNPP)	OE-2, OE-6
RLC-03	Fuga de datos personales por fallas de autorización (LFPDPPP)	OE-4, OE-5
RLC-04	Indisponibilidad o degradación del PSC (Cincel)	OE-3
ROP-01	Curva de aprendizaje TDD en módulos criptográficos	OE-3, OE-4, OE-5
ROP-02	Scope creep en interfaces gráficas	OE-1, OE-2
ROP-03	Ruptura de cadencia de Pair Programming	Transversal (OE-1 a OE-6)

3.7 Diseño de Arquitectura del Sistema

Esta sección documenta las decisiones arquitectónicas que estructuran el prototipo y articula su correspondencia con los requerimientos funcionales y no funcionales especificados en §3.3 y §3.4. La arquitectura se construye alrededor de los atributos de calidad priorizados en §3.7.1, se materializa en el estilo hexagonal seleccionado en §3.7.3, y se enforce mediante el sistema de tipos de Rust y la organización del workspace de Cargo descrita en §3.7.6. Las vistas adicionales —de contexto, componentes, despliegue y seguridad— se derivan del modelo C4 propuesto por Brown [64] y del modelo 4+1 de Kruchten [65], adaptados al alcance del prototipo. Las decisiones arquitectónicas significativas se registran en formato ADR (Architecture Decision Record) propuesto por Nygard [66] en §3.7.10, y la coherencia interna del capítulo se verifica mediante la matriz de trazabilidad de §3.7.11.

3.7.1 Drivers Arquitectónicos: Atributos de Calidad

La arquitectura del sistema no se selecciona por adherencia a una moda tecnológica sino por su capacidad para satisfacer un conjunto explícito de atributos de calidad derivados de los requerimientos no funcionales del §3.4.2. La norma ISO/IEC 25010:2011 [67] clasifica los atributos de calidad de software en ocho características principales (adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad), de las cuales se priorizan aquellas que tienen mayor impacto sobre el dominio jurídico-criptográfico que aborda el prototipo.

El primer atributo prioritario es la seguridad, condicionada por los requerimientos RNF-01 (aislamiento lógico), RNF-02 (gestión segura de credenciales) y RNF-03 (gestión de material criptográfico). La seguridad no se concibe como una capa añadida al final del diseño sino como una propiedad trans-

versal que la arquitectura debe garantizar desde sus fundamentos. La adopción de Rust como lenguaje del backend, la separación estricta de material criptográfico mediante el patrón envelope encryption (véase §3.11.4), y el aislamiento de los puertos criptográficos mediante traits intercambiables son decisiones arquitectónicas que materializan este atributo.

El segundo atributo prioritario es la integridad y trazabilidad, condicionada por RNF-04 (inmutabilidad de registros críticos) y RNF-11 (conformidad con la NOM-151). Los documentos gestionados por el sistema poseen valor probatorio bajo los Artículos 33 del Código de Comercio y 89-bis del mismo ordenamiento, así como bajo la NOM-151-SCFI-2016. Esto exige que la arquitectura garantice la imposibilidad de modificación retroactiva de la bitácora de auditoría —implementada mediante el encadenamiento SHA-256 descrito en §3.8.1—, así como la conservación criptográficamente verificable de los hashes y sellos de tiempo de cada documento.

El tercer atributo es la evolvabilidad, definida por Ford, Parsons y Kua como la capacidad de un sistema para cambiar guiado [68]. El prototipo se diseña para que la sustitución de cualquier sistema externo —el Prestador de Servicios de Certificación (Cincel por SeguriData o CECOBAN), el motor de base de datos, el proveedor de correo— requiera únicamente la implementación de un nuevo adaptador, sin modificación del dominio. Esta propiedad se garantiza mediante la regla de dependencias de la arquitectura hexagonal, formalizada en §3.7.4.

El cuarto atributo es la testabilidad, condicionada por la metodología Extreme Programming adoptada en §3.1, que prescribe Test-Driven Development como práctica central. Una arquitectura es testeable cuando permite ejercitar la lógica de dominio sin requerir infraestructura real (bases de datos, servicios de red, sistemas externos). La separación explícita del dominio respecto a sus dependencias mediante puertos permite inyectar adaptadores de prueba (mocks o stubs) durante la ejecución de la suite de pruebas, manteniendo el dominio agnóstico respecto a la infraestructura.

El quinto atributo es la mantenibilidad —entendida como bajo acoplamiento y alta cohesión—, y el sexto es la observabilidad, esencial para la operación del sistema en producción. Atributos de menor prioridad, pero considerados son la performance (en tanto el sistema deba responder a consultas interactivas en menos de 200 ms), y la portabilidad (limitada al despliegue en sistemas Linux POSIX-compliant).

La [tabla 3.75](#) formaliza la priorización adoptada y su mapeo a los requerimientos no funcionales que la justifican. Esta priorización guía todas las decisiones arquitectónicas posteriores: ante un trade-off entre dos atributos en tensión, se favorece aquel de mayor prioridad.

Tabla 3.75: Priorización de atributos de calidad y su mapeo a requerimientos no funcionales.

Prioridad	Atributo de calidad	RNFs que lo motivan
1 (más alta)	Seguridad	RNF-01, RNF-02, RNF-03
2	Integridad y trazabilidad	RNF-04, RNF-11
3	Evolvabilidad	Sustitución de PSC, BD, SMTP
4	Testabilidad	XP/TDD (§3.1)
5	Mantenibilidad	Bajo acoplamiento, alta cohesión
6 (más baja)	Observabilidad y performance	Operación productiva

3.7.2 Vista de Contexto del Sistema

La vista de contexto, correspondiente al primer nivel del modelo C4 de Brown [64], representa al sistema como una caja negra que interactúa con actores humanos y sistemas externos. Su propósito es establecer las fronteras del alcance del prototipo y los protocolos de integración antes de descender a los detalles internos.

La Figura 4 presenta los tres tipos de actores humanos contemplados —socio (abogado titular), paralegal (rol operativo) y cliente (firmante de contratos de prestación de servicios)— y los sistemas externos con los que el sistema se integra: el Prestador de Servicios de Certificación Cincel para sellado de tiempo conforme al RFC 3161 y la NOM-151-SCFI-2016, el proveedor de correo electrónico SMTP para notificaciones, y los navegadores web utilizados como cliente de la aplicación. Las flechas etiquetan el protocolo o estándar que regula cada interacción.

Figura 4. Vista de contexto del sistema: actores humanos y sistemas externos con sus protocolos de integración.

El sistema queda explícitamente fuera del alcance respecto a la firma criptográfica realizada por entidades distintas a las partes contractuales (por ejemplo, no se integra con el SAT para emisión de e.firma; las razones se discuten en §3.11.2). Las integraciones con sistemas judiciales públicos (Tribunales, Fiscalías) tampoco están en alcance del prototipo, aunque la arquitectura permite añadirlas en el futuro como adaptadores driving adicionales sin modificar el dominio.

3.7.3 Selección del Estilo Arquitectónico

La selección del estilo arquitectónico es la decisión más estructurante del diseño porque condiciona todas las decisiones subsiguientes sobre organización del código, manejo de dependencias y estrategia de pruebas. Para el presente proyecto se evaluaron seis alternativas mayores, todas documentadas en la literatura canónica de arquitectura de software: arquitectura en capas tradicional (N-tier), Clean Architecture de Martin [69], Onion Architecture de Palermo, arquitectura hexagonal o de puertos y adaptadores de Cockburn [70], microservicios [71] y monolito modular sin patrón hexagonal.

La arquitectura en capas tradicional fue descartada por su acoplamiento estructural del dominio a la persistencia: en este estilo, el dominio depende típicamente del repositorio (que a su vez depende del motor de base de datos), lo que impide aislar la lógica de negocio para pruebas sin levantar infraestructura real. Este acoplamiento contradice directamente los atributos de testabilidad y evolvabilidad priorizados en §3.7.1, y resulta especialmente inadecuado para un proyecto que adopta TDD como práctica central.

Los microservicios fueron descartados por desproporción operativa respecto al alcance. Newman documenta el conjunto de prerequisites operacionales que un sistema basado en microservicios necesita —orquestración, descubrimiento de servicios, gestión de configuración distribuida, observabilidad cross-service, pipelines de despliegue independientes por servicio, infraestructura de comunicación inter-servicios— [71]. Para un prototipo desarrollado por un equipo de dos personas en un periodo académico, esta complejidad es claramente desproporcionada respecto al beneficio. Fowler ha argumentado explícitamente la conveniencia de iniciar con un monolito modular y extraer microservicios solo cuando la escala lo justifique [72].

Las tres arquitecturas restantes —Clean, Onion y Hexagonal— comparten el mismo principio fundamental: la regla de dependencias hacia adentro, que establece que las capas internas (dominio) no deben conocer las capas externas (infraestructura). Difieren en su formulación y en el grado de subdivisión que prescriben. Clean Architecture introduce cuatro capas concéntricas (Entities, Use

Cases, Interface Adapters, Frameworks & Drivers); Onion Architecture introduce variantes de dos a cinco capas según el autor; Hexagonal —en su formulación original de Cockburn— establece únicamente la distinción entre el dominio (interior) y los adaptadores (exterior), conectados por puertos (interfaces de entrada y salida).

Se selecciona la arquitectura hexagonal por tres razones específicas. Primero, su formulación es la más simple: dos categorías (puertos driving y puertos driven) capturan toda la diversidad de adaptadores necesarios sin obligar a clasificaciones más finas que el alcance no amerita. Segundo, la metáfora del hexágono comunica explícitamente que los adaptadores son intercambiables: el dominio queda en el centro y alrededor pueden orbitar tantos adaptadores driving (REST, CLI, WebSocket) y driven (PostgreSQL, Redis, Cincel) como sea necesario. Tercero, la correspondencia con el sistema de tipos de Rust es directa: los puertos se materializan como traits y los adaptadores como structs con bloques impl, y el compilador garantiza mecánicamente la regla de dependencias mediante el grafo de dependencias del workspace de Cargo (véase §3.7.6).

La [tabla 3.76](#) sintetiza la comparativa entre las seis alternativas consideradas, evaluándolas contra los atributos de calidad priorizados en §3.7.1. La decisión final se enuncia formalmente como ADR-01 en §3.7.10.

Tabla 3.76: Comparativa de estilos arquitectónicos contra atributos de calidad priorizados.

Estilo arquitectónico	Testabili- dad	Evolvabili- dad	Adecuación al alcance
N-tier (capas tradicional)	Baja	Baja	Inadecuada (acoplamiento BD)
Clean Architecture	Alta	Alta	Adecuada pero más prescriptiva
Onion Architecture	Alta	Alta	Equivalente a Hexagonal pero menos clara
Hexagonal (Cockburn)	Alta	Alta	Óptima para el alcance
Microservicios	Media	Alta	Desproporcionada operativamente
Monolito modular sin hexagonal	Media	Media	Insuficiente para enforce dependencias

3.7.4 Arquitectura Hexagonal: Definición y Aplicación

La arquitectura hexagonal, propuesta originalmente por Alistair Cockburn en 2005 [70] y refinada por Vernon [73] en el contexto del Domain-Driven Design [74], organiza una aplicación de software en tres capas concéntricas: el dominio en el centro, la aplicación en la capa intermedia, y los adaptadores en la capa exterior.

El dominio contiene los conceptos centrales del problema modelado: entidades (Case, Document, User, Signature, Hearing, AuditEvent, Certificate), value objects (Nuc, ProcessStage, DocumentHash) y servicios de dominio (lógica de negocio que no pertenece naturalmente a una entidad específica). La regla fundamental que gobierna esta capa es que su código no importa absolutamente nada de las capas externas: depende exclusivamente de la biblioteca estándar de Rust y de un conjunto mínimo de crates de bajo nivel (serde para serialización, thiserror para errores tipados, time para manejo de fechas, uuid para identificadores únicos, y zeroize para el borrado garantizado del material secreto en memoria al liberarlo).

La capa de aplicación contiene los casos de uso del sistema: `SignContract`, `RegisterCase`, `ScheduleHearing`, `VerifySignature`, `TransitionCaseStage`, `QueryAuditLog`, entre otros. Cada caso de uso orquesta las entidades del dominio para satisfacer una intención específica del usuario, y declara las dependencias que requiere del exterior mediante traits (los puertos). La aplicación depende exclusivamente del dominio.

Los adaptadores son las implementaciones concretas que conectan el sistema con el mundo exterior. Se clasifican en dos categorías: los puertos driving (también llamados `primary` o `inbound`) son aquellos por los que el sistema recibe peticiones —el adaptador REST construido con Axum, los handlers de WebSocket para notificaciones en tiempo real, y opcionalmente un adaptador de línea de comandos para tareas administrativas y procesos batch—; los puertos driven (también llamados `secondary` u `outbound`) son aquellos a través de los cuales el sistema invoca servicios externos —repositorios sobre PostgreSQL vía `sqlx`, almacén de sesiones sobre Redis vía `fred`, servicios criptográficos sobre la biblioteca `ring`, cliente RFC 3161 hacia Cincel vía `reqwest`, y dispatcher SMTP vía `lettre`.

La regla de dependencias se enuncia formalmente así: ningún módulo de una capa interior puede importar símbolos de una capa exterior. El dominio no importa de aplicación ni de adaptadores; la aplicación no importa de adaptadores. Esta regla se garantiza mecánicamente mediante el grafo de dependencias del workspace de Cargo (véase §3.7.6): si un desarrollador intentara importar `infrastructure::PostgresCaseRepository` desde un módulo del `crate domain`, el compilador rechazaría la compilación por dependencia inexistente, dado que el `Cargo.toml` del `crate domain` no declara a `infrastructure` entre sus dependencias.

La [figura 3.19](#) presenta la arquitectura hexagonal aplicada al prototipo en su forma conceptual. La versión completa, con todos los puertos nombrados como traits específicos y todos los adaptadores nombrados como structs específicas, se presenta en la Figura A.2 del Anexo A.

Arquitectura Hexagonal: capas y regla de dependencias

Las dependencias apuntan hacia el centro;
el dominio no conoce a los adaptadores

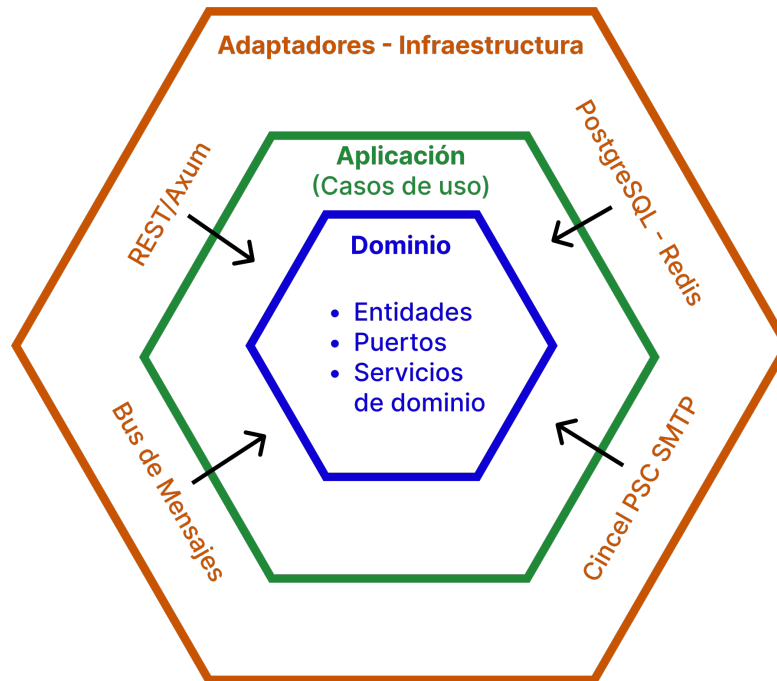


Figura 3.19: Arquitectura Hexagonal: capas conceptuales y regla de dependencias. Las dependencias siempre apuntan hacia el dominio.

El compilador de Rust verifica en tiempo de compilación que `PostgresCaseRepository` cumple con el contrato declarado por el trait `CaseRepository`. Cualquier divergencia (firma incorrecta, tipo de retorno incompatible, ausencia de un método requerido) impide la compilación del programa. Esta verificación mecánica elimina una categoría completa de errores que en lenguajes con tipado dinámico solo se manifestarían en tiempo de ejecución.

3.7.5 Vista de Componentes y Bounded Contexts

La vista de componentes desciende un nivel respecto a la vista de contexto y muestra la organización interna del sistema en términos de bounded contexts —contextos limitados— derivados directamente de las siete agrupaciones funcionales declaradas en §3.2. El concepto de bounded context, introducido por Eric Evans en el contexto de Domain-Driven Design [74], denota una zona del modelo dentro de la cual los conceptos del lenguaje ubicuo tienen un significado específico y consistente.

El sistema se descompone en siete bounded contexts: Identidad y Acceso (autenticación, RBAC, MFA), Gestión de Casos (expedientes y transiciones procesales), Calendario y Plazos (audiencias

y plazos del CNPP), Documentos y Custodia (carga, clasificación y cadena de custodia), Servicios Criptográficos (firma, sellado y verificación), Firma de Contratos (firma con sellado de tiempo) y Operaciones y Auditoría (tableros de control, informes y bitácora). Servicios Criptográficos es un servicio compartido del que dependen tanto Documentos y Custodia como Firma de Contratos.

Vista de Componentes

Vista de los siete contextos limitados y sus dependencias principales

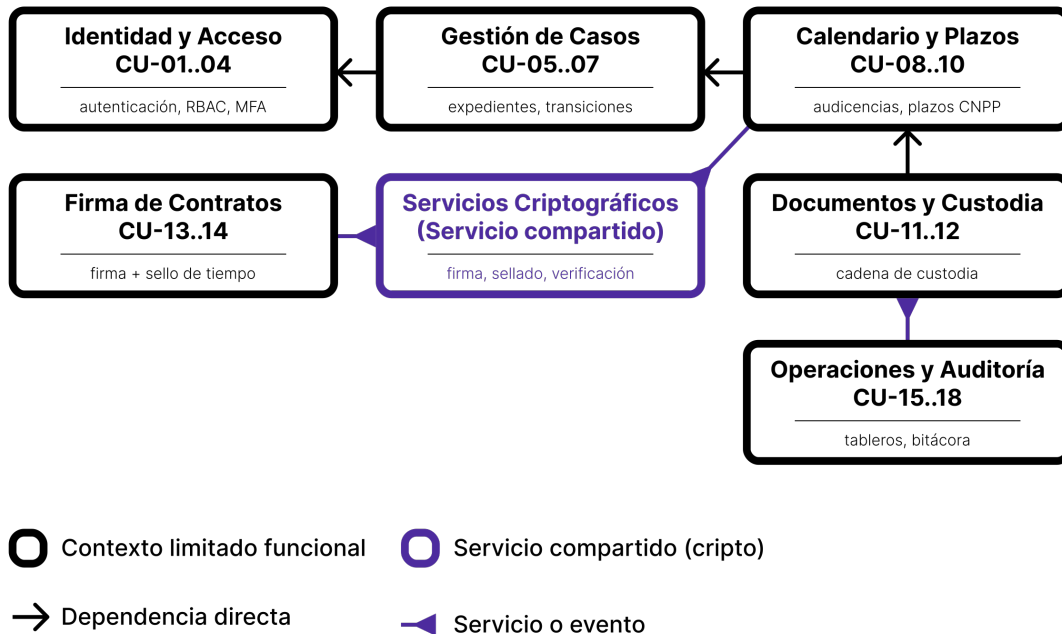


Figura 3.20: Vista de componentes: siete contextos limitados del sistema y sus dependencias principales.

El mapeo entre cada bounded context, los casos de uso que implementa y el crate del workspace de Cargo donde reside su código se presenta en la [tabla 3.77](#). Esta correspondencia permite que las modificaciones a un bounded context queden contenidas dentro de un crate específico, facilitando la atribución de cambios y la asignación de responsabilidades.

Tabla 3.77: Mapeo entre contextos delimitados, casos de uso y módulos del workspace de Cargo.

Contexto delimitado	Casos de uso	Crate / módulo
Identidad y Acceso	CU-01..04	domain::identity, application::identity
Gestión de Casos	CU-05..07	domain::case, application::case
Calendario y Plazos	CU-08..10	domain::calendar, application::calendar
Documentos y Custodia	CU-11..12	domain::document, application::document
Servicios Criptográficos	(transversal)	domain::crypto, infrastructure::crypto
Firma de Contratos	CU-13..14	domain::contract, application::contract
Operaciones y Auditoría	CU-15..18	domain::audit, application::audit

3.7.6 Estructura del Workspace de Cargo

El proyecto se organiza como un workspace de Cargo —el sistema de compilación y gestión de paquetes de Rust— compuesto por cinco crates miembros con responsabilidades estrictamente delimitadas y dependencias permitidas que enforce mecánicamente la regla de la arquitectura hexagonal. Esta estructura se documenta en el `Cargo.toml` raíz del proyecto y se valida en cada compilación.

Los cinco crates son: `domain` (entidades, value objects, traits/puertos y errores de dominio; sus únicas dependencias externas son `serde`, `thiserror`, `time`, `uuid` y `zeroize`), `application` (casos de uso, comandos y consultas; depende exclusivamente de `domain` y de `async-trait`), `infrastructure` (adaptadores driven que implementan los traits del dominio; depende de `domain` y `application`), `web` (adaptadores driving HTTP contruidos con Axum, middleware Tower, DTOs y serialización HTTP; depende de `application` y `domain`), y `bin` (composition root que instancia los adaptadores concretos y los inyecta en los casos de uso; depende de todos los anteriores). La [figura 3.21](#) representa esta estructura y las dependencias que el grafo de Cargo permite o prohíbe.

Estructura del espacio de trabajo

Los crates miembros y sus dependencias permitidas.
La regla de dependencias se verifica mecánicamente por el compilador.

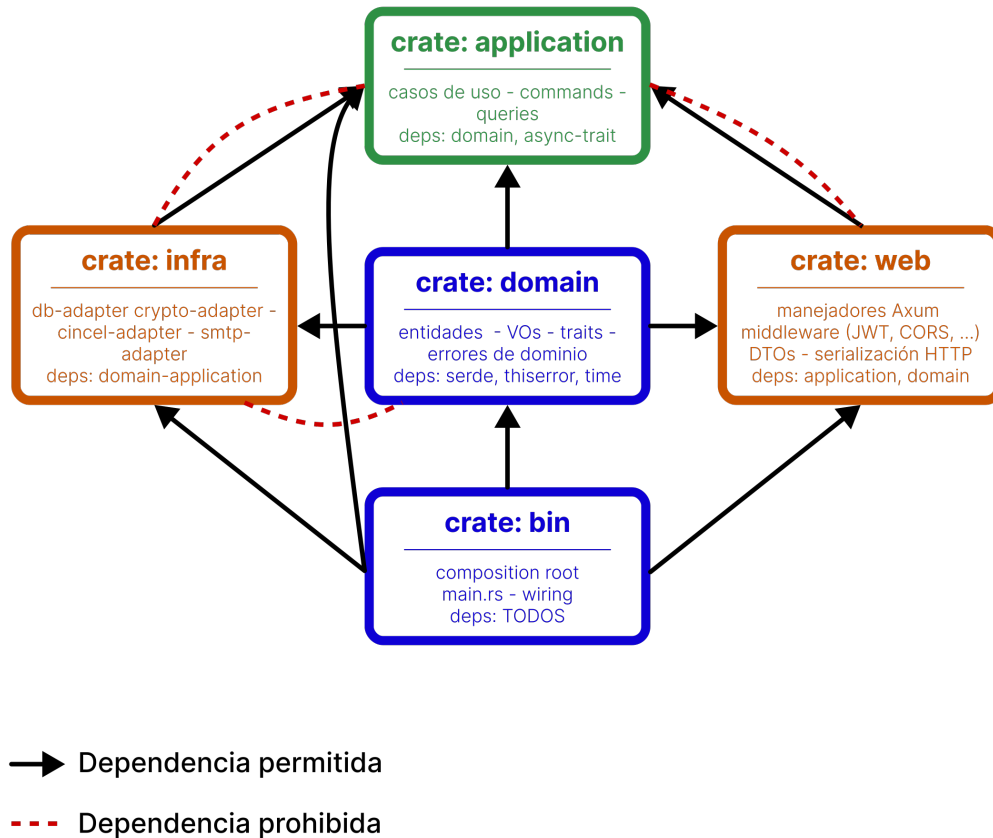


Figura 3.21: Estructura del workspace de Cargo: cinco crates miembros con dependencias permitidas (flechas negras) y prohibidas (flechas rojas punteadas).

La consecuencia técnica de esta organización es que las dependencias prohibidas no son una convención que el equipo deba recordar respetar: son una imposibilidad mecánica. Si un módulo del crate domain intenta importar un símbolo del crate infrastructure mediante una declaración `use infrastructure::PostgresCaseRepository`, el compilador emite el error «unresolved import» porque el Cargo.toml de domain no incluye infrastructure entre sus dependencias. La regla de dependencias se verifica mecánicamente por el compilador en cada build.

El sistema de Cargo features se utiliza para alternar adaptadores intercambiables en tiempo de compilación. Por ejemplo, la feature `cinsel-tsa` activa el adaptador para el PSC Cinsel, mientras que una futura feature `seguridata-tsa` activaría un adaptador alternativo para el PSC SeguriData. La selección se realiza al compilar el binario, manteniendo un único path de código para cada despliegue específico y evitando la complejidad de un sistema de inyección de dependencias en runtime.

3.7.7 Stack Tecnológico

Esta subsección documenta y justifica las tecnologías concretas seleccionadas para implementar el sistema. Cada elección se sustenta en los atributos de calidad priorizados en §3.7.1 y se compara explícitamente con sus alternativas serias del ecosistema actual. Las decisiones más estructurantes se registran adicionalmente como ADRs formales en §3.7.10.

Justificación de Rust como lenguaje del backend

El backend del prototipo se implementa en Rust [75]. La selección responde principalmente a los atributos de seguridad e integridad priorizados en §3.7.1, en cuanto Rust elimina por construcción —mediante su sistema de ownership y verificación en tiempo de compilación— las clases de errores de memoria que han sido históricamente la fuente principal de vulnerabilidades explotables en software de sistemas. Microsoft ha documentado que aproximadamente el 70 % de las vulnerabilidades reportadas en sus productos a lo largo de una década correspondieron a errores de manejo de memoria [76]; un porcentaje similar ha sido reportado por el equipo de seguridad de Chromium. Rust elimina categóricamente estos errores sin recurrir a un recolector de basura, lo que lo distingue de Go, Java o cualquier lenguaje con runtime managed.

Frente a Go, Rust ofrece tipos algebraicos (Result, Option, enums con datos asociados) que permiten modelar errores y casos especiales como parte del sistema de tipos, en lugar de mediante valores centinela o múltiples valores de retorno como hace Go. Esta expresividad es particularmente valiosa para un dominio en el que los errores deben ser explícitos y exhaustivos —una operación de firma puede fallar por certificado revocado, por reloj fuera de sincronía con la TSA, por documento que excede el tamaño máximo, entre otros casos—, y donde olvidar manejar uno de ellos no debe ser silencioso. Rust también ofrece zero-cost abstractions, permitiendo construir abstracciones de alto nivel (traits genéricos, iteradores) sin penalización en tiempo de ejecución, lo que Go renuncia a soportar por su modelo de simplicidad explícita.

Frente a Java o Kotlin con Spring, Rust evita el costo del runtime JVM —tiempo de arranque del orden de segundos, huella de memoria elevada, dependencia de un intérprete de bytecode— que es desproporcionado para un prototipo desarrollado en un equipo de desarrollo individual. Frente a Node.js con TypeScript, Rust proporciona tipado estático verificado en compilación (TypeScript es type erasure: los tipos desaparecen en runtime y no protegen contra errores reales del mundo) y rendimiento criptográfico sustancialmente superior, esencial para un sistema cuyo eje son las operaciones de firma, hash y cifrado intensivo. Frente a Python con FastAPI, Rust elimina el Global Interpreter Lock que limita la concurrencia real de Python, y ofrece un ecosistema criptográfico que ha sido sometido a auditorías formales independientes (particularmente la biblioteca ring, que utilizamos para las primitivas).

Una propiedad operacional adicional de Rust pertinente al alcance del prototipo es que el binario resultante es ejecutable único, sin dependencia de runtime, intérprete ni gestor de paquetes en producción. El despliegue se reduce a copiar un archivo de aproximadamente 15 MB al servidor objetivo, lo que simplifica la operación y reduce significativamente la superficie de ataque del entorno desplegado.

Backend: Axum sobre el ecosistema tokio, tower y hyper

Como framework web se selecciona Axum, desarrollado por el equipo de tokio. Axum se construye sobre tres capas estándar de facto en el ecosistema Rust: tokio como runtime asíncrono multi-thread

con scheduler work-stealing, tower como abstracción de middleware componible basada en el trait `Service<Request, Response>`, y hyper como implementación de los protocolos HTTP/1.1 y HTTP/2. Esta estratificación permite que cada nivel de la pila se evalúe y evolucione independientemente, y que los componentes desarrollados para tower puedan reutilizarse fuera del contexto HTTP.

Axum se distingue de alternativas como actix-web o rocket por no ser opinado respecto a la organización del código de la aplicación, lo que facilita la implementación del estilo arquitectónico hexagonal seleccionado en §3.7.3 sin tener que combatir las convenciones del framework. Los handlers de Axum reciben sus dependencias mediante extractores tipados (extractors), permitiendo inyectar los casos de uso de la capa de aplicación de manera explícita y verificable en compilación.

El pipeline de middleware Tower se configura para ejecutar, en orden, los siguientes interceptores transversales: CORS para el control de origen cruzado de peticiones del navegador; Rate Limiting con ventana deslizante para protección contra abuso; Tracing con propagación de identificador de petición para observabilidad; autenticación JWT para verificación de identidad del solicitante; logging estructurado para auditoría operativa. Cada middleware es composable —su ejecución se especifica declarativamente al construir el router— y verificable en compilación por el sistema de tipos.

Frontend: Astro con Svelte Islands

El frontend del sistema se implementa con Astro como meta-framework y Svelte como librería de componentes interactivos, siguiendo el patrón Islands Architecture propuesto por Saylor-Miller y Miller [77]. Bajo este patrón, las páginas se renderizan como HTML estático por defecto, y solo los componentes que requieren interactividad (calendario jurídico, dashboard, interfaz de verificación de firma) se hidratan selectivamente como islas reactivas. Esta estrategia minimiza el JavaScript enviado al navegador a aquello estrictamente necesario, mejorando los tiempos de carga y reduciendo la superficie de ataque del cliente.

Frente a Next.js con React, Astro y Svelte tienen tres ventajas específicas para el alcance del prototipo. Primero, Svelte compila a JavaScript imperativo sin Virtual DOM ni runtime de framework, produciendo bundles considerablemente más ligeros que React. Segundo, la reactividad de Svelte es sintáctica —se declara con `$:` y asignaciones a variables— lo que reduce el boilerplate frente al modelo de hooks de React. Tercero, Astro está diseñado desde su concepción para emitir el mínimo JavaScript posible, mientras que Next.js parte del modelo de SPA hidratada completa y solo recientemente ha introducido los Server Components para mitigar ese problema.

Frente a SvelteKit puro, Astro se elige porque las páginas mayoritariamente informativas del sistema (landing, documentación, vistas de solo lectura) no se benefician de hidratación completa; ahí Astro emite HTML estático directamente. SvelteKit, en cambio, hidrata todas las páginas como SPA, perdiendo la oportunidad de servir contenido estático puro. La combinación Astro + Svelte aprovecha lo mejor de ambos: contenido estático por defecto y reactividad rica solo donde se necesita.

Persistencia híbrida

El sistema emplea un modelo de persistencia híbrido que combina PostgreSQL como base de datos relacional para los datos con valor probatorio, con Redis como almacén clave-valor en memoria para datos volátiles y comunicación en tiempo real. La justificación completa de esta separación, los esquemas de cada motor, y el mecanismo de encadenamiento criptográfico SHA-256 que garantiza la inmutabilidad de la bitácora de auditoría se documentan en §3.8 (Modelado de Base de Datos Híbrida). Los crates utilizados para acceder a cada motor son `sqlx` para PostgreSQL —que ofrece

verificación de queries SQL en tiempo de compilación contra el esquema real— y fred para Redis —cliente asíncrono con soporte nativo para pub/sub.

Inventario completo de crates

El proyecto utiliza un conjunto curado de crates externos del ecosistema Rust, todos publicados en crates.io con versiones pinneadas en los archivos `Cargo.toml` de cada crate del workspace. Los crates se agrupan por capa arquitectónica para facilitar la auditoría y para enforce las reglas de dependencias entre capas discutidas en §3.7.6. La [tabla 3.78](#) enumera los crates principales del proyecto, agrupados por capa y con sus respectivas funciones. La versión mínima soportada del compilador (Minimum Supported Rust Version, MSRV) es Rust 1.78 estable.

Tabla 3.78: Inventario de crates externos utilizados por el proyecto, agrupados por capa arquitectónica.

Capa	Crate	Función
Dominio	serde	Serialización/deserialización tipada
Dominio	thiserror	Errores tipados por capa
Dominio	time	Manejo de fechas con zona horaria
Dominio	uuid	Identificadores únicos universales
Dominio	zeroize	Borrado de material secreto en memoria al liberar (higiene de memoria)
Aplicación	async-trait	Traits asíncronos
Infraestructura cripto	ring	Hash, AES-GCM, HMAC (auditado)
Infraestructura cripto	x509-cert + der	Manejo de certificados X.509 RFC 5280
Infraestructura cripto	rsa	Firma digital RSA-3072 PKCS#1 v1.5
Infraestructura cripto	argon2	Hashing de contraseñas (RFC 9106)
Infraestructura cripto	totp-rs	Autenticación multifactor TOTP (RFC 6238)
Infraestructura cripto	cms + x509-tsp	Parseo del token de sellado de tiempo RFC 3161 (CMS SignedData y TSTInfo)
Infraestructura datos	sqlx	Cliente PostgreSQL con verificación en compile-time
Infraestructura datos	fred	Cliente Redis asíncrono con pub/sub
Infraestructura datos	fs2	Bloqueo advisory de archivos compartidos entre procesos
Infraestructura red	reqwest	Cliente HTTP asíncrono (Cincel TSA)
Infraestructura red	lettre	Cliente SMTP con TLS y SASL
Web/HTTP	axum	Framework web sobre tokio/tower/hyper
Web/HTTP	tower / tower-http	Middleware composable
Web/HTTP	tokio	Runtime asíncrono multi-thread

Continúa en la página siguiente

Capa	Crate	Función
Interfaz de línea de comandos	clap	Análisis declarativo de argumentos y subcomandos
Observabilidad	tracing + tracing-subscriber	Logs estructurados con spans
Validación	validator	Validación de DTOs HTTP
Configuración	config + dotenvy	Carga de configuración desde entorno
Errores (boundaries)	anyhow	Errores polimórficos en <code>main.rs</code> y tests
Testing	mockall, proptest, rstest, wiremock	Mocks, pruebas basadas en propiedades, fixtures, mock HTTP

3.7.8 Patrones Arquitectónicos Transversales

Esta subsección documenta cuatro patrones que atraviesan transversalmente todas las capas de la arquitectura. Su tratamiento explícito y consistente es lo que permite que el sistema mantenga coherencia operativa en escala: errores tipados, concurrencia estructurada, inyección de dependencias por composición, y observabilidad estructurada.

Manejo de errores

El sistema adopta una estrategia de errores tipados por capa, implementada con la macro `derive` de `thiserror`. Cada capa define su propio tipo de error con variantes específicas: `DomainError` captura las violaciones de invariantes de negocio (caso inválido, transición prohibida, firma rechazada); `ApplicationError` envuelve errores de orquestación de casos de uso e incluye `DomainError` como variante; los crates de infraestructura definen errores específicos de su backend (`PostgresError`, `CinzelError`, `SmtplibError`); finalmente, `ApiError` representa errores HTTP y mapea desde las capas anteriores mediante el `trait IntoResponse` de `Axum`.

Esta tipificación contrasta con el uso indiscriminado de `anyhow`, que se reserva exclusivamente para el binario principal (`bin/main.rs`) y para las pruebas, donde la conveniencia del polimorfismo de errores supera la necesidad de discriminar tipos. En cambio, en los crates de dominio, aplicación e infraestructura, los errores deben ser exhaustivos y discriminables porque cada caller necesita decidir qué hacer ante cada variante específica.

Modelo de concurrencia asíncrona

El sistema utiliza `tokio` como runtime asíncrono multi-thread con `scheduler work-stealing`, configurado por defecto con tantos worker threads como núcleos lógicos disponibles. Las operaciones I/O bound (consultas a PostgreSQL, llamadas a Cinzel, envíos SMTP) se ejecutan asíncronamente sin bloquear los workers, permitiendo concurrencia masiva con un consumo modesto de threads del sistema operativo. La concurrencia entre tareas se estructura mediante `tokio::spawn` con joins explícitos—siguiendo el principio de `structured concurrency`— para garantizar que ninguna tarea escape del scope de su parent y produzca leaks o panics no manejados.

Las operaciones que pueden quedar bloqueadas indefinidamente (invocaciones HTTP a sistemas externos, esperas a locks distribuidos) se envuelven con `tokio::time::timeout` para garantizar

terminación acotada. Las operaciones cancelables utilizan `CancellationToken` propagado desde el handler raíz, permitiendo que un cliente que cierra la conexión HTTP libere inmediatamente todos los recursos asociados a su petición. Los traits `Send + Sync` requeridos para los puertos garantizan en tiempo de compilación que cualquier implementación es segura para usar entre threads, eliminando una categoría completa de errores de concurrencia.

Inyección de dependencias por composición

La inyección de dependencias se realiza por composición explícita en el `composition root` —el archivo `bin/main.rs`—, sin recurrir a un contenedor de DI en runtime. Los casos de uso reciben sus dependencias como parámetros genéricos restringidos por traits (`impl Trait`) o como trait objects (`Arc<dyn Trait>`). Por ejemplo, el caso de uso `SignContract` se define como `SignContract<R, S, T>` donde `R`: `CaseRepository`, `S`: `DocumentSigner`, `T`: `TimestampService`; `main.rs` construye `PostgresCaseRepository`, `RsaPkcs1Signer` y `CinzelTsaAdapter`, los inyecta en `SignContract`, y registra el resultado en el router de Axum. Esta estrategia es completamente verificable en tiempo de compilación —si un adaptador no satisface el contrato del puerto, el programa no compila— y elimina la categoría de errores característicos de los frameworks DI (dependencias no resueltas, ciclos detectados solo en runtime, configuraciones mal escritas).

Observabilidad estructurada

El sistema utiliza el ecosistema tracing para producir logs estructurados con spans tipados. Cada petición HTTP genera un span raíz con un `request_id` propagado a través de todas las invocaciones asíncronas mediante el contexto de tokio. Los logs se emiten en formato JSON line-delimited (JSONL), facilitando su ingesta por agregadores como Vector, Loki o Elasticsearch. Los niveles de verbosidad son configurables por módulo a través de la variable de entorno `RUST_LOG`, permitiendo aumentar temporalmente el detalle de una capa específica sin reiniciar el servicio. Para trazabilidad distribuida, el `crate tracing-opentelemetry` permite exportar spans a backends compatibles (Jaeger, Tempo) sin modificar el código de la aplicación. La política de redacción de información sensible (referencia cruzada a RNF-03) está implementada como un layer personalizado del subscriber, que aplica patrones regex sobre los campos antes de su emisión para sanitizar secretos y material criptográfico que pudieran filtrarse.

3.7.9 Vista de Seguridad: Fronteras de Confianza

Esta subsección desarrolla la vista de seguridad arquitectónica, complementaria al diseño criptográfico detallado en §3.11. Mientras §3.11 documenta los algoritmos criptográficos específicos (RSA-3072, AES-256-GCM, SHA-256, RFC 3161, PKI con CA interna) y sus parámetros, esta subsección documenta cómo esos mecanismos se distribuyen entre las distintas zonas de confianza del sistema y qué controles se aplican en cada cruce de frontera.

El modelo adoptado sigue el principio de defensa en profundidad formalizado por Saltzer y Schroeder [78] y desarrollado operativamente en el OWASP Application Security Verification Standard [79]. La Figura A.1 en el Anexo A presenta el modelo completo con cuatro zonas de confianza y los controles de seguridad aplicados en cada cruce.

El sistema se modela como cuatro zonas de confianza separadas por tres fronteras explícitas. La primera zona, la red pública, comprende todo aquello que no está bajo control directo del sistema: el

navegador del usuario final, los actores adversarios potenciales, y los servicios externos como Cincel PSC y los proveedores SMTP que aunque sean legítimos no operan dentro del perímetro de seguridad del prototipo. La segunda zona, el perímetro de aplicación, contiene el reverse proxy con terminación TLS y la API pública expuesta. La tercera zona, el núcleo aplicativo, contiene el binario Rust con sus tres módulos internos (web, application, domain). La cuarta y más confiable zona, datos persistentes y secretos, contiene PostgreSQL, Redis y el subsistema de gestión de claves criptográficas.

El cruce de la primera frontera —de la red pública al perímetro de aplicación— se controla mediante terminación TLS 1.3 con suites de cifrado modernas; rate limiting con ventana deslizante para protección contra abuso de la API; un Web Application Firewall que filtra patrones de ataque conocidos; y cabeceras CORS estrictas que restringen el origen de las peticiones legítimas. La superficie expuesta a esta frontera es deliberadamente mínima: solo los endpoints HTTPS estrictamente necesarios.

El cruce de la segunda frontera —del perímetro al núcleo aplicativo— se controla mediante autenticación basada en JWT (JSON Web Token) firmado por la propia aplicación; validación exhaustiva de toda entrada del usuario antes de cualquier procesamiento; y autorización basada en roles (RBAC) que restringe qué operaciones puede invocar cada rol (socio, paralegal, cliente). Cualquier petición que no satisfaga las tres verificaciones es rechazada con un código de error apropiado y registrada en la bitácora de auditoría.

El cruce de la tercera frontera —del núcleo aplicativo a datos persistentes y secretos— se controla mediante conexiones TLS internas entre el binario Rust y los motores de PostgreSQL y Redis; uso obligatorio de consultas parametrizadas mediante sqlx (que verifica en compile-time la conformidad de cada query con el esquema, eliminando la posibilidad de inyección SQL); y cifrado en reposo de todos los documentos mediante el patrón envelope encryption con KEK protegida por el módulo de gestión de claves (cuyo diseño detallado se documenta en §3.11.4).

Los servicios externos se modelan como entidades de la zona no confiable a las que el sistema invoca explícitamente con canales seguros: un PSC externo futuro para sellado de tiempo (RFC 3161) utilizará TLS y verificación de la firma del sello recibido; la TSA local de demostración opera sin salir del perímetro del prototipo; la integración con el proveedor SMTP utiliza TLS más autenticación SASL. En ningún caso se confía en estos servicios para operaciones de seguridad sin verificación local del resultado: un sello recibido se verifica criptográficamente contra la cadena de certificados del emisor antes de ser aceptado e incorporado al expediente.

El modelo de amenazas STRIDE de Shostack [80] se aplicó como herramienta de revisión sistemática durante el diseño para verificar que cada una de las seis categorías de amenazas (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) tuviese al menos un control asignado. La matriz resultante demuestra que la combinación de los controles enumerados ofrece cobertura para todas las categorías, con énfasis particular en Tampering (mediante el encadenamiento SHA-256 de la bitácora) y Repudiation (mediante el sellado de tiempo independiente del PSC).

3.7.10 Decisiones Arquitectónicas Significativas (ADRs)

Las decisiones arquitectónicas más estructurantes del sistema se registran formalmente como Architecture Decision Records (ADRs) en el formato propuesto por Michael Nygard [66]. Cada ADR documenta de forma autocontenida cuatro elementos: el contexto del problema que motivó la decisión, las alternativas consideradas, la decisión adoptada, y las consecuencias —tanto positivas como

negativas— de la decisión. Los seis ADRs siguientes cubren las decisiones cuya reversión implicaría una refactorización significativa.

ADR-01. Adopción de Arquitectura Hexagonal

Contexto: el sistema requiere alta testabilidad (XP/TDD) y evolvabilidad (sustitución de PSC, motor de BD). Alternativas: N-tier, Clean, Onion, Hexagonal, Microservicios, Monolito Modular sin patrón hexagonal. Decisión: Monolito Modular con Arquitectura Hexagonal de Cockburn. Consecuencias positivas: dominio aislado y testeable, adaptadores intercambiables, regla de dependencias verificable mecánicamente. Consecuencias negativas: ligero overhead de indirección por cada operación que cruza una capa, requiere disciplina del equipo para no introducir atajos.

ADR-02. Rust como lenguaje del backend

Contexto: prototipo de seguridad con operaciones criptográficas intensivas. Alternativas: Go, Java/-Kotlin con Spring, Node.js con TypeScript, Python con FastAPI. Decisión: Rust. Consecuencias positivas: seguridad de memoria sin GC, sistema de tipos algebraicos, ecosistema criptográfico auditado, binario único sin runtime. Consecuencias negativas: curva de aprendizaje del ownership y borrow checker, ecosistema más pequeño que Java o Node, tiempos de compilación más largos.

ADR-03. Axum como framework web

Contexto: necesidad de un framework HTTP que no impusiera estructura incompatible con el estilo hexagonal. Alternativas: actix-web, rocket, warp. Decisión: Axum sobre tokio/tower/hyper. Consecuencias positivas: framework no opinado, ecosistema tower reutilizable para otros servicios, mantenido por el equipo de tokio. Consecuencias negativas: API ergonómica menos madura que actix-web, documentación todavía en evolución.

ADR-04. Astro + Svelte Islands para el frontend

Contexto: páginas mayoritariamente informativas con islas de interactividad rica (calendario, dashboard, verificación de firma). Alternativas: Next.js + React, SvelteKit puro, Remix. Decisión: Astro como meta-framework con Svelte como librería de componentes interactivos. Consecuencias positivas: HTML estático por defecto, JavaScript mínimo enviado al cliente, Svelte sin Virtual DOM. Consecuencias negativas: dos sistemas de templating (Astro y Svelte) que el desarrollador debe conocer, ecosistema de componentes Svelte más pequeño que React.

ADR-05. Monolito Modular sobre Microservicios

Contexto: equipo de desarrollo de dos personas en periodo académico, alcance del prototipo no requiere despliegue distribuido. Alternativas: descomposición en microservicios desde el inicio, monolito modular. Decisión: Monolito Modular siguiendo la recomendación *MonolithFirst* de Fowler [72]. Consecuencias positivas: simplicidad operativa, despliegue como binario único, sin overhead de comunicación inter-servicios, facilidad de refactorización. Consecuencias negativas: escalado monolítico (ya aceptable para el alcance del prototipo), imposibilidad de desplegar componentes independientemente.

ADR-06. Persistencia híbrida PostgreSQL + Redis

Contexto: el sistema maneja tanto datos con valor probatorio (que requieren persistencia transaccional duradera) como datos volátiles para sesiones, caché y notificaciones en tiempo real. Alternativas: PostgreSQL exclusivamente, Redis exclusivamente, PostgreSQL + Redis. Decisión: PostgreSQL para datos persistentes con responsabilidades específicas en §3.8.1, Redis para datos volátiles con TTL conforme a §3.8.2. Consecuencias positivas: cada motor utilizado para lo que mejor hace, baja latencia para sesiones y notificaciones, integridad transaccional para operaciones críticas. Consecuencias negativas: necesidad de operar dos motores de almacenamiento, mayor complejidad operativa, necesidad de coherencia eventual entre ambos.

3.7.11 Trazabilidad Arquitectura ↔ Requerimientos

Esta subsección presenta la matriz de trazabilidad que cierra el Capítulo 3 demostrando que cada requerimiento funcional y no funcional cuenta con un componente arquitectónico responsable, y que ningún componente arquitectónico carece de justificación en un requerimiento. La [tabla 3.79](#) sintetiza este mapeo en una vista de alto nivel; la matriz completa, con los diecisiete casos de uso (CU-01 a CU-17) y los catorce requerimientos no funcionales (RNF-01 a RNF-14 de §3.4), se entrega como anexo electrónico complementario al documento.

Tabla 3.79: Mapeo de alto nivel entre componentes arquitectónicos y los requerimientos que satisfacen.

Componente arquitectónico	Requerimientos atendidos
Bounded Context «Identidad y Acceso»	CU-01..04, RNF-01, RNF-02
Bounded Context «Gestión de Casos»	CU-05..07, RNF-04 (parcial)
Bounded Context «Calendario y Plazos»	CU-08..10
Bounded Context «Documentos y Custodia»	CU-11..12, RNF-03, RNF-04
Bounded Context «Servicios Criptográficos»	CU-11..14, RNF-03
Bounded Context «Firma de Contratos»	CU-13..14
Bounded Context «Operaciones y Auditoría»	CU-15..17, RNF-04, RF-20
Crate domain con traits (puertos)	Soporta evolvabilidad transversal
Workspace de Cargo con regla de dependencias	Mantenibilidad transversal
Pipeline Tower middleware	RF-01 (autenticación y sesiones), seguridad cross-cutting
Modelo de fronteras de confianza (§3.7.9)	RNF-01, RNF-02, RNF-03
Subsistema observabilidad (tracing)	RF-20 (bitácora de actividad)

Fitness functions

Siguiendo el enfoque de arquitecturas evolutivas formalizado por Ford, Parsons y Kua [68], la coherencia entre la arquitectura prescrita y la arquitectura efectiva del código se verifica mediante un conjunto de fitness functions: pruebas automatizadas que comprueban propiedades arquitectónicas que no podrían verificarse mediante pruebas funcionales tradicionales. Estas fitness functions se ejecutan como parte del pipeline de Integración Continua y, en caso de fallo, bloquean la integración del cambio en la rama principal.

Las fitness functions implementadas para el prototipo son: (1) verificación de que ningún módulo del crate domain importa símbolos de los crates infrastructure ni web (verificada por el compilador en cada build); (2) verificación de que todo handler HTTP envuelve su lógica en un span de tracing (verificada con un lint personalizado de clippy); (3) verificación de que ningún log emitido contiene literales que matcheen patrones de claves criptográficas (verificada con un test que ejecuta toda la suite e inspecciona los logs producidos); (4) verificación de que cada trait que define un puerto driven tiene al menos una implementación concreta y al menos un mock para tests (verificada por inspección del workspace en cada build); (5) verificación de que el binario compilado no excede 25 MB (verificada por el pipeline de release).

3.8 Modelado de Base de Datos Híbrida

El sistema emplea un modelo de persistencia híbrido que combina **PostgreSQL** (base de datos relacional) con **Redis** (almacén clave-valor en memoria), con responsabilidades estrictamente delimitadas. Esta decisión responde a los requisitos funcionales y no funcionales identificados en el protocolo, particularmente RF-20 (bitácora de actividad), RNF-04 (integridad append-only) y la necesidad de notificaciones en tiempo real.

3.8.1 PostgreSQL: Datos Persistentes con Valor Probatorio

PostgreSQL se selecciona como motor relacional principal por las siguientes razones: (1) soporte nativo de **JSONB** para metadatos variables de expedientes, lo que permite modelar la diversidad del sistema acusatorio oral mexicano (CNPP, Art. 211-215) sin sacrificar la integridad referencial; (2) **RULE** para enforce de append-only en la bitácora de auditoría, impidiendo operaciones UPDATE y DELETE a nivel de motor; (3) extensiones criptográficas nativas (pgcrypto); (4) tipo de dato INET para registro de direcciones IP en la bitácora.

La bitácora de auditoría (audit_log) implementa un mecanismo de **encadenamiento SHA-256**: cada registro incluye un campo sha256_chain calculado como el hash SHA-256 (véase §2.3.1) del valor sha256_chain del registro anterior concatenado con los datos del registro actual, es decir, $\text{sha256_chain}(n) = \text{SHA-256}(\text{sha256_chain}(n - 1) \parallel \text{registro}(n))$. Esto crea una cadena verificable de integridad que permite detectar cualquier manipulación retroactiva, cumpliendo con el Art. 33 del Código de Comercio (retención mínima de 10 años).

Decisión de diseño: Single-Tenant

El sistema se diseña para un solo despacho legal (single-tenant), eliminando la complejidad de aislamiento multi-tenant contemplada en el protocolo original (cuyos requerimientos se reformulan

en RNF-01 y RNF-05 como aislamiento y privacidad de la información del despacho dentro de su instancia). Esta simplificación es coherente con el alcance del prototipo y reduce significativamente la superficie de ataque.

3.8.2 Redis: Datos Volátiles y Comunicación en Tiempo Real

Redis se emplea exclusivamente para datos efímeros que **no requieren persistencia a largo plazo ni tienen valor probatorio**:

- **Sesiones JWT**: almacenamiento de tokens de sesión con TTL configurable (24h por defecto), permitiendo invalidación inmediata por revocación.
- **Caché de permisos**: materialización temporal de las autorizaciones del usuario (TTL 5 minutos) para evitar consultas repetidas a PostgreSQL en cada petición.
- **Pub/Sub para notificaciones**: canal de comunicación entre el backend y los clientes WebSocket. Cuando ocurre un evento relevante (nuevo documento, firma completada, deadline próximo), se publica en un canal Redis que el adaptador WebSocket consume y retransmite al navegador del usuario.
- **Rate limiting**: contadores con ventana deslizante para protección contra abuso de la API.
- **Tokens temporales**: tokens de un solo uso para restablecimiento de contraseña y verificación MFA (TTL 15 minutos).

3.8.3 Diagrama del Modelo de Datos

La [figura 3.22](#) presenta el modelo de datos completo, mostrando las tablas de PostgreSQL (izquierda) con sus relaciones y las estructuras de Redis (derecha) con sus políticas de TTL.

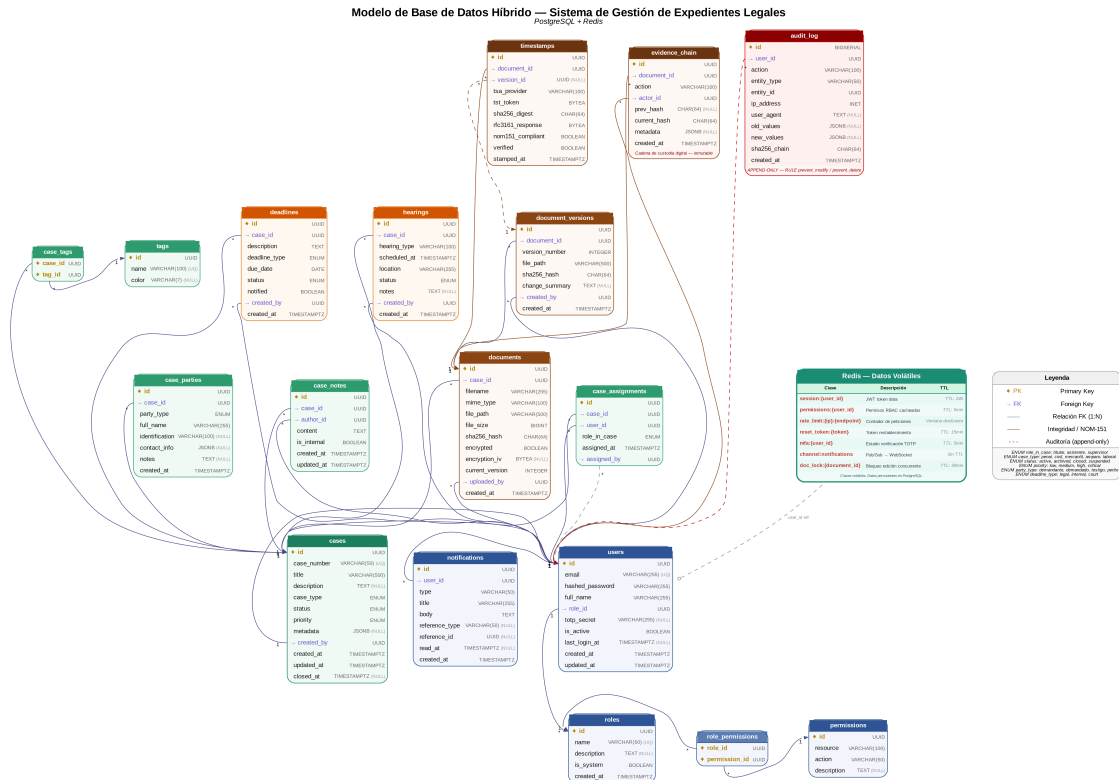


Figura 3.22: Modelo de datos híbrido: PostgreSQL para datos persistentes con valor probatorio, Redis para datos volátiles.

Nótese que la tabla `audit_log` está resaltada en rojo para indicar su naturaleza especial: es la única tabla del sistema con restricción `RULE` que impide `UPDATE` y `DELETE`. Su campo `sha256_chain` implementa el encadenamiento criptográfico descrito en la [sección 3.8.1](#). El campo `metadata` JSONB en la tabla `cases` permite almacenar atributos específicos de cada tipo de procedimiento sin necesidad de alterar el esquema relacional.

3.9 Diagrama de Despliegue y Stack Tecnológico

La [figura 3.23](#) presenta la vista de despliegue del sistema, ilustrando la interacción entre las capas del frontend, el backend y los servicios de infraestructura.

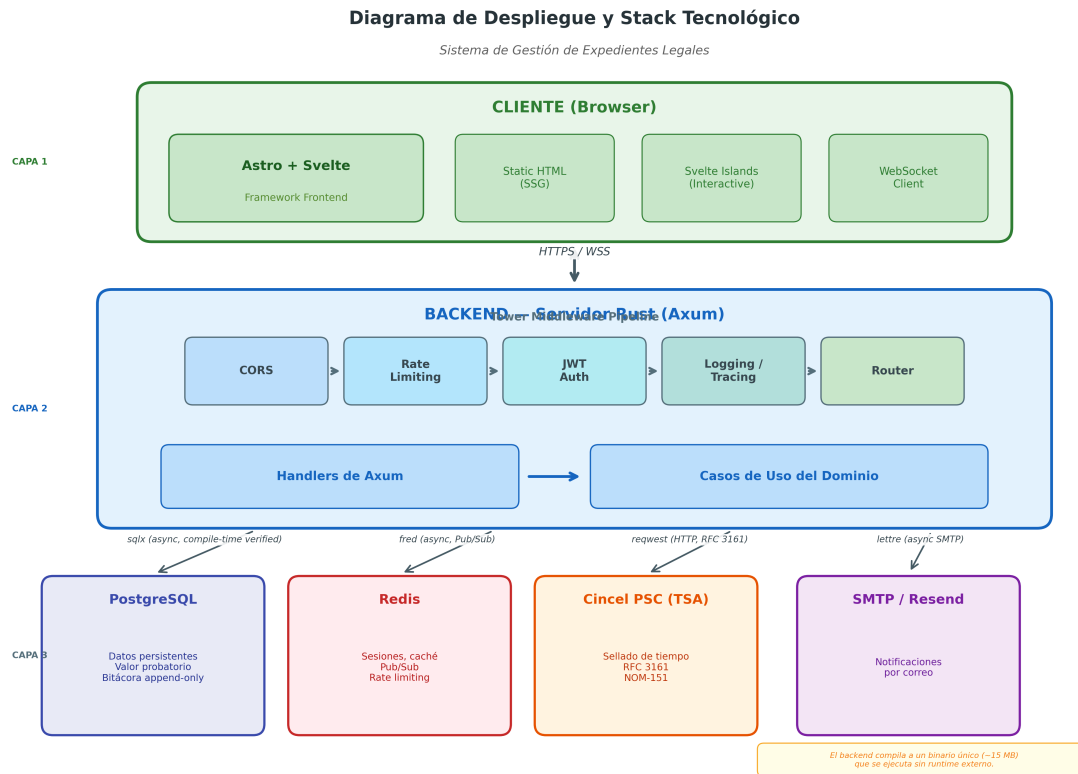


Figura 3.23: Diagrama de despliegue: Astro+Svelte (cliente) → Axum/Rust (backend) → PostgreSQL + Redis + servicios externos.

El backend Rust compila a un **binario único** de aproximadamente 15 MB que incluye todas las dependencias. El binario Rust se ejecuta directamente sin runtime externo, eliminando la necesidad de intérpretes, servidores web adicionales o gestores de paquetes. Esto simplifica significativamente el despliegue y reduce la superficie de ataque del servidor.

La capa de **Tower Middleware** opera como un pipeline composable donde cada middleware es una capa que envuelve al siguiente. El flujo de una petición HTTP es: CORS → Rate Limiting → Autenticación JWT → Logging → Handler de Axum → Dominio. Esta composición es declarativa y verificada en tiempo de compilación por el sistema de tipos de Rust.

La comunicación con **Cincel PSC** para sellado de tiempo RFC 3161 se realiza a través de request (cliente HTTP asíncrono). El diseño del puerto TimestampService como trait abstracto permite sustituir Cincel por cualquier otra TSA (SeguriData, CECOBAN; véase §2.3.3.4) implementando únicamente un nuevo adaptador, sin modificar el dominio ni los demás componentes del sistema.

Las conexiones a **PostgreSQL** se gestionan mediante sqlx, que ofrece verificación de queries SQL en tiempo de compilación contra el esquema real de la base de datos. Las conexiones a **Redis** se gestionan mediante fred, un cliente asíncrono compatible con cluster que soporta pub/sub nativo para las notificaciones en tiempo real.

3.10 Modelo de Datos Jurídico-Técnico

La fundamentación del proceso penal expuesta en el marco teórico tiene una consecuencia directa sobre el diseño del prototipo: la base de datos no puede ser un conjunto de tablas genéricas, sino que debe reflejar fielmente las dependencias y restricciones que el CNPP impone. Esta sección articula los vínculos explícitos entre la teoría jurídica y los requerimientos funcionales.

Un caso en el prototipo no es un registro estático, sino una entidad que evoluciona a través de los estados que el CNPP define como etapas [1]. La transición de un estado a otro solo es posible mediante la carga del documento procesal habilitante: de Investigación a Intermedia únicamente mediante el escrito de acusación; de Intermedia a Juicio Oral mediante el auto de apertura. El historial de transiciones debe mantenerse de forma inmutable como parte del expediente, dado que un tribunal revisor puede requerir verificar la secuencia procesal completa.

3.11 Diseño Criptográfico del Prototipo

Esta sección documenta las decisiones de diseño criptográfico adoptadas para el prototipo, fundamentadas en los conceptos teóricos expuestos en el capítulo 2 y en las restricciones impuestas por el marco legal mexicano. Cada decisión se justifica con referencia a los estándares y publicaciones del NIST, las RFC del IETF y las guías de OWASP que las sustentan.

3.11.1 Nivel de Seguridad Objetivo

El prototipo adopta un nivel de fortaleza de seguridad de 128 bits conforme a NIST SP 800-57 Part 1 Rev 5 [37]. Este nivel unificado exige que cada primitivo criptográfico ofrezca al menos 128 bits de resistencia: SHA-256 proporciona 128 bits de resistencia a colisiones, AES-256 proporciona 256 bits de fortaleza, y RSA-3072 proporciona aproximadamente 128 bits. La elección de 128 bits como objetivo coherente garantiza que ningún eslabón de la cadena criptográfica sea más débil que los demás. NIST considera este nivel seguro al menos hasta 2030 para aplicaciones de uso general.

La Tabla 2 del NIST SP 800-57 [37] clasifica los niveles de fortaleza en función de los tamaños de clave recomendados para cada familia de algoritmos. El nivel de 112 bits, correspondiente a RSA-2048, está clasificado como legacy-use: aceptable para procesar información existente, pero no recomendado para proteger información nueva más allá de 2030. El nivel de 128 bits, correspondiente a RSA-3072, AES-128 o AES-256 y SHA-256, es el primer nivel que NIST clasifica como aceptable para uso general sin restricción temporal. Dado que el prototipo gestiona documentos con valor probatorio que pueden requerirse como prueba judicial años después de su firma, la elección del nivel de 128 bits proporciona un margen de seguridad adecuado frente a la evolución del poder computacional. Cabe señalar que la e.firma del SAT utiliza RSA-2048 (112 bits), un nivel inferior al adoptado por el prototipo; la libertad de elegir parámetros que otorga la CA interna permite superar ese piso normativo.

La coherencia entre primitivos se verifica de la siguiente manera: SHA-256 proporciona 128 bits de resistencia a colisiones y 256 bits de resistencia a preimagen; RSA-3072 proporciona aproximadamente 128 bits de fortaleza para firma digital; AES-256 proporciona 256 bits de fortaleza para cifrado simétrico, superando el nivel objetivo pero seleccionado por ser el tamaño estándar de clave para GCM en aplicaciones de nivel 128 bits o superior; y Argon2id con hash de salida de 32 bytes proporciona 256 bits de resistencia a preimagen para el almacenamiento de contraseñas. Ningún

primitivo del sistema presenta una fortaleza inferior a 128 bits, lo que garantiza la ausencia de un eslabón débil en la cadena criptográfica. Esta propiedad de coherencia es especialmente relevante en un sistema que combina múltiples servicios criptográficos —integridad, confidencialidad, no repudio y autenticación—, ya que un atacante racional concentraría su esfuerzo en el primitivo más débil.

3.11.2 Firma Digital

El esquema de firma digital emplea RSA-3072 con SHA-256 y padding PKCS#1 v1.5, con sha256-WithRSAEncryption como OID del algoritmo de firma. Se descartó ECDSA P-256 tras verificar que la Política de Certificados de la ACR2-SE [42] establece que los certificados emitidos bajo la jerarquía PKI mexicana deben contener claves públicas RSA con sha256-WithRSAEncryption como algoritmo de firma; ECDSA no está contemplado para certificados legalmente reconocidos bajo PSC mexicano. La elección de PKCS#1 v1.5 sobre PSS se justifica por la alineación con el esquema que usa la PKI mexicana real (ACR2-SE y e.firma), lo que facilita la migración futura a certificados emitidos por un PSC sin rediseñar el verificador.

Los certificados X.509 v3 de los socios del despacho son emitidos por una Autoridad Certificadora (CA) interna del prototipo, implementada con OpenSSL scriptado en una topología de una sola capa: un certificado raíz autofirmado con vigencia de cinco años firma directamente los certificados de los socios con vigencia de un año. Esta elección expone al equipo al ciclo de vida completo de una PKI —generación de la clave raíz, emisión de CSRs (Certificate Signing Request, solicitudes de firma de certificado), firma de certificados, validación de cadena y revocación— que una integración con la e.firma del SAT o un PSC comercial ocultaría detrás de una API. El módulo de firma acepta cualquier certificado X.509 RSA + SHA-256, de modo que en un despliegue productivo la CA interna podría sustituirse por la e.firma o un PSC autorizado sin modificar la lógica de verificación.

La validación del estado de revocación se realiza mediante una Lista de Certificados Revocados (CRL) conforme a RFC 5280 [44], publicada como archivo PEM en el endpoint GET /pki/crl.pem con vigencia de siete días. La CRL se regenera bajo demanda tras cada revocación y periódicamente antes de su expiración. Se descartó OCSP por la escala reducida del prototipo (~2–5 certificados), donde la CRL es de tamaño despreciable y no requiere un servicio persistente adicional.

La elección de una CA interna sobre la e.firma del SAT o un PSC comercial responde a múltiples consideraciones. En primer lugar, las firmas generadas con certificados de la CA interna emplean los mismos mecanismos técnicos que las generadas con certificados de la e.firma o de un PSC: RSA-3072 con SHA-256, estructura X.509 v3 y verificación de cadena. Lo que cambia es el emisor y, con él, el nivel de confianza jurídica atribuible al certificado. Esta elección permite demostrar integridad y autoría dentro del alcance académico, pero no debe presentarse como equivalente a una firma avanzada. En segundo lugar, un sello RFC 3161 emitido por un PSC autorizado puede aportar la independencia temporal prevista por la NOM-151 [14], [27]; un sello emitido por la TSA interna conserva el mismo flujo criptográfico, pero permanece dentro de la frontera de confianza del prototipo.

En tercer lugar, usar la e.firma habría obligado a que el socio subiera su archivo .key y su contraseña al prototipo, o a implementar firma del lado del cliente parseando el formato propietario del SAT. Ambas rutas introducen un problema de custodia de claves que contradice el objetivo declarado de no repudio: si el servidor custodia la clave privada del firmante, la firma pierde su propiedad vinculante porque no puede atribuirse exclusivamente a su titular. Con la CA interna, el ciclo de vida de las claves queda bajo control del prototipo y es auditable. Finalmente, montar la CA internamente expone al equipo al ciclo de vida completo de una PKI —generación de la clave raíz, emisión de CSRs, firma de certificados con extensiones X.509 v3, construcción de la cadena de confianza, validación

y revocación— que una integración con la e.firma o un PSC comercial ocultaría detrás de una API externa. Este aprendizaje es parte del valor formativo declarado del Trabajo Terminal.

La implementación de la CA se realiza con OpenSSL scriptado en lugar de herramientas como step-ca o EJBCA. OpenSSL expone cada paso del proceso de forma transparente y auditable: la generación de la clave raíz (openssl genrsa), la creación del certificado autofirmado (openssl req -x509), la firma de solicitudes de certificado (openssl x509 -req -CA) y la generación de CRLs (openssl ca -gencrl) se encapsulan en aproximadamente cuatro invocaciones cuyo resultado es verificable con openssl x509 -text. step-ca automatiza y oculta esa mecánica detrás de una API REST, lo cual es deseable en producción pero contraproducente para un ejercicio académico. Además, OpenSSL no introduce dependencias adicionales al stack del prototipo. Respecto a las vigencias, el certificado raíz tiene una validez de cinco años para sobrevivir ampliamente al ciclo de vida del proyecto y al período de evaluación, mientras que los certificados de los socios tienen una validez de un año, alineada con la práctica estándar de la industria y suficiente para demostrar que el prototipo contempla la renovación de certificados.

3.11.3 Sellado de Tiempo

El sellado de tiempo se diseña detrás del puerto `TimestampService`, con `LocalOpenSSLTsa` como adaptador actual de demostración y un adaptador de PSC externo como sustitución futura. En ambos casos el flujo opera conforme al protocolo RFC 3161 [27]: se calcula el hash SHA-256 del documento firmado, se solicita el sello al emisor configurado y se almacena la respuesta `TimeStampResp` en formato ASN.1 DER como artefacto independiente (.tsr). Esta abstracción permite sustituir la TSA local por Cincel, SeguriData, CECOBAN u otra TSA compatible sin modificar la lógica central. La equivalencia es de protocolo, no de confianza: únicamente el camino hacia un PSC autorizado y contratado puede producir la constancia externa prevista por la NOM-151-SCFI-2016.

Para facilitar la verificación independiente por un tercero (tribunal, contraparte, perito), el prototipo genera un paquete ZIP descargable que contiene el documento original, la firma RSA del socio, el sello de tiempo .tsr, los certificados del firmante y de la CA raíz, e instrucciones con comandos OpenSSL para la verificación completa. Este diseño garantiza que la verificación no dependa de la disponibilidad de la plataforma.

La selección inicial de Cincel como proveedor de sellado de tiempo obedeció a tres factores. Primero, Cincel es un PSC acreditado por la Secretaría de Economía para la emisión de Sellos Digitales de Tiempo (SDT) y constancias de conservación de mensajes de datos (CCMD), con acreditación publicada en el Diario Oficial de la Federación el 24 de noviembre de 2023 [47]. Segundo, ofrece un entorno de sandbox y recursos de integración. Tercero, la interfaz documentada permite enviar el hash SHA-256 y recuperar el artefacto ASN.1 DER conforme a RFC 3161 [27], [81], incluidos estados de procesamiento diferido que el prototipo maneja mediante reintentos configurables. La experiencia de integración corrigió la segunda premisa: aunque el registro se completó, el sandbox requiere pago con precios no transparentes y la API no ofreció respuesta y estabilidad suficientes para una campaña repetible. En consecuencia, Cincel no se utiliza como evidencia de esta entrega; permanece únicamente como adaptador externo opcional, mientras la TSA interna verifica el flujo completo sin convertir la disponibilidad comercial en requisito de desarrollo.

El sello de tiempo se almacena como la respuesta cruda `TimeStampResp` en formato ASN.1 DER (extensión .tsr) en lugar de empaquetarlo en una estructura CAdES-T o embebido en un PDF (PAdES-T). Esta decisión se justifica por tres razones. Primera, con artefactos independientes cada componente del flujo criptográfico —documento, firma, sello, certificado— puede inspeccionarse

y verificarse por separado con herramientas estándar; el comando `openssl ts -verify` opera directamente sobre archivos `.tsr` sin necesidad de parsear contenedores CMS ni estructuras PDF. Segunda, CAdES-T requeriría construir estructuras CMS con atributos no firmados y PAdES-T requeriría manipular los mecanismos internos del formato PDF (ByteRange, diccionarios de firma y guardados incrementales), tareas de ingeniería de formatos que consumirían tiempo del cronograma sin aportar al objetivo criptográfico del Trabajo Terminal. Tercera, el contenedor no altera el token RFC 3161 subyacente ni la identidad de su emisor: el alcance jurídico depende de la autoridad y su cadena de confianza, no de almacenar el token como `.tsr`, CAdES-T o PAdES-T [14].

El flujo de verificación completo del sello de tiempo procede en cuatro pasos: (1) se parsea la `TimeStampResp` para extraer el `TimeStampToken`; (2) se verifica que `messageImprint`, `hashedMessage` coincida con el hash SHA-256 recalculado del documento original; (3) se verifica la firma de la TSA sobre el token; y (4) se valida su certificado contra el ancla de confianza correspondiente. Para la TSA interna, esa ancla es la CA del prototipo; para una constancia externa, debe ser la cadena del PSC emisor hasta la ACR2-SE [42]. Con OpenSSL, el proceso se ejecuta con `openssl ts -verify -data documento.pdf -in sello.tsr -CAfile ancla.pem`. Un despliegue productivo podría migrar a CAdES-T o PAdES-T sin cambiar el token subyacente ni el flujo de obtención.

3.11.4 Cifrado en Reposo

Los documentos almacenados se cifran con AES-256-GCM conforme a NIST SP 800-38D [50]. Cada operación utiliza un nonce (número de uso único) aleatorio de 96 bits generado con un CSPRNG (Generador de Números Pseudoaleatorios Criptográficamente Seguro), cuya probabilidad de colisión es despreciable cuando cada documento emplea su propia clave (NIST SP 800-38D §8.3 establece un límite de 2^{32} invocaciones por clave). La gestión de claves sigue un esquema de envelope encryption: una Clave de Cifrado de Clave (KEK), almacenada en una variable de entorno del servidor, envuelve una Clave de Cifrado de Datos (DEK) generada aleatoriamente para cada documento conforme a NIST SP 800-38F [52]. Esta separación permite rotar la KEK re-cifrando únicamente las DEKs envueltas, sin necesidad de re-cifrar los documentos del repositorio. Los Datos Adicionales Autenticados (AAD) de GCM incluyen el identificador del documento y su versión, lo que previene ataques de intercambio (swap) y de reversión (rollback) entre documentos.

La elección de un nonce aleatorio sobre un nonce de contador responde a la prioridad de evitar el modo de fallo más catastrófico de GCM: la reutilización de un par (clave, nonce). Un nonce de contador requiere mantener un estado persistente que se incremente atómicamente en cada operación de cifrado; si ese estado se reinicia por un error de software, una restauración de respaldo o una condición de carrera, el contador repite un valor y el esquema GCM se rompe irrecuperablemente, exponiendo tanto el texto claro como la clave de autenticación. Un nonce aleatorio de 96 bits generado con un CSPRNG elimina esta dependencia de estado por diseño: cada invocación es independiente y no requiere coordinación con invocaciones anteriores. NIST SP 800-38D [50] §8.3 establece que con nonces aleatorios de 96 bits el número total de invocaciones por clave no debe exceder 2^{32} (aproximadamente 4,300 millones) para mantener la probabilidad de colisión por debajo de $\sim 2^{-33}$. Con envelope encryption, donde cada documento emplea su propia DEK, el número de invocaciones por clave es efectivamente uno, lo que hace la probabilidad de colisión exactamente cero en la práctica.

La arquitectura de envelope encryption se seleccionó sobre dos alternativas: una clave única compartida para todo el repositorio y una derivación de claves mediante HKDF. Una sola clave compartida concentra el riesgo —su compromiso expone la totalidad de la información— y hace la rotación

prohibitivamente costosa porque cada documento debe re-cifrarse. Un esquema basado en HKDF derivaría una clave por documento a partir de una clave maestra y un contexto único (por ejemplo, el identificador del documento), lo que evita almacenar claves envueltas pero presenta dos desventajas: si la clave maestra se compromete, todas las claves derivadas caen sin posibilidad de revocar una sola; y la unicidad absoluta del contexto de derivación reintroduce un problema de estado similar al del nonce de contador. Envelope encryption aísla el daño por diseño: el compromiso de una DEK solo expone un documento, y el compromiso de la KEK requiere además acceso al almacenamiento cifrado para descifrar las DEKs. Este es el patrón estándar de la industria, implementado por AWS KMS, Google Cloud KMS y Azure Key Vault [52].

El flujo de cifrado procede de la siguiente manera al almacenar un documento: (1) se genera una DEK aleatoria de 32 bytes con un CSPRNG; (2) se genera un nonce aleatorio de 12 bytes; (3) se cifra el documento con AES-256-GCM utilizando la DEK, el nonce y los AAD (`document_id + version`), obteniendo el ciphertext y un authentication tag de 16 bytes; (4) se genera un segundo nonce de 12 bytes para la envoltura; y (5) se cifra la DEK con la KEK usando AES-256-GCM. El paquete cifrado del documento se almacena en formato nonce (12 bytes) || ciphertext || tag (16 bytes), y la DEK envuelta sigue el mismo formato. El flujo de descifrado invierte el proceso: primero se descifra la DEK con la KEK verificando la integridad del tag, y después se descifra el documento con la DEK recuperada. Si cualquiera de los dos tags falla la verificación, la operación se rechaza íntegramente, lo que previene la entrega de documentos corrompidos o manipulados.

La rotación de la KEK es una operación ligera que no requiere re-cifrar ningún documento. Al generar una nueva KEK con un CSPRNG, se itera sobre las DEKs del repositorio: cada una se descifra con la KEK vigente y se re-cifra con la nueva. Dado que las DEKs son bloques de 32 bytes, esta operación es de tamaño fijo y se ejecuta en tiempo constante por documento, independientemente del tamaño del documento protegido. Los documentos en sí no se tocan: sus DEKs no cambian, solo cambia su envoltura. Esta separación es la ventaja principal de envelope encryption sobre un esquema de clave única: la rotación opera sobre claves (rápida, de tamaño fijo) y no sobre documentos (potencialmente de varios megabytes). Para el prototipo, basta con implementar y documentar el mecanismo de rotación; ejecutar una rotación periódica queda fuera del alcance de la demostración.

3.11.5 Autenticación y Control de Acceso

Las contraseñas de los usuarios se almacenan hasheadas con Argon2id (RFC 9106) [59], [60], la variante híbrida recomendada por OWASP [61] y NIST SP 800-63B Rev 4 [56] para nuevas implementaciones. Los parámetros adoptados son: memoria $m = 256$ MiB (262 144 KiB), iteraciones $t = 4$, paralelismo $p = 1$, sal de 16 bytes y hash de salida de 32 bytes. En el hardware de referencia (AMD Ryzen 7 7730U, 16 hilos lógicos) cinco corridas promediaron 607.8 ms, dentro de la banda objetivo de 0.5–1.0 segundo por intento legítimo. Si el hardware de despliegue difiere, la medición debe repetirse antes de la puesta en producción.

Los clientes del despacho cuentan adicionalmente con autenticación de doble factor mediante TOTP (RFC 6238) [19], [82] con HMAC-SHA1, códigos de 6 dígitos, período de 30 segundos y secreto de 20 bytes, alcanzando el nivel AAL2 de NIST SP 800-63B [56]. Se descartó WebAuthn/FIDO2 (AAL3) por la barrera de adopción que representa para clientes de un despacho legal que requiere solo un smartphone con una app autenticadora estándar. Los socios, por su parte, se autentican mediante certificados digitales X.509 emitidos por la CA interna (sección 3.11.2), lo que provee un nivel de aseguramiento equivalente a AAL3 sin hardware dedicado adicional.

La selección de Argon2id sobre bcrypt se fundamenta en la propiedad de dureza de memoria

(memory-hardness). bcrypt, diseñado en 1999, es resistente a fuerza bruta en CPU gracias a su costo iterativo, pero utiliza únicamente aproximadamente 4 KiB de memoria por invocación, lo que permite a un atacante con GPUs o ASICs ejecutar miles de instancias en paralelo a bajo costo por unidad de intento. Argon2id obliga a reservar 256 MiB por cada intento de hash en la configuración calibrada, lo que hace que escalar el ataque con hardware masivo sea prohibitivamente caro porque la memoria DRAM es el recurso que peor escala en paralelo. Argon2 fue el ganador de la Password Hashing Competition (PHC) en julio de 2015 [59], una competencia abierta de dos años con 24 candidatos evaluados por un panel de criptógrafos. Esta validación pública es análoga a la que AES recibió del concurso NIST en 2001 y aporta confianza en que el diseño ha sido ampliamente escrutado.

Argon2 define tres variantes que difieren en su resistencia a distintos vectores de ataque. Argon2d maximiza la resistencia a ataques con GPU y ASIC, pero es vulnerable a ataques de canal lateral si el atacante puede observar los patrones de acceso a memoria del proceso. Argon2i es resistente a ataques de canal lateral al usar un patrón de acceso independiente de los datos, pero ofrece ligeramente menos resistencia a hardware especializado. Argon2id es la variante híbrida: ejecuta Argon2i en el primer pase para proteger contra canal lateral, y Argon2d en los pases subsiguientes para maximizar la resistencia a GPU. RFC 9106 [60] §4 recomienda explícitamente Argon2id para hashing de contraseñas por esta combinación de protecciones. Los parámetros adoptados ($m = 256$ MiB, $t = 4$, $p = 1$, sal de 16 bytes, hash de 32 bytes) fueron seleccionados por calibración del hardware de referencia; la sal de 128 bits excede el mínimo de 64 bits de RFC 9106 y es el valor recomendado por OWASP.

La selección de HMAC-SHA1 como función base del TOTP merece una justificación explícita dado que SHA-1 está desaconsejado para firmas digitales desde 2017 por la demostración práctica de colisiones. Sin embargo, la seguridad de HMAC no depende de la resistencia a colisiones de la función hash subyacente, sino de la pseudoaleatoriedad de su función de compresión interna. NIST SP 800-107 Rev 1 [57] confirma que HMAC-SHA-1 sigue siendo aceptable para autenticación de mensajes y generación de códigos. Adicionalmente, HMAC-SHA1 es el único algoritmo soportado universalmente por todas las aplicaciones autenticadoras del mercado; Google Authenticator históricamente solo soporta SHA-1, lo que haría que la adopción de HMAC-SHA256 o HMAC-SHA512 excluyera a una parte significativa de los dispositivos de los clientes del despacho.

El flujo de registro (enrollment) del segundo factor procede de la siguiente manera: (1) el cliente inicia sesión con usuario y contraseña; (2) el sistema genera un secreto de 20 bytes con un CSPRNG; (3) se codifica como URI otpauth (otpauth://totp/Despacho:cliente@email?secret=BASE32(secret)) y se presenta como código QR; (4) el cliente escanea con su aplicación autenticadora; (5) el cliente ingresa el código actual para confirmar la configuración; y (6) el sistema almacena el secreto cifrado con AES-256-GCM y genera ocho códigos de recuperación de un solo uso, hashados con Argon2id antes de almacenarse. Durante la autenticación, el sistema calcula el TOTP esperado para tres ventanas temporales (t_{0-1} , t_0 , t_{0+1}), lo que tolera un desfase de reloj de hasta 30 segundos entre servidor y dispositivo. Se aplica rate limiting de cinco intentos fallidos consecutivos con bloqueo temporal de 15 minutos para prevenir fuerza bruta en línea sobre el espacio de 10^6 combinaciones (6 dígitos).

3.11.6 Postura Legal del Prototipo

La firma generada por el prototipo constituye una firma electrónica simple conforme al artículo 89 del Código de Comercio [13], dado que los certificados no provienen de un PSC autorizado (artículo 97, requisito 5). Su argumentación probatoria se apoya en: (1) la admisibilidad de la firma simple como prueba, ya que el artículo 89 no condiciona la admisibilidad a la intervención de un PSC; y (2) la cláusula de equivalencia funcional del artículo 100, que permite argumentar la fiabilidad

del método con base en estándares reconocidos internacionalmente (RSA-3072 conforme a NIST SP 800-57 [37] y SHA-256 conforme a FIPS 180-4 [25]). Un sello emitido por un PSC autorizado añadiría fecha cierta e independencia de tercero conforme al marco de la NOM-151; el token de la TSA interna demuestra el mecanismo RFC 3161, pero no añade esa garantía externa. La distinción práctica respecto de una firma avanzada radica en la carga de la prueba: quien invoca la firma simple debe acreditar la fiabilidad del método, mientras que la firma avanzada goza de presunción legal. En un despliegue productivo, la sustitución de la CA interna por un PSC autorizado convertiría la firma en avanzada sin cambios arquitectónicos, dado que el módulo de firma acepta cualquier certificado X.509 RSA + SHA-256.

La arquitectura del prototipo está diseñada para que la transición de firma simple a firma avanzada sea una operación de configuración, no de rediseño. El módulo de firma define una interfaz abstracta que acepta cualquier certificado X.509 con clave RSA y algoritmo sha256WithRSAEncryption. En un despliegue productivo, el despacho podría sustituir la CA interna por certificados emitidos por un PSC autorizado como SeguriData [45] o CECOBAN [46], convirtiendo la firma en avanzada conforme al artículo 97 del Código de Comercio [13]; o bien integrar la e.firma del SAT si los socios ya disponen de ella, sin modificar la lógica de verificación. De forma análoga, el módulo de sellado de tiempo define una interfaz abstracta que permite sustituir Cincel por cualquier TSA que implemente RFC 3161 [27]. Esta portabilidad del emisor es un acierto arquitectónico deliberado: el prototipo demuestra la corrección de los mecanismos criptográficos con una infraestructura soberana del despacho, mientras que la producción puede adoptar la infraestructura regulada sin tocar el código de verificación.

CAPÍTULO 4

Implementación del prototipo

Este capítulo documenta la implementación del prototipo cuyo análisis y diseño se presentó en el Capítulo 3. La materialización sigue fielmente las decisiones estructurales de ese capítulo: el sistema se construyó como un workspace de Cargo de cinco crates con la regla de dependencias de la arquitectura hexagonal impuesta por el grafo de compilación (sección 3.7.6), y cada uno de los servicios criptográficos especificados en el diseño criptográfico (sección 3.11) se materializó como un par puerto–adaptador: un trait en el crate domain y una struct concreta en el crate infrastructure, tal como anticipó la definición de la arquitectura hexagonal (sección 3.7.4). El capítulo se organiza en seis secciones. La primera describe la estructura real del workspace y el andamiaje de calidad que lo rodea: perfil de compilación, configuración, telemetría, integración continua y política de dependencias. La segunda recorre los diez servicios criptográficos implementados, identificando para cada uno su puerto, su adaptador, los crates empleadas y las decisiones técnicas registradas. La tercera presenta la interfaz de línea de comandos que expone los casos de uso. La cuarta documenta la aplicación HTTP y su persistencia local cifrada. La quinta reporta los resultados de la exploración del sellado de tiempo, el componente que el análisis de riesgos señaló como el de mayor exposición (sección 3.6.2). La sexta registra las divergencias respecto al diseño del Capítulo 3 y las decisiones de arquitectura documentadas durante la construcción.

4.1 Estructura del workspace y andamiaje

Actualización al cierre (17 de agosto de 2026). El crate web expone una rebanada vertical Axum: carga y persistencia cifrada de documentos, firma y sellado con la TSA local, verificación, exportación del paquete de evidencia y comprobación de la bitácora. Las sesiones, RBAC, PostgreSQL, Redis y la interfaz de usuario permanecen pendientes.

El prototipo es un workspace de Cargo con cinco crates miembros bajo el directorio crates/, exactamente los cinco que definió la estructura del workspace en el diseño (sección 3.7.6): domain, application, infrastructure, web y bin. La dirección de dependencias apunta siempre hacia el dominio: domain no depende de ningún otro crate del workspace; application depende solo de domain; infrastructure depende de domain y application; web puede depender únicamente de application y domain; y bin —la raíz de composición, que produce el binario despacho-cli— depende de todos los anteriores y es el único crate autorizado a usar anyhow para errores polimórficos en la frontera del proceso. El crate web adapta HTTP al puerto DocumentWorkflow, sin conocer los adaptadores criptográficos ni el repositorio concretos. La tabla 4.1 resume el tamaño de cada crate; ningún archivo del workspace supera las 400 líneas de código —regla de diseño registrada en CONTRIBUTING.md—, siendo uno de los mayores crates/infrastructure/tests/local_tsa_rfc3161.rs (381 líneas).

Tabla 4.1: Tamaño de los crates del workspace en líneas de código. Los tests unitarios viven embebidos en `src`; los de integración, en el directorio `tests/` de cada crate.

Crate	Responsabilidad	Líneas en <code>src</code>	Líneas en <code>tests/</code>
<code>domain</code>	Valores, puertos (traits) y errores; sin E/S ni crates criptográficos	2 625	—
<code>application</code>	Casos de uso genéricos sobre los puertos	2 969	1 694
<code>infrastructure</code>	Adaptadores driven concretos	3 770	2 570
<code>web</code>	Adaptador driving HTTP y DTO de contrato	367	212
<code>bin</code>	Raíz de composición, CLI y servidor HTTP	1 892	2 835

La regla de dependencias no descansa en la disciplina del equipo sino en dos mecanismos verificables. El primero es el propio grafo de Cargo: como se argumentó en el diseño ([sección 3.7.6](#)), una importación prohibida produce un error de compilación, no una advertencia de revisión. El segundo es una prueba de arquitectura, `crates/bin/tests/architecture.rs`, que lee el manifiesto del crate `domain` y falla si este declara dependencia alguna de `application`, `infrastructure` o `web`; esta prueba protege contra la degradación gradual del manifiesto, que el compilador no reportaría mientras los símbolos prohibidos no se usen. Una segunda prueba estructural, `crates/bin/tests/port_coverage.rs`, recorre el workspace y exige que cada `pub trait` del dominio —cada puerto— tenga al menos un adaptador (`impl <Trait> for`) en `infrastructure` y al menos un mock generado con `mockall` en los tests; para detectar un escáner degradado, la prueba usa puertos centinela (`AuditLog`, `DocumentHasher`, `KeyManager`) cuya ausencia delataría un análisis roto.

La versión mínima soportada del compilador es Rust 1.78, declarada con el campo `rust-version` en el `Cargo.toml` raíz y verificada en integración continua, no fijando el `toolchain` local. Esta MSRV condiciona el conjunto de crates criptográficos: como registra el ADR [docs/adr/0001-msrv-and-dependency-pinning.md](#), los crates de certificados y sellado se fijan a la familia construida alrededor de `der 0.7` —`x509-cert 0.2`, `cms 0.2`, `x509-tsp 0.1` y `rsa 0.9`—, porque las líneas mayores siguientes (`x509-cert 0.3`, `rsa 0.10`) exigen un compilador más nuevo; `ring` se fija a 0.17.12 o posterior, la primera versión que corrige el aviso RUSTSEC-2025-0009. Los pines viven en `[workspace.dependencies]` del manifiesto raíz, de modo que todos los crates miembros reciben el mismo conjunto compatible. El perfil de release aplica `strip = true` y `lto = true`, y los perfiles de desarrollo elevan `opt-level = 3` solo para los núcleos criptográficos (`argon2`, `blake2`, `num-bigint-dig`, `rsa`) para que la suite de pruebas siga siendo viable sin optimizar todo el árbol.

La configuración se carga desde variables de entorno con `dotenvy` (archivo `.env` local, nunca versionado; la plantilla comentada `.env.example` sí lo está). La [tabla 4.2](#) lista las variables. Dos decisiones de manejo de secretos son transversales: la carga de configuración registra en la bitácora únicamente la *presencia* de cada variable, jamás su valor; y todo material secreto leído del entorno o de la entrada estándar (la KEK, la credencial del proveedor de sellado, las contraseñas) aterriza en buffers `Zeroizing` que sobrescriben su contenido al liberarse, conforme al ADR [docs/adr/0003-zeroize-secret-material-in-domain.md](#).

Tabla 4.2: Variables de entorno del prototipo (plantilla `.env.example`).

Variable	Propósito
RUST_LOG	Filtro de severidad de la telemetría (por defecto <code>info</code>).
CINCEL_BASE_URL	URL base del proveedor remoto de sellado de tiempo.
CINCEL_API_KEY	Credencial del proveedor de sellado (secreto; se llena localmente).
DATABASE_URL	Cadena de conexión PostgreSQL, reservada para la capa de persistencia.
KEK_BASE64	Clave de cifrado de claves: 32 bytes aleatorios en base64 (secreto).
NEW_KEK_BASE64	KEK de reemplazo; la lee únicamente <code>vault rotate-kek</code> .
AUDIT_LOG_PATH	Ruta de la bitácora encadenada (por defecto <code>audit-log.jsonl</code>).
PKI_CA_DIR	Directorio de trabajo de la CA interna (por defecto <code>pki-ca</code>).
TSA_DIR	Directorio de trabajo de la TSA local (por defecto <code>pki-tsa</code>).

La telemetría usa `tracing` y `tracing-subscriber` con filtro por entorno, materializando la observabilidad estructurada prevista en el diseño ([sección 3.7.8](#)). Todos los diagnósticos se emiten por la salida de error estándar, de modo que la salida estándar queda limpia para resultados y tuberías de shell. La promesa de que ningún secreto aparece en los diagnósticos no es una convención: la prueba `crates/bin/tests/log_redaction.rs` ejecuta los flujos con el nivel de log más verboso e inspecciona lo emitido, fallando si algún material secreto conocido aparece en la salida de error.

La integración continua ejecuta siete jobs sobre cada push y cada pull request ([tabla 4.3](#)). El job de `MSRV` es hoy informativo (`continue-on-error`): su comentario en el workflow explica que, mientras el conjunto de dependencias no esté congelado, una dependencia transitiva que eleve su versión mínima de compilador debe hacerse visible sin bloquear la tubería, y que la marca debe retirarse cuando el conjunto se estabilice para volver vinculante la garantía. El umbral de cobertura lo impone `scripts/coverage-gate.sh`: 90 % de líneas por crate sobre `domain`, `application` e `infrastructure` —los crates con lógica criptográfica, en línea con el requerimiento RNF-13—, mientras que `bin` se reporta sin bloquear.

Tabla 4.3: Jobs del workflow de integración continua.

Job	Herramienta	Criterio de aprobación
<code>format</code>	<code>cargo fmt</code>	Formato canónico en todo el workspace.
<code>lint</code>	<code>cargo clippy</code>	Cero advertencias (<code>-D warnings</code>).
<code>test</code>	<code>cargo test</code>	Suite completa del workspace en verde.
<code>msrv</code>	<code>cargo check</code>	Compila con Rust 1.78 (informativo).
<code>coverage</code>	<code>cargo-llvm-cov</code>	≥ 90 % de líneas por crate criptográfico.
<code>deny</code>	<code>cargo-deny</code>	Avisos, licencias, duplicados y fuentes en regla.
<code>binary-size</code>	<code>cargo build --release</code>	<code>despacho-cli</code> ≤ 25 MB.

El diseño comprometió cinco fitness functions como verificaciones automatizadas de los atributos de calidad ([sección 3.7.11](#)) [68]. La [tabla 4.4](#) muestra dónde vive cada una en el repositorio. Cuatro de las cinco diseñadas se implementaron directamente; la función que exige un span de `tracing`

por handler HTTP sigue pendiente aun cuando los handlers ya existen, y en su lugar se incorporó el umbral de cobertura por crate como función de aptitud adicional.

Tabla 4.4: Fitness functions implementadas y su ubicación en el repositorio.

Id	Propiedad verificada	Verificación implementada
FF-1	El dominio no depende de capas externas	Grafo de Cargo en cada build y <code>crates/bin/tests/architecture.rs</code> .
FF-2	Ningún log contiene material secreto	<code>crates/bin/tests/log_redaction.rs</code> al nivel de log más verboso.
FF-3	Todo puerto tiene adaptador y mock	<code>crates/bin/tests/port_coverage.rs</code> con puertos centinela.
FF-4	Cobertura ≥ 90 % en crates criptográficos	<code>scripts/coverage-gate.sh</code> sobre el reporte de <code>cargo-llvm-cov</code> .
FF-5	Binario release ≤ 25 MB	Job <code>binary-size</code> del workflow de integración continua.

La política de dependencias se aplica con `cargo-deny` (configuración versionada en `deny.toml`) y `cargo-audit` (`audit.toml`): el grafo se analiza con todas las features activas; solo se admiten licencias permisivas de una lista explícita; las versiones duplicadas se reportan sin bloquear; y la única fuente de crates permitida es el índice de `crates.io`. Existe exactamente una excepción de seguridad declarada en el repositorio: el aviso RUSTSEC-2023-0071 [83], un canal lateral de temporización en las operaciones de clave privada del `crate rsa`, aceptado bajo el modelo de amenaza del prototipo —la firma corre localmente por línea de comandos y no existe un oráculo de temporización observable por red— con el razonamiento completo registrado en `docs/adr/0002-rsa-signing-crate-and-advisory.md`, al que apuntan de vuelta ambos archivos de configuración.

4.2 Servicios criptográficos

El diseño criptográfico del Capítulo 3 (sección 3.11) especificó los servicios de integridad, confidencialidad, no repudio y autenticación del prototipo. Esta sección los recorre como diez servicios implementados, S1 a S10, cada uno con la misma anatomía: un puerto (`trait`) en `domain`, un adaptador en `infrastructure` y un caso de uso en `application` que los orquesta. La tabla 4.5 presenta el mapa completo. Dos convenciones atraviesan todos los servicios. Primera: los rechazos criptográficos son *resultados* tipados (enumeraciones como `SignatureVerification` o `CertificateValidation`), nunca errores; el camino de error queda reservado a fallos del backend o a evidencia inevaluable, lo que permite construir políticas futuras —por ejemplo, bloqueo de cuenta— sin confundir un intento fallido con una falla del sistema. Segunda: la delegación a la herramienta `openssl` como subproceso ocurre en exactamente tres puntos —la CA interna, la TSA local y la etapa criptográfica del verificador RFC 3161—; todo lo demás es Rust nativo sobre `ring`, `rsa`, `sha2`, `x509-cert`, `cms`, `x509-tsp`, `argon2` y `totp-rs`.

El manejo de errores materializa el patrón de errores tipados por capa que el diseño estableció (sección 3.7.8). El crate de dominio define `DomainError` (`crates/domain/src/error.rs`)

con 26 variantes tipadas mediante `thiserror`, una por condición de fallo con significado propio—longitud de digest inválida, payload sellado malformado, CRL vencida, nombre de entrada de archivo inutilizable—. Una variante merece mención explícita: `AuthenticationFailed`, el rechazo de un descifrado autenticado, es deliberadamente opaca y no revela si falló la clave, el tag o los datos, negando a un adversario la retroalimentación que distingue esos casos. Los errores de los backends de infraestructura (`CryptoError`, `TsaError`) se convierten hacia el dominio mediante conversiones `From`, y la capa de aplicación añade en `ApplicationError` solo lo que le es propio: el fallo genérico de un puerto, un archivo vault malformado y un certificado recién emitido que no supera su validación posterior a la emisión.

Tabla 4.5: Servicios criptográficos implementados: puertos del dominio y adaptadores de infraestructura.

Id	Servicio	Puerto(s) del dominio	Adaptador(es)
S1	Hash de documentos	<code>DocumentHasher</code>	<code>RingSha256Hasher</code>
S2	Cifrado autenticado y envelope encryption	<code>AuthenticatedCipher</code> , <code>KeyManager</code>	<code>RingAesGcmCipher</code> , <code>EnvelopeKeyManager</code>
S3	PKI interna	<code>CertificateAuthority</code> , <code>CertificateValidator</code>	<code>OpensslCaAdapter</code> , <code>X509ChainValidator</code>
S4	Firma digital	<code>DocumentSigner</code> , <code>SignatureVerifier</code>	<code>RsaPkcs1Signer</code> , <code>RsaPkcs1Verifier</code>
S5	Sellado de tiempo	<code>TimestampService</code> , <code>TimestampVerifier</code>	<code>LocalOpensslTsa</code> , <code>CincelTsaAdapter</code> , <code>Rfc3161Verifier</code>
S6	Verificación integral	(compone los de S1, S3, S4 y S5)	caso de uso <code>VerifyDocument</code>
S7	Hash de contraseñas	<code>PasswordHasher</code>	<code>Argon2idHasher</code>
S8	Segundo factor TOTP	<code>TotpProvider</code> , <code>RecoveryCodeGenerator</code>	<code>TotpRsProvider</code> , <code>RandomRecoveryCodeGenerator</code>
S9	Cadena de auditoría	<code>AuditLog</code> , <code>Clock</code>	<code>FileAuditLog</code> , <code>InMemoryAuditLog</code> , <code>SystemClock</code>
S10	Paquete de evidencia	<code>ArchiveWriter</code>	<code>StoredZipWriter</code>

4.2.1 S1. Hash de documentos con SHA-256

El puerto `DocumentHasher` (`crates/domain/src/crypto/haser.rs`) es el más simple del sistema y sirve para ilustrar el patrón que repiten todos los demás: un `trait` del dominio que declara el contrato criptográfico sin nombrar backend alguno. El Listado 4.1 lo reproduce íntegro.

```
1 /// Computes SHA-256 digests of document content.
2 ///
```

```

3  /// Adapters implement this port with a concrete cryptographic backend. Both
4  /// operations must produce the same digest for the same input bytes.
5  pub trait DocumentHasher {
6      /// Hashes a byte slice already held in memory.
7      fn hash_bytes(&self, data: &[u8]) -> Sha256Digest;
8
9      /// Hashes a stream by consuming it in fixed-size blocks, so inputs of
10     /// any length are digested without loading them fully into memory.
11     ///
12     /// # Errors
13     ///
14     /// Returns [`DomainError::StreamRead`] when reading from `reader` fails.
15     fn hash_stream(&self, reader: &mut dyn Read) -> Result<Sha256Digest,
16     DomainError>;
17 }

```

Listado 4.1: Puerto de hashing del dominio (crates/domain/src/crypto/haser.rs).

El adaptador RingSha256Hasher (crates/infrastructure/src/hashing.rs) lo implementa con el crate ring 0.17. La variante de flujo consume la entrada en bloques de 64 KiB, de modo que el consumo de memoria es plano para entradas de cualquier tamaño, y reintenta las lecturas interrumpidas por señales del sistema operativo. El resultado es el value object Sha256Digest, un arreglo fijo de 32 bytes con constructores que rechazan longitudes o codificaciones hexadecimales inválidas. Siguiendo la práctica de desarrollo guiado por pruebas del proyecto, los primeros tests del adaptador son los vectores oficiales de FIPS 180-4 [25], complementados con una verificación de interoperabilidad contra sha256sum y con propiedades verificadas por generación aleatoria de casos. El caso de uso HashDocument expone el servicio a la interfaz de usuario.

4.2.2 S2. Cifrado autenticado y envelope encryption

La confidencialidad en reposo diseñada en [sección 3.11.4](#) se implementa con dos puertos. El primero, AuthenticatedCipher (crates/domain/src/crypto/cipher.rs), declara las operaciones de sellado y apertura autenticados; el material de clave cruza la interfaz como slices de bytes para que el crate de dominio permanezca libre de dependencias criptográficas, mientras los buffers propietarios viven en contenedores Zeroizing del lado del llamador. Su adaptador, RingAesGcmCipher (crates/infrastructure/src/encryption.rs), implementa AES-256-GCM con ring [50], [51]: en cada operación de sellado se extrae un nonce fresco de 96 bits del CSPRNG del sistema, y la salida adopta el formato fijo nonce (12 bytes) || criptograma || tag (16 bytes), representado por el value object SealedPayload, que rechaza cualquier secuencia menor a 28 bytes. Todo rechazo de descifrado se colapsa deliberadamente en el error opaco AuthenticationFailed, que no revela si falló la clave, el tag o los datos.

Los datos adicionales autenticados ligando cada criptograma a la identidad y versión del documento, tal como exigió el diseño para prevenir ataques de intercambio y de reversión. La función de dominio document_aad (Listado 4.2) construye ese vínculo: 16 bytes crudos del UUID del documento seguidos de la versión como 4 bytes big-endian.

```

1  /// Builds the additional authenticated data that binds a sealed document to
2  /// its identity: the 16 raw bytes of the document id UUID followed by the

```

```

3  /// document version as 4 big-endian bytes.
4  ///
5  /// Sealing a document with this AAD makes decryption fail unless the caller
6  /// asks for exactly the same document and version. That prevents
7  /// substituting one document's ciphertext for another's (the id would
8  /// differ) and replaying an older version of the same document (the version
9  /// would differ).
10 pub fn document_aad(id: DocumentId, version: DocumentVersion) -> [u8;
    DOCUMENT_AAD_LEN] {
11     let mut aad = [0u8; DOCUMENT_AAD_LEN];
12     aad[..16].copy_from_slice(id.as_uuid().as_bytes());
13     aad[16..].copy_from_slice(&version.get().to_be_bytes());
14     aad
15 }

```

Listado 4.2: Construcción del AAD que liga el criptograma a su documento (crates/domain/src/crypto/cipher.rs).

El segundo puerto, `KeyManager` (crates/domain/src/crypto/keys.rs), gobierna el esquema de envelope encryption [52]: generar una DEK, envolverla bajo la KEK, desenveloparla y re-envolverla bajo una KEK nueva. Su adaptador `EnvelopeKeyManager` genera una DEK aleatoria de 32 bytes por documento y la sella con el mismo cifrador AES-256-GCM, pero con AAD vacío: la decisión, documentada en el propio código, es que la DEK envuelta ya queda ligada a su documento por el archivo que la transporta, y mantener la envoltura independiente de la identidad permite rotar la KEK re-envolviendo todas las claves sin necesidad de saber a qué documento pertenece cada una. El caso de uso `EncryptDocument` serializa el resultado en el formato de archivo DVL1 (tabla 4.6), definido normativamente en crates/application/src/vault.rs; `DecryptDocument` reconstruye el AAD con la identidad y versión que el llamador afirma, de modo que un identificador o versión incorrectos fallan exactamente igual que un criptograma alterado; y `RotateKek` re-envuelve únicamente la DEK, copiando intactos los bytes del documento sellado, con un costo independiente del tamaño del documento —la propiedad de rotación que el diseño exigió en sección 3.11.4.

Tabla 4.6: Formato del archivo vault DVL1 (definición normativa en crates/application/src/vault.rs).

Desplazamiento	Tamaño	Campo
0	5	Bytes mágicos ASCII DVL1.
5	4	Longitud N (big-endian) de la DEK envuelta.
9	N	DEK envuelta: nonce criptograma tag, sellada bajo la KEK con AAD vacío.
$9 + N$	resto	Documento sellado: nonce criptograma tag, sellado bajo la DEK con AAD = identificador del documento (16 bytes UUID) versión (4 bytes big-endian).

4.2.3 S3. Infraestructura de clave pública interna

La CA interna de una sola capa especificada en [sección 3.11.2](#) se implementó como un conjunto de scripts versionados bajo `pki/` —`init-ca.sh`, `issue-cert.sh`, `revoke.sh`, `gen-crl.sh`— que operan la herramienta OpenSSL con configuración declarativa (`openssl.cnf`). Los parámetros coinciden con el diseño: claves RSA de 3 072 bits, firma `sha256WithRSAEncryption` en todos los casos, certificado raíz autofirmado con vigencia de 1 825 días (cinco años), certificados de entidad final con vigencia de 365 días y CRL con vigencia de siete días conforme a RFC 5280 [44]. La política de emisión obliga a que país y organización del certificado emitido coincidan con los de la CA, ignora las extensiones solicitadas en el CSR (`copy_extensions = none`) y aplica un retroceso deliberado de cinco minutos al inicio de vigencia —práctica común de las autoridades públicas— para que un certificado sea utilizable de inmediato aun con desfase de reloj entre máquinas. El puerto `CertificateAuthority` se implementa en `OpenSSLAdapter` (`crates/infrastructure/src/certificates/authority.rs`), que ejecuta esos scripts como subprocesos con el directorio de trabajo inyectado vía `PKI_CA_DIR` y normaliza los números de serie a hexadecimal en mayúsculas antes de invocar subproceso alguno. El caso de uso `IssueCertificate` rehúsa reportar éxito hasta validar el certificado recién emitido contra la raíz, de modo que una CA mal inicializada falla en la emisión y no en el primer uso. Los scripts operan con disciplina defensiva —`set -euo pipefail`, verificación previa de que `openssl` está disponible, inicialización idempotente que rehúsa sobrescribir una CA existente— y el material de claves privadas jamás se versiona: vive únicamente en el directorio de trabajo en tiempo de ejecución, con el `.gitignore` del repositorio como red de seguridad adicional; el `README.md` del directorio documenta el orden de uso, los comandos de verificación independiente y la versión de OpenSSL (3.5.7) con la que fueron probados.

La validación es el puerto `CertificateValidator`, implementado por `X509ChainValidator` (`crates/infrastructure/src/certificates/validator.rs`). Las crates `x509-cert 0.2` y `der 0.7` solo *parsean* X.509 y deliberadamente no verifican firmas, así que la verificación criptográfica se implementó re-codificando a DER la estructura *to-be-signed* parseada y comprobando la firma PKCS#1 v1.5 con `rsa` y `sha2` contra la clave pública del emisor —la misma técnica para la firma de la CRL sobre su `tbsCertList`—, decisión registrada en `docs/adr/0004-certificate-validation-implementation.md` y correcta porque DER es una codificación canónica: decodificar y re-codificar entrada DER válida reproduce los bytes firmados. El validador ejecuta sus comprobaciones en orden fijo y reporta exactamente el primer fallo: (1) confianza del emisor —nombre, algoritmo y firma—; (2) ventana de vigencia del certificado; (3) ventana de vigencia *del emisor*, porque un emisor fuera de su propia vigencia no ancla confianza en ese instante; y (4) revocación contra la CRL opcional. El contrato de revocación es estricto: si se presenta una CRL, cualquier defecto de la lista —ilegible, no firmada por el emisor, o vencida en su `nextUpdate`— es un error duro y nunca un pase permisivo; omitir la CRL es una decisión explícita del llamador que el reporte de verificación hace visible. El instante de evaluación siempre es un timestamp explícito, lo que permite a las pruebas controlar el reloj.

4.2.4 S4. Firma digital RSA-3072

El esquema de firma es el diseñado en [sección 3.11.2](#): RSA-3072 con SHA-256 y padding PKCS#1 v1.5. Los puertos son `DocumentSigner` y `SignatureVerifier` (`crates/domain/src/crypto/signature.rs`); nunca se firma el documento sino su digest SHA-256, calculado por el puer-

to de hashing, y el rechazo de una verificación es un resultado tipado, no un error. Los adaptadores `RsaPkcs1Signer` y `RsaPkcs1Verifier` (`crates/infrastructure/src/signing.rs`) usan los crates `rsa 0.9` y `sha2 0.10` con la feature `oid`, que liga SHA-256 a su identificador en la estructura `DigestInfo` del padding, de modo que los 32 bytes ya calculados se firman sin hashear dos veces. El firmador rechaza en construcción cualquier módulo menor a 3 072 bits; acepta la clave privada en PEM PKCS#8 o PKCS#1, la retiene en un buffer `Zeroizing`, la parsea únicamente durante la operación de firma y la descarta al salir del ámbito, y su representación de depuración omite el material (`private key pem withheld`). El verificador acepta como material de clave tanto un certificado X.509 —del que extrae el `SubjectPublicKeyInfo`— como una clave pública SPKI en PEM; una firma cuya longitud no coincide con el tamaño de la clave se rechaza como malformada, y cualquier comprobación fallida se colapsa en `MismatchedDocumentOrKey`, porque un fallo de verificación no distingue un documento alterado de una clave ajena. La elección del crate `rsa` —con el aviso RUSTSEC-2023-0071 aceptado y un plan de contingencia hacia `ring` si el riesgo se volviera inaceptable— está registrada en `docs/adr/0002-rsa-signing-crate-and-advisory.md`.

4.2.5 S5. Sellado de tiempo RFC 3161

El sellado de tiempo conserva el contrato del diseño (sección 3.11.3): el puerto `TimestampService` recibe un digest SHA-256 y devuelve el token como bytes DER opacos, y el puerto `TimestampVerifier` verifica un token contra el digest esperado y un ancla de confianza en PEM [27]. Existen dos adaptadores emisores, intercambiables sin tocar el dominio. `LocalOpensslTsa` (`crates/infrastructure/src/timestamp/local.rs`) es una autoridad de sellado local de desarrollo y demostración que ejecuta `openssl ts -query` y `openssl ts -reply` como subprocessos contra el directorio de trabajo preparado por `pki/issue-tsa-cert.sh`; el paso de respuesta se serializa entre hilos y procesos con un lock advisory exclusivo sobre el archivo `serial.lock` (crate `fs2`), porque OpenSSL avanza el archivo de serie con una secuencia leer-incrementar-escribir no atómica que, sin serialización, acuñaría números de serie duplicados. `CinzelTsaAdapter` (`crates/infrastructure/src/timestamp/cinzel.rs`) es el camino remoto hacia el sandbox del PSC [81], implementado con `reqwest 0.12` en modo bloqueante sobre `rustls`; envía el digest por POST a `/api/v1/timestamps` con la credencial en el header `X-Api-Key`, maneja las respuestas de token inmediato, procesamiento diferido con sondeo (`RetryPolicy` configurable) y rechazo, y mantiene la credencial en un buffer `Zeroizing` excluido de la depuración y de todo mensaje de error. Los resultados de la exploración con ambos emisores se reportan en sección 4.5.

La verificación, `Rfc3161Verifier` (`crates/infrastructure/src/timestamp/verify.rs`), opera en dos etapas conforme a `docs/adr/0005-rfc3161-verification-strategy.md`. La etapa nativa decodifica el DER con `cms 0.2` y `x509-tsp 0.1` —aceptando una respuesta `TimeStampResp` completa o un token CMS desnudo—, extrae el `TSTInfo`, compara el message imprint contra el digest esperado exigiendo tanto el OID `id-sha256` (2.16.840.1.101.3.4.2.1) como la igualdad de bytes, y lee el tiempo de generación en formato RFC 3339; las entradas indecodificables y los desajustes de imprint se diagnostican aquí, con causas precisas y sin lanzar subprocesso alguno. La etapa delegada encarga la comprobación de la firma CMS y de la cadena de certificados a `openssl ts -verify`, montando los archivos en un directorio temporal privado (modo 0700, nombre impredecible) que se elimina en todo camino de retorno; la aceptación produce `Valid` con el instante de generación, y un rechazo produce `UntrustedToken` con el diagnóstico de la herramienta. Implementar a mano la verificación CMS —atributos firmados, comprobaciones de tipo de contenido, construcción de cadena— habría añadido una superficie criptográfica sustancial que el prototipo no debe poseer,

mientras que la dependencia de la herramienta `openssl` ya existía para la CA interna.

4.2.6 S6. Verificación integral

El caso de uso `VerifyDocument` (`crates/application/src/verification.rs`) compone cuatro puertos —`DocumentHasher`, `SignatureVerifier`, `CertificateValidator` y `TimestampVerifier`— en un único reporte sobre un flujo de entrada, materializando el flujo de verificación en cuatro pasos que el diseño describió en [sección 3.11.3](#). El reporte contiene cuatro componentes (integridad, firma, certificado y sello de tiempo), cada uno con estado `Passed`, `Failed` o `Skipped` y una causa legible. La semántica es deliberadamente conservadora: los componentes cuya evidencia no se aportó se reportan omitidos *con su razón*, nunca desaparecen en silencio, y no cuentan contra el veredicto; el veredicto es `NotValid` si cualquier componente ejercitado falló ([Listado 4.3](#)). La integridad compara el digest recomputado contra el que atestigua el token de sellado —sin token no existe referencia independiente y el componente se omite—, y el verificador revisa el imprint *antes* que la cadena de confianza, de modo que un token no confiable aún puede confirmar que el digest coincide (integridad aprobada con sello fallido). El estado del certificado habla únicamente del instante de evaluación: que esté hoy expirado o revocado no invalida por sí mismo una firma producida cuando era válido, y el texto del reporte lo hace explícito, dejando la lectura jurídica al operador. Los errores —flujo ilegible, PEM imparseable, CRL no confiable o vencida— no son resultados: abortan la verificación.

```

1  let components = [&integrity, &signature, &certificate, &timestamp];
2  let verdict = if components
3      .iter()
4      .any(|component| component.status == ComponentStatus::Failed)
5      {
6          Verdict::NotValid
7      } else {
8          Verdict::Valid
9      };
10
11  Ok(VerificationReport {
12      document_digest_hex: digest.to_hex(),
13      integrity,
14      signature,
15      certificate,
16      timestamp,
17      verdict,
18  })
19 }
```

Listado 4.3: Veredicto de la verificación integral sobre los cuatro componentes (`crates/application/src/verification.rs`).

4.2.7 S7. Hash de contraseñas con Argon2id

El puerto `PasswordHasher` (`crates/domain/src/crypto/password.rs`) declara el contrato del almacenamiento de contraseñas: cada hash usa una sal aleatoria fresca y produce una cadena en formato

PHC, y una cadena almacenada ilegible es un error (`MalformedPasswordHash`) que jamás se confunde con un intento de acceso fallido. El adaptador `Argon2idHasher` (`crates/infrastructure/src/password.rs`) lo implementa con el crate `argon2 0.5` y sales de `getrandom`, con los parámetros calibrados que fijó el diseño en [sección 3.11.5](#) declarados como constantes ([Listado 4.4](#)): 262 144 KiB de memoria (256 MiB), cuatro iteraciones, un carril, sal de 16 bytes y salida de 32 bytes. El PHC resultante inicia con `$argon2id$v=19$` y contiene `m=262144, t=4, p=1`, propiedad fijada por prueba. En la demostración de cierre, el hardware de referencia promedió 607.8 ms sobre cinco corridas, dentro de la banda objetivo inclusiva de 500 a 1 000 milisegundos, y emite un veredicto `BelowBand`, `InBand` o `AboveBand`. Si el hardware de despliegue difiere, la medición debe repetirse.

```

1  /// Memory cost in KiB (256 MiB), calibrated on the reference host.
2  const MEMORY_KIB: u32 = 262144;
3
4  /// Number of passes over the memory.
5  const ITERATIONS: u32 = 4;
6
7  /// Degree of parallelism (lanes).
8  const PARALLELISM: u32 = 1;
9
10 /// Length in bytes of the derived hash.
11 const OUTPUT_LEN: usize = 32;
12
13 /// Length in bytes of the random salt drawn for every hash.
14 const SALT_LEN: usize = 16;

```

Listado 4.4: Parámetros de Argon2id declarados como constantes (`crates/infrastructure/src/password.rs`).

4.2.8 S8. Segundo factor TOTP y códigos de recuperación

El puerto `TotpProvider` (`crates/domain/src/crypto/totp.rs`) declara el enrolamiento y la verificación del segundo factor; su adaptador `TotpRsProvider` (`crates/infrastructure/src/totp.rs`) usa el crate `totp-rs 5` con las features `otpauth` y `zeroize`, configurada con los parámetros del diseño ([sección 3.11.5](#)): HMAC-SHA1, códigos de seis dígitos, paso de 30 segundos, desfase aceptado de un paso a cada lado (ventana efectiva de unos 90 segundos) y secreto de 20 bytes —igual a la salida de HMAC-SHA1, como recomienda RFC 4226 [\[58\]](#)— con un mínimo aceptado de 16 bytes [\[19\]](#). El URI de aprovisionamiento `otpauth://` embebe el secreto codificado en base32 sin relleno [\[84\]](#); por ello, el value object `TotpEnrollment` guarda sus tres campos —secreto crudo, secreto en base32 y URI— en buffers `Zeroizing`. La comparación de códigos ya es de tiempo constante dentro de `totp-rs`, por lo que no se añade otra capa. Una limitación quedó registrada de forma explícita: la verificación es sin estado, así que un código ya aceptado sigue siendo aceptable durante el resto de su ventana; la exigencia de un solo uso de RFC 6238 pertenece al llamador del puerto cuando exista un almacén persistente por cuenta, decisión razonada en `docs/adr/0008-totp-single-use-enforcement.md` —simular la protección con estado en memoria daría la apariencia de defensa contra reproducción sin su sustancia.

El enrolamiento completo lo orquesta el caso de uso `EnrollTotp`, que además emite los ocho códigos de recuperación de un solo uso previstos por el diseño. El generador

RandomRecoveryCodeGenerator produce códigos de diez símbolos en la forma XXXXX-XXXXX sobre un alfabeto de 32 símbolos que excluye los caracteres confundibles I, L, O y U; cada símbolo aporta cinco bits extraídos sin sesgo de módulo, para un total de 50 bits de entropía por código. Los códigos en claro se muestran una única vez (viajan como Zeroizing); lo que se almacena es el value object RecoveryCodeSet: ocho ranuras con hashes PHC producidos por el mismo puerto PasswordHasher de S7, cuya operación consume verifica un código contra las ranuras no usadas y consume la que coincide.

4.2.9 S9. Cadena de auditoría

La bitácora de auditoría es lógica de dominio pura (crates/domain/src/audit/). Cada evento registra secuencia, timestamp, actor, acción y recurso, y la cadena se define por la regla

$$chain(n) = \text{SHA-256}(chain(n-1) \parallel \text{bytes canónicos}(n)),$$

con un valor génesis fijo de 32 bytes cero para la primera entrada. La codificación canónica serializa la secuencia como 8 bytes big-endian y cada campo variable con un prefijo de longitud de 4 bytes big-endian, de modo que dos eventos distintos jamás producen los mismos bytes; el timestamp se normaliza siempre a UTC y su forma RFC 3339 es la única representación textual que cubre la cadena. El Listado 4.5 muestra el cálculo del eslabón. El hashing pasa por el puerto DocumentHasher, así que ningún backend criptográfico entra al crate de dominio.

```

1 pub const GENESIS_PREVIOUS: Sha256Digest = Sha256Digest::from_array([0u8;
   SHA256_LEN]);
2
3 /// Computes the chain value of an event given the previous chain value.
4 ///
5 /// # Errors
6 ///
7 /// Propagates the canonical-encoding failures of
8 /// [AuditEvent::canonical_bytes`].
9 pub fn chain_digest(
10     hasher: &dyn DocumentHasher,
11     previous: &Sha256Digest,
12     event: &AuditEvent,
13 ) -> Result<Sha256Digest, DomainError> {
14     let canonical = event.canonical_bytes()?;
15     let mut linked = Vec::with_capacity(SHA256_LEN + canonical.len());
16     linked.extend_from_slice(previous.as_bytes());
17     linked.extend_from_slice(&canonical);
18     Ok(hasher.hash_bytes(&linked))
19 }
```

Listado 4.5: Regla de encadenamiento de la bitácora (crates/domain/src/audit/chain.rs).

La verificación recomputa cada eslabón desde el génesis y devuelve la cadena como válida o rota, señalando el índice del primer eslabón que no coincide. Esta construcción detecta la alteración de cualquier campo, la inserción de un evento forjado y el borrado o reordenamiento en cualquier posición interior. Su garantía tiene un límite documentado y fijado por prueba: el truncamiento de la cola *no* se

detecta, porque todo prefijo de una cadena válida es a su vez una cadena válida desde el mismo génesis; detectarlo exige anclar el valor de cadena más reciente fuera del log, responsabilidad que se delega deliberadamente a la capa de persistencia conforme a docs/adr/0007-audit-chain-anchoring.md. El adaptador persistente, `FileAuditLog` (crates/infrastructure/src/audit/file.rs), almacena un objeto JSON por línea con los campos del evento y el valor de cadena en hexadecimal; el `append` lee solo la última línea para enlazar y escribe en modo de anexo, con la secuencia leer-luego-escribir protegida por un lock advisory exclusivo (`fs2`) sobre un archivo hermano, de modo que escritores concurrentes —hilos o procesos— se serializan en lugar de bifurcar la cadena. Un adaptador en memoria, `InMemoryAuditLog`, aplica la misma regla para pruebas y procesos efímeros, y el puerto `Clock` (`SystemClock` en infraestructura) suministra el tiempo, lo que permite a las pruebas controlarlo.

4.2.10 S10. Paquete de evidencia

El caso de uso `ExportEvidencePackage` (crates/application/src/evidence.rs) construye el paquete autocontenido que el diseño prometió en sección 3.11.3: un archivo ZIP con el documento, su firma separada (`<doc>.sig`), su sello de tiempo (`<doc>.tsr`), el certificado del firmante (`certificado.pem`), la raíz emisora (`ca.pem`), la CRL vigente (`crl.pem`), opcionalmente la cadena de la autoridad de sellado (`tsha-chain.pem`) y un archivo `INSTRUCCIONES.md` generado desde una plantilla versionada, que documenta la verificación independiente completa en cuatro pasos con comandos literales de `openssl` —incluida la versión local de la herramienta con que se redactaron—. La regla del ancla del sello es explícita: si `tsha-chain.pem` viaja en el paquete, la comprobación del token se ancla en esa cadena; en caso contrario, en `ca.pem`, pues la autoridad interna de sellado encadena a la misma raíz.

Los cuatro pasos de las instrucciones reproducen, con herramientas ajenas al prototipo, la verificación integral de S6:

- **Integridad:** `openssl dgst -sha256` sobre el documento, comparado carácter por carácter con el resumen registrado en las propias instrucciones.
- **Firma:** extracción de la clave pública del certificado del firmante (`openssl x509 -pubkey`) y verificación de la firma separada con `openssl dgst -sha256 -verify`.
- **Certificado:** concatenación de `ca.pem` y `crl.pem`, seguida de `openssl verify -crl_check`, con la advertencia de que la CRL refleja el estado al momento de exportar y de que un certificado hoy expirado o revocado no invalida por sí solo una firma producida cuando era válido.
- **Sello de tiempo:** `openssl ts -verify` contra el ancla resuelta por la regla anterior, con la inspección del token disponible vía `openssl ts -reply -in <sello>-text`.

El puerto `ArchiveWriter` lo implementa `StoredZipWriter` (crates/infrastructure/src/archive.rs), una implementación propia del subconjunto mínimo del formato ZIP [85]: entradas almacenadas sin compresión (método 0), un encabezado local por entrada, directorio central y registro de fin de directorio, con CRC-32 calculado bit a bit (polinomio reflejado `0xEDB88320`, verificado contra el valor de comprobación estándar `0xCB43926`). La salida es determinista —orden de entradas del llamador y timestamps fijos a la época DOS—, de modo que archivar dos veces el mismo contenido produce bytes idénticos. La alternativa de depender del `crate zip` se descartó en docs/adr/0006-evidence-package-format.md: su línea vigente exige un compilador más

nuevo que la MSRV y su build por defecto arrastra backends de compresión y criptografía innecesarios, mientras que aquí la compresión no aporta nada —los miembros pesan pocos kilobytes y la fidelidad byte a byte importa más que el tamaño—. La seguridad de extracción se garantiza en el dominio: el value object `ArchiveEntry` restringe los nombres a un subconjunto ASCII conservador sin separadores de ruta, así que un archivo extraído jamás escribe fuera de su directorio destino.

4.3 La interfaz de línea de comandos

El primer adaptador driving entregado por el prototipo es la interfaz de línea de comandos `despacho-cli`, construida con `clap 4` en modo `derive` (`crates/bin/src/cli.rs`). El diseño anticipó un adaptador de línea de comandos para tareas administrativas junto a los adaptadores REST y WebSocket (sección 3.7.5); el orden de construcción privilegió la CLI porque permitió ejercitar primero los servicios criptográficos de extremo a extremo y después reutilizarlos desde HTTP. La bandera global `--json` conmuta las operaciones administrativas a un objeto JSON legible por máquina. La [tabla 4.7](#) presenta el catálogo completo de subcomandos con sus banderas reales, y la [tabla 4.8](#) el cableado de cada grupo hacia los casos de uso de `application` y los adaptadores concretos que la raíz de composición inyecta.

Con `--json`, cada handler imprime exactamente un objeto JSON en la salida estándar —por ejemplo, `crypto hash` emite `{algorithm, digest, file}` y `verify` emite el reporte completo con los cuatro componentes y el veredicto—, mientras los diagnósticos van siempre a la salida de error. Los códigos de salida están pensados para que los scripts bifurquen directamente: el proceso termina con código distinto de cero cuando `verify` produce un veredicto no válido, cuando `auth verify-password` reporta una contraseña que no coincide, cuando `auth totp verify` rechaza el código, cuando `audit verify-chain` encuentra la cadena rota —imprimiendo antes el JSON de fallo con el índice del primer eslabón roto— y cuando `package export` rehúsa exportar. Los subcomandos de lectura de la bitácora (`verify-chain`, `show`) rehúsan operar sobre un archivo inexistente: reportar una ruta equivocada o un log borrado como una cadena vacía válida frustraría el propósito de la verificación.

La raíz de composición (`crates/bin/src/main.rs`) es el único lugar del sistema donde los adaptadores concretos se instancian e inyectan en los casos de uso, y el único crate que usa `anyhow`, tal como estableció el patrón de inyección de dependencias por composición del diseño (sección 3.7.8). Ahí —y no en los adaptadores, que quedan libres de acceso al entorno— se leen los secretos: la KEK de KEK_BASE64 a un buffer `Zeroizing`, la clave privada del firmante desde su archivo PEM, la credencial del proveedor de sellado. La raíz concentra también salvaguardas operativas que no pertenecen a ningún servicio individual: `sign` verifica la firma recién producida contra el certificado *antes* de escribir `<FILE>.sig`, de modo que una clave que no corresponde al certificado falla ahí y no produce una firma inverificable; `timestamp` relee el token recibido y comprueba que cubra el digest calculado antes de escribir el archivo `.tsr`; `vault rotate-kek` escribe a un archivo temporal y renombra encima, de modo que un fallo a la mitad deja el original intacto; y `package export` somete los artefactos al mismo caso de uso `VerifyDocument` de la verificación integral —CRL y token incluidos— y rehúsa exportar si el veredicto no es válido, nombrando cada componente fallido, porque un paquete que fallaría sus propias instrucciones de verificación no debe existir.

La integración de todos los servicios se demuestra con el guion `scripts/demo.sh`, que compila el binario y recorre el ciclo completo de evidencia en un directorio temporal: creación de la autoridad interna, del firmante y de la autoridad de sellado; hash del documento de muestra; cifrado

Tabla 4.7: Catálogo de subcomandos de `despacho-cli`. La bandera global `--json` aplica a todos.

Comando y banderas	Efecto
<code>serve --signer-cert <CERT>--signer-key <KEY>[--bind <ADDR>] [--data-dir <DIR>]</code>	Inicia la aplicación HTTP local con persistencia cifrada y TSA OpenSSL.
<code>crypto hash <FILE></code>	Resumen SHA-256 del archivo.
<code>vault encrypt <FILE>--doc-id <UUID>[--version <N>]</code>	Cifra con envelope encryption; escribe <FILE>.enc.
<code>vault decrypt <PACKAGE>--doc-id <UUID>[--version <N>] [--out <PATH>]</code>	Descifra un archivo DVLT1; sin <code>--out</code> escribe el claro a la salida estándar.
<code>vault rotate-kek --file <PATH></code>	Re-envuelve la DEK bajo la KEK de <code>NEW_KEK_BASE64</code> .
<code>pki [--scripts-dir <DIR>] init-ca</code>	Crea la CA raíz interna (idempotente).
<code>pki issue --cn <CN></code>	Emite un certificado de entidad final.
<code>pki revoke --serial <HEX></code>	Revoca por número de serie.
<code>pki gen-crl</code>	Genera la CRL en PEM.
<code>pki show <CERT></code>	Inspecciona sujeto, emisor, serie y vigencia.
<code>sign <FILE>--cert <PATH>--key <PATH></code>	Firma el digest del archivo; escribe <FILE>.sig.
<code>timestamp <INPUT>[--mock] [--out <PATH>] [--pki-dir <DIR>]</code>	Solicita un sello RFC 3161; escribe <INPUT>.tsr. Con <code>--mock</code> usa la TSA local; sin ella, el proveedor remoto.
<code>verify <FILE>--sig <PATH>--cert <PATH>--ca <PATH>[--tsr] [--crl] [--tsa-cert] [--at <RFC3339>]</code>	Verificación integral de cuatro componentes; <code>--at</code> evalúa el certificado a un instante dado.
<code>package export <FILE>--sig --tsr --cert --ca --crl [--tsa-chain] --out <PATH></code>	Exporta el paquete de evidencia, previa verificación de los artefactos.
<code>auth calibrate</code>	Mide el costo real de Argon2id contra la banda de 500–1 000 ms.
<code>auth hash-password</code>	Hash PHC de una contraseña leída de la entrada estándar.
<code>auth verify-password --hash <PHC></code>	Verifica una contraseña contra un PHC almacenado.
<code>auth totp enroll --user <EMAIL>--secret-out <PATH></code>	Enrola el segundo factor y emite los códigos de recuperación.
<code>auth totp verify --code <CODE>--secret-file <PATH></code>	Verifica un código TOTP.
<code>audit append --action <STRING>--resource <STRING></code>	Anexa un evento encadenado a la bitácora.
<code>audit verify-chain</code>	Verifica la cadena completa desde el génesis.
<code>audit show</code>	Lista los eventos en orden de inserción.

Tabla 4.8: Correspondencia entre comandos, casos de uso de application y adaptadores inyectados por la raíz de composición.

Comando	Caso(s) de uso	Adaptadores inyectados
serve	LocalDocumentWorkflow	repositorio JSON local, bitácora, cifrado, firma, validación, TSA local y ZIP
crypto hash	HashDocument	RingSha256Hasher
vault *	EncryptDocument, DecryptDocument, RotateKek	RingAesGcmCipher, EnvelopeKeyManager
pki *	InitializeCa, IssueCertificate, RevokeCertificate, GenerateCrl, InspectCertificate	OpensslCaAdapter, X509ChainValidator
sign	SignDocument + VerifySignature	RsaPkcs1Signer, RsaPkcs1Verifier
timestamp	TimestampDocument	LocalOpensslTsa o CincelTsaAdapter
verify	VerifyDocument	RingSha256Hasher, RsaPkcs1Verifier, X509ChainValidator, Rfc3161Verifier
package export	VerifyDocument + ExportEvidencePackage	los cuatro anteriores más StoredZipWriter
auth calibrate, hash-password, verify-password	CalibratePassword Hashing, HashPassword, VerifyPassword	Argon2idHasher
auth totp *	EnrollTotp, VerifyTotp	TotpRsProvider, RandomRecoveryCodeGenerator, Argon2idHasher
audit *	AppendAuditEvent, VerifyAuditChain, ShowAuditLog	FileAuditLog, RingSha256Hasher, SystemClock

en reposo con corrupción deliberada de un byte y observación del rechazo; firma y obtención del sello de tiempo; verificación integral con los cuatro componentes válidos; alteración de una copia del documento con el consecuente fallo del componente de firma; exportación del paquete de evidencia y su verificación con solo `openssl` y `unzip`; revocación del certificado y su detección; calibración de autenticación y códigos de un solo uso; y bitácora encadenada con anexo, verificación, manipulación y nueva verificación. La transcripción íntegra de una ejecución real, capturada el 12 de julio de 2026 sobre OpenSSL 3.5.7, está versionada en `docs/demo-transcript.md`. Una segunda demostración, `scripts/api-demo.sh`, recorre el mismo núcleo a través de HTTP y se describe en la sección siguiente. El entorno, los comandos y el resumen de ambos recorridos quedaron asentados en `docs/verification-report.md`.

4.4 Aplicación HTTP y persistencia local

La superficie HTTP se construyó como un adaptador driving en `crates/web`: sus handlers convierten cabeceras, UUID, cuerpos binarios y respuestas JSON, pero solo conocen el puerto Document Workflow de `application`. La implementación `LocalDocumentWorkflow` orquesta los puertos ya probados de hash, cifrado, envoltura de claves, firma, validación X.509, sellado RFC 3161, verificación, archivo ZIP, reloj y auditoría. La raíz de composición del comando `serve` selecciona explícitamente `LocalOpenSSLtsa`; por ello, el camino de ejecución de la aplicación no lee la configuración de Cincel ni puede degradarse silenciosamente hacia un proveedor externo.

El flujo comienza con `POST /api/v1/documents`. El cuerpo es el documento binario, limitado a 16 MiB; las cabeceras `X-Document-Name` y `X-Actor` aportan el nombre seguro para archivo y la etiqueta de auditoría. La aplicación crea un UUID y la versión uno, calcula SHA-256, cifra mediante el formato DVL1 ligado al par identidad–versión y persiste el registro. `X-Actor` no es una credencial ni una autorización: únicamente atribuye el evento, por lo que la escucha predeterminada se limita a `127.0.0.1` y la exposición multiusuario queda condicionada a sesiones y RBAC.

El repositorio `FileDocumentRepository` escribe un JSON por UUID bajo `runtime-data/documents`; los campos binarios se codifican en base64 y el contenido documental se conserva exclusivamente dentro del paquete cifrado. Cada actualización se prepara en un archivo temporal y se publica por reemplazo atómico, mientras un lock advisory evita carreras entre escritores. La bitácora encadenada se mantiene en `runtime-data/audit.jsonl`. Esta persistencia permite demostrar reinicio y consistencia local sin afirmar equivalencia con PostgreSQL y Redis, que siguen siendo los adaptadores previstos para producción.

Tabla 4.9: Contrato de la aplicación HTTP local.

Ruta	Responsabilidad
GET /healthz	Diagnóstico del proceso.
POST /api/v1/documents	Carga, cifra y persiste la versión inicial; responde 201.
POST /api/v1/documents/<id>/seal	Firma el digest, obtiene un token de la TSA local, lo verifica y almacena evidencia inmutable.
POST /api/v1/documents/<id>/verify	Descifra y reporta integridad, firma, certificado/CRL, sello y veredicto.
GET /api/v1/documents/<id>/evidence	Exporta el ZIP interoperable tras una verificación satisfactoria.
GET /api/v1/audit/verify	Verifica el encadenamiento y reporta el primer índice roto.

Sellar es una transición de una sola vez: antes de guardar, el workflow descifra el paquete, comprueba que coincide con el digest almacenado, firma, verifica que la llave corresponde al certificado, solicita el token local y lo verifica contra la cadena de la TSA. Verificar reutiliza el reporte de cuatro componentes del caso de uso existente. Exportar vuelve a verificar y materializa el texto claro solamente en la respuesta ZIP, junto con firma, certificado, CA, CRL, token y cadena de la TSA. Los errores se traducen a una envoltura JSON estable: entrada inválida en 400 o 422, ausencia en 404, transición incompatible en 409 y fallos internos redactados en 500.

El guion `scripts/api-demo.sh` genera una PKI temporal, desactiva expresamente las variables de Cincel, inicia `serve` sobre un puerto efímero y recorre carga, comprobación de ausencia del texto claro en el repositorio, sellado, verificación, descarga del ZIP y auditoría. Después extrae la evidencia y verifica con herramientas ajenas a la aplicación la igualdad del documento, la firma, el certificado con CRL y el sello RFC 3161. El contrato operativo completo se entrega en `docs/http-api.md` y la decisión arquitectónica en `docs/adr/0011-local-document-workflow.md`.

4.5 Resultados de la exploración del sellado de tiempo

El análisis de riesgos identificó la integración con la API del PSC como el riesgo técnico de mayor exposición (RTC-01, [tabla 3.61](#)), y previó como contingencia un servicio sustituto que simulara las respuestas de la TSA para no bloquear el desarrollo. El registro en el entorno de Cincel pudo completarse, pero el riesgo se materializó durante el consumo: la API no mostró estabilidad suficiente para sostener una campaña repetible y el sandbox requiere pago, por lo que no es una dependencia determinista ni económicamente neutra para la suite o la demostración ([sección 3.6.6](#)). La respuesta de implementación fue más fuerte que la contingencia prevista: en lugar de un simulador, se construyó una *autoridad de sellado real* operada localmente con OpenSSL, que emite tokens RFC 3161 auténticos, criptográficamente completos y verificables con herramientas independientes.

La autoridad local se prepara con el script `pki/issue-tsa-cert.sh`: la CA interna emite un certificado de TSA RSA-3072 con vigencia de 365 días cuya extensión `extendedKeyUsage` es crítica y contiene únicamente `timeStamping`, como exige RFC 3161 [27]. La configuración `pki/tsa.cnf` fija los parámetros de operación: digests de solicitud y de firma SHA-256, exactitud declarada de un segundo (`accuracy = secs:1`), sin garantía de orden entre tokens, números de serie desde

1000, y una política de emisión identificada por el OID 1.3.6.1.4.1.13762.3 —un arco privado del árbol de empresas de IANA, declarado explícitamente como política de demostración y no como política de producción registrada; las solicitudes que pidan cualquier otra política se rehúsan—. El adaptador `LocalOpenSSLTsa` ejerce esta autoridad vía el comando `timestamp --mock`, y las pruebas de integración `crates/infrastructure/tests/local_tsa_rfc3161.rs` recorren el ciclo completo contra los scripts reales en directorios temporales, cotejando los tokens emitidos con la línea de comandos de `openssl`; ninguna de esas pruebas está marcada como ignorada. La demostración de extremo a extremo del prototipo emite sus sellos con esta autoridad, y la verificación independiente del paquete de evidencia (`openssl ts -verify -data documento -in documento.tsr -CAfile ca.pem`) los acepta.

El camino remoto hacia el PSC no se eliminó: permanece como una integración explícita del comando `timestamp` cuando no se pasa `--mock`. En ese modo, el binario exige `CINCEL_BASE_URL` y `CINCEL_API_KEY` no vacías en el entorno —los valores en blanco de la plantilla no configuran nada— y construye el `CinzelTsaAdapter` con un tiempo límite de 15 segundos por solicitud y una política de reintento de cinco sondeos cada dos segundos. Debe decirse con precisión qué estatus tiene ese adaptador: su contrato HTTP —envío del digest por POST, estados `completed`, `processing` y `rejected`, sondeo del recurso diferido— es un contrato asumido por el adaptador y cubierto por un stub local, no un contrato confirmado por una campaña externa. El stub (`crates/infrastructure/tests/cinzel_stub.rs`) cubre el token inmediato, el procesamiento diferido, los rechazos con detalle, los estados HTTP de error, los cuerpos indecodificables, la autoridad inalcanzable y la promesa de que la credencial jamás se filtra a los mensajes de error. La prueba de humo contra el sandbox real, `the_real_sandbox_issues_a_token`, permanece ignorada: la API no ofreció garantía operativa suficiente y el sandbox requiere pago con precios no transparentes, por lo que no forma parte de la evidencia de esta entrega.

El balance honesto de la exploración es entonces el siguiente. El acceso al proveedor no produjo una ejecución suficientemente estable y repetible para declarar confirmado su contrato HTTP ni para medir el criterio de éxito de OE-3; tampoco existe un precio transparente que permita autorizar una campaña. Por decisión del proyecto, la confirmación empírica del algoritmo de firma de la TSA comercial y la tasa externa quedan fuera del alcance actual. La limitación quedó *acotada arquitectónicamente*: el flujo RFC 3161 completo —solicitud, emisión, almacenamiento del token DER y verificación en dos etapas— está demostrado de extremo a extremo con una autoridad real, el verificador `Rfc3161Verifier` es agnóstico de la autoridad emisora porque verifica cualquier token contra el ancla de confianza suministrada, y el intercambio de emisor sigue siendo una sustitución explícita detrás del puerto `TimestampService`, sin tocar el dominio ni la lógica de verificación. Esta portabilidad no equipara las garantías: un token de la TSA interna prueba el funcionamiento técnico bajo la CA del prototipo, pero no constituye una constancia independiente emitida por un PSC autorizado.

4.6 Divergencias respecto al diseño y decisiones registradas

Durante la construcción, las decisiones con consecuencias arquitectónicas se registraron como Architecture Decision Records versionados en el directorio `docs/adr/` del repositorio, en el formato de Nygard [66] —contexto, decisión, estatus y consecuencias—. Estos once registros son distintos y complementarios de los ADR de diseño presentados en [sección 3.7.10](#): aquellos capturaron las decisiones de arquitectura previas a la construcción; estos capturan las decisiones tomadas al materializarla, y el

código y la configuración los citan por su ruta. La [tabla 4.10](#) los resume.

Tabla 4.10: Architecture Decision Records registrados en docs/adr/ durante la implementación.

Archivo	Decisión
0001-msrv-and-dependency-pinning.md	MSRV Rust 1.78 y pin de los crates criptográficos a la familia der 0.7 (x509-cert 0.2, cms 0.2, x509-tsp 0.1, rsa 0.9; ring $\geq$ 0.17.12).
0002-rsa-signing-crate-and-advisory.md	Firmar con el crate rsa; aviso RUSTSEC-2023-0071 aceptado bajo el modelo de amenaza local y registrado en deny.toml y audit.toml.
0003-zeroize-secret-material-in-domain.md	Admitir zeroize en el dominio y declarar todo campo portador de secretos como buffer Zeroizing.
0004-certificate-validation-implementation.md	Validar X.509 re-codificando a DER la estructura to-be-signed parseada y verificando la firma con rsa y sha2.
0005-rfc3161-verification-strategy.md	Verificar tokens RFC 3161 en dos etapas: decodificación e imprint nativos; firma CMS y cadena delegadas a openssl ts -verify.
0006-evidence-package-format.md	Escribir el paquete de evidencia con un writer ZIP propio de entradas almacenadas, determinista y sin dependencia nueva.
0007-audit-chain-anchoring.md	Mantener la regla de cadena y documentar que el truncamiento de cola no es detectable; el anclaje de la cabeza se delega a la capa de persistencia.
0008-totp-single-use-enforcement.md	Mantener la verificación TOTP sin estado; la exigencia de un solo uso pertenece al llamador cuando exista un almacén persistente.
0009-local-timestamp-authority.md	Excluir la campaña externa no reproducible y demostrar RFC 3161 con una TSA OpenSSL local, sin atribuirle equivalencia jurídica.
0010-http-adapter-foundation.md	Introducir Axum como adaptador driving, conservar la dirección de dependencias y comenzar por el contrato de salud.
0011-local-document-workflow.md	Orquestar el flujo documental detrás de un puerto, persistir solo contenido cifrado y seleccionar explícitamente la TSA local en la aplicación.

Las divergencias sustantivas entre lo diseñado en el Capítulo 3 y lo construido son las siguientes; se listan de forma explícita porque un documento que las omitiera daría una imagen más limpia pero menos útil del proceso de ingeniería.

- **Biblioteca de firma.** El diseño situó los servicios criptográficos, de manera general, sobre la biblioteca ring ([sección 3.7.4](#)). En la implementación, ring quedó a cargo de SHA-256 y AES-256-GCM, pero la firma se materializó con el crate rsa —el adaptador conserva el nombre RsaPkcs1Signer que el diseño ya anticipaba en [sección 3.7.8](#)— porque interoperaba directamente con las claves y certificados PEM que produce la CA interna vía x509-cert y

sha2. El costo fue aceptar el aviso RUSTSEC-2023-0071 [83], con plan de contingencia de regresar la firma a ring si el riesgo se volviera inaceptable (docs/adr/0002).

- **Autoridad de sellado demostrada.** El diseño delegó el sellado de tiempo en el sandbox de Cincel (sección 3.11.3); la vía demostrada de extremo a extremo es la TSA local con OpenSSL. El adaptador remoto está implementado y probado contra stub, pero la inestabilidad observada de la API y el costo del sandbox impidieron una campaña externa reproducible, como detalla sección 4.5.
- **Dependencias del dominio.** El diseño declaró que el dominio dependería exclusivamente de `serde`, `thiserror`, `time` y `uuid` (sección 3.7.4); la implementación añadió `zeroize`, aceptado porque es un crate utilitario sin E/S ni superficie de framework que codifica una propiedad de seguridad del dominio —este valor se borra al liberarse— y no un detalle de infraestructura (docs/adr/0003).
- **Verificación de firmas X.509.** El diseño asumió el manejo de certificados con `x509-cert` y `der` (sección 3.7.7) sin anticipar que esas crates no verifican firmas; la verificación se implementó por re-codificación manual de la estructura `to-be-signed` (docs/adr/0004), aceptando únicamente `sha256WithRSAEncryption`.
- **Escritura del paquete ZIP.** El diseño previó un paquete ZIP descargable sin fijar el mecanismo; se implementó un writer propio del subconjunto de entradas almacenadas en lugar de depender del crate `zip`, cuya línea vigente rompería la MSRV (docs/adr/0006).
- **Superficie driving y persistencia.** De los tres adaptadores driving previstos (sección 3.7.5), el prototipo entrega la línea de comandos y una rebanada HTTP completa del flujo de evidencia; el frontend Astro sigue como andamiaje mínimo intencional. La persistencia PostgreSQL y Redis del diseño (sección 3.8) aún no se ejerce: los documentos se almacenan cifrados en JSON con reemplazo atómico y la bitácora en JSON Lines encadenado. El repositorio local demuestra el puerto y la continuidad entre reinicios, pero no se presenta como almacenamiento de producción.
- **Fitness functions.** De las cinco funciones de aptitud diseñadas (sección 3.7.11), la que exige un span de `tracing` por handler HTTP sigue diferida aunque los handlers ya están implementados; en su lugar se incorporó el umbral de cobertura del 90 % por crate criptográfico (FF-4, tabla 4.4).
- **Controles de autenticación diferidos.** La exigencia de un solo uso por código TOTP y los controles de limitación de intentos descritos en sección 3.11.5 pertenecen al llamador del puerto y quedan diferidos hasta que exista el almacén persistente de estado de autenticación por cuenta (docs/adr/0008); el riesgo residual —la reproducción de un código observado durante el resto de su ventana de unos 90 segundos— queda documentado en el propio registro.

Ninguna de estas divergencias contradice la arquitectura del Capítulo 3; todas se resolvieron dentro de sus reglas —detrás de un puerto, registradas en un ADR versionado y verificadas por el andamiaje de la sección 4.1—, lo que constituye en sí mismo una validación empírica de la propiedad de evolvabilidad que motivó la elección de la arquitectura hexagonal [70].

CAPÍTULO 5

Pruebas y resultados

Este capítulo reporta la validación del prototipo cuya implementación documentó el Capítulo 4. La estrategia de pruebas es la que fijó la metodología del Capítulo 3: desarrollo guiado por pruebas [63], con la regla —registrada en `CONTRIBUTING.md`— de que la lógica criptográfica y de dominio llega acompañada de sus pruebas en el mismo cambio, y de que los vectores de prueba oficiales de cada estándar son las primeras pruebas cuando existen. Sobre esa base, la suite se organiza como una pirámide de cinco niveles. En la base, las pruebas unitarias del `crate domain` ejercitan tipos de valor y puertos con dobles de prueba deterministas escritos a mano, sin ninguna biblioteca criptográfica; las del `crate application` verifican la orquestación de los casos de uso con mocks generados con `mockall`; y las del `crate infrastructure` ejecutan la criptografía real, comenzando por los vectores oficiales de FIPS 180-4, del modo GCM, de RFC 6238 y de RFC 4226. El segundo nivel son las pruebas de propiedades, que verifican invariantes sobre entradas generadas aleatoriamente [86]. El tercero son las pruebas de integración de infraestructura, que usan como fixtures los mismos scripts versionados de `pki/` que opera producción, con instantes de evaluación derivados de las ventanas de vigencia parseadas —nunca de esperas de reloj—, de modo que son deterministas. El cuarto son las pruebas de extremo a extremo del `crate bin`, que ejecutan el binario compilado `despacho-cli` como subproceso y observan salidas, archivos y códigos de terminación. El quinto son las fitness functions FF-1 a FF-5 (tabla 4.4) [68], que protegen propiedades arquitectónicas en cada integración.

El resultado global de la suite es de 382 pruebas en verde, cero fallos y una prueba ignorada: la prueba de humo contra el sandbox del proveedor externo de sellado. Se conserva para documentar el adaptador, pero no se ejecuta como requisito de la entrega porque la API no ofreció estabilidad suficiente y el sandbox es de pago con precios no transparentes (sección 4.5). Las demostraciones reproducibles utilizan la TSA local y no dependen de red ni de un proveedor comercial. El `crate web` añade tres pruebas de contrato que recorren carga, sellado, verificación, exportación y errores sobre el workflow inyectado; la persistencia local y la orquestación se prueban además en sus capas respectivas. Este resultado se reprodujo el 17 de agosto de 2026; el corte de herramientas, comandos y resultados está versionado en `docs/verification-report.md`. La cobertura de líneas se mide con `cargo-llvm-cov` y se aplica como umbral en integración continua (job coverage, tabla 4.3): al menos 90 % de líneas por cada uno de los tres crates con lógica criptográfica —`domain`, `application` e `infrastructure`—, conforme al requerimiento RNF-13. El capítulo se organiza en cuatro secciones: la matriz de pruebas por servicio criptográfico (sección 5.1), los resultados criptográficos específicos —vectores oficiales, frescura de nonces, mutaciones, propiedades y calibración— (sección 5.2), la verificación independiente con herramientas ajenas al prototipo (sección 5.3), y la cobertura alcanzada junto con el balance contra los criterios de éxito del Capítulo 1 (sección 5.4).

5.1 Matriz de pruebas

La suite consta de 383 funciones de prueba distribuidas como muestra la [tabla 5.1](#): 198 unitarias embebidas en `src` y 185 de integración en los directorios `tests/` de cada crate. La única función que no se ejecuta en la corrida ordinaria es `the_real_sandbox_issues_a_token`, marcada como ignorada porque consume el sandbox externo inestable y de pago; permanece disponible para una ejecución manual controlada. Las pruebas nuevas fijan el modelo y puerto del registro documental, la orquestación completa, el repositorio de archivos —incluido el rechazo de un UUID interno que no coincide con la ruta consultada— y el contrato HTTP.

Tabla 5.1: Distribución de las funciones de prueba por crate del workspace.

Crate	Dobles de prueba empleados	Unitarias	Integración	Total
domain	hashers falsos deterministas, sin criptografía real	69	—	69
application	mocks mockall de los puertos	39	43	82
infrastructure	criptografía real y herramientas del sistema	86	87 ^a	173
bin	el binario compilado como subproceso	3	52	55
web	contrato HTTP con workflow inyectado	1	3	4
Total		198	185	383

^a Incluye la prueba ignorada contra el sandbox del proveedor de sellado.

La [tabla 5.2](#) presenta la matriz de pruebas por servicio criptográfico, usando la misma identificación S1 a S10 del catálogo de servicios implementados ([tabla 4.5](#)). Cada función de prueba se adscribe al servicio cuyo puerto, caso de uso, adaptador o subcomando ejercita; 355 funciones quedan adscritas a los diez servicios y las 28 restantes son transversales: el workflow documental, su repositorio y contrato HTTP, la validez del árbol completo de la interfaz de línea de comandos, las conversiones de error entre capas, la lectura de la versión local de OpenSSL que se incrusta en el instructivo del paquete de evidencia, y las fitness functions de arquitectura, de cobertura de puertos y de no fuga de secretos en los diagnósticos. La columna de tipos señala qué categorías especiales de prueba posee cada servicio, además de las unitarias y de integración clásicas que todos tienen: **V** = vectores oficiales del estándar, **P** = propiedades sobre entradas generadas aleatoriamente, **M** = mutación de artefactos (detección de alteraciones), **I** = interoperabilidad con herramientas independientes, **E** = extremo a extremo sobre el binario compilado.

Tabla 5.2: Matriz de pruebas por servicio criptográfico: número de funciones de prueba adscritas y tipos presentes.

Id	Servicio	Pruebas	Tipos
S1	Hash de documentos (SHA-256)	24	V, P, M, I, E
S2	Cifrado autenticado y envelope encryption	54	V, P, M, E
S3	PKI interna: emisión, revocación y validación	51	M, I, E
S4	Firma digital RSA-3072	36	M, I, E
S5	Sellado de tiempo RFC 3161	50 ^a	M, I, E
S6	Verificación integral de cuatro componentes	22	M, E
S7	Hash de contraseñas (Argon2id)	15	E
S8	Segundo factor TOTP y códigos de recuperación	23	V, I, E
S9	Cadena de auditoría	52	P, M, E
S10	Paquete de evidencia	28	V, M, I, E
Subtotal adscrito a servicios		355	
Transversales (workflow, HTTP, CLI y fitness)		27	

^a Incluye la prueba ignorada del sandbox del proveedor; las 49 restantes se ejecutan en cada corrida.

El criterio del objetivo OE-6 exige que la matriz contenga, por lo menos, un caso positivo y uno negativo por caso de uso. La [tabla 5.3](#) nombra un caso representativo de cada signo por servicio; como documenta el inventario, todos los servicios poseen varios casos negativos adicionales —el patrón general de la suite es que cada camino de rechazo tipado del dominio tiene al menos una prueba que lo provoca—. El inventario completo de las 383 funciones de prueba, función por función y con la descripción de lo que cada una fija, se entrega en el Anexo E.

Tabla 5.3: Caso positivo y caso negativo representativos por servicio criptográfico.

Id	Caso positivo	Caso negativo
S1	Los cuatro vectores de FIPS 180-4 se reproducen, y el digest calculado por flujo con cortes irregulares coincide con el calculado en memoria.	Un fallo de lectura del flujo se reporta como error con diagnóstico; un archivo inexistente termina el proceso con código distinto de cero.
S2	El ciclo cifrar–descifrar restaura el documento byte a byte, por propiedad y de extremo a extremo; tras rotar la KEK, el documento solo se descifra bajo la llave nueva.	Cualquier byte alterado del paquete cifrado, una KEK equivocada, la versión desplazada en uno o el identificador de otro documento producen el mismo rechazo opaco.

Continúa en la página siguiente

Id	Caso positivo	Caso negativo
S3	Ciclo completo por el binario real: el certificado emitido encadena a la raíz y, tras revocar y regenerar la CRL, se reporta revocado con su número de serie.	Una CRL de una autoridad ajena se rechaza como no confiable; una CRL vencida en su <code>nextUpdate</code> es un error duro, nunca un pase permisivo.
S4	Firmar y verificar es válido tanto con el certificado como con la clave pública SPKI; las codificaciones PKCS#1 y PKCS#8 de una misma clave firman idéntico.	Una clave de 2 048 bits se rechaza en construcción; un byte alterado del documento o de la firma produce el rechazo tipado de no correspondencia.
S5	Un token recién emitido verifica como válido con hora plausible, igual que el token CMS desnudo extraído de la respuesta.	Un token sobre otro digest es discrepancia de huella, detectada sin lanzar subprocesos; bytes aleatorios o DER truncado son token malformado; un ancla ajena deja el token como no confiable.
S6	Con los cuatro componentes aprobados el veredicto es válido; la evidencia no aportada se reporta omitida con su razón y no envenena el veredicto.	Cada componente se hace fallar por separado —firma rechazada, certificado expirado o revocado, emisor no confiable, huella discrepante— y el veredicto pasa a no válido nombrando la causa.
S7	La cadena PHC emitida registra los parámetros fijados por el diseño y la contraseña correcta verifica como coincidencia.	La contraseña incorrecta termina con código distinto de cero; un hash almacenado corrupto es un error, nunca una simple no coincidencia.
S8	Los dieciséis vectores oficiales se reproducen y un código vigente derivado del secreto enrolado se acepta de extremo a extremo.	Un código a dos pasos de distancia se rechaza, igual que un secreto demasiado corto o una codificación base32 inválida; cada código de recuperación funciona exactamente una vez y su reuso se rechaza.
S9	Treinta y dos anexados concurrentes desde ocho hilos se serializan en una sola cadena válida con secuencias únicas.	Editar un campo de una línea rompe la verificación exactamente en su índice; una bitácora ausente es un error, no una cadena vacía válida.
S10	Los comandos del instructivo del paquete, extraídos y ejecutados literalmente, aceptan el documento, la firma, el certificado y el token.	La exportación se rehúsa ante una firma alterada o una CRL ajena, sin escribir el ZIP; un byte volteado en un miembro extraído lo detectan los propios comandos y el CRC del formato.

Una convención del inventario merece mención explícita porque determina cómo se lee: cada función de prueba se nombra como una oración conductual que enuncia la garantía comprobada,

de modo que el inventario del Anexo E funciona como una especificación ejecutable del comportamiento del sistema —el nombre declara la garantía y la función la comprueba—. Dos ejemplos: `version_rejects_zero` y `an_empty_log_verifies_as_valid`.

5.2 Resultados criptográficos

5.2.1 Vectores oficiales

Todos los vectores oficiales adoptados están en verde. La [tabla 5.4](#) los resume. Para SHA-256 se reproducen los cuatro vectores de ejemplo de FIPS 180-4 [25]: la cadena vacía, el mensaje «abc», el mensaje de dos bloques y un millón de caracteres «a» —este último consumido por flujo, lo que ejercita a la vez la interfaz de streaming del adaptador—. El vector «abc» se reutiliza además de extremo a extremo: la prueba del subcomando `crypto hash` verifica que el binario imprime exactamente `ba7816bf...` ante ese contenido. Para AES-256-GCM se adoptaron los casos de prueba TC13 a TC16 de la especificación del modo GCM de McGrew y Viega [87], la base de NIST SP 800-38D [50]; los mismos valores figuran en las suites del programa de validación CAVP del NIST. El adaptador expone una operación de sellado con nonce inyectable exclusivamente para reproducir los vectores; las cuatro pruebas de cifrado exigen el criptograma y el tag exactos, dos de descifrado exigen la autenticación y el texto claro exactos, y una prueba de respuesta conocida negativa voltea el último byte del tag y exige el rechazo autenticado.

Tabla 5.4: Vectores oficiales reproducidos por la suite de pruebas.

Estándar y fuente	Vectores reproducidos	Pruebas
FIPS 180-4, SHA-256 [25]	Cadena vacía (<code>e3b0c442...</code>), «abc» (<code>ba7816bf...</code>), mensaje de dos bloques (<code>248d6a61...</code>) y un millón de «a» por flujo (<code>cdc76e5c...</code>).	4
Casos TC13–TC16 del modo GCM [50], [87]	AES-256-GCM: cifrado de los cuatro casos con criptograma y tag exactos; descifrado autenticado de TC14 y TC16; caso negativo con el último byte del tag volteado.	7
RFC 6238, apéndice B [19]	Los seis vectores TOTP SHA-1 de ocho dígitos (T desde 59 hasta 20 000 000 000, secreto ASCII de referencia).	1 (6 vectores)
RFC 4226, apéndice D [58]	Los diez vectores HOTP de seis dígitos (contadores 0 a 9), como verificación cruzada del conteo de dígitos de producción.	1 (10 vectores)
CRC-32 del formato ZIP	Valor de comprobación estándar: <code>crc32</code> de <code>"123456789"</code> igual a <code>0xCBf43926</code> , y <code>crc32</code> de la cadena vacía igual a 0.	1

Para la autenticación multifactor se reproducen íntegros los dos apéndices normativos: los seis vectores TOTP de RFC 6238 con SHA-1 y ocho dígitos, y los diez vectores HOTP de RFC 4226, que cruzan el conteo de seis dígitos usado en producción. Para la firma digital RSA-3072 con padding

PKCS#1 v1.5 no se adoptaron vectores oficiales —los publicados fijan claves de otros tamaños—; su lugar lo ocupan dos verificaciones equivalentes: el determinismo del esquema, fijado por la prueba de que las codificaciones PKCS#1 y PKCS#8 de una misma clave producen firmas byte a byte idénticas, y la interoperabilidad bidireccional con OpenSSL que documenta la [sección 5.3](#).

5.2.2 Frescura de nonces

El criterio del objetivo OE-4 exige cero reutilizaciones de nonce en una suite de 10 000 cifrados. La prueba `nonces_are_distinct_across_ten_thousand_seals` ejecuta 10 000 operaciones de sellado consecutivas sobre el cifrador real y registra el nonce de 96 bits de cada una en un conjunto: la prueba falla si algún nonce se repite. El resultado es de 10 000 nonces distintos en 10 000 operaciones, consistente con la generación por CSPRNG fresco en cada sellado que estableció el diseño ([sección 3.11.4](#)). La misma familia de garantías de frescura está fijada por pruebas en el resto del sistema: dos DEK generadas nunca coinciden, dos enrolamientos TOTP extraen secretos distintos, cada hash de contraseña usa una sal aleatoria nueva —dos hashes de la misma contraseña difieren y ambos verifican—, los ocho códigos de recuperación de un lote no se repiten, y ocho solicitudes concurrentes a la autoridad de sellado local producen ocho números de serie distintos.

5.2.3 Detección de mutaciones

Las pruebas de mutación alteran un artefacto criptográfico real —un byte o un bit a la vez— y exigen el rechazo. El resultado global es la detección del 100 % de las mutaciones ensayadas, en todos los servicios donde se aplicaron. La pieza central es la prueba exhaustiva del cifrado autenticado, `any_single_byte_flip_breaks_authentication`: recorre en bucle *cada* byte del payload sellado —nonce, criptograma y tag, uno por uno— y verifica que voltear cualquiera de ellos hace fallar la apertura con el error opaco de autenticación; para este servicio la afirmación de detección total no es muestral sino exhaustiva sobre todas las posiciones del artefacto. Alrededor de ese núcleo, la suite ensaya mutaciones sobre cada tipo de evidencia:

- **Firma digital:** un byte alterado del documento firmado, o un byte alterado de la firma, produce el rechazo tipado de no correspondencia; el mismo caso se ensaya de extremo a extremo, donde el componente de firma de la verificación integral falla con su causa y el veredicto pasa a no válido.
- **Certificado:** corromper el último byte DER de un certificado —su firma— hace que el validador reporte emisor no confiable aunque la estructura siga parseando; la prueba fija que la confianza depende de la verificación criptográfica y no del parseo.
- **Token de sellado:** un token DER truncado o sustituido por bytes aleatorios se diagnostica como malformado; un token legítimo evaluado contra un ancla ajena queda como no confiable, conservando el diagnóstico de la herramienta.
- **Cadena de auditoría:** `any_single_corruption_is_detected_at_its_index` corrompe un bit de un byte de cualquier campo —valor de cadena, secuencia, actor, acción o recurso— de cualquier entrada de bitácoras generadas aleatoriamente, y exige que la verificación se rompa exactamente en el índice alterado; el mismo comportamiento se fija a mano sobre el archivo JSONL y de extremo a extremo por el subcomando `audit verify-chain`.

- **Paquete cifrado y paquete de evidencia:** un byte alterado del archivo DVL1 produce el rechazo autenticado en el subcomando `vault decrypt`; un byte alterado de un miembro del ZIP lo detecta la herramienta independiente `unzip` por el CRC; un documento corrompido dentro del paquete extraído es rechazado por los comandos `openssl` del propio instructivo; y la exportación se rehúsa a escribir un paquete cuya firma fue alterada, nombrando el componente fallido.

Este conjunto satisface el criterio de detección del 100 % de alteraciones de un byte del objetivo OE-4 y materializa la mitigación comprometida en el análisis de riesgos para el compromiso del esquema de cifrado (RTC-02, [tabla 3.62](#)).

5.2.4 Propiedades verificadas por generación aleatoria de casos

Cinco invariantes se verifican con `proptest` sobre cientos de entradas generadas aleatoriamente por corrida, en la tradición de las herramientas de *property-based testing* [86]:

1. **Round-trip del cifrado autenticado:** para toda llave de 32 bytes, todo AAD de hasta 63 bytes y todo texto claro de hasta 1 023 bytes, abrir lo sellado devuelve el texto claro original.
2. **Determinismo de SHA-256:** para entradas arbitrarias de hasta 4 095 bytes, el digest es idéntico entre corridas y entre la interfaz de bloque y la de flujo.
3. **Sensibilidad de SHA-256:** voltear cualquier bit de cualquier byte de la entrada cambia el digest.
4. **Validez de la cadena de auditoría:** cualquier secuencia aleatoria de 0 a 11 eventos anexados produce una cadena que verifica como válida.
5. **Detección puntual de corrupción:** una corrupción de un bit en cualquier campo de cualquier entrada de la bitácora se detecta exactamente en su índice.

Las dos últimas propiedades son el complemento probabilístico de las pruebas de mutación de la [sección 5.2.3](#): no fijan un caso concreto sino el invariante completo del que esos casos son instancias.

5.2.5 Concurrencia

Los dos adaptadores que comparten recursos del sistema de archivos entre procesos —la bitácora encadenada y la autoridad de sellado local— se probaron bajo concurrencia real, no simulada. Ocho hilos que anexan cuatro eventos cada uno sobre el mismo archivo de bitácora, liberados simultáneamente con una barrera, producen 32 entradas con secuencias únicas y una sola cadena que verifica como válida: el lock exclusivo entre procesos serializa los escritores en lugar de permitir que bifurquen la cadena, y la misma garantía se fija para instancias del adaptador en procesos separados que escriben por turnos. En la autoridad de sellado, ocho solicitudes simultáneas producen ocho tokens con números de serie todos distintos —la condición de carrera sobre el archivo de serie de OpenSSL que el adaptador debe serializar ([sección 4.2.5](#))—, cuatro verificaciones simultáneas del mismo token concluyen todas válidas gracias al directorio de trabajo privado de cada una, y ninguna operación deja archivos de trabajo residuales en el directorio de la autoridad.

5.2.6 Calibración de Argon2id

El diseño exige que el costo por intento de Argon2id quede entre 0.5 y 1.0 segundos en el hardware de despliegue (sección 3.11.5). El caso de uso de calibración (subcomando `auth calibrate`) promedia cinco corridas de hash sobre una muestra fija y clasifica la media contra la banda objetivo inclusiva. En el hardware de referencia (AMD Ryzen 7 7730U, 16 hilos lógicos), la calibración de la demostración de cierre del 17 de agosto de 2026 fijó $m = 256$ MiB, $t = 4$, $p = 1$ y reportó 607.8 milisegundos, es decir, un veredicto dentro de la banda. Si el hardware final difiere, la misma medición debe repetirse.

La lectura operativa de este resultado es que el criterio queda cubierto para el hardware de referencia y que el ajuste está localizado en un único punto del adaptador. El hardware final debe verificarse por separado porque la memoria y el tiempo por hash cambian con el perfil de despliegue. El resto del servicio no depende de la banda: el formato PHC con los parámetros registrados, la coincidencia y no coincidencia de contraseñas, el tratamiento del hash corrupto como error y la ausencia de la contraseña en los diagnósticos están fijados por pruebas que pasan en cualquier hardware.

5.3 Verificación independiente

Un principio transversal de la suite es que ningún resultado criptográfico se acepta únicamente porque el software propio lo apruebe: cada artefacto que el prototipo produce se verifica también con herramientas independientes, y cada artefacto que las herramientas independientes producen se verifica con el software propio. La herramienta de referencia es OpenSSL, en la versión 3.5.7 (9 de junio de 2026) del entorno de pruebas; la versión queda registrada en tres lugares: en las pruebas —que la leen con `openssl version`—, en el instructivo de cada paquete de evidencia exportado, y en la transcripción de la demostración. Este cruce sistemático materializa la mitigación comprometida en el análisis de riesgos para la discrepancia de digests ante el sellado de tiempo (RLC-01, tabla 3.66), que exigía auditar los resúmenes generados en Rust contra herramientas certificadas de línea de comandos. La tabla 5.5 resume las comprobaciones cruzadas que la suite ejecuta como subprocesos en cada corrida.

Tabla 5.5: Comprobaciones de interoperabilidad ejecutadas por la suite con herramientas independientes.

Artefacto	Comprobación cruzada
Resumen SHA-256	El digest del hasher propio coincide con la salida de sha256sum del sistema sobre un archivo de 123 457 bytes.
Firma propia	openssl dgst -sha256 -verify acepta las firmas del prototipo (Verified OK); se ensaya en integración, de extremo a extremo tras sign, y sobre el paquete de evidencia.
Firma ajena	Una firma generada con openssl dgst -sha256 -sign es aceptada por el verificador propio: la interoperabilidad es bidireccional.
Certificados y CRL	openssl verify -crl_check concuerda con el validador propio en el certificado vigente y en el revocado; openssl verify -atime concuerda en la cadena vigente y en el emisor expirado.
Sellos de tiempo	openssl ts -verify acepta los tokens de la autoridad local, verificados por digest y por documento; openssl ts -reply -text se usa para leer los números de serie emitidos.
Paquete ZIP	unzip -t acepta el paquete y la extracción restaura cada byte, incluida una entrada con los 256 valores de byte; con un byte alterado, unzip -t falla nombrando el CRC.
Código TOTP	En la demostración, el código se recomputa con base32, od y openssl dgst -sha1 -mac HMAC —sin el software propio— y el verificador del prototipo lo acepta.
Instructivo del paquete	La prueba de extremo a extremo del paquete extrae los comandos de INSTRUCCIONES.md y los ejecuta literalmente: las tres comprobaciones pasan sobre el paquete íntegro y rechazan el miembro corrompido.

La evidencia operativa de este enfoque comprende dos demostraciones de extremo a extremo. scripts/demo.sh (sección 4.3), cuya transcripción íntegra —capturada el 12 de julio de 2026 sobre OpenSSL 3.5.7 y re-ejecutada sin errores con posterioridad— está versionada en docs/demo-transcript.md, recorre la CLI. scripts/api-demo.sh levanta la aplicación en un puerto efímero, recorre carga, sellado, verificación, exportación y auditoría a través de HTTP, comprueba que el texto claro no aparece en el repositorio y valida el ZIP descargado con OpenSSL y unzip; las variables de Cincel se eliminan expresamente del entorno. Las demostraciones no son vitrinas: cada condición esperada aborta el guion si no se cumple. Los extractos siguientes reproducen tres momentos de la primera transcripción; se abrevian las rutas del directorio temporal y del binario, sin alterar ninguna salida. El primero es el reporte de la verificación integral con los cuatro componentes en verde:

```
+ despacho-cli verify document.txt --sig document.txt.sig
  --tsr document.txt.tsr --cert socia-demo.crt.pem
  --ca ca.crt.pem --crl crl.pem
document:    document.txt
```

```

digest:      fb5c0a6097be9246860fa82c89db3576e8a4ba7e...
integrity:    passed: recomputed digest matches the digest
               the timestamp token attests
signature:    passed: signature verifies over the document
               digest with the signer certificate
certificate:  passed: status at the evaluation time: valid
               (revocation checked against the supplied list)
timestamp:    passed: token accepted under the trust anchor;
               generated at 2026-07-12T10:19:41Z
verdict:      valid

```

Listado 5.1: Verificación integral con evidencia íntegra (extracto de docs/demo-transcript.md).

El segundo es la verificación del paquete de evidencia exportado usando únicamente OpenSSL, sin el software que lo generó —el mismo procedimiento que seguiría un perito o la contraparte procesal, reproducido paso a paso en el Anexo C—:

```

+ openssl dgst -sha256 document.txt
SHA2-256(document.txt)= fb5c0a6097be9246860fa82c89db3576...
+ openssl x509 -in certificado.pem -pubkey -noout
  -out firmante.pub.pem
+ openssl dgst -sha256 -verify firmante.pub.pem
  -signature document.txt.sig document.txt
Verified OK
+ openssl verify -crl_check -CAfile ca-y-crl.pem certificado.pem
certificado.pem: OK
+ openssl ts -verify -data document.txt -in document.txt.tsr
  -CAfile ca.pem
Using configuration from /etc/pki/tls/openssl.cnf
Verification: OK

```

Listado 5.2: Verificación del paquete de evidencia con solo OpenSSL (extracto de docs/demo-transcript.md).

El tercero es la manipulación de la bitácora encadenada: la sustitución de una acción por otra de la misma longitud —de modo que solo el encadenamiento criptográfico puede notarlo— y su detección en el índice exacto:

```

+ despacho-cli audit verify-chain
audit chain valid (2 entries)
+ sed -i s/demo.sello/demo.XXXXX/ audit-log.jsonl
+ (expected to fail) despacho-cli audit verify-chain
Error: audit chain broken at index 1
OK: command was rejected, as expected

```

Listado 5.3: Detección de la manipulación de la bitácora (extracto de docs/demo-transcript.md).

La misma transcripción registra la revocación del certificado del firmante y su efecto sobre la verificación: el componente de certificado falla reportando `revoked (serial 1000)` y el reporte añade que ese estado habla solo del instante de evaluación y no invalida por sí mismo una firma

producida durante la vigencia —la lectura jurídica cuidadosa que el diseño exigió y que las pruebas fijan textualmente en tres niveles: caso de uso, interfaz de línea de comandos y demostración—.

5.4 Cobertura y criterios de éxito

La [tabla 5.6](#) presenta la cobertura de líneas final por crate, medida con `cargo-llvm-cov` sobre la suite completa y reproducida en el corte del 17 de agosto de 2026. El umbral del 90 % que exige el requerimiento RNF-13 se aplica en integración continua a los tres crates con lógica criptográfica mediante `scripts/coverage-gate.sh`, y los tres lo superan. Los crates `bin` y `web` se reportan sin umbral: son fronteras de entrada y composición cuyo valor se complementa con pruebas de contrato y guiones E2E. Parte del margen sobre el umbral proviene de pruebas dedicadas a rutas de error de los adaptadores —una bitácora cuya ruta es un directorio, una línea almacenada con timestamp imparseable, respuestas del proveedor de sellado sin token o con base64 inválido, material PEM corrupto en el firmador y el verificador—, de modo que los caminos de fallo están tan ejercitados como los de éxito.

Tabla 5.6: Cobertura de líneas por crate medida con `cargo-llvm-cov` y umbral aplicado en integración continua.

Crate	Líneas cubiertas	Líneas instrumentadas	Cobertura	Umbral en CI
<code>domain</code>	937	974	96.2 %	90 % (bloqueante)
<code>application</code>	1 488	1 561	95.3 %	90 % (bloqueante)
<code>infrastructure</code>	2 009	2 151	93.4 %	90 % (bloqueante)
<code>bin</code>	833	1 008	82.6 %	informativo
<code>web</code>	168	223	75.3 %	informativo
Workspace	5 435	5 917	91.9 %	—

Sobre los umbrales conviene una precisión: el criterio de éxito del objetivo OE-6 pide cobertura de al menos 80 % en los módulos criptográficos, mientras que el requerimiento RNF-13 pide 90 %. La integración continua aplica el más estricto de los dos, y los resultados superan ambos. El requerimiento RNF-14 —que el 100 % de las pruebas de los módulos criptográficos pase antes de considerar completado un incremento— se cumple por construcción: el job de pruebas de integración continua bloquea la rama principal ante cualquier fallo, y la suite cierra con 382 de 382 pruebas ejecutadas en verde, más una prueba externa ignorada.

La [tabla 5.7](#) confronta los resultados con los criterios de éxito verificables que el Capítulo 1 fijó para los objetivos específicos validables en esta fase (OE-3 a OE-6, [tabla 1.1](#)), desglosados por subcriterio y con un estado explícito: cumplido, parcial o diferido.

Tabla 5.7: Resultados contra los criterios de éxito de los objetivos OE-3 a OE-6.

OE	Criterio de éxito	Resultado medido	Estado
OE-3	Firma RSA-3072 con SHA-256 verificable de forma independiente sobre el 100 % de los contratos firmados.	Toda firma se verifica antes de escribirse; interoperabilidad bidireccional con OpenSSL; paquete de evidencia verificable sin el software propio.	Cumplido
OE-3	Sello de tiempo RFC 3161 vinculado al hash del documento.	Tokens emitidos sobre el digest SHA-256, verificados en dos etapas y aceptados por <code>openssl ts -verify</code> ; el binario comprueba que el token cubre el digest antes de almacenarlo.	Cumplido
OE-3	Tasa de verificación exitosa ≥ 99 % sobre sellos emitidos por el sandbox de Cincel.	No medible: el registro no garantizó una API operativa, la campaña requeriría un sandbox de pago con precios no transparentes y la evidencia externa fue excluida de la entrega. La TSA local sí queda verificada de extremo a extremo.	Fuera del alcance actual
OE-4	Detección del 100 % de alteraciones de un byte mediante pruebas de mutación.	Todas las mutaciones ensayadas fueron detectadas; para el cifrado autenticado la verificación es exhaustiva sobre cada byte del payload (sección 5.2.3).	Cumplido
OE-4	Nonce único por operación y cero reutilizaciones en una suite de 10 000 cifrados.	10 000 sellados consecutivos sin repetición del nonce de 96 bits (sección 5.2.2).	Cumplido
OE-4	Envelope encryption con DEK por documento.	DEK aleatoria de 32 bytes por documento envuelta bajo la KEK; rotación de KEK sin recifrar el documento, fijada por prueba.	Cumplido
OE-5	Argon2id calibrado a 0.5–1.0 s por intento en el hardware de referencia ($m = 256$ MiB, $t = 4$, $p = 1$).	Cinco corridas en AMD Ryzen 7 7730U promediaron 607.8 ms y emitieron veredicto dentro de la banda; repetir si cambia el hardware de despliegue (sección 5.2.6).	Cumplido*

Continúa en la página siguiente

OE	Criterio de éxito	Resultado medido	Estado
OE-5	MFA basado en TOTP con ventana de tolerancia ± 1 período.	Vectores oficiales de RFC 6238 y RFC 4226 en verde; desfase de ± 1 paso aceptado y ± 2 rechazado; enrolamiento y verificación de extremo a extremo.	Cumplido
OE-5	Control de acceso RBAC con cuatro roles diferenciados (Owner, Litigante, Paralegal, Cliente).	La superficie HTTP documental existe, pero todavía no autentica sesiones ni aplica permisos; X-Actor es solo una etiqueta de auditoría.	Diferido
OE-6	Cobertura de pruebas ≥ 80 % en los módulos criptográficos.	Entre 93.4 y 96.2 % por crate criptográfico; el umbral aplicado en CI es el 90 % de RNF-13, más estricto que el criterio.	Cumplido
OE-6	Al menos 15 pruebas de extremo a extremo que cubran los flujos de firma y verificación.	22 pruebas ejecutan el binario real sobre esos flujos (sign, timestamp, verify y package export), además de la demostración íntegra.	Cumplido
OE-6	Matriz de pruebas funcionales con, por lo menos, un caso positivo y uno negativo por caso de uso.	Tabla 5.3 por servicio; inventario completo, función por función, en el Anexo E.	Cumplido

El balance debe leerse con la misma franqueza con que se construyó la tabla. Los objetivos OE-4 y OE-6 se cumplen en todos sus subcriterios. OE-3 queda parcialmente cumplido porque su criterio externo fue excluido de esta entrega; OE-5 conserva dos pendientes operativos precisos y acotados:

- **Evidencia externa de PSC (OE-3).** La autoridad de sellado local demuestra el flujo RFC 3161 completo —solicitud, emisión, token DER, verificación en dos etapas y aceptación por herramientas independientes—. La tasa de verificación externa no se mide porque la API de Cincel no ofreció garantía operativa y el sandbox requiere un pago no transparente. El adaptador remoto queda probado contra un stub y disponible para una futura sustitución consciente, pero no bloquea el desarrollo ni se presenta como evidencia de esta entrega.
- **RBAC de cuatro roles (OE-5).** La superficie HTTP documental ya existe, pero opera como aplicación local: X-Actor etiqueta la auditoría y no prueba identidad ni permisos. Los cimientos de autenticación sobre los que se montarán las sesiones —hash de contraseñas, segundo factor y códigos de recuperación— están implementados y probados; JWT, RBAC, la exigencia de un solo uso por código TOTP y la limitación de intentos requieren un almacén persistente por cuenta y permanecen diferidos (sección 4.6).
- **Banda de calibración de Argon2id (OE-5).** El hardware de referencia quedó dentro de banda

con $m = 256$ MiB, $t = 4$, $p = 1$ y una media de 607.8 ms. El asterisco de la tabla recuerda que un hardware de despliegue distinto exige repetir la medición ([sección 5.2.6](#)).

En síntesis, la validación deja los servicios criptográficos y la rebanada HTTP local respaldados por 382 pruebas en verde con una cobertura del 91.9 % del workspace, vectores oficiales reproducidos para cada primitivo que los publica, detección total en las campañas de mutación, y verificación independiente bidireccional con OpenSSL 3.5.7. También deja explícito que la evidencia externa de un PSC no forma parte de esta entrega y que la TSA local no se presenta como equivalente jurídico. Los pendientes técnicos son sesiones y RBAC, persistencia PostgreSQL/Redis y la interfaz de usuario; ya no incluyen la orquestación documental ni su contrato HTTP básico. Las conclusiones generales del trabajo se presentan en el [Capítulo 6](#).

CAPÍTULO 6

Conclusiones

Este capítulo cierra el documento: contrasta los resultados de la fase de implementación, documentada en los Capítulos 4 y 5, con los objetivos y criterios de éxito planteados en el Capítulo 1; enuncia las contribuciones del trabajo; delimita con honestidad lo que el prototipo todavía no demuestra; y organiza el trabajo futuro en horizontes de corto, mediano y largo plazo.

6.1 Resultados obtenidos

El presente Trabajo Terminal diseñó la arquitectura de un sistema de gestión de expedientes penales para despachos jurídicos mexicanos e implementó y demostró su núcleo criptográfico completo junto con una rebanada vertical de aplicación HTTP. Los diez servicios criptográficos cuya construcción se documenta en el Capítulo 4 quedaron codificados en Rust sobre la arquitectura hexagonal diseñada en el Capítulo 3, materializada en un workspace de Cargo de cinco crates cuya regla de dependencias verifica mecánicamente el compilador en cada compilación (véase §3.7.6):

- integridad de documentos mediante SHA-256 [25];
- confidencialidad en reposo mediante AES-256-GCM [50] con envelope encryption [52] y rotación de la clave maestra sin recifrado de documentos;
- firma digital RSA-3072 con SHA-256 sobre certificados X.509;
- sellado de tiempo conforme a RFC 3161 [27];
- operación de una CA interna: emisión, revocación y validación de certificados con CRL conforme a RFC 5280 [44];
- verificación integral de cuatro componentes (integridad, firma, estado del certificado y sello de tiempo);
- exportación de paquetes de evidencia verificables por terceros sin el software del prototipo;
- hashing de contraseñas mediante Argon2id [60];
- segundo factor TOTP [19] con códigos de recuperación de un solo uso;
- bitácora de auditoría encadenada criptográficamente.

El funcionamiento de estos servicios no se afirma: se demuestra con la evidencia presentada en el Capítulo 5. La suite de 383 pruebas automatizadas (198 unitarias y 185 de integración) se ejecuta en verde —382 en cada corrida y una ignorada que conserva la sonda del sandbox externo, sin formar parte del requisito de entrega— e incorpora, como primeras pruebas de cada primitivo, los vectores oficiales correspondientes: los de FIPS 180-4 [25] para SHA-256, los casos conocidos de

AES-256-GCM que subyacen a NIST SP 800-38D [50], y los apéndices de vectores de RFC 6238 [19] y RFC 4226 [58]. La cobertura de líneas se mantiene igual o superior al 90 % en los tres crates criptográficos (*domain*, *application* e *infrastructure*), umbral que la integración continua impone en cada cambio y que excede el criterio de éxito del 80 % fijado para OE-6 (tabla 1.1). Las pruebas de mutación detectaron el 100 % de las alteraciones de un byte —incluido un barrido exhaustivo que voltea uno a uno cada byte del criptograma sellado— y una suite de 10 000 cifrados consecutivos no registró repetición alguna de nonce, con lo que se satisface el criterio de éxito de OE-4. Finalmente, dos demostraciones de extremo a extremo, ejecutadas sobre OpenSSL 3.5.7, recorren el ciclo de vida completo por CLI y HTTP —creación de la autoridad interna, persistencia cifrada, firma, sellado, verificación, exportación y bitácora— y culminan verificando el paquete de evidencia exportado utilizando exclusivamente `openssl` y `unzip`.

El diseño se fundamentó en un análisis riguroso del marco legal mexicano aplicable a las transacciones entre particulares: el Código de Comercio [13] (artículos 89, 89 bis, 93, 93 bis, 97 y 100), el Código Federal de Procedimientos Civiles [38] (artículo 210-A), el Código Nacional de Procedimientos Penales [1] (artículos 356 y 380) y la NOM-151-SCFI-2016 [14]. Este análisis permitió establecer que la firma electrónica simple, respaldada por un sello de tiempo de un PSC autorizado, constituye una estrategia probatoria válida y defendible en sede judicial, conforme a la cláusula de equivalencia funcional del artículo 100 del Código de Comercio.

La arquitectura hexagonal dejó de ser una intención de diseño para convertirse en una propiedad verificada del código: cada puerto se materializa como un `trait` de Rust, cada adaptador como una `struct` intercambiable, y dos de las *fitness functions* definidas en §3.7.11 vigilan en cada integración que el dominio no dependa de ninguna otra capa y que todo puerto cuente con un adaptador concreto y un doble de pruebas. La separación se manifiesta en dos hechos comprobados: el módulo de firma acepta cualquier certificado X.509 RSA + SHA-256 con independencia de su emisor, y el verificador de sellos de tiempo valida cualquier token RFC 3161 contra el ancla de confianza que se le presente. Ambas propiedades permiten sustituir la infraestructura criptográfica del prototipo (CA interna, TSA local) por infraestructura regulada (e.firma del SAT, PSC comercial) sin modificar la lógica de verificación.

6.2 Contribuciones

La contribución principal de este trabajo es un puente documentado —y ahora ejecutable— entre la teoría criptográfica contemporánea (NIST SP 800-57 [37], RFC 9106 [60], RFC 6238 [19]) y los requisitos específicos del marco legal mexicano para documentos electrónicos con valor probatorio. Cada decisión de diseño criptográfico se justificó con referencia tanto a los estándares internacionales que la sustentan como a los artículos del Código de Comercio y el CNPP que determinan su admisibilidad en sede judicial, y las decisiones tomadas durante la implementación quedaron registradas en registros de decisiones de arquitectura versionados junto al código, en el directorio `docs/adr/` del repositorio, de modo que el racional de cada compromiso técnico es auditable con el solo repositorio.

Como contribución adicional, el trabajo establece que la distinción entre firma electrónica simple y avanzada en el derecho mexicano no es de admisibilidad —ambas son admisibles conforme al artículo 89 del Código de Comercio— sino de carga de la prueba. Esta precisión, documentada con referencia a los artículos 97 y 100, permite argumentar que un prototipo académico con CA interna genera evidencia criptográfica válida sin necesidad de contratar un PSC para la emisión de certificados de firmante, siempre que el sello de tiempo provenga de un PSC autorizado.

Una tercera contribución es metodológica: la exigencia de que toda salida criptográfica del sistema

sea verificable con herramientas independientes se tradujo en pruebas de interoperabilidad que acompañan a cada servicio —las firmas se validan con `openssl dgst`, los certificados con `openssl verify`, los sellos con `openssl ts -verify` y los paquetes con `unzip`— y en un paquete de evidencia cuyas instrucciones de verificación la propia suite de pruebas ejecuta literalmente antes de darlas por buenas.

6.3 Limitaciones

El prototipo presenta las siguientes limitaciones que deben considerarse al interpretar sus resultados. En primer lugar, opera bajo una arquitectura de inquilino único (single-tenant): cada instancia sirve a un solo despacho, y un despliegue comercial requeriría aislamiento de datos y de claves entre despachos. En segundo lugar, la CA interna emite certificados que constituyen firma electrónica simple, no avanzada; esto es suficiente para la demostración académica, pero un despliegue productivo debería evaluar la migración a certificados emitidos por un PSC autorizado para obtener la presunción legal del artículo 97 [13].

En tercer lugar, el registro en el sandbox de Cincel se completó, pero la API no ofreció respuesta y estabilidad suficientes para ejecutar de forma repetible la prueba de humo y la campaña externa; además, el uso del sandbox requiere pago con precios no transparentes. Por ello el algoritmo de firma de su TSA comercial y la tasa de verificación sobre una muestra controlada no se miden y quedan fuera del alcance de esta entrega. El adaptador remoto implementa un contrato HTTP documentado en el propio código y se prueba contra un stub HTTP local que lo ejercita tal como está escrito. Entre tanto, la TSA local basada en OpenSSL demuestra el flujo RFC 3161 completo —solicitud, emisión, verificación e interoperabilidad con `openssl ts -verify`— bajo un certificado de sellado emitido por la CA interna. Este resultado acredita el funcionamiento técnico, pero no aporta independencia de tercero ni sustituye una constancia NOM-151 emitida por un PSC autorizado.

En cuarto lugar, la calibración empírica de Argon2id en el hardware de referencia fijó $m = 256$ MiB, $t = 4$, $p = 1$ y arrojó una media de 607.8 milisegundos por intento sobre cinco corridas, dentro de la banda objetivo de 0.5–1.0 segundos definida en §3.11.5. El resultado cierra la calibración para este perfil de hardware; si el despliegue final utiliza otra máquina, debe repetirse con el mismo comando antes de la puesta en producción.

En quinto lugar, la aplicación HTTP entregada opera de forma local y no multiusuario: X-Actor es una etiqueta de auditoría, no una credencial. Los documentos se persisten cifrados en archivos JSON con reemplazo atómico, pero el prototipo aún no almacena estado en PostgreSQL ni en Redis y no implementa autenticación de sesiones con JWT ni RBAC. Dos salvaguardas quedan condicionadas a esa persistencia de producción: la imposición de un solo uso para los códigos TOTP, que exige recordar el último paso temporal aceptado (`docs/adr/0008-totp-single-use-enforcement.md`), y el anclaje externo de la cabeza de la bitácora encadenada, sin el cual la verificación no puede detectar el truncamiento de las entradas más recientes (`docs/adr/0007-audit-chain-anchoring.md`).

Finalmente, la CRL del prototipo tiene una vigencia de siete días, lo que introduce una ventana teórica de exposición entre la revocación de un certificado y su detección por los verificadores; en un entorno productivo esta ventana podría reducirse complementando la CRL con un responder OCSP. Asimismo, el aviso de seguridad RUSTSEC-2023-0071 —un canal lateral de temporización en las operaciones de clave privada de la biblioteca de firma RSA— se aceptó de forma explícita bajo el modelo de amenaza del prototipo, donde la firma se ejecuta localmente y no existe un oráculo de temporización observable por red; el razonamiento completo queda registrado en `docs/adr/0002-`

rsa-signing-crate-and-advisory.md.

6.4 Trabajo futuro

Las líneas de trabajo futuro se organizan en tres horizontes. En el corto plazo: (1) sustituir el repositorio local por la capa de persistencia híbrida con PostgreSQL y Redis diseñada en §3.8, lo que habilita además la imposición de un solo uso para los códigos TOTP y el anclaje de la cabeza de la bitácora de auditoría; (2) ampliar la API REST construida con Axum con sesiones, consultas y versionado adicional, y desarrollar la interfaz web con Astro y Svelte; (3) activar el control de acceso RBAC en tiempo de ejecución, con autenticación de sesiones mediante JWT, para los cuatro roles diferenciados del sistema. La integración con un PSC externo queda como sustitución futura del adaptador, condicionada a un contrato estable y precios verificables, no como requisito para continuar. La calibración Argon2id queda cerrada para el hardware de referencia y se repetirá únicamente si cambia el perfil de despliegue.

En el mediano plazo, orientado a fortalecer la postura de seguridad del sistema: (1) integrar un módulo de seguridad de hardware (HSM) o un servicio de gestión de claves en la nube (KMS) para el almacenamiento de la KEK, eliminando la dependencia de variables de entorno; (2) implementar un responder OCSP que complemente la CRL para reducir la ventana de exposición ante revocaciones; (3) evaluar la migración del formato de sello de tiempo de .tsr independiente a CAdES-T o PAdES-T para ofrecer paquetes de verificación autocontenidos; y (4) incorporar WebAuthn/FIDO2 como opción progresiva de MFA para clientes con dispositivos compatibles, alcanzando el nivel AAL3 de NIST SP 800-63B [56].

En el largo plazo, orientado a la viabilidad comercial: (1) rediseñar la arquitectura para soportar múltiples inquilinos (multi-tenant) con aislamiento criptográfico por despacho, donde cada tenant disponga de su propia KEK y su propio espacio de certificados; (2) migrar los certificados de firmante de la CA interna a un PSC autorizado como SeguriData [45] o CECOBAN [46] para convertir la firma en avanzada conforme al artículo 97 del Código de Comercio [13], beneficiándose de la portabilidad demostrada por la arquitectura de puertos y adaptadores; y (3) extender la bitácora encadenada ya implementada con anclajes periódicos de su cabeza —replicación a almacenamiento independiente o contrafirma mediante sellos de tiempo— para cerrar la brecha de truncamiento documentada y fortalecer la cadena de custodia digital ante instancias judiciales.

La fase concluida deja, en suma, un núcleo criptográfico cuyos resultados puede reproducir y verificar cualquier tercero con herramientas estándar, y una aplicación local que ya consume los mismos puertos desde la CLI y desde HTTP. La siguiente ampliación puede concentrarse en identidad, autorización, persistencia de producción e interfaz de usuario sin trasladar criptografía ni reglas de negocio a la frontera web.

Referencias

- [1] Cámara de Diputados del H. Congreso de la Unión. «Código Nacional de Procedimientos Penales.» Texto Vigente, Última Reforma DOF 19-02-2021. (2021), dirección: <https://www.diputados.gob.mx/LeyesBiblio/pdf/CNPP.pdf>.
- [2] Justia México. «Nuevo Sistema de Justicia Penal,» Justia México Derecho Penal. (2024), dirección: <https://mexico.justia.com/derecho-penal/nuevo-sistema-de-justicia-penal/>.
- [3] Consejo de la Judicatura Federal. «Acuerdo General del Pleno del Consejo de la Judicatura Federal que regula la integración y trámite de expediente electrónico y el uso de videoconferencias,» Suprema Corte de Justicia de la Nación. (2020), dirección: https://www.scjn.gob.mx/sites/default/files/normativa/electronico/documentos/143196_5.pdf.
- [4] Poder Judicial de la Ciudad de México. «Expedientes Digitales con Total Legalidad,» PJCDMX. (2023), dirección: https://www.poderjudicialcdmx.gob.mx/expedientes_digital_es_legalidad/.
- [5] «Humanismo: Cero papel en el Poder Judicial del Estado de México,» Heraldo del Estado de México. (mar. de 2026), dirección: <https://hgrupoeditorial.com/humanismo-25/>.
- [6] A. Bello-Hernández, «Hacia una teoría sobre la e-justice o justicia digital: instrucciones para armar,» *Cuestiones Constitucionales: Revista Mexicana de Derecho Constitucional*, n.º 46, págs. 177-214, 2022. dirección: https://www.scielo.org.mx/scielo.php?script=sci_arttext&pid=S1405-91932022000100177.
- [7] ESET Latinoamérica. «ESET Security Report 2025: El estado de la ciberseguridad en empresas de Latinoamérica,» ESET. (2025), dirección: <https://www.welivesecurity.com/es/informes/eset-security-report-2025-ciberseguridad-empresas-latinoamerica/>.
- [8] ESET Latinoamérica. «Las amenazas más detectadas en Latinoamérica en el primer semestre 2024,» ESET. (2024), dirección: <https://www.eset.com/py/acerca-de-eset/sala-de-prensa/comunicados-de-prensa/articulos-de-prensa/las-amenazas-mas-detectadas-en-latinoamerica-en-el-primer-semestre-2024/>.
- [9] LexLatin. «Ciberseguridad: ¿Cómo las firmas legales protegen los datos de sus clientes en la era de las legaltech?» LexLatin. (2023), dirección: <https://lexlatin.com/reportajes/ciberseguridad-firmas-protegen-datos-clientes-legaltech>.
- [10] American Bar Association. «2023 Cybersecurity TechReport,» ABA Legal Technology Resource Center. (2023), dirección: https://www.americanbar.org/groups/law_practice/resources/tech-report/2023/2023-cybersecurity-techreport/.
- [11] IBM Security. «Cost of a Data Breach Report 2024,» IBM. (2024), dirección: <https://www.clcio.com/blog/data-breach-lawyers/>.

- [12] Enthec. «Ciberseguridad en despachos de abogados,» Enthec. (2025), dirección: <https://enthec.com/ciberseguridad-despachos-abogados/>.
- [13] Cámara de Diputados del H. Congreso de la Unión. «Código de Comercio.» Texto Vigente, Última Reforma DOF 28-03-2018. (2018), dirección: <https://www.diputados.gob.mx/LeyesBiblio/pdf/CCom.pdf>.
- [14] Secretaría de Economía. «NOM-151-SCFI-2016, Requisitos que deben observarse para la conservación de mensajes de datos y digitalización de documentos,» Diario Oficial de la Federación. (2016), dirección: https://www.dof.gob.mx/normasOficiales/6499/seecoll_C/seecoll_C.html (visitado 15-09-2025).
- [15] Mifiel. «Prueba electrónica en México: el valor probatorio del mensaje de datos,» Blog Mifiel. (sep. de 2025), dirección: <https://blog.mifiel.com/prueba-electronica/>.
- [16] UBT Compliance. «Cómo documentar una prueba digital y su cadena de custodia,» UBT Compliance. (jul. de 2025), dirección: <https://ubtcompliance.com/blog/cadena-de-custodia-prueba-digital/>.
- [17] Poder Judicial de la Federación. «Guía para la valoración judicial de la prueba pericial en materia de análisis de video digital,» Órgano de Administración Judicial, México. (), dirección: https://www.oaj.gob.mx/PJD/PJD_resources/guias/lib/P01008.pdf.
- [18] Cámara de Diputados del H. Congreso de la Unión. «Ley Federal de Protección de Datos Personales en Posesión de los Particulares.» Texto Vigente, Última Reforma DOF 25-06-2018. (2018), dirección: <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>.
- [19] D. M'Raihi, S. Machani, M. Pei y J. Rydell, «TOTP: Time-Based One-Time Password Algorithm,» Internet Engineering Task Force, RFC 6238, mayo de 2011. dirección: <https://www.rfc-editor.org/rfc/rfc6238>.
- [20] International Organization for Standardization. «ISO/IEC 27001:2022, Information security, cybersecurity and privacy protection — Information security management systems — Requirements,» ISO/IEC. (2022), dirección: <https://www.iso.org/standard/27001> (visitado 15-09-2025).
- [21] Themis Solutions Inc. «Clio: Legal Practice Management Software,» Clio. (2025), dirección: <https://www.clio.com/> (visitado 22-09-2025).
- [22] 8am. «Legal Case Management Software and Solutions,» MyCase. (2025), dirección: <https://www.mycase.com/> (visitado 17-09-2025).
- [23] PracticePanther. «Law Practice Management Software,» PracticePanther. (2025), dirección: <https://www.practicepanther.com> (visitado 17-09-2025).
- [24] C. Paar y J. Pelzl, *Understanding Cryptography: A Textbook for Students and Practitioners*. Berlin, Heidelberg: Springer-Verlag, 2010. dirección: <https://link.springer.com/book/10.1007/978-3-642-04101-3>.
- [25] National Institute of Standards and Technology, «Secure Hash Standard (SHS),» National Institute of Standards y Technology, inf. téc. NIST FIPS 180-4, ago. de 2015. doi: [10.6028/NIST.FIPS.180-4](https://doi.org/10.6028/NIST.FIPS.180-4).
- [26] National Institute of Standards and Technology, «Advanced Encryption Standard (AES),» National Institute of Standards y Technology, inf. téc. NIST FIPS 197, nov. de 2001, Updated May 2023. doi: [10.6028/NIST.FIPS.197-upd1](https://doi.org/10.6028/NIST.FIPS.197-upd1).

- [27] C. Adams, P. Cain, D. Pinkas y R. Zuccherato, «Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP),» Internet Engineering Task Force, RFC 3161, ago. de 2001. dirección: <https://www.rfc-editor.org/rfc/rfc3161>.
- [28] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7.^a ed. Upper Saddle River, NJ, USA: Pearson, 2017.
- [29] Archivo General del Estado de Puebla. «La evolución del Sistema Penal Acusatorio,» Artículos de Investigación. (2020), dirección: https://archivogeneraldelestado.puebla.gob.mx/articulos-de-investigacion/item/download/4_b2f24cd4244c77bfbe5c964ab5545.
- [30] Instituto de Estudios Judiciales de la CDMX. «Unidades de Gestión Judicial en el Sistema Acusatorio,» IEJ CDMX. (2018), dirección: <https://www.iejcdmx.gob.mx/wp-content/uploads/UGAS070518.pdf>.
- [31] La Salle Bajío, «El sistema acusatorio penal mexicano,» *De La Salle, Revista de Derecho*, n.º 13, 2015. dirección: https://www.lasallebajio.edu.mx/delasalle/contenidos/revistas/derecho/numero_13/maestros_elsistema.html.
- [32] Órgano de Administración Judicial. «La e-Justicia en el Consejo de la Judicatura Federal,» OAJ. (2023), dirección: <https://apps.cjf.gob.mx/dgetd/assets/PDF/ebook.pdf>.
- [33] M. A. García Ramírez, «Etapas de Investigación en el CNPP,» en *Código Nacional de Procedimientos Penales Comentado*, UNAM, 2015, cap. 5. dirección: <https://archivos.juridicas.unam.mx/www/bjv/libros/11/5262/5.pdf>.
- [34] Poder Judicial del Estado de Guanajuato. «Los sujetos procesales en el juicio oral penal,» PJ Guanajuato. (2015), dirección: <https://www.poderjudicial-gto.gob.mx/pdfs/2encuentro/LIC.%20MYRNA%20EDITH%20HIGAREDA%20LORENZO.pdf>.
- [35] Gobierno de México. «¿Quiénes son los actores del nuevo Sistema de Justicia Penal?» gob.mx. (2022), dirección: <https://www.gob.mx/justiciapenal/articulos/quienes-son-los-actores-del-nuevo-sistema-de-justiciapenal>.
- [36] Órgano de Administración Judicial. «Guía de apoyo para el estudio del Código Nacional de Procedimientos Penales,» OAJ. (2023), dirección: <https://www.oaj.gob.mx/documentos/guiacnpp.pdf>.
- [37] E. Barker, «Recommendation for Key Management: Part 1 – General,» National Institute of Standards y Technology, NIST Special Publication 800-57 Part 1, Rev. 5, mayo de 2020. doi: 10.6028/NIST.SP.800-57pt1r5. dirección: <https://doi.org/10.6028/NIST.SP.800-57pt1r5>.
- [38] Cámara de Diputados del H. Congreso de la Unión. «Código Federal de Procedimientos Civiles.» Texto Vigente, Última Reforma DOF 07-06-2023, Diario Oficial de la Federación. (2023), dirección: <https://www.diputados.gob.mx/LeyesBiblio/pdf/CFPC.pdf>.
- [39] Cámara de Diputados del H. Congreso de la Unión. «Ley de Firma Electrónica Avanzada.» Texto Vigente, DOF 11-01-2012, Diario Oficial de la Federación. (2012), dirección: <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFEA.pdf>.
- [40] Secretaría de Economía. «Directorio de Prestadores de Servicios de Certificación,» psc.economia.gob.mx. (2026), dirección: <https://psc.economia.gob.mx/directorio.html>.

- [41] Mifiel. «API para constancias de conservación NOM-151,» Blog Mifiel. (2024), dirección: <https://blog.mifiel.com/api-nom-151-constancias-conservacion/>.
- [42] Secretaría de Economía. «Política de Certificados de la Autoridad Certificadora Raíz Segunda de la Secretaría de Economía (ACR2-SE),» Secretaría de Economía. (), dirección: https://psc.economia.gob.mx/docs_l/Politica_de_Certificados.pdf.
- [43] Cámara de Diputados del H. Congreso de la Unión. «Reglamento del Código de Comercio en Materia de Prestadores de Servicios de Certificación.» Texto Vigente, Diario Oficial de la Federación. (), dirección: https://www.diputados.gob.mx/LeyesBiblio/regley/Reg_CCom_MPSC.pdf.
- [44] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley y W. Polk, «Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile,» Internet Engineering Task Force, RFC 5280, mayo de 2008. dirección: <https://www.rfc-editor.org/rfc/rfc5280>.
- [45] SeguriData. «Política de Certificados para Infraestructura de Clave Pública,» SeguriData PSC. (), dirección: <https://psc.seguridata.com/docs/doc06.pdf>.
- [46] CECOBAN. «Política de Certificación de la Agencia Certificadora,» CECOBAN S.A. de C.V. (), dirección: <https://www.cecoban.org.mx/AgenciaSE/Soporte/Politicasy20de%20Certificacion.pdf>.
- [47] comparasoftware.com. «Cinel: Acreditado como el Primer Prestador de Servicios de Certificación nativo en la nube.» (2026), dirección: <https://www.comparasoftware.com/cinel>.
- [48] A. J. Menezes, P. C. van Oorschot y S. A. Vanstone, *Handbook of Applied Cryptography*. Boca Raton, FL, USA: CRC Press, 1996. dirección: <https://cacr.uwaterloo.ca/hac/>.
- [49] M. Dworkin, «Recommendation for Block Cipher Modes of Operation: Methods and Techniques,» National Institute of Standards y Technology, NIST Special Publication 800-38A, dic. de 2001. doi: [10.6028/NIST.SP.800-38A](https://doi.org/10.6028/NIST.SP.800-38A). dirección: <https://doi.org/10.6028/NIST.SP.800-38A>.
- [50] M. Dworkin, «Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,» National Institute of Standards y Technology, NIST Special Publication 800-38D, nov. de 2007. dirección: <https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-38d.pdf>.
- [51] D. McGrew, «An Interface and Algorithms for Authenticated Encryption,» Internet Engineering Task Force, RFC 5116, ene. de 2008. dirección: <https://www.rfc-editor.org/rfc/rfc5116>.
- [52] M. Dworkin, «Recommendation for Block Cipher Modes of Operation: Methods for Key Wrapping,» National Institute of Standards y Technology, NIST Special Publication 800-38F, dic. de 2012. doi: [10.6028/NIST.SP.800-38F](https://doi.org/10.6028/NIST.SP.800-38F). dirección: <https://doi.org/10.6028/NIST.SP.800-38F>.
- [53] National Institute of Standards and Technology, «Digital Signature Standard (DSS),» National Institute of Standards y Technology, NIST FIPS 186-5, feb. de 2023. doi: [10.6028/NIST.FIPS.186-5](https://doi.org/10.6028/NIST.FIPS.186-5). dirección: <https://doi.org/10.6028/NIST.FIPS.186-5>.

- [54] International Telecommunication Union. «Information technology – Open Systems Interconnection – The Directory: Public-key and attribute certificate frameworks,» ITU-T Rec. X.509. (oct. de 2019), dirección: <https://www.itu.int/rec/T-REC-X.509-201910-I>.
- [55] National Institute of Standards and Technology, «Digital Identity Guidelines: Authentication and Lifecycle Management,» National Institute of Standards y Technology, NIST Special Publication 800-63B, jun. de 2017. dirección: <https://pages.nist.gov/800-63-3/sp800-63b.html>.
- [56] National Institute of Standards and Technology, «Digital Identity Guidelines: Authentication and Lifecycle Management,» National Institute of Standards and Technology, NIST Special Publication 800-63B Rev. 4, ago. de 2024. dirección: <https://pages.nist.gov/800-63-4/sp800-63b.html>.
- [57] Q. Dang, «Recommendation for Applications Using Approved Hash Algorithms,» National Institute of Standards and Technology, NIST Special Publication 800-107 Rev. 1, ago. de 2012. doi: [10.6028/NIST.SP.800-107r1](https://doi.org/10.6028/NIST.SP.800-107r1).
- [58] D. M'Raihi, M. Bellare, F. Hoornaert, D. Naccache y O. Ranen, «HOTP: An HMAC-Based One-Time Password Algorithm,» Internet Engineering Task Force, RFC 4226, dic. de 2005. dirección: <https://www.rfc-editor.org/rfc/rfc4226>.
- [59] A. Biryukov, D. Dinu y D. Khovratovich. «Argon2: the memory-hard function for password hashing and other applications.» Password Hashing Competition winner. (2015), dirección: <https://www.password-hashing.net/>.
- [60] A. Biryukov, D. Dinu, D. Khovratovich y S. Josefsson, «Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications,» Internet Research Task Force, RFC 9106, sep. de 2021. dirección: <https://www.rfc-editor.org/rfc/rfc9106>.
- [61] OWASP Foundation. «Password Storage Cheat Sheet,» OWASP Cheat Sheet Series. (2024), dirección: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html.
- [62] National Institute of Standards and Technology, «Security and Privacy Controls for Information Systems and Organizations,» National Institute of Standards y Technology, inf. téc. NIST Special Publication 800-53, Rev. 5, 2020. dirección: <https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final> (visitado 19-09-2025).
- [63] K. Beck y C. Andres, *Extreme Programming Explained: Embrace Change*, 2.^a ed. Boston, MA, USA: Addison-Wesley, 2004.
- [64] S. Brown, *The C4 Model for Visualising Software Architecture: Definitions, notation and examples*. Leanpub, 2024. dirección: <https://c4model.com>.
- [65] P. Kruchten, «The 4+1 View Model of Architecture,» *IEEE Software*, vol. 12, n.º 6, págs. 42-50, nov. de 1995. doi: [10.1109/52.469759](https://doi.org/10.1109/52.469759).
- [66] M. Nygard. «Documenting Architecture Decisions,» cognitect.com. (nov. de 2011), dirección: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>.

- [67] International Organization for Standardization. «ISO/IEC 25010:2011, Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models,» ISO/IEC. (2011), dirección: <https://www.iso.org/standard/35733.html>.
- [68] N. Ford, R. Parsons y P. Kua, *Building Evolutionary Architectures: Automated Software Governance*, 2.^a ed. Sebastopol, CA, USA: O'Reilly Media, 2023.
- [69] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Upper Saddle River, NJ, USA: Prentice Hall, 2017.
- [70] A. Cockburn. «Hexagonal Architecture.» (2005), dirección: <https://alistair.cockburn.us/hexagonal-architecture/>.
- [71] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2.^a ed. Sebastopol, CA, USA: O'Reilly Media, 2021.
- [72] M. Fowler. «MonolithFirst.» (2015), dirección: <https://martinfowler.com/bliki/MonolithFirst.html>.
- [73] V. Vernon, *Implementing Domain-Driven Design*. Upper Saddle River, NJ, USA: Addison-Wesley Professional, 2013.
- [74] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley Professional, 2003.
- [75] S. Klabnik y C. Nichols, *The Rust Programming Language*, 2.^a ed. San Francisco, CA, USA: No Starch Press, 2023. dirección: <https://doc.rust-lang.org/book/>.
- [76] Microsoft Security Response Center. «A proactive approach to more secure code.» (2019), dirección: <https://msrc.microsoft.com/blog/2019/07/a-proactive-approach-to-more-secure-code/>.
- [77] K. Sylor-Miller y J. Miller. «Islands Architecture.» (2019), dirección: <https://jasonformat.com/islands-architecture/>.
- [78] J. H. Saltzer y M. D. Schroeder, «The Protection of Information in Computer Systems,» *Proceedings of the IEEE*, vol. 63, n.º 9, págs. 1278-1308, 1975. doi: [10.1109/PROC.1975.9939](https://doi.org/10.1109/PROC.1975.9939).
- [79] OWASP Foundation. «Application Security Verification Standard (ASVS) v4.0.3,» OWASP. (2021), dirección: <https://owasp.org/www-project-application-security-verification-standard/>.
- [80] A. Shostack, *Threat Modeling: Designing for Security*. Indianapolis, IN, USA: Wiley, 2014.
- [81] Cincel. «Documentación API v3: Timestamps NOM-151,» Cincel Certificadora Digital. (2024), dirección: <https://docs.cincel.digital/v3/timestamps/nom151/>.
- [82] OWASP Foundation. «Multifactor Authentication Cheat Sheet,» OWASP Cheat Sheet Series. (2024), dirección: https://cheatsheetseries.owasp.org/cheatsheets/Multifactor_Authentication_Cheat_Sheet.html.
- [83] RustSec Advisory Database. «RUSTSEC-2023-0071: Marvin Attack: potential key recovery through timing side channels.» (2023), dirección: <https://rustsec.org/advisories/RUSTSEC-2023-0071.html>.

- [84] S. Josefsson, «The Base16, Base32, and Base64 Data Encodings,» Internet Engineering Task Force, RFC 4648, oct. de 2006. dirección: <https://www.rfc-editor.org/rfc/rfc4648>.
- [85] PKWARE, Inc. «APPNOTE.TXT — .ZIP File Format Specification, Version 6.3.10.» (2022), dirección: <https://pkware.cachefly.net/webdocs/casestudies/APPNOTE.TXT>.
- [86] K. Claessen y J. Hughes, «QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs,» en *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00)*, ACM, 2000, págs. 268-279. doi: [10.1145/351240.351266](https://doi.org/10.1145/351240.351266).
- [87] D. A. McGrew y J. Viega. «The Galois/Counter Mode of Operation (GCM),» Submission to the NIST Modes of Operation Process. (2004), dirección: <https://csrc.nist.gov/csrc/media/projects/block-cipher-techniques/documents/bcm/proposed-modes/gcm/gcm-spec.pdf>.
- [88] The OpenSSL Project. «OpenSSL Documentation: openssl-dgst, openssl-verify, openssl-ts.» (2026), dirección: <https://docs.openssl.org/>.

Glosario de Términos

- AAD** Additional Authenticated Data (Datos Adicionales Autenticados). Información que no se cifra pero cuya integridad se protege mediante la etiqueta de autenticación del cifrado autenticado GCM.
- AAL** Authentication Assurance Level (Nivel de Aseguramiento de Autenticación). Clasificación del NIST SP 800-63B que define tres niveles de confianza en la autenticación: AAL1 (factor único), AAL2 (dos factores) y AAL3 (dos factores con hardware criptográfico).
- ACR2-SE** Autoridad Certificadora Raíz Segunda de la Secretaría de Economía. Raíz de la jerarquía PKI mexicana bajo la cual operan los PSC autorizados.
- AEAD** Authenticated Encryption with Associated Data (Cifrado Autenticado con Datos Asociados). Esquema criptográfico que provee simultáneamente confidencialidad, integridad y autenticación del mensaje. AES-GCM es una instancia de AEAD.
- AES** Advanced Encryption Standard (Estándar de Cifrado Avanzado). Algoritmo de cifrado simétrico por bloques estandarizado en FIPS 197. AES-256 utiliza claves de 256 bits.
- API** Application Programming Interface (Interfaz de Programación de Aplicaciones). Conjunto de definiciones y protocolos que permite la comunicación entre componentes de software.
- Argon2id** Variante híbrida de la función de hashing de contraseñas Argon2, ganadora de la Password Hashing Competition (2015) y estandarizada en RFC 9106. Combina resistencia a ataques de canal lateral (Argon2i) con resistencia a hardware especializado (Argon2d).
- ASIC** Application-Specific Integrated Circuit (Circuito Integrado de Aplicación Específica). Hardware diseñado para ejecutar una tarea específica, utilizado en ataques de fuerza bruta contra funciones hash.
- ASN.1** Abstract Syntax Notation One (Notación de Sintaxis Abstracta Uno). Estándar internacional para definir estructuras de datos criptográficas, utilizado en certificados X.509 y sellos de tiempo RFC 3161.
- backend** Capa lógica del servidor que procesa la lógica de negocio, el acceso a datos y los servicios criptográficos, en contraposición al frontend que interactúa con el usuario.
- bcrypt** Función de hashing de contraseñas diseñada en 1999, basada en el cifrador Blowfish. Incorpora un factor de costo ajustable pero carece de dureza de memoria configurable.
- CA** Certificate Authority (Autoridad Certificadora). Entidad de confianza responsable de emitir, renovar y revocar certificados digitales dentro de una infraestructura de clave pública (PKI).
- CAdES-T** CMS Advanced Electronic Signatures with Timestamp. Formato de firma electrónica avanzada que incorpora un sello de tiempo dentro de una estructura CMS (Cryptographic Message Syntax).

- CAVP** Cryptographic Algorithm Validation Program (programa de validación de algoritmos criptográficos) del NIST, que publica suites de vectores de prueba oficiales para verificar la corrección de implementaciones de algoritmos aprobados.
- CI** Integración continua (Continuous Integration). Práctica de ingeniería que compila y ejecuta las verificaciones automatizadas de un proyecto sobre cada cambio propuesto, antes de integrarlo a la rama principal del repositorio.
- CLI** Interfaz de línea de comandos (Command-Line Interface). Adaptador que expone las operaciones de un sistema mediante comandos de texto ejecutados en una terminal, con entradas y salidas aptas para la automatización con scripts.
- CMS** Sintaxis de Mensajes Criptográficos (Cryptographic Message Syntax). Formato estándar del IETF para datos firmados o cifrados; los tokens de sellado de tiempo RFC 3161 son estructuras CMS de tipo SignedData.
- crate** Unidad de compilación y distribución de código en el ecosistema de Rust. Un crate puede ser una biblioteca o un binario, declara sus dependencias en un manifiesto y se publica y versiona de forma independiente.
- CRC-32** Cyclic Redundancy Check (verificación por redundancia cíclica) de 32 bits; suma de comprobación empleada por el formato ZIP para detectar corrupción accidental de los datos almacenados. Detecta errores de transmisión o almacenamiento, pero no es un mecanismo criptográfico frente a adversarios.
- CRL** Certificate Revocation List (Lista de Certificados Revocados). Documento firmado por una CA que enumera los certificados revocados antes de su fecha de expiración, conforme a RFC 5280.
- CSPRNG** Cryptographically Secure Pseudo-Random Number Generator (Generador de Números Pseudoaleatorios Criptográficamente Seguro). Generador cuya salida es computacionalmente indistinguible de datos aleatorios verdaderos.
- CSR** Certificate Signing Request (Solicitud de Firma de Certificado). Mensaje enviado a una CA para solicitar la emisión de un certificado digital, conteniendo la clave pública del solicitante.
- DEK** Data Encryption Key (Clave de Cifrado de Datos). En un esquema de envelope encryption, clave simétrica generada aleatoriamente para cifrar un documento individual.
- DER** Distinguished Encoding Rules. Reglas de codificación binaria para estructuras ASN.1, utilizadas en certificados X.509 y sellos de tiempo.
- DOF** Diario Oficial de la Federación. Órgano del Gobierno de México para la publicación de leyes, reglamentos y acuerdos oficiales.
- e.firma** Certificado de firma electrónica avanzada emitido por el Servicio de Administración Tributaria (SAT) para personas físicas y morales en México. Utiliza RSA-2048 con SHA-256.
- ECDSA** Elliptic Curve Digital Signature Algorithm (Algoritmo de Firma Digital de Curva Elíptica). Esquema de firma basado en la dificultad del logaritmo discreto sobre curvas elípticas.
- endpoint** Punto de acceso de una API, identificado por una URL y un método HTTP (por ejemplo, GET /pki/crl.pem).

- envelope encryption** Cifrado de envoltente. Esquema de gestión de claves donde una clave maestra (KEK) cifra claves de datos individuales (DEK), cada una asociada a un documento específico.
- FIPS** Federal Information Processing Standards (Estándares Federales de Procesamiento de Información). Conjunto de estándares del gobierno de Estados Unidos para sistemas informáticos. FIPS 180-4 define SHA-256; FIPS 197 define AES.
- fitness function** Función de aptitud arquitectónica. Prueba automatizada que verifica de forma continua y objetiva una característica estructural de la arquitectura (por ejemplo, la dirección de las dependencias o la ausencia de secretos en las bitácoras), de modo que su degradación rompa la integración continua.
- framework** Marco de trabajo o plataforma de desarrollo que proporciona una estructura predefinida para construir aplicaciones de software.
- GCM** Galois/Counter Mode (Modo Galois/Contador). Modo de operación de cifrado por bloques que provee cifrado autenticado (AEAD), especificado en NIST SP 800-38D.
- GPU** Graphics Processing Unit (Unidad de Procesamiento Gráfico). Procesador diseñado para cómputo paralelo masivo, utilizado también en ataques de fuerza bruta contra funciones hash.
- hash** Resumen criptográfico o huella digital. Valor de longitud fija producido por una función hash criptográfica a partir de una entrada de longitud arbitraria. Cualquier modificación en la entrada produce un hash completamente diferente.
- HKDF** HMAC-based Key Derivation Function (Función de Derivación de Claves basada en HMAC). Mecanismo para derivar múltiples claves criptográficas a partir de una clave maestra y un contexto único.
- HMAC** Hash-based Message Authentication Code (Código de Autenticación de Mensajes basado en Hash). Construcción criptográfica que combina una función hash con una clave secreta para verificar integridad y autenticidad.
- HOTP** HMAC-Based One-Time Password (Contraseña de Un Solo Uso basada en HMAC). Algoritmo definido en RFC 4226 que genera códigos de un solo uso a partir de un secreto compartido y un contador.
- HSM** Hardware Security Module (módulo de seguridad de hardware). Dispositivo dedicado a custodiar claves criptográficas y a ejecutar operaciones con ellas sin exponer el material de clave al sistema anfitrión.
- IETF** Internet Engineering Task Force. Organismo internacional que desarrolla y publica estándares de protocolos de Internet, incluyendo las RFC.
- IV** Initialization Vector (Vector de Inicialización). Valor aleatorio o único utilizado junto con una clave para iniciar un algoritmo de cifrado, garantizando que el mismo texto plano produzca diferentes textos cifrados.
- JSON** Notación de Objetos de JavaScript (JavaScript Object Notation). Formato textual ligero de intercambio de datos estructurados, empleado por el prototipo para la salida legible por máquina de la interfaz de línea de comandos y para la bitácora en formato JSON Lines.

- JSONL** JSON Lines: formato de texto en el que cada línea del archivo es un objeto JSON independiente; lo emplea la bitácora de auditoría del prototipo para persistir un evento encadenado por línea.
- JWT** JSON Web Token. Estándar abierto (RFC 7519) para la creación de tokens de acceso firmados digitalmente en formato JSON, utilizados para autenticación y autorización.
- KEK** Key Encryption Key (Clave de Cifrado de Clave). En un esquema de envelope encryption, clave maestra utilizada exclusivamente para cifrar y descifrar las DEKs.
- KMS** Key Management Service (servicio de gestión de claves). Servicio, generalmente en la nube, que centraliza la generación, custodia, rotación y control de acceso de claves criptográficas.
- legacy-use** Uso heredado. Clasificación del NIST para algoritmos o tamaños de clave que son aceptables para procesar información existente pero no recomendados para proteger información nueva.
- memory-hardness** Dureza de memoria. Propiedad de una función criptográfica que requiere una cantidad configurable de memoria durante su ejecución, encareciendo proporcionalmente los ataques con hardware paralelo.
- MFA** Multi-Factor Authentication (Autenticación por Múltiples Factores). Mecanismo que requiere la presentación de al menos dos categorías de evidencia de identidad: algo que el usuario sabe, posee o es.
- middleware** Capa de software intermedia que intercepta y procesa peticiones HTTP entre el cliente y la lógica de la aplicación, realizando funciones transversales como autenticación, registro de actividad o limitación de frecuencia.
- mock** Doble de prueba que implementa la interfaz de una dependencia y permite programar sus respuestas y verificar las invocaciones recibidas; se emplea para probar la orquestación de un componente en aislamiento.
- MSRV** Versión mínima soportada de Rust (Minimum Supported Rust Version). Versión más antigua del compilador de Rust con la que un proyecto se compromete a compilar, declarada en el manifiesto del workspace y verificada en integración continua.
- NIST** National Institute of Standards and Technology (Instituto Nacional de Estándares y Tecnología). Organismo del gobierno de Estados Unidos que publica estándares criptográficos y guías de seguridad.
- NOM** Norma Oficial Mexicana. Regulación técnica de observancia obligatoria emitida por dependencias del gobierno federal de México. NOM-151-SCFI-2016 establece los requisitos para la conservación de mensajes de datos.
- nonce** Número de uso único (del inglés number used once). Valor que no debe repetirse con la misma clave en operaciones criptográficas. En AES-GCM, su reutilización compromete catastróficamente el esquema.
- OCSP** Online Certificate Status Protocol (Protocolo de Estado de Certificado en Línea). Protocolo que permite consultar en tiempo real el estado de revocación de un certificado ante un servidor autorizado.

- OID** Object Identifier (Identificador de Objeto). Secuencia numérica única que designa un algoritmo, atributo u objeto en el estándar ASN.1. Ejemplo: 1.2.840.113549.1.1.11 identifica sha256WithRSAEncryption.
- OWASP** Open Worldwide Application Security Project. Proyecto abierto de referencia internacional en seguridad de aplicaciones web, que publica guías de mejores prácticas y listas de vulnerabilidades.
- PAdES-T** PDF Advanced Electronic Signatures with Timestamp. Formato de firma electrónica avanzada embebida dentro de un documento PDF, con sello de tiempo incorporado.
- PEM** Privacy-Enhanced Mail. Formato de codificación Base64 para certificados, claves criptográficas y listas de revocación, delimitado por encabezados como —BEGIN CERTIFICATE—.
- PHC** Password Hashing Competition (Competencia de Hashing de Contraseñas). Concurso criptográfico internacional celebrado entre 2013 y 2015 con 24 candidatos, cuyo ganador fue Argon2.
- PKCS#1** Public-Key Cryptography Standards #1. Estándar que define el formato y los esquemas de firma y cifrado RSA, incluyendo los esquemas de relleno (padding) v1.5 y PSS.
- PKI** Public Key Infrastructure (Infraestructura de Clave Pública). Conjunto de entidades, políticas y procedimientos para crear, gestionar, distribuir y revocar certificados digitales.
- PSC** Prestador de Servicios de Certificación. Entidad autorizada por la Secretaría de Economía de México para emitir certificados digitales, sellos de tiempo y constancias de conservación bajo la jerarquía ACR2-SE.
- rate limiting** Limitación de frecuencia. Mecanismo de seguridad que restringe el número de peticiones o intentos que un usuario puede realizar en un período de tiempo, previniendo ataques de fuerza bruta.
- RBAC** Role-Based Access Control (Control de Acceso Basado en Roles). Modelo de autorización que asigna permisos a roles predefinidos y asocia cada usuario a uno o más roles.
- REST** Representational State Transfer (Transferencia de Estado Representacional). Estilo arquitectónico para diseñar servicios web basado en recursos identificados por URLs y operados mediante métodos HTTP estándar.
- RFC** Request for Comments (Solicitud de Comentarios). Serie de documentos técnicos publicados por el IETF que definen estándares y protocolos de Internet. Cada RFC se identifica con un número único (por ejemplo, RFC 3161).
- RSA** Rivest-Shamir-Adleman. Algoritmo de criptografía asimétrica basado en la dificultad de factorizar números enteros grandes, utilizado para firma digital y cifrado de clave pública.
- salt** Sal. Valor aleatorio único que se concatena con la contraseña antes de aplicar la función hash, garantizando que contraseñas idénticas produzcan hashes diferentes.
- sandbox** Entorno de pruebas aislado que replica las funcionalidades de un sistema en producción sin afectar datos reales ni generar efectos legales.
- SAT** Servicio de Administración Tributaria. Órgano del gobierno de México responsable de la recaudación fiscal y emisor de la e.firma para personas físicas y morales.

- SHA-256** Secure Hash Algorithm de 256 bits. Función hash criptográfica de la familia SHA-2 estandarizada en FIPS 180-4, que produce un resumen de 256 bits (32 bytes) y proporciona 128 bits de resistencia a colisiones.
- SP** Special Publication (Publicación Especial). Serie de documentos técnicos del NIST que contienen guías, recomendaciones y estándares de seguridad. Ejemplo: NIST SP 800-57.
- spike** Exploración técnica acotada en tiempo cuyo objetivo es reducir la incertidumbre de una decisión de diseño o de integración antes de comprometer el desarrollo, produciendo evidencia ejecutable en lugar de funcionalidad terminada.
- stub** Doble de prueba que sustituye a un servicio externo respondiendo con datos prefabricados; en las pruebas del prototipo, un servidor HTTP local simula al proveedor de sellado de tiempo.
- TDD** Test-Driven Development (desarrollo guiado por pruebas). Disciplina en la que primero se escribe una prueba automatizada que falla, después el código mínimo que la hace pasar y finalmente se refactoriza, de modo que toda la funcionalidad queda cubierta por pruebas desde su origen.
- TOTP** Time-Based One-Time Password (Contraseña de Un Solo Uso basada en Tiempo). Algoritmo definido en RFC 6238 que genera códigos temporales a partir de un secreto compartido y la hora actual, utilizado como segundo factor de autenticación.
- TSA** Time-Stamp Authority (Autoridad de Sellado de Tiempo). Entidad de confianza que emite sellos de tiempo criptográficos conforme al protocolo RFC 3161, acreditando la existencia de un documento en un momento determinado.
- UUID** Identificador Único Universal (Universally Unique Identifier), valor de 128 bits empleado para identificar recursos con probabilidad de colisión despreciable.
- WebAuthn** Web Authentication. Estándar del W3C y FIDO Alliance para autenticación sin contraseña basada en criptografía de clave pública, mediante llaves de seguridad físicas o biometría del dispositivo.
- workspace** Espacio de trabajo de Cargo que agrupa varios crates bajo un manifiesto raíz común, compartiendo el archivo de bloqueo de versiones y el directorio de compilación, y permitiendo declarar dependencias y perfiles una sola vez para todos los miembros.
- X.509** Estándar de la ITU-T que define el formato de los certificados digitales de clave pública, incluyendo la estructura de la cadena de confianza, las extensiones y los perfiles de listas de revocación (CRL).
- ZIP** Formato contenedor que agrupa uno o más archivos, con o sin compresión; el paquete de evidencia del prototipo emplea su subconjunto almacenado, sin compresión.

APÉNDICE A

Diagramas detallados de arquitectura

Este anexo presenta las versiones detalladas de los diagramas arquitectónicos cuya densidad de información requiere la orientación apaisada para mantener la legibilidad de todas sus etiquetas. Las figuras de este anexo se referencian desde las subsecciones correspondientes del cuerpo del Capítulo 3.

A.1 Vista detallada de Fronteras de Confianza

Modelo completo de defensa en profundidad: cuatro zonas de confianza (red pública, perímetro de aplicación, núcleo aplicativo, datos persistentes y secretos), tres fronteras con sus respectivos controles de seguridad (TLS 1.3, Rate Limiting, WAF/CORS, Autenticación JWT, Validación de input, Autorización RBAC, Conexiones TLS internas, Consultas parametrizadas vía sqlx, Cifrado en reposo mediante envelope encryption), y los servicios externos (Cincel PSC, Proveedor SMTP) ubicados en la zona no confiable con sus canales seguros explícitos.

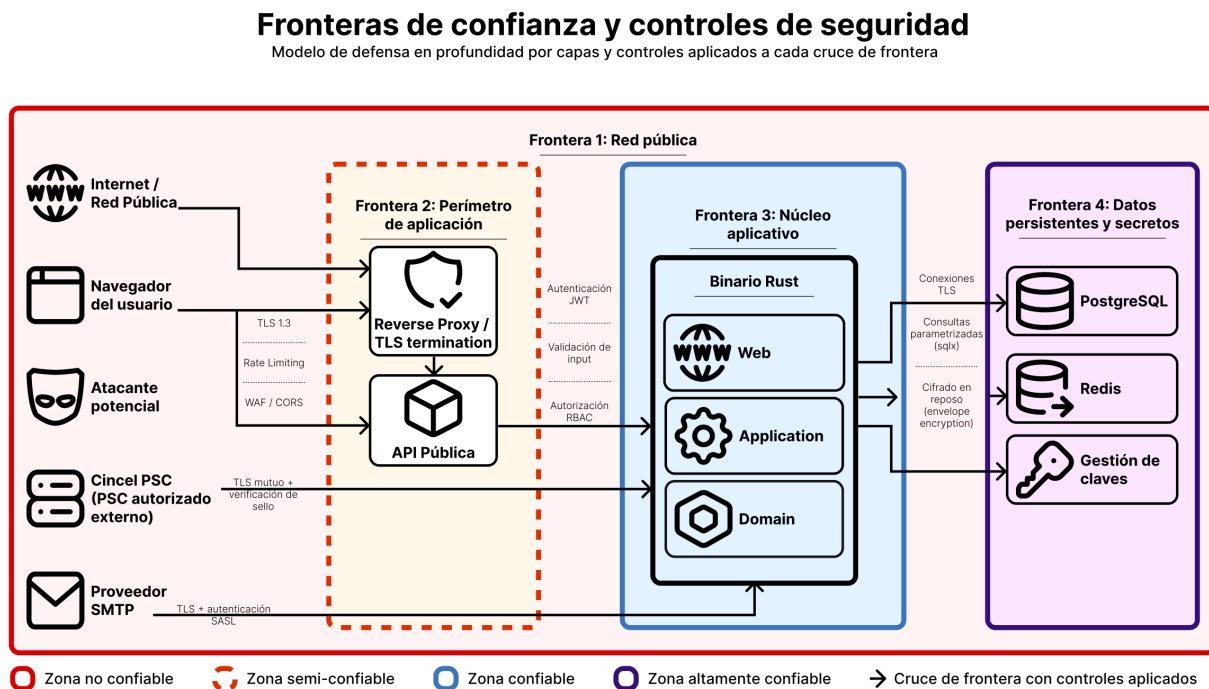


Figura A.1: Modelo de defensa en profundidad por capas y controles aplicados a cada cruce de frontera.

A.2 Arquitectura Hexagonal completa con puertos y adaptadores nombrados

Implementación detallada de la arquitectura hexagonal del prototipo: todos los puertos driving del sistema (RestApiPort sobre Axum, CommandLinePort sobre Clap, WebSocketPort sobre Axum WebSocket), todos los puertos driven (CaseRepository, DocumentRepository, DocumentSigner, TimestampService, NotificationDispatcher, AuditLogger, SessionStore) y sus adaptadores concretos correspondientes. Cada puerto se materializa como un trait de Rust en el crate domain, y cada adaptador como una struct con bloque impl en el crate infrastructure.

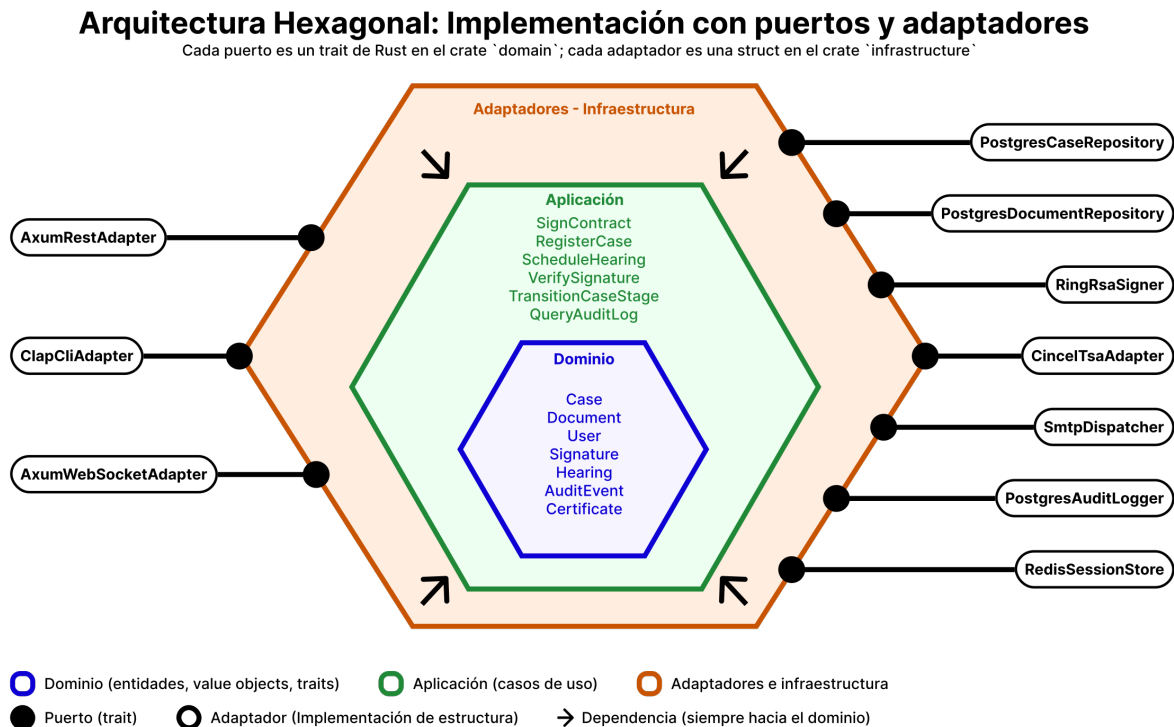


Figura A.2: Arquitectura hexagonal completa: cada puerto es un trait de Rust en el crate domain; cada adaptador es una struct en el crate infrastructure.

APÉNDICE B

Manual de la interfaz de línea de comandos

Este anexo es el manual de referencia de `despacho-cli`, la interfaz de línea de comandos del prototipo. El binario lo produce el `crate bin` del workspace de Cargo, que opera como raíz de composición de la arquitectura hexagonal: cada subcomando corresponde a exactamente un caso de uso de la capa de aplicación, y su manejador instancia los adaptadores concretos de infraestructura y los inyecta en dicho caso de uso. El árbol de mandos está definido en `crates/bin/src/cli.rs`. Las salidas que acompañan a los ejemplos son reales: provienen de la transcripción versionada `docs/demo-transcript.md`, capturada el 12 de julio de 2026 con `despacho-cli 0.1.0` y `OpenSSL 3.5.7`; en ellas, `$REPO` abrevia la raíz del repositorio y `$WORK` el directorio temporal de trabajo de aquella ejecución.

B.1 Convenciones generales

El árbol completo de subcomandos es el siguiente; los corchetes denotan elementos opcionales y los ángulos, valores que aporta el usuario:

```
despacho-cli [--json]
  serve --signer-cert <PATH> --signer-key <PATH>
    [--bind <ADDR>] [--data-dir <DIR>] [--ca-cert <PATH>]
    [--crl <PATH>] [--tsa-config <PATH>] [--tsa-dir <DIR>]
  crypto hash <FILE>
  vault encrypt <FILE> --doc-id <UUID> [--version <N>]
  vault decrypt <PACKAGE> --doc-id <UUID> [--version <N>] [--out <PATH>]
  vault rotate-kek --file <PATH>
  pki [--scripts-dir <DIR>] {init-ca | issue --cn <CN> |
    revoke --serial <HEX> | gen-crl | show <CERT>}
  sign <FILE> --cert <PATH> --key <PATH>
  timestamp <INPUT> [--mock] [--out <PATH>] [--pki-dir <DIR>]
  verify <FILE> --sig <PATH> --cert <PATH> --ca <PATH> [--tsr <PATH>]
    [--crl <PATH>] [--tsa-cert <PATH>] [--at <RFC3339>]
  package export <FILE> --sig <PATH> --tsr <PATH> --cert <PATH>
    --ca <PATH> --crl <PATH> [--tsa-chain <PATH>] --out <PATH>
  auth {calibrate | hash-password | verify-password --hash <PHC>}
  auth totp enroll --user <EMAIL> --secret-out <PATH>
  auth totp verify --code <CODE> --secret-file <PATH>
  audit {append --action <STRING> --resource <STRING> |
    verify-chain | show}
```

Tres convenciones gobiernan a los subcomandos administrativos. Primera, la separación de flujos: los resultados se escriben en la salida estándar (`stdout`) y los diagnósticos en el flujo de

error (`stderr`), de modo que la salida pueda encadenarse en tuberías sin contaminación; el nivel de diagnóstico se controla con la variable de entorno `RUST_LOG`. Segunda, la bandera global `--json`: cuando está activa, cada operación finita imprime un objeto JSON legible por máquina en lugar del texto para humanos; `vault decrypt` sin `--out` escribe el claro crudo y `serve` es un proceso persistente cuyo contrato se consume por HTTP. Tercera, los códigos de salida: el proceso termina con código distinto de cero ante cualquier error y también —de forma deliberada, para que los scripts puedan bifurcar directamente— cuando un resultado criptográfico es un rechazo: un veredicto de verificación no válido, una contraseña que no coincide, un código TOTP rechazado, una cadena de auditoría rota o un paquete de evidencia que no verifica antes de exportarse. La bandera `--help` está disponible en todos los niveles; `--version`, en el mando raíz.

El binario carga un archivo `.env` del directorio de trabajo cuando existe (nunca se versiona; la plantilla comentada es `.env.example`). La [tabla B.1](#) resume las variables reconocidas; de ellas solo se registra en el diagnóstico su presencia, jamás su valor.

Tabla B.1: Variables de entorno leídas por `despacho-cli`. En las variables del proveedor remoto de sellado, un valor en blanco cuenta como ausente.

Variable	Propósito	Default
<code>RUST_LOG</code>	Filtro del diagnóstico emitido por <code>stderr</code> .	<code>info</code>
<code>KEK_BASE64</code>	Llave de cifrado de llaves: 32 bytes aleatorios codificados en base64; la leen <code>vault</code> y <code>serve</code> .	—
<code>NEW_KEK_BASE64</code>	Llave de reemplazo; la lee solo <code>vault rotate-kek</code> .	—
<code>PKI_CA_DIR</code>	Directorio de trabajo de la CA interna.	<code>pki-ca</code>
<code>TSA_DIR</code>	Directorio de trabajo de la TSA local, junto al de la CA.	<code>pki-tsa</code>
<code>CINCEL_BASE_URL</code>	URL base del proveedor remoto de sellado de tiempo.	—
<code>CINCEL_API_KEY</code>	Credencial del proveedor remoto (secreto).	—
<code>AUDIT_LOG_PATH</code>	Archivo de la bitácora de auditoría.	<code>audit-log.jsonl</code>
<code>USER</code>	Actor que registra <code>audit append</code> .	<code>unknown</code>
<code>DATABASE_URL</code>	Cadena de conexión a base de datos, reservada para la persistencia futura; solo se registra su presencia en el diagnóstico.	—

B.2 Comando `serve`: aplicación HTTP local

```
despacho-cli serve --signer-cert <PATH> --signer-key <PATH>
  [--bind <ADDR>] [--data-dir <DIR>] [--ca-cert <PATH>]
  [--crl <PATH>] [--tsa-config <PATH>] [--tsa-dir <DIR>]
```

El comando compone el workflow documental con persistencia local cifrada, bitácora encadenada y la TSA OpenSSL. Escucha por defecto en `127.0.0.1:3000`, almacena bajo `runtime-data` y exige el certificado y la llave privada del firmante. Las rutas permiten cargar, sellar, verificar y exportar evidencia, además de comprobar la auditoría; su contrato, cabeceras y errores se documentan

en docs/http-api.md. La composición no consulta Cincel. X-Actor solo etiqueta la bitácora y no reemplaza autenticación ni RBAC.

B.3 Grupo crypto: integridad de contenido

B.3.1 crypto hash

```
despacho-cli [--json] crypto hash <FILE>
```

Calcula el resumen SHA-256 del archivo indicado y lo imprime en hexadecimal en minúsculas. El único argumento, <FILE> (posicional, ruta, obligatorio), se consume como flujo en bloques de 64 KiB, por lo que el uso de memoria es constante sin importar el tamaño de la entrada. Un archivo inexistente termina el proceso con código distinto de cero.

```
+ $REP0/target/debug/despacho-cli crypto hash document.txt
fb5c0a6097be9246860fa82c89db3576e8a4ba7e554f3954f148f8bdfdd27ad9
```

Con `--json`, la salida es un objeto con los campos `algorithm` (siempre "sha-256"), `digest` y `file`.

B.4 Grupo vault: cifrado en reposo

Los tres subcomandos de `vault` materializan el cifrado autenticado en reposo con AES-256-GCM y envelope encryption: cada documento se cifra con una llave de datos (DEK) fresca, que a su vez se almacena envuelta bajo la llave de cifrado de llaves (KEK) leída de la variable `KEK_BASE64`. El criptograma queda ligado a la identidad y versión del documento mediante los datos adicionales autenticados (AAD), de modo que descifrar con un identificador o versión distintos falla igual que un criptograma alterado. La [tabla B.2](#) lista las opciones.

Tabla B.2: Opciones de los subcomandos del grupo `vault`.

Opción	Tipo	Default	Descripción
vault encrypt			
<FILE>	ruta	obligatorio	Archivo en claro; el resultado se escribe en <FILE>.enc.
--doc-id	cadena (UUID)	obligatorio	Identidad del documento a la que se liga el criptograma.
--version	entero	1	Versión del documento ligada al criptograma.
vault decrypt			
<PACKAGE>	ruta	obligatorio	Paquete cifrado producido por <code>encrypt</code> .
--doc-id	cadena (UUID)	obligatorio	Identidad esperada dentro del paquete.
--version	entero	1	Versión esperada dentro del paquete.
--out	ruta	stdout	Destino del texto claro.
vault rotate-kek			
--file	ruta	obligatorio	Paquete cuya DEK se re-envuelve in situ.

`vault rotate-kek` lee la KEK vigente de `KEK_BASE64` y su reemplazo de `NEW_KEK_BASE64`, re-envuelve únicamente la llave de datos y copia intactos los bytes del documento sellado, por lo que su costo criptográfico es independiente del tamaño del documento (la E/S de copiar el paquete sí crece con él); la escritura ocurre sobre un archivo temporal que después se renombra, de modo que un fallo a la mitad deja el original intacto. El ejemplo siguiente muestra el ciclo con detección de alteraciones: se cifra el documento, se corrompe un byte del paquete, el descifrado se rechaza con el error opaco de autenticación, se restaura el byte y se rota la KEK.

```
+ $REPO/target/debug/despacho-cli vault encrypt document.txt --doc-id
  00000000-0000-4000-8000-000000000001 --version 1
wrote document.txt.enc
+ $REPO/scripts/flip-byte.sh document.txt.enc 512
flipped byte at offset 512 of document.txt.enc: 0x45 -> 0xba (size 3678 bytes,
  unchanged)
+ (expected to fail) $REPO/target/debug/despacho-cli vault decrypt document.txt.
  enc --doc-id 00000000-0000-4000-8000-000000000001 --out rechazado.txt
Error: decryption of document.txt.enc failed

Caused by:
  authenticated decryption failed
+ $REPO/scripts/flip-byte.sh document.txt.enc 512
flipped byte at offset 512 of document.txt.enc: 0xba -> 0x45 (size 3678 bytes,
  unchanged)
+ $REPO/target/debug/despacho-cli vault rotate-kek --file document.txt.enc
rewrapped the data key in document.txt.enc
```

Con `--json`: `encrypt` emite `package_path`; `decrypt` con `--out` emite `out`; `rotate-kek` emite `rewritten`. `decrypt` sin `--out` escribe el texto claro crudo a `stdout` incluso bajo `--json`.

B.5 Grupo pki: autoridad certificadora interna

El grupo `pki` administra la autoridad certificadora interna ejecutando como subprocesos los scripts versionados del directorio `pki/` (detallados en el Anexo D). La opción de grupo `--scripts-dir` (ruta, default `pki`) indica dónde viven los scripts; el directorio de trabajo de la CA se toma de `PKI_CA_DIR` (default `pki-ca` bajo el directorio actual). La [tabla B.3](#) lista las opciones.

Tabla B.3: Opciones de los subcomandos del grupo `pki`.

Opción	Tipo	Default	Descripción
<code>--scripts-dir</code>	ruta	<code>pki</code>	Directorio con los scripts de la autoridad (opción del grupo).
pki init-ca (sin opciones propias)			
pki issue			
<code>--cn</code>	cadena	obligatorio	Common name del sujeto del certificado.
pki revoke			
<code>--serial</code>	cadena (hex)	obligatorio	Serial del certificado, tal como lo imprimen <code>issue</code> y <code>show</code> .
pki gen-crl (sin opciones propias)			
pki show			
<code><CERT></code>	ruta	obligatorio	Certificado a inspeccionar.

`init-ca` crea la raíz autofirmada y es idempotente: re-ejecutarlo sobre una CA existente no la reemplaza. `issue` emite un certificado de entidad final y rehúsa reportar éxito hasta validar el certificado fresco contra la raíz. `revoke` marca el serial en la base de datos de la CA; la revocación solo se vuelve visible para terceros tras regenerar la lista con `gen-crl`. `show` imprime sujeto, emisor, serial y ventana de vigencia de cualquier certificado.

```
+ $REPO/target/debug/despacho-cli pki --scripts-dir $REPO/pki init-ca
certificate authority ready at $WORK/pki-ca
root certificate: $WORK/pki-ca/ca.crt.pem
+ $REPO/target/debug/despacho-cli pki --scripts-dir $REPO/pki issue --cn "Socia
  Demo"
issued certificate serial 1000
  certificate: $WORK/pki-ca/certs/socia-demo.crt.pem
  private key: $WORK/pki-ca/private/socia-demo.key.pem (keep secret, never commit)
+ $REPO/target/debug/despacho-cli pki --scripts-dir $REPO/pki revoke --serial 1000
revoked certificate serial 1000
run 'pki gen-crl' to publish an updated revocation list
+ $REPO/target/debug/despacho-cli pki --scripts-dir $REPO/pki gen-crl
wrote $WORK/pki-ca/crl/crl.pem
```

Con la bandera `--json`: `init-ca` emite `ca_dir` y `root_certificate`; `issue` emite `serial`, `certificate` y `private_key`; `revoke` emite `revoked_serial`; `gen-crl` emite `crl`; `show` emite `subject`, `issuer`, `serial`, `not_before` y `not_after`.

B.6 Comando `sign`: firma digital

```
despacho-cli [--json] sign <FILE> --cert <PATH> --key <PATH>
```

Firma el resumen SHA-256 del documento con RSA PKCS#1 v1.5 y escribe la firma separada en `<FILE>.sig`. `<FILE>` (posicional, ruta) es el documento a firmar; `--cert` (ruta, obligatoria) es el certificado del firmante y `--key` (ruta, obligatoria) su llave privada en PEM, en codificación PKCS#8 o PKCS#1; llaves con módulo menor a 3 072 bits se rechazan al construir el firmador. Antes de escribir el archivo de firma, el comando verifica la firma recién producida contra el certificado presentado: una llave que no corresponde al certificado falla en ese punto en lugar de producir una firma unverificable.

```
+ $REPO/target/debug/despacho-cli sign document.txt --cert $WORK/pki-ca/certs/
  socia-demo.crt.pem --key $WORK/pki-ca/private/socia-demo.key.pem
digest: fb5c0a6097be9246860fa82c89db3576e8a4ba7e554f3954f148f8bdfdd27ad9
signature: document.txt.sig
```

Con `--json`, la salida contiene `digest`, `signature_path` y `certificate_subject`.

B.7 Comando `timestamp`: sellado de tiempo

```
despacho-cli [--json] timestamp <INPUT> [--mock] [--out <PATH>]
  [--pki-dir <DIR>]
```

Solicita un sello de tiempo RFC 3161 sobre el resumen SHA-256 del archivo `<INPUT>` (posicional, ruta) y escribe el token DER en `<INPUT>.tsr`, o en la ruta dada con `--out`. Con la bandera `--mock` (booleana), la autoridad es la TSA local de OpenSSL: su directorio de trabajo se lee de `TSA_DIR` (default `pki-tsa`, junto al directorio de la CA) y la configuración `tsa.cnf` se busca en `--pki-dir` (ruta, default `pki`). Sin `--mock`, se usa el proveedor remoto: el comando exige `CINCEL_BASE_URL` y `CINCEL_API_KEY` no vacías en el entorno, con tiempo límite de 15 segundos por solicitud y hasta cinco sondeos cada dos segundos cuando el proveedor difiere la emisión; si faltan, aborta indicando que se configuren esas variables o se use `--mock`. En ambos casos, tras obtener el token el comando lo relea y exige que cubra exactamente el resumen calculado antes de escribir archivo alguno: una autoridad que respondiera con un token inutilizable falla aquí sin dejar un archivo espurio.

```
+ $REPO/target/debug/despacho-cli timestamp document.txt --mock --pki-dir $REPO/
  pki
digest: fb5c0a6097be9246860fa82c89db3576e8a4ba7e554f3954f148f8bdfdd27ad9
token: document.txt.tsr
generated at: 2026-07-12T10:19:41Z
```

Con `--json`, la salida contiene `digest`, `token_path` y `generated_at`.

B.8 Comando verify: verificación integral

```
despacho-cli [--json] verify <FILE> --sig <PATH> --cert <PATH>
  --ca <PATH> [--tsr <PATH>] [--crl <PATH>] [--tsa-cert <PATH>]
  [--at <RFC3339>]
```

Produce un reporte de cuatro componentes sobre el documento: integridad (el resumen recomputado contra el que atestigua el token de sello), firma, estado del certificado del firmante y sello de tiempo. La [tabla B.4](#) lista las opciones.

Tabla B.4: Opciones del comando verify.

Opción	Tipo	Default	Descripción
<FILE>	ruta	obligatorio	Documento original.
--sig	ruta	obligatorio	Firma separada.
--cert	ruta	obligatorio	Certificado del firmante.
--ca	ruta	obligatorio	Certificado de la autoridad emisora.
--tsr	ruta	—	Token de sello de tiempo; sin él, los componentes que dependen del token se reportan omitidos con su razón.
--crl	ruta	—	Lista de revocación; sin ella no se comprueba revocación y el reporte lo dice.
--tsa-cert	ruta	--ca	Ancla de confianza para el token; por defecto el propio emisor, que ancla a las autoridades internas.
--at	instante RFC 3339	ahora	Instante de evaluación del estado del certificado.

La semántica del veredicto es estricta: un componente cuya evidencia no se aportó se reporta omitido (*skipped*) con su razón y no cuenta contra el veredicto; el veredicto es *not valid* si cualquier componente ejercitado falló, y en ese caso el proceso termina con código distinto de cero. El estado del certificado habla únicamente del instante de evaluación: el propio reporte aclara que un certificado hoy expirado o revocado no invalida por sí mismo una firma efectuada mientras era válido, lectura que corresponde a quien valora la evidencia. El primer ejemplo muestra una verificación íntegra; el segundo, la misma evidencia después de revocar el certificado y regenerar la CRL.

```
+ $REPO/target/debug/despacho-cli verify document.txt --sig document.txt.sig --tsr
  document.txt.tsr --cert $WORK/pki-ca/certs/socia-demo.crt.pem --ca $WORK/pki-
  ca/ca.crt.pem --crl $WORK/pki-ca/crl/crl.pem
document:    document.txt
digest:      fb5c0a6097be9246860fa82c89db3576e8a4ba7e554f3954f148f8bdfdd27ad9
integrity:   passed: recomputed digest matches the digest the timestamp token
             attests
signature:   passed: signature verifies over the document digest with the signer
             certificate
certificate: passed: status at the evaluation time: valid (revocation checked
             against the supplied list)
```

```
timestamp:    passed: token accepted under the trust anchor; generated at
               2026-07-12T10:19:41Z
verdict:      valid
```

```
+ (expected to fail) $REPO/target/debug/despacho-cli verify document.txt --sig
  document.txt.sig --tsr document.txt.tsr --cert $WORK/pki-ca/certs/socia-demo.
  crt.pem --ca $WORK/pki-ca/ca.crt.pem --crl $WORK/pki-ca/crl/crl.pem
document:     document.txt
digest:       fb5c0a6097be9246860fa82c89db3576e8a4ba7e554f3954f148f8bdfdd27ad9
integrity:    passed: recomputed digest matches the digest the timestamp token
               attests
signature:    passed: signature verifies over the document digest with the signer
               certificate
certificate:   failed: status at the evaluation time: revoked (serial 1000); this
               status speaks for the evaluation time only and does not by itself invalidate a
               signature made while the certificate was valid
timestamp:    passed: token accepted under the trust anchor; generated at
               2026-07-12T10:19:41Z
verdict:      not valid
Error: the verification verdict is not valid
```

Con `--json`, la salida contiene `document`, `digest`, un objeto `{status, detail}` por cada uno de los componentes `integrity`, `signature`, `certificate` y `timestamp`, y el campo `verdict`.

B.9 Grupo package: paquete de evidencia

```
despacho-cli [--json] package export <FILE> --sig <PATH> --tsr <PATH>
  --cert <PATH> --ca <PATH> --crl <PATH> [--tsa-chain <PATH>]
  --out <PATH>
```

Construye el paquete de evidencia autocontenido: un archivo ZIP con el documento, su firma, su sello de tiempo, los certificados, la lista de revocación y el instructivo `INSTRUCCIONES.md` de verificación independiente (la guía completa se reproduce en el Anexo C). Todas las opciones son rutas obligatorias salvo `--tsa-chain` (ruta, opcional), que incluye la cadena de certificados de la autoridad de sellado cuando está disponible; si viaja, las instrucciones anclan la comprobación del token en esa cadena, y si no, en el certificado del emisor. Antes de escribir el ZIP, el comando ejecuta la misma verificación integral de `verify` sobre los artefactos, incluidos CRL y token, y rehúsa exportar si el veredicto no es válido, nombrando cada componente fallido: el paquete nunca puede fallar sus propias instrucciones.

```
+ $REPO/target/debug/despacho-cli package export document.txt --sig document.txt.
  sig --tsr document.txt.tsr --cert $WORK/pki-ca/certs/socia-demo.crt.pem --ca
  $WORK/pki-ca/ca.crt.pem --crl $WORK/pki-ca/crl/crl.pem --tsa-chain $WORK/pki-
  tsa/tsa-chain.pem --out evidencia.zip
digest:       fb5c0a6097be9246860fa82c89db3576e8a4ba7e554f3954f148f8bdfdd27ad9
package:      evidencia.zip
instructions inside the package: INSTRUCCIONES.md
```

Con `--json`, la salida contiene `digest`, `package` e `instructions`.

B.10 Grupo auth: credenciales y segundo factor

El grupo `auth` cubre el hashing de contraseñas con Argon2id, su calibración empírica y el segundo factor TOTP con códigos de recuperación. Las contraseñas nunca viajan como argumentos: se leen de la entrada estándar hacia búferes que se borran de memoria al liberarse. La [tabla B.5](#) lista las opciones.

Tabla B.5: Opciones de los subcomandos del grupo `auth`.

Opción	Tipo	Default	Descripción
auth calibrate (sin opciones; no lee contraseña)			
auth hash-password (contraseña por <code>stdin</code>)			
auth verify-password (contraseña por <code>stdin</code>)			
<code>--hash</code>	cadena (PHC)	obligatorio	Hash almacenado contra el que se verifica.
auth totp enroll			
<code>--user</code>	cadena (correo)	obligatorio	Identificador de la cuenta que se inscribe.
<code>--secret-out</code>	ruta	obligatorio	Archivo que recibe el secreto base32 (modo 0600).
auth totp verify			
<code>--code</code>	cadena	obligatorio	Código de un solo uso a verificar.
<code>--secret-file</code>	ruta	obligatorio	Archivo con el secreto base32 escrito al inscribir.

`calibrate` mide el costo real del hash en el hardware local: ejecuta cinco corridas de Argon2id sobre una muestra fija y clasifica la media contra la banda objetivo inclusiva de 500 a 1 000 milisegundos, con veredicto por debajo, dentro o por encima de la banda. En el hardware de referencia la configuración vigente es $m = 262144$, $t = 4$, $p = 1$ y la media observada fue 607.8 ms. `hash-password` emite la cadena PHC resultante, que registra esos parámetros (`$argon2id$v=19$m=262144,t=4,p=1$...`). `verify-password` responde `match` o `mismatch` —con código de salida distinto de cero en el segundo caso—, y un hash almacenado ilegible es un error, nunca un `mismatch`. `totp enroll` genera el secreto compartido, imprime el URI de aprovisionamiento `otpauth` y los ocho códigos de recuperación de un solo uso, y escribe el secreto base32 en el archivo indicado. `totp verify` acepta un desfase de un paso de 30 segundos a cada lado; la verificación es sin estado y el rechazo de reutilización dentro de la ventana se difiere a la capa de persistencia, según consta en `docs/adr/0008-totp-single-use-enforcement.md`.

```
+ $REPO/target/debug/despacho-cli auth calibrate
mean over 5 runs: 607.8 ms (inside the 500-1000 ms target band)
+ $REPO/target/debug/despacho-cli auth totp enroll --user demo@example.com --
  secret-out totp-secret.b32
otpauth-uri: otpauth://totp/despacho:demo%40example.com?secret=
  IGAPAF202XDMIG6QGZU2YITPCD44P3&issuer=despacho
```

```
secret-base32: IGAPAF202XDMIG6QGFZU2YITPDCD44P3
recovery codes (each works exactly once):
  GQVMH-DCWJS
  9QBY1-NXNJ3
  MY4P3-CX4HZ
  0CE8T-KBNSS
  31D0P-GDAD8
  B6B6N-DBGAJ
  ZV2T3-B35JJ
  C329Q-TTBEE
secret written to totp-secret.b32
+ despacho-cli auth totp verify --code (computed with openssl)
accepted
```

Con `--json`: `calibrate` emite `runs`, `mean_ms`, `band_min_ms`, `band_max_ms` y `verdict`; `hash-password` emite `phc`; `verify-password` emite `match`; `totp enroll` emite `otpauth_uri`, `secret_base32`, el arreglo `recovery_codes` y `secret_path`; `totp verify` emite `accepted`.

B.11 Grupo audit: bitácora encadenada

El grupo `audit` opera la bitácora de auditoría de solo anexo: un archivo JSON Lines cuya ruta se toma de `AUDIT_LOG_PATH`, donde cada entrada almacena, además de sus campos, un valor de cadena SHA-256 que la enlaza con la anterior desde un génesis fijo. `append` registra un evento con las opciones `--action` y `--resource` (cadenas, obligatorias); el actor se toma de la variable `USER` y el instante del reloj del sistema, e imprime la secuencia y el valor de cadena asignados, de modo que la terminal del operador retenga un registro informal de la cabeza de la cadena. `verify-chain` recomputa cada eslabón desde el génesis y nombra el índice del primer eslabón roto; `show` lista las entradas en orden de inserción. Los subcomandos de lectura rehúsan operar sobre una bitácora inexistente —reportar una ruta equivocada o un archivo borrado como cadena vacía frustraría la verificación—; solo `append` crea el archivo. La verificación no detecta el truncado de la cola, limitación documentada en `docs/adr/0007-audit-chain-anchoring.md`.

```
+ $REPO/target/debug/despacho-cli audit append --action demo.firma --resource
  document.txt.sig
appended entry 0 with chain
  e8b3fefc3966e47a695260f2b098dd73eff0f7da9e9b4692314df873b1e083fa
+ $REPO/target/debug/despacho-cli audit append --action demo.sello --resource
  document.txt.tsr
appended entry 1 with chain
  ffc1a151ecc7e5976d24b0bbfaaaed4c9230a8ed597938e24e6c228b2a0a5581
+ $REPO/target/debug/despacho-cli audit verify-chain
audit chain valid (2 entries)
+ $REPO/target/debug/despacho-cli audit show
0 2026-07-12T10:19:42.272903635Z eddndev demo.firma document.txt.sig
  e8b3fefc3966e47a695260f2b098dd73eff0f7da9e9b4692314df873b1e083fa
1 2026-07-12T10:19:42.275499975Z eddndev demo.sello document.txt.tsr
  ffc1a151ecc7e5976d24b0bbfaaaed4c9230a8ed597938e24e6c228b2a0a5581
+ sed -i s/demo.sello/demo.XXXXX/ $WORK/audit-log.jsonl
```

```
+ (expected to fail) $REPO/target/debug/despacho-cli audit verify-chain  
Error: audit chain broken at index 1
```

Con `--json`: `append` emite `sequence`, `timestamp`, `actor`, `action`, `resource` y `chain`; `show` emite un arreglo de objetos con esos mismos campos; `verify-chain` emite `{valid, entries}` cuando la cadena es válida o `{valid: false, first_broken_index}` —impreso antes de terminar con código distinto de cero— cuando está rota.

APÉNDICE C

Guía de verificación independiente con OpenSSL

Cada paquete de evidencia exportado por el prototipo (comando `package export`, véase el Anexo B) viaja con un instructivo `INSTRUCCIONES.md` que permite a un tercero —un perito, la contraparte procesal o un tribunal— verificar el documento, su firma, el estado del certificado y el sello de tiempo usando únicamente la herramienta de línea de comandos `openssl`, sin el software que generó el paquete. El instructivo se genera a partir de la plantilla versionada `crates/application/src/evidence/INSTRUCCIONES.template.md`, cuyos marcadores se rellenan al momento de exportar con los datos concretos del paquete (tabla C.1). Este anexo reproduce esa guía paso a paso y explica qué demuestra cada comando; con ello se materializa el criterio de verificabilidad independiente del objetivo OE-3. Las salidas mostradas provienen de la ejecución real registrada en `docs/demo-transcript.md`.

Tabla C.1: Marcadores de la plantilla del instructivo y valor con que se rellenan al exportar.

Marcador	Valor al exportar
{{DOCUMENTO}}	Nombre base del documento original.
{{FIRMA}}	Nombre del archivo de firma separada (<documento>.sig).
{{SELLO}}	Nombre del archivo del sello de tiempo (<documento>.tsr).
{{DIGEST_SHA256}}	Resumen SHA-256 del documento, registrado al momento de exportar.
{{ANCLA_SELLO}}	Ancla de confianza del token: <code>tsa-chain.pem</code> si viaja en el paquete, <code>ca.pem</code> en caso contrario.
{{OPENSSL_VERSION}}	Línea de versión de la herramienta <code>openssl</code> de la máquina exportadora.

C.1 Requisitos y versión de referencia

La verificación requiere únicamente dos herramientas presentes en prácticamente cualquier sistema operativo de escritorio o servidor: `unzip` (o cualquier extractor de archivos ZIP) para abrir el paquete, y `openssl` [88] para ejecutar las cuatro comprobaciones. La versión de referencia es **OpenSSL 3.5.7**: con ella se probaron los scripts de la autoridad interna (Anexo D) y sobre ella se capturó la transcripción de la demostración. El propio instructivo cita la línea de versión exacta de la máquina exportadora —en la ejecución de referencia, `OpenSSL 3.5.7 9 Jun 2026`—, de modo que el lector sepa contra qué herramienta se redactaron los comandos y pueda atribuir a la versión cualquier diferencia menor en el formato de las salidas. Todos los comandos se ejecutan dentro del directorio donde se extrajo el paquete.

La verificación tampoco requiere acceso a la red: las anclas de confianza (el certificado de la autoridad emisora y, cuando viaja, la cadena de la autoridad de sellado) y la lista de revocación están contenidas en el propio paquete. La contraparte de esa autosuficiencia es temporal y el instructivo la hace explícita: la lista de revocación incluida refleja el estado al momento de exportar, de modo que quien necesite el estado actual del certificado debe solicitar una lista vigente a la autoridad emisora.

C.2 Estructura del paquete

El paquete es un archivo ZIP con entradas planas, sin directorios, escrito en el subconjunto almacenado del formato ZIP [85]: método 0 (sin compresión), sumas CRC-32 correctas por entrada y salida determinista, con nombres restringidos a un subconjunto ASCII sin separadores de ruta, de modo que la extracción jamás escribe fuera del directorio destino. Las razones de esta implementación propia y mínima constan en docs/adr/0006-evidence-package-format.md. La [tabla C.2](#) describe cada entrada; el nombre del documento conserva su nombre base original y define los nombres de la firma y del sello.

Tabla C.2: Contenido del paquete de evidencia. El documento de la ejecución de referencia se llama `document.txt`.

Entrada	Contenido
<code><documento></code>	El documento original, byte a byte.
<code><documento>.sig</code>	La firma digital separada: RSA PKCS#1 v1.5 sobre el resumen SHA-256 del documento.
<code><documento>.tsr</code>	El sello de tiempo RFC 3161 emitido sobre el resumen SHA-256 del documento.
<code>certificado.pem</code>	El certificado X.509 del firmante.
<code>ca.pem</code>	El certificado de la autoridad emisora, raíz de la confianza del paquete.
<code>crl.pem</code>	La lista de revocación publicada por la autoridad al momento de exportar.
<code>tsa-chain.pem</code>	La cadena de certificados de la autoridad de sellado de tiempo (presente solo cuando estaba disponible al exportar).
<code>INSTRUCCIONES.md</code>	El instructivo de verificación con los comandos de este anexo ya instanciados.

La extracción con cualquier herramienta ZIP estándar restaura los miembros byte a byte:

```
+ unzip -o evidencia.zip -d evidencia
Archive:  evidencia.zip
  extracting: evidencia/document.txt
  extracting: evidencia/document.txt.sig
  extracting: evidencia/document.txt.tsr
  extracting: evidencia/certificado.pem
  extracting: evidencia/ca.pem
  extracting: evidencia/crl.pem
  extracting: evidencia/tsa-chain.pem
  extracting: evidencia/INSTRUCCIONES.md
```

El encabezado del instructivo extraído en la ejecución de referencia ilustra cómo quedan instanciados los marcadores:

```
# Instrucciones de verificación independiente
```

```
Este paquete de evidencia permite verificar el documento `document.txt`  
sin el software que lo generó. Todos los comandos usan únicamente la  
herramienta de línea de órdenes `openssl`; el paquete se generó en una  
máquina con `OpenSSL 3.5.7 9 Jun 2026 (Library: OpenSSL 3.5.7 9 Jun  
2026)`. Ejecute los comandos dentro del directorio donde extrajo el  
paquete.
```

C.3 Paso 1: integridad del documento

El primer paso recalcula el resumen SHA-256 [25] del documento extraído:

```
openssl dgst -sha256 document.txt
```

La salida debe coincidir carácter por carácter con el resumen registrado en el instructivo al momento de exportar. En la ejecución de referencia:

```
+ openssl dgst -sha256 document.txt  
SHA2-256(document.txt)=  
fb5c0a6097be9246860fa82c89db3576e8a4ba7e554f3954f148f8bdfdd27ad9
```

Qué demuestra. Que el documento contenido en el paquete es, bit a bit, el mismo sobre el que se registró el resumen: cualquier alteración posterior a la exportación —por mínima que sea— produce un resumen distinto. Este paso establece la referencia sobre la que descansan los tres siguientes, pues tanto la firma como el sello de tiempo se emitieron sobre ese mismo resumen.

C.4 Paso 2: firma digital

El segundo paso extrae la clave pública del certificado del firmante y verifica la firma separada sobre el documento:

```
openssl x509 -in certificado.pem -pubkey -noout > firmante.pub.pem  
openssl dgst -sha256 -verify firmante.pub.pem \  
-signature document.txt.sig document.txt
```

La salida esperada es `Verified OK`; cualquier otra salida significa que la firma no corresponde al documento y a la clave del certificado. En la ejecución de referencia:

```
+ openssl x509 -in certificado.pem -pubkey -noout -out firmante.pub.pem  
+ openssl dgst -sha256 -verify firmante.pub.pem -signature document.txt.sig  
document.txt  
Verified OK
```

Qué demuestra. Que la firma fue producida con la clave privada correspondiente a la clave pública contenida en `certificado.pem`, y que cubre exactamente este documento. Junto con el paso

1, esto vincula el contenido íntegro del documento con la identidad declarada en el certificado: es la base técnica del no repudio, porque solo quien posee la clave privada pudo generar esa firma.

C.5 Paso 3: certificado del firmante y revocación

El tercer paso comprueba que el certificado del firmante encadena a la autoridad emisora incluida y consulta su estado de revocación contra la lista incluida, conforme a RFC 5280 [44]:

```
cat ca.pem crl.pem > ca-y-crl.pem
openssl verify -crl_check -CAfile ca-y-crl.pem certificado.pem
```

La salida esperada es `certificado.pem: OK`. En la ejecución de referencia:

```
+ openssl verify -crl_check -CAfile ca-y-crl.pem certificado.pem
certificado.pem: OK
```

Qué demuestra. Que el certificado del firmante fue efectivamente emitido por la autoridad raíz del paquete (la firma de la autoridad sobre el certificado se sostiene), que se encuentra dentro de su ventana de vigencia y que no aparece listado en la lista de revocación incluida. El propio instructivo advierte dos matices que el verificador debe tener presentes: la lista de revocación refleja el estado *al momento de la exportación* y tiene fecha de caducidad, por lo que para conocer el estado actual debe solicitarse una lista vigente a la autoridad emisora; y un certificado hoy expirado o revocado no invalida por sí solo una firma efectuada mientras el certificado era válido —esa lectura corresponde a quien valora la evidencia.

C.6 Paso 4: sello de tiempo

El cuarto paso verifica el sello de tiempo RFC 3161 [27]: que el token cubre este documento y que fue firmado por una autoridad de sellado que encadena al ancla de confianza incluida en el paquete. El ancla es `tsa-chain.pem` cuando esa entrada viaja en el paquete; en su ausencia, el instructivo ancla en `ca.pem`, pues las autoridades internas del prototipo encadenan a la misma raíz:

```
openssl ts -verify -data document.txt -in document.txt.tsr \
  -CAfile tsa-chain.pem
```

La salida esperada termina en `Verification: OK`. En la ejecución de referencia el ancla empleada fue `ca.pem`; ambas invocaciones son equivalentes en ese caso porque la cadena de la TSA local es una copia del certificado raíz:

```
+ openssl ts -verify -data document.txt -in document.txt.tsr -CAfile ca.pem
Using configuration from /etc/pki/tls/openssl.cnf
Verification: OK
```

Para inspeccionar el contenido aseverado por la autoridad de sellado:

```
openssl ts -reply -in document.txt.tsr -text
```

Esta lectura muestra, en texto plano, los campos de la estructura `TSTInfo` que el token protege bajo la firma de la autoridad: el identificador de la política de sellado, el algoritmo y el valor del

message imprint —que debe coincidir con el resumen del paso 1—, el número de serie del token y el instante de generación (*genTime*). En los sellos emitidos por la autoridad local del prototipo, la política es el OID de demostración 1.3.6.1.4.1.13762.3 y la exactitud aseverada es de un segundo (véase el Anexo D).

Qué demuestra. Que una autoridad de sellado de tiempo, cuya cadena de certificados se sostiene hasta el ancla de confianza del paquete, dio fe de que el resumen SHA-256 del documento existía en el instante aseverado dentro del token. Combinado con el paso 1, esto otorga fecha cierta al contenido exacto del documento: el documento no pudo haberse creado ni modificado después de ese instante sin invalidar el sello.

C.7 Resumen: qué demuestra cada comprobación

Tabla C.3: Correspondencia entre cada paso de la guía, su salida esperada y la propiedad que demuestra.

Paso	Comando central	Salida esperada	Propiedad demostrada
1	<code>openssl dgst -sha256</code>	resumen idéntico al registrado	Integridad: el documento no fue alterado después de la exportación.
2	<code>openssl dgst -verify</code>	Verified OK	Autenticidad y no repudio: la firma corresponde a este documento y a la clave del certificado incluido.
3	<code>openssl verify -crl_check certificado.pem: OK</code>		Confianza y vigencia: el certificado encadena a la autoridad emisora y no está revocado según la lista incluida.
4	<code>openssl ts -verify</code>	Verification: OK	Fecha cierta: una autoridad de sellado atestiguó la existencia del documento en el instante aseverado.

C.8 Comportamiento ante evidencia alterada

Los cuatro pasos no son ceremonias: cada uno rechaza activamente la evidencia manipulada. Si se altera un solo byte del documento extraído, el paso 1 produce un resumen distinto del registrado, el paso 2 responde con un fallo de verificación en lugar de Verified OK y el paso 4 rechaza el token porque el *message imprint* ya no corresponde al documento presentado; la suite de pruebas del proyecto fija exactamente este comportamiento alterando un byte de un miembro extraído y comprobando que

los comandos documentados lo rechazan (`crates/bin/tests/package_cli.rs`). La corrupción del propio archivo ZIP se detecta antes, en la extracción, mediante la suma CRC-32 almacenada por entrada, que `unzip` comprueba con `unzip -t`.

La transcripción de la demostración ilustra el efecto sobre el resumen: tras voltear el byte en el desplazamiento 100 de una copia del documento, el resumen recomputado cambió de `fb5c0a60...` a `19914d87...`, y la verificación reportó fallidos los componentes de integridad, firma y sello, conservando aprobado únicamente el estado del certificado —que no depende del contenido del documento—.

C.9 Autonomía respecto del sistema

La verificación descrita no requiere el prototipo en ningún punto: ni el binario `despacho-cli`, ni la base de datos, ni la disponibilidad de la plataforma que exportó el paquete. Todo lo necesario —documento, firma, sello, certificados, lista de revocación e instructivo— viaja dentro del ZIP, y las cuatro comprobaciones se ejecutan con herramientas de terceros ampliamente auditadas. Esta autonomía es deliberada: el valor probatorio de la evidencia no puede depender de que el sistema que la produjo siga existiendo u operando al momento de valorarla.

Dos salvaguardas respaldan la garantía. Primera, el comando `package export` ejecuta la verificación integral completa sobre los artefactos antes de escribir el ZIP y rehúsa exportar un paquete cuyo veredicto no sea válido, de modo que ningún paquete emitido puede fallar sus propias instrucciones. Segunda, la suite de pruebas del proyecto extrae un paquete real, toma los comandos de `INSTRUCCIONES.md` y los ejecuta literalmente contra los archivos extraídos (`crates/bin/tests/package_cli.rs`), comprobando además que la corrupción de un solo byte del documento extraído es detectada por los comandos documentados; la etapa novena de la transcripción `docs/demo-transcript.md` reproduce el mismo recorrido de extremo a extremo.

APÉNDICE D

Scripts de la autoridad certificadora interna

La autoridad certificadora interna del prototipo y su autoridad de sellado de tiempo local se operan con cinco scripts de bash y dos archivos de configuración de OpenSSL, todos versionados en el directorio `pki/` del repositorio y probados con OpenSSL 3.5.7. Los scripts comparten tres disciplinas: se ejecutan con `set -euo pipefail` (cualquier fallo intermedio aborta), verifican que `openssl` esté disponible antes de operar, y leen el directorio de trabajo de la CA de la variable de entorno `PKI_CA_DIR` (por defecto `pki-ca` bajo el directorio actual). El directorio de trabajo es un artefacto de tiempo de ejecución: contiene las llaves privadas y la base de datos de la autoridad, se crea con permisos restrictivos (`umask 077` al generar llaves, `chmod 700` sobre `private/`) y jamás se versiona; como red de seguridad, el `.gitignore` de la raíz ignora `pki/**/*.key` y `pki/**/*.pem`. El modelo de confianza es una jerarquía de una sola capa: una raíz autofirmada firma directamente certificados de entidad final, la CRL y el certificado de la TSA, conforme a X.509 y RFC 5280 [44].

Estos mismos scripts son los que ejecuta el prototipo: el adaptador de infraestructura de la autoridad los invoca como subprocesos desde los subcomandos `pki` de la interfaz de línea de comandos (véase el Anexo B), y la suite de pruebas los usa como fixtures reales en directorios temporales, de modo que las pruebas ejercitan exactamente la autoridad de producción.

D.1 Inventario y parámetros criptográficos comunes

Tabla D.1: Archivos del directorio pki/ y su propósito.

Archivo	Propósito
init-ca.sh	Crea la CA raíz: llave privada, certificado autofirmado y base de datos de la autoridad.
issue-cert.sh	Emite un certificado de entidad final firmado por la CA para el common name dado.
revoke.sh	Marca como revocado un certificado emitido, en la base de datos de la CA.
gen-crl.sh	Genera la lista de certificados revocados (CRL) en formato PEM.
issue-tsa-cert.sh	Emite el certificado de la TSA local y prepara su directorio de trabajo.
openssl.cnf	Configuración de la CA: layout del directorio, política de emisión y extensiones X.509 v3.
tsa.cnf	Configuración de la TSA para openssl ts -reply y extensiones del certificado de sellado.
README.md	Guía de operación: orden de uso, ejemplo completo y comandos de verificación independiente.

Los parámetros criptográficos son idénticos en toda la jerarquía y están alineados con el nivel de fortaleza de 128 bits adoptado por el diseño criptográfico (sección 3.11.1): todas las llaves son RSA de 3 072 bits generadas con openssl genpkey y todas las firmas usan SHA-256, con sha256WithRSAEncryption como algoritmo de firma. La [tabla D.2](#) resume los valores exactos.

Tabla D.2: Parámetros criptográficos y de vigencia de la autoridad interna.

Parámetro	Valor
Algoritmo y tamaño de llave (raíz, entidad final y TSA)	RSA 3 072 bits
Digesto de firma en certificados, CRL y sellos	SHA-256
Vigencia del certificado raíz autofirmado	1 825 días (5 años)
Vigencia de los certificados de entidad final	365 días (1 año)
Vigencia del certificado de la TSA	365 días (1 año)
Vigencia de cada CRL (default_crl_days)	7 días
Retrodatación del inicio de vigencia (notBefore)	5 minutos
Serial inicial de certificados y de números de CRL	1000
Serial inicial de tokens de la TSA	1000
Política de sellado de la TSA (OID)	1.3.6.1.4.1.13762.3
Exactitud del tiempo aseverado en cada sello	1 segundo

El sujeto por defecto (sección dn_defaults de openssl.cnf) es una identidad de prototipo para un despacho jurídico mexicano: C=MX, ST=Ciudad de Mexico, L=Ciudad de Mexico,

0=Despacho Juridico Demo, con OU y CN propios de cada pieza (autoridad, personal del despacho o autoridad de sellado).

D.2 init-ca.sh: creación de la CA raíz

El script crea el layout completo del directorio de trabajo (certs/, crl/, csr/, newcerts/ y private/ con permisos 700) e inicializa la base de datos de la autoridad: index.txt vacío, serial en 1000 y crlnumber en 1000. Después genera la llave privada RSA 3 072 bajo umask 077 y autofirma el certificado raíz por 1 825 días con las extensiones v3_root_ca. El script es idempotente y defensivo: sobre una CA ya inicializada no hace nada y lo reporta; rehúsa sobrescribir llave o certificado existentes; y un estado inconsistente —existe la llave pero no el certificado, o viceversa— es un error explícito.

```
1 # Generate the CA private key with restrictive permissions.
2 (
3     umask 077
4     openssl genpkey -quiet -algorithm RSA \
5         -pkeyopt "rsa_keygen_bits:$KEY_BITS" -out "$CA_KEY"
6 )
7
8 # Self-sign the root certificate. The subject comes from the
9 # dn_defaults section of openssl.cnf.
10 openssl req -config "$OPENSSL_CNF" -new -x509 -sha256 \
11     -days "$CA_DAYS" -extensions v3_root_ca \
12     -key "$CA_KEY" -out "$CA_CERT"
```

Listado D.1: Generación de la llave raíz y autofirma del certificado en pki/init-ca.sh.

D.3 issue-cert.sh: emisión de certificados de entidad final

Recibe como único argumento el common name del sujeto, restringido por una expresión regular a un subconjunto ASCII seguro (letras, dígitos, espacios y . _ @ -, iniciando con letra o dígito) para poder incrustarlo en el argumento -subj y en nombres de archivo sin escape. Del common name deriva un identificador en minúsculas con los espacios convertidos a guiones (Juan Perez → juan-perez) que nombra la llave, el CSR y el certificado. Genera la llave RSA 3 072, crea la solicitud de firma y la firma con la CA por 365 días usando las extensiones v3_end_entity; el país y la organización del sujeto deben coincidir con los de la CA porque la política de emisión así lo exige. El script rehúsa reemitir si ya existen el certificado o la llave del mismo identificador.

Una decisión operativa merece nota: el inicio de vigencia se retrodata cinco minutos, como acostumbran las autoridades públicas, para que el certificado sea utilizable de inmediato aun cuando la máquina emisora y la máquina que valida difieran ligeramente en su reloj; sin la retrodatación, una validación efectuada en el mismo segundo de la emisión puede caer antes de notBefore y reportar el certificado como aún no válido.

```
1 # Backdate the start of validity by a few minutes, as public authorities
2 # commonly do, so the certificate is immediately usable even when the
```

```

3 # issuing and the relying machine disagree slightly on the time (clock
4 # skew). Without this, a validation performed in the same second as the
5 # issuance can land before notBefore and report the certificate as not
6 # yet valid.
7 START_DATE="$(date -u -d '5 minutes ago' +%Y%m%d%H%M%SZ)"
8
9 # Sign the request with the CA using the end-entity extensions.
10 openssl ca -config "$OPENSSL_CNF" -batch -notext -md sha256 \
11     -startdate "$START_DATE" -days "$CERT_DAYS" -extensions v3_end_entity \
12     -in "$CSR_FILE" -out "$CERT_FILE"

```

Listado D.2: Retrodatación del inicio de vigencia y firma por la CA en pki/issue-cert.sh.

D.4 revoke.sh: revocación

Recibe la ruta del certificado a revocar, lee su número de serie con `openssl x509 -noout -serial` y consulta la base de datos `index.txt` de la CA —un archivo separado por tabuladores con estado, expiración, fecha de revocación, serial, archivo y sujeto, donde el estado R significa revocado— antes de intentar la revocación. Si el certificado ya está revocado, lo reporta y termina con éxito, de modo que el script puede re-ejecutarse sin riesgo. La revocación se asienta con `openssl ca -revoke` y solo se hace visible a terceros cuando se regenera la CRL con `gen-crl.sh`.

```

1 # index.txt is tab separated: status, expiry, revocation date,
2 # serial, file name, subject. Status R means already revoked.
3 if awk -F '\t' -v s="$SERIAL" '$1 == "R" && $4 == s { found = 1 } END { exit !
4     found }' \
5     "$PKI_CA_DIR/index.txt"; then
6     printf 'Certificate with serial %s is already revoked (nothing to do).\n' "
7         $SERIAL"
8     exit 0
9 fi
10 openssl ca -config "$OPENSSL_CNF" -revoke "$CERT_PATH"

```

Listado D.3: Consulta del estado y revocación en pki/revoke.sh.

D.5 gen-crl.sh: generación de la CRL

Produce la lista de certificados revocados en formato PEM a partir de la base de datos de la CA. Cada CRL es válida siete días (`default_crl_days = 7` en `openssl.cnf`), por lo que debe regenerarse al menos semanalmente y después de cada revocación; el validador del prototipo trata una CRL vencida como error duro, nunca como un pase permisivo. La única extensión de la lista es `authorityKeyIdentifier` (sección `crl_ext`). Al terminar, el script reporta las fechas `lastUpdate` y `nextUpdate` de la lista recién emitida y cuenta los certificados revocados que contiene.

```

1 openssl ca -config "$OPENSSL_CNF" -gencrl -out "$CRL_FILE"
2
3 printf 'CRL generated successfully.\n'
4 printf '  CRL file: %s\n' "$CRL_FILE"
5 openssl crl -noout -lastupdate -nextupdate -in "$CRL_FILE" | sed 's/^/ /'
6 REVOKED_COUNT="$(awk -F '\t' ' $1 == "R" ' "$PKI_CA_DIR/index.txt" | wc -l)"
7 printf '  Revoked certificates listed: %s\n' "$REVOKED_COUNT"

```

Listado D.4: Emisión y reporte de la CRL en `pki/gen-crl.sh`.

D.6 `issue-tsa-cert.sh`: certificado de la autoridad de sellado

Emite el certificado de la TSA local y prepara su directorio de trabajo, tomado de la variable `TSA_DIR` (por defecto `pki-tsa`, junto al directorio de la CA). Genera la llave RSA 3072 de la TSA, crea el CSR con el sujeto `OU=Autoridad de Sellado de Tiempo, CN=TSA Interna Despacho Juridico Demo` y lo firma con la CA por 365 días, con la misma retrodatación de cinco minutos, pero con una diferencia esencial: las extensiones no provienen de `openssl.cnf` sino de `tsa.cnf` (`-extfile tsa.cnf -extensions v3_tsa`), porque RFC 3161 [27] exige que el `extendedKeyUsage` de un certificado de sellado sea crítico y contenga `id-kp-timeStamping`. Finalmente prepara los dos archivos que `openssl ts -reply` lee en cada emisión de token: el archivo `serial` de tokens (inicia en 1000) y `tsa-chain.pem`, la cadena que se adjunta a cada respuesta, que es una copia del certificado raíz de la CA.

```

1 # Sign the request with the CA. The extensions come from tsa.cnf, not
2 # from openssl.cnf, so the certificate carries the timestamping
3 # extendedKeyUsage instead of the end-entity one.
4 openssl ca -config "$OPENSSL_CNF" -batch -notext -md sha256 \
5     -startdate "$START_DATE" -days "$CERT_DAYS" -extfile "$TSA_CNF" -extensions
6     v3_tsa \
7     -in "$CSR_FILE" -out "$CERT_FILE"
8
9 # Files "openssl ts -reply" reads: the running serial number of issued
10 # tokens and the chain appended to every response.
11 [ -f "$SERIAL_FILE" ] || printf '1000\n' > "$SERIAL_FILE"
12 cp "$PKI_CA_DIR/ca.crt.pem" "$CHAIN_FILE"

```

Listado D.5: Firma con extensiones de `tsa.cnf` y preparación del directorio TSA en `pki/issue-tsa-cert.sh`.

D.7 `openssl.cnf`: configuración de la CA

El archivo define la sección `internal_ca` con el layout del directorio de trabajo (resuelto desde `$ENV : PKI_CA_DIR`, por lo que el archivo no es utilizable de forma independiente de los scripts) y los valores por defecto para la emisión, que son `default_md = sha256`, `default_days = 365` y `default_crl_days = 7`. Tres directivas endurecen la emisión: `copy_extensions = none`

garantiza que solo se apliquen las extensiones definidas en el propio archivo y nunca las que el solicitante incluya en su CSR; la política `policy_internal` exige que el país y la organización del certificado emitido coincidan con los de la CA (`match`) y que el `common name` esté presente (`supplied`); y `unique_subject = no` permite reemitir para un sujeto cuyo certificado anterior fue revocado. La sección `req` fija `default_bits = 3072`, `default_md = sha256` y `string_mask = utf8only`. Los tres juegos de extensiones X.509 v3 son los siguientes:

```

1 [ v3_root_ca ]
2 # Extensions for the self-signed root CA certificate.
3 basicConstraints      = critical, CA:true
4 keyUsage              = critical, keyCertSign, cRLSign
5 subjectKeyIdentifier  = hash
6
7 [ v3_end_entity ]
8 # Extensions for issued end-entity certificates: people or services
9 # that sign documents and authenticate against internal systems.
10 basicConstraints      = critical, CA:false
11 keyUsage              = critical, digitalSignature, nonRepudiation
12 extendedKeyUsage      = clientAuth, emailProtection
13 authorityKeyIdentifier = keyid:always
14 subjectKeyIdentifier  = hash
15
16 [ crl_ext ]
17 # Extensions attached to every generated CRL.
18 authorityKeyIdentifier = keyid:always

```

Listado D.6: Extensiones X.509 v3 definidas en `pki/openssl.cnf`.

La raíz declara `CA:true` con uso de llave restringido a firmar certificados y CRL; las entidades finales declaran `CA:false` con `digitalSignature` y `nonRepudiation` críticos —los usos que sustentan la firma de documentos— más autenticación de cliente y protección de correo como usos extendidos.

D.8 tsa.cnf: configuración de la autoridad de sellado

La sección `internal_tsa` gobierna a `openssl ts -reply`, la orden con la que la TSA local emite cada token; el directorio se resuelve desde `$ENV:TSA_DIR`. Todos los digestos son SHA-256, tanto los aceptados en las solicitudes como el usado para firmar la respuesta. La política estampada en cada token es el OID 1.3.6.1.4.1.13762.3: el arco 1.3.6.1.4.1 es el árbol de empresas privadas de la IANA y el valor es una política de demostración del prototipo, no una política de producción registrada; las solicitudes que pidan cualquier otra política se rechazan. El tiempo aseverado es exacto a un segundo (`accuracy = secs:1`), las respuestas no garantizan orden entre tokens (`ordering = no`) y se mantienen mínimas (`tsa_name = no`, `ess_cert_id_chain = no`).

```

1 # All digests are SHA-256: the imprint digests accepted in requests and
2 # the digest used for the response signature.
3 digests      = sha256
4 signer_digest = sha256

```

```

5
6 # Policy identifier stamped into every token. The arc 1.3.6.1.4.1 is
7 # the IANA private-enterprise tree; the value below is a demo policy
8 # for this prototype, not a registered production policy. Requests
9 # asking for any other policy are refused.
10 default_policy = 1.3.6.1.4.1.13762.3
11
12 # The asserted time is accurate to one second and responses carry no
13 # ordering guarantee between tokens.
14 accuracy = secs:1
15 ordering = no

```

Listado D.7: Parámetros de emisión de tokens en `pki/tsa.cnf`.

El mismo archivo define las extensiones `v3_tsa` del certificado de la autoridad de sellado, que difieren de las de entidad final en el punto que RFC 3161 exige: el uso extendido de llave `timeStamping` marcado como crítico, junto con `basicConstraints = critical`, `CA:false` y `keyUsage = critical`, `digitalSignature`.

D.9 Flujo típico de operación

El orden de uso es el siguiente (los pasos de la autoridad certificadora están documentados en `pki/README.md`; el paso de la TSA lo añade `issue-tsa-cert.sh`):

1. `init-ca.sh`, una sola vez, para crear la raíz y la base de datos de la autoridad.
2. `issue-cert.sh "Nombre Apellido"`, una vez por cada firmante que requiera certificado.
3. `issue-tsa-cert.sh`, una sola vez, si se empleará el sellado de tiempo local.
4. `revoke.sh ruta/al/certificado.crt.pem`, cuando una llave se compromete o el vínculo con el firmante termina.
5. `gen-crl.sh`, después de cada revocación y al menos cada siete días, para que los verificadores dispongan siempre de una lista vigente.

En la operación normal del prototipo estos pasos no se ejecutan a mano: el adaptador de la autoridad en el crate de infraestructura invoca `init-ca.sh`, `issue-cert.sh`, `revoke.sh` y `gen-crl.sh` como subprocesos desde los subcomandos `pki` del binario, inyectando `PKI_CA_DIR`; y la TSA local invoca `openssl ts -query` y `openssl ts -reply` contra `tsa.cnf` con `TSA_DIR` inyectada, serializando las respuestas concurrentes con un candado consultivo sobre un archivo del directorio TSA, porque OpenSSL avanza el archivo de seriales con una secuencia leer-incrementar-escribir no atómica. Las pruebas de integración conducen estos mismos scripts en directorios temporales y contrastan sus productos con la línea de comandos de OpenSSL (`crates/infrastructure/tests/openssl_ca_adapter.rs`, `certificate_validation.rs` y `local_tsa_rfc3161.rs`).

Cualquier producto de la autoridad puede auditarse sin el prototipo, con los comandos documentados en `pki/README.md`: `openssl verify -CAfile` para el encadenamiento a la raíz, `openssl verify -crl_check` —concatenando raíz y CRL— para el estado de revocación, y `openssl crl -noout -text` para inspeccionar la lista; la guía completa de verificación de un paquete de evidencia se presenta en el Anexo C.

APÉNDICE E

Matriz de pruebas y evidencia de ejecución

Este anexo cataloga la suite completa de pruebas automatizadas del prototipo y documenta la evidencia de su ejecución. La matriz responde al criterio de éxito del objetivo OE-6 —una matriz de pruebas funcionales con al menos un caso positivo y uno negativo por caso de uso— y al requerimiento RNF-13, que exige una cobertura igual o mayor al 90 % en los módulos criptográficos. El anexo se organiza en tres partes: una nota metodológica que explica cómo leer la matriz, la matriz misma con una fila por archivo o módulo de prueba, y la evidencia de ejecución (resumen de la corrida, cobertura por crate, calibración de Argon2id, transcripción de la demostración y re-ejecución en integración continua).

E.1 Nota metodológica

El workspace contiene **383 funciones de prueba**: 382 se ejecutan en verde en cada corrida y una se declara ignorada (`#[ignore]`) porque es una prueba de humo contra el sandbox del proveedor externo de sellado de tiempo. El registro se completó, pero la API observada no ofrece estabilidad suficiente para una prueba determinista y el sandbox requiere pago; por ello, la prueba ignorada se conserva como referencia del adaptador y no se agenda como campaña de entrega. El resto del adaptador correspondiente se prueba contra un stub HTTP local, de modo que la suite regular no depende de ningún servicio de red ni incurre en costo externo.

Los nombres de las funciones de prueba son frases completas en `snake_case` (en inglés, como todo identificador del código fuente) que describen el comportamiento verificado: por ejemplo, `a_code_is_accepted_once_and_rejected_on_reuse` documenta por sí misma la regla de un solo uso de los códigos de recuperación. Este estilo convierte al inventario de pruebas en la *especificación ejecutable* del prototipo: la lectura de los nombres de una suite enumera las obligaciones de comportamiento del módulo correspondiente, y la corrida en verde certifica que todas se cumplen. Por esa razón la matriz no parafrasea cada prueba: agrupa las 383 funciones por crate y por archivo o módulo, indica su tipo y cita dos o tres nombres representativos por fila. El inventario íntegro se reproduce con `cargo test --workspace` sobre el repositorio.

La estrategia por capa sigue la arquitectura hexagonal: en `domain` las pruebas usan dobles deterministas escritos a mano (el crate no depende de ninguna biblioteca criptográfica); en `application` usan mocks o fakes de los puertos para probar la orquestación y la propagación de errores; en `infrastructure` ejercitan los adaptadores con criptografía real y herramientas del sistema; en `web` verifican el contrato HTTP sobre un workflow inyectado; y en `bin` ejecutan el binario compilado `despacho-cli` como subprocesso. La [tabla E.1](#) define la taxonomía de tipos empleada en la matriz y sus abreviaturas.

Tabla E.1: Taxonomía de tipos de prueba empleada en la matriz y abreviaturas usadas.

Tipo (abrev.)	Descripción
Unitaria (Unit.)	Prueba de una unidad en aislamiento: tipos de valor, reglas de dominio, orquestación de casos de uso con mocks, o rutas de error de un adaptador.
Vectores (Vect.)	Prueba de respuesta conocida (<i>known-answer test</i>) contra vectores publicados en estándares: FIPS 180-4 para SHA-256 [25], los casos 13–16 de la especificación de GCM en que se basa NIST SP 800-38D [50], RFC 6238 [19], RFC 4226 [58] y el valor de comprobación de CRC-32.
Propiedad (Prop.)	Prueba basada en propiedades con proptest: la propiedad debe sostenerse para entradas generadas aleatoriamente.
Mutación (Mut.)	Alteración deliberada de un byte o un bit de un artefacto (documento, firma, certificado, payload cifrado, línea de bitácora) para comprobar que la verificación lo rechaza y señala el punto exacto.
Interoperabilidad (Interop.)	Verificación cruzada con software independiente del propio: openssl, sha256sum y unzip.
Integración (Integr.)	Adaptador real contra recursos locales: archivos compartidos, subprocessos, hilos concurrentes o un stub HTTP en 127.0.0.1.
E2E	Extremo a extremo: la prueba ejecuta el binario compilado como subprocesso y examina salidas, archivos producidos y códigos de salida.
Arquitectura (Arq.)	<i>Fitness function</i> que fija una regla estructural del sistema [68]: dependencias del dominio (FF-1), no fuga de secretos en logs (FF-2) y cobertura de puertos con adaptador y mock (FF-3).

E.2 Matriz de pruebas

La [tabla E.2](#) resume la distribución de las 383 funciones por crate y la [tabla E.3](#) presenta la matriz completa: una fila por archivo o módulo de prueba, agrupadas por crate. La columna **N** indica el número de funciones de prueba del archivo; la suma de la columna es 383. Las rutas se abrevian respecto de la raíz de cada crate: `src/` señala pruebas unitarias embebidas en el módulo (`#[cfg(test)]`) y `tests/` señala binarios de prueba de integración. En la fila del stub del proveedor de sellado, la anotación (+1) corresponde a la única prueba ignorada.

Tabla E.2: Distribución de las 383 funciones de prueba por crate. El conteo de integración de infrastructure incluye la prueba ignorada.

Crate	Unitarias (src/)	Integración (tests/)	Total
domain	69	0	69
application	39	43	82
infrastructure	86	87	173
bin	3	52	55
web	1	3	4
Total	198	185	383

Tabla E.3: Matriz de pruebas del prototipo: una fila por archivo o módulo de prueba, con el número de funciones (N), el tipo conforme a la [tabla E.1](#) y casos representativos.

Módulo / archivo	N	Tipo	Casos representativos
Crate domain — 69 pruebas unitarias en src/			
src/audit/chain.rs	10	Unit.	an_intact_chain_verifies_as_valid truncating_the_tail_is_not_detected_ by_verification_alone
src/audit/event.rs	6	Unit.	canonical_bytes_follow_the_ documented_layout a_non_utc_timestamp_is_normalized_to_ utc
src/audit/port.rs	1	Unit.	port_is_object_safe_and_round_trips_ an_entry
src/clock.rs	1	Unit.	port_is_object_safe_and_returns_the_ configured_instant
src/crypto/archive.rs	7	Unit.	names_with_path_separators_are_ rejected ordinary_file_names_are_accepted
src/crypto/ certificate.rs	5	Unit.	stale_crl_error_carries_the_missed_ update_time summary_reports_the_validity_window
src/crypto/cipher.rs	6	Unit.	sealed_payload_rejects_bytes_shorter_ than_nonce_and_tag document_aad_changes_with_the_version
src/crypto/digest.rs	6	Unit.	from_bytes_rejects_wrong_length hex_round_trip_is_stable
src/crypto/document.rs	5	Unit.	version_rejects_zero next_increments_version

Continúa en la página siguiente

Módulo / archivo	N	Tipo	Casos representativos
src/crypto/haser.rs	4	Unit.	hash_stream_reports_matching_digest_for_same_input hash_stream_surfaces_read_failures
src/crypto/keys.rs	3	Unit.	wrapped_dek_round_trips_its_bytes wrapped_dek_rejects_bytes_shorter_than_a_sealed_payload
src/crypto/recovery.rs	6	Unit.	a_code_is_accepted_once_and_rejected_on_reuse each_code_works_exactly_once_until_the_set_is_exhausted
src/crypto/signature.rs	6	Unit.	signature_debug_hides_the_bytes ports_are_object_safe_and_round_trip
src/crypto/timestamp.rs	3	Unit.	each_rejection_cause_is_reported_explicitly ports_are_object_safe_and_round_trip
Crate application — 39 unitarias en src/ y 43 en tests/, con dobles de los puertos			
src/audit.rs	8	Unit.	append_takes_the_timestamp_from_the_clock_and_returns_the_stored_entry verify_reports_the_first_index_whose_chain_does_not_match
src/auth/password.rs	6	Unit.	calibration_with_an_instant_hasher_reports_below_band band_edges_are_inclusive
src/auth/totp.rs	4	Unit.	enrollment_bundles_provisioning_material_and_eight_recovery_codes verify_totp_surfaces_backend_errors
src/hashing.rs	2	Unit.	streams_the_reader_through_the_port_and_returns_its_digest
src/signing.rs	7	Unit.	sign_document_signs_the_digest_of_the_streamed_content verify_signature_passes_a_rejection_through_unchanged
src/timestamping.rs	5	Unit.	timestamp_document_requests_a_token_over_the_streamed_digest verify_timestamp_checks_the_token_over_the_recomputed_digest
src/vault.rs	7	Unit.	serialized_layout_starts_with_magic_and_length parse_rejects_a_wrapped_key_length_beyond_the_file
tests/document_record.rs	2	Unit.	a_pending_record_becomes_sealed_once a_record_rejects_a_name_that_cannot_enter_an_evidence_archive

Continúa en la página siguiente

Módulo / archivo	N	Tipo	Casos representativos
tests/document_repository_port.rs	1	Unit.	the_repository_port_is_object_safe_and_round_trips_a_record
tests/document_workflow.rs	4	Integr.	upload_seal_verify_export_and_audit_form_one_workflow a_document_cannot_be_sealed_twice
tests/evidence_package.rs	7	Unit.	the_package_holds_every_artifact_in_order_under_the_documented_names the_instructions_fill_every_placeholder
tests/pki_use_cases.rs	10	Unit.	issue_reports_success_only_after_the_certificate_chains_to_the_root validate_propagates_a_stale_crl_error
tests/vault_use_cases.rs	6	Unit.	encrypt_seals_with_a_fresh_dek_and_the_document_aad rotate_rewraps_the_dek_and_copies_the_sealed_document_through
tests/verify_document.rs	6	Unit.	all_supplied_components_passing_yield_a_valid_verdict without_a_token_integrity_and_timestamp_are_skipped_with_reasons
tests/verify_document_failures.rs	7	Unit.	an_imprint_mismatch_fails_integrity_and_timestamp a_revoked_certificate_reports_the_matched_serial
Crate infrastructure — 86 unitarias en src/ y 87 de integración en tests/ (una ignorada)			
src/archive.rs	6	Unit., vect.	crc32_matches_the_standard_check_value output_is_deterministic
src/audit/file.rs	6	Unit.	stored_lines_are_json_objects_with_an_rfc3339_utc_timestamp a_line_that_is_not_json_surfaces_as_a_storage_failure
src/audit/memory.rs	4	Unit.	appended_entries_get_consecutive_sequences_and_verify the_first_entry_links_to_the_genesis_value
src/certificates/authority.rs	6	Unit.	revoking_an_unissued_serial_is_rejected serials_normalize_to_uppercase
src/certificates/parse.rs	7	Unit.	a_pem_block_with_the_wrong_label_is_rejected serial_hex_strips_the_der_sign_padding_byte

Continúa en la página siguiente

Módulo / archivo	N	Tipo	Casos representativos
src/certificates/ validator.rs	2	Unit.	unparseable_certificate_bytes_are_a_ hard_error inspect_rejects_unparseable_bytes
src/clock.rs	1	Unit.	now_is_utc_and_does_not_go_backwards
src/encryption.rs	3	Unit.	seal_then_open_round_trips seal_rejects_a_short_key
src/envelope.rs	6	Unit.	unwrap_with_the_wrong_kek_fails_ opaquely rewrap_moves_the_dek_to_the_new_kek
src/error.rs	3	Unit.	a_crypto_error_becomes_an_ application_port_error a_tsa_error_becomes_an_application_ port_error
src/hashing.rs	9	Vect., prop., interop.	hashes_one_million_a_bytes_streamed flipping_one_bit_changes_the_digest digest_matches_sha256sum_output
src/password.rs	4	Unit.	phc_string_records_the_fixed_ argon2id_parameters malformed_stored_hash_is_an_error_ not_a_mismatch
src/recovery.rs	3	Unit.	codes_have_the_grouped_human_ typeable_shape codes_within_a_batch_are_distinct
src/signing.rs	8	Unit.	constructor_rejects_a_corrupt_pkcs1_ block verifier_rejects_non_utf8_material
src/timestamp/ cincel.rs	2	Unit.	the_debug_form_redacts_the_api_key trailing_slashes_in_the_base_url_are_ trimmed
src/timestamp/local.rs	4	Unit.	nonzero_exits_map_to_rejected_with_ the_step_name a_serial_lock_whose_directory_is_ absent_is_unreachable
src/timestamp/ verify.rs	4	Unit.	truncated_der_is_a_malformed_token_ with_a_diagnosis token_info_rejects_undecodable_bytes
src/tools.rs	1	Interop.	the_local_openssl_reports_a_version_ line
src/totp.rs	7	Vect.	eight_digit_codes_match_the_rfc6238_ appendix_b_sha1_vectors verification_accepts_exactly_one_ step_of_skew_on_each_side

Continúa en la página siguiente

Módulo / archivo	N	Tipo	Casos representativos
tests/aead_properties.rs	6	Prop., mut.	any_single_byte_flip_breaks_authentication nonces_are_distinct_across_ten_thousand_seals
tests/aes_gcm_vectors.rs	7	Vect.	encrypt_vector_tcl6_with_aad decrypt_vector_tcl4_with_wrong_tag_fails
tests/audit_log.rs	8	Integr., prop., mut.	concurrent_appends_serialize_into_one_valid_chain any_single_corruption_is_detected_at_its_index
tests/certificate_validation.rs	11	Interop., mut.	openssl_verify_agrees_on_the_valid_and_revoked_cases corrupting_one_signature_byte_makes_the_issuer_untrusted
tests/cinzel_stub.rs	13 (+1)	Integr.	a_processing_submission_is_polled_until_the_token_arrives the_api_key_travels_in_the_header_and_never_into_errors ignorada: the_real_sandbox_issues_a_token
tests/file_document_repository.rs	6	Integr.	insert_and_find_round_trip_a_pending_record stored_identity_must_match_the_requested_path
tests/issuer_validity_window.rs	2	Interop.	expired_issuer_is_rejected_and_openssl_agrees chain_is_valid_while_both_windows_are_open_and_openssl_agrees
tests/local_tsa_rfc3161.rs	13	Integr., interop.	a_requested_token_is_accepted_by_openssl_ts_verify concurrent_requests_yield_distinct_serial_numbers
tests/openssl_ca_adapter.rs	5	Integr.	init_ca_creates_the_root_and_is_idempotent revoke_then_generate_crl_marks_the_certificate_revoked
tests/password_hashing.rs	2	Unit.	the_default_hasher_produces_the_fixed_argon2id_parameters a_stored_hash_with_out_of_range_parameters_is_malformed_not_a_mismatch

Continúa en la página siguiente

Módulo / archivo	N	Tipo	Casos representativos
tests/rsa_signing.rs	11	Interop., mut.	signer_output_verifies_with_the_openssl_command_line an_openssl_signature_verifies_as_valid the_signer_debug_form_withholds_the_private_key
tests/zip_archive.rs	2	Interop., mut.	unzip_accepts_the_archive_and_extraction_restores_every_byte unzip_detects_a_flipped_byte_through_the_crc
Crate bin (despacho-cli) — 3 unitarias en src/ y 52 E2E en tests/ sobre el binario compilado			
src/cli.rs	1	Unit.	command_tree_is_valid
src/timestamp_cmd.rs	2	Unit.	the_token_suffix_keeps_the_original_extension the_default_tsa_directory_sits_next_to_the_ca_directory
tests/architecture.rs	1	Arq.	domain_has_no_workspace_dependencies
tests/audit_cli.rs	7	E2E	a_corrupted_line_makes_verify_chain_fail_naming_the_index verifying_a_missing_log_fails_naming_the_path
tests/auth_cli.rs	6	E2E	hash_password_then_verify_password_round_trip totp_enroll_writes_the_secret_and_a_current_code_verifies
tests/crypto_hash.rs	3	E2E, vect.	prints_the_lowercase_hex_digest fails_when_the_file_does_not_exist
tests/help.rs	1	E2E	help_succeeds_and_names_the_binary
tests/log_redaction.rs	3	Arq.	vault_encrypt_never_logs_the_kek hash_password_never_logs_the_password
tests/package_cli.rs	6	E2E, interop., mut.	the_documented_openssl_commands_verify_the_extracted_package export_refuses_a_tampered_signature_naming_the_component
tests/pki_cli.rs	3	E2E	init_issue_show_revoke_and_gen_crl_work_end_to_end revoking_an_unknown_serial_fails
tests/port_coverage.rs	1	Arq.	every_domain_port_has_an_infrastructure_adapter_and_a_mock

Continúa en la página siguiente

Módulo / archivo	N	Tipo	Casos representativos
tests/sign_cli.rs	4	E2E, interop.	sign_writes_a_signature_that_openssl_verifies sign_rejects_a_key_that_does_not_match_the_certificate
tests/timestamp_cli.rs	3	E2E, interop.	timestamp_with_mock_writes_a_verifiable_token_next_to_the_input timestamp_without_provider_settings_suggests_the_mock_authority
tests/vault_roundtrip.rs	5	E2E, mut.	encrypt_then_decrypt_round_trips_byte_identical decrypting_a_corrupted_package_fails_with_an_authentication_error
tests/verify_cli.rs	9	E2E, mut.	an_intact_document_verifies_valid_on_every_component a_revoked_certificate_reports_its_serial_from_the_fresh_crl
Crate web — 1 unitaria en src/ y 3 de contrato en tests/			
src/lib.rs	1	Unit.	health_endpoint_reports_ready
tests/document_api.rs	3	Integr.	the_document_routes_expose_the_complete_workflow upload_requires_actor_and_document_name_headers an_unknown_document_maps_to_not_found

El crate web conserva la prueba de `/healthz` y añade tres pruebas de contrato sobre el workflow inyectado. Sesiones, RBAC y persistencia PostgreSQL/Redis permanecen fuera de este corte; el repositorio local cifrado se prueba en `infrastructure`.

Dos lecturas transversales de la matriz merecen mención. Primera, todo servicio criptográfico cuenta con al menos un caso positivo y al menos un caso negativo, como exige el criterio de éxito de OE-6; los casos negativos dominan numéricamente porque el valor probatorio del sistema depende de que los rechazos sean correctos y diagnósticos. Segunda, las pruebas de mutación de un byte materializan el criterio de OE-4 (detección del 100 % de alteraciones de un byte): la suite altera deliberadamente documentos firmados, firmas, certificados, payloads cifrados, miembros del paquete de evidencia y líneas de la bitácora, y en todos los casos verifica el rechazo; la prueba `nonces_are_distinct_across_ten_thousand_seals` cubre además la exigencia de cero reutilizaciones de nonce en una suite de 10 000 cifrados.

E.3 Evidencia de ejecución

E.3.1 Resumen de la corrida

La ejecución de cargo `test --workspace` reporta **382 pruebas en verde, 0 fallos y 1 ignorada**, distribuidas en 39 binarios de prueba. Sobre un árbol ya compilado, la suite completa termina en

segundos; las pruebas más costosas son las que ejecutan subprocessos reales de `openssl` y del binario propio. Este costo bajo es deliberado: los instantes de evaluación de los certificados se derivan de la ventana de validez parseada del propio certificado y no de esperas de reloj, de modo que la suite es determinista y puede correrse en cada cambio sin fricción.

Los conteos y la cobertura de este anexo se reproducen con tres comandos sobre la raíz del repositorio, los mismos que ejecuta el flujo de integración continua:

```
cargo test --workspace
cargo llvm-cov --workspace --json --summary-only \
  --output-path coverage.json
scripts/coverage-gate.sh coverage.json
```

E.3.2 Cobertura de líneas por crate

La cobertura de líneas se mide con `cargo-llvm-cov` y se aplica como umbral de integración continua mediante el script versionado `scripts/coverage-gate.sh`: los tres crates que concentran la lógica criptográfica (`domain`, `application`, `infrastructure`) deben mantener una cobertura igual o mayor al 90 %, en cumplimiento de RNF-13 (y con margen sobre el umbral del 80 % que fija el criterio de éxito de OE-6). Los crates `bin` y `web` se reportan sin umbral por tratarse de fronteras de entrada y composición, cuyo valor se complementa con pruebas de contrato y demostraciones E2E. La [tabla E.4](#) presenta la salida del gate.

Tabla E.4: Cobertura de líneas por crate medida con `cargo-llvm-cov` y umbral aplicado en integración continua.

Crate	Líneas cubiertas	Líneas totales	Cobertura	Umbral
<code>domain</code>	937	974	96.2 %	$\geq 90 \%$
<code>application</code>	1 488	1 561	95.3 %	$\geq 90 \%$
<code>infrastructure</code>	2 009	2 151	93.4 %	$\geq 90 \%$
<code>bin</code>	833	1 008	82.6 %	sin umbral (informativo)
<code>web</code>	168	223	75.3 %	sin umbral (informativo)
Total del workspace	5 435	5 917	91.9 %	—

E.3.3 Calibración de Argon2id en el hardware de referencia

El comando `auth calibrate` mide el costo real del hash de contraseñas con los parámetros fijos del prototipo (Argon2id con $m = 256$ MiB, $t = 4$, $p = 1$; véase §3.11.5) contra la banda objetivo de 0.5–1.0 segundos por intento [60], [61]. En el hardware de referencia AMD Ryzen 7 7730U la calibración de la demostración de cierre del 17 de agosto de 2026 reportó una **media de 607.8 ms sobre cinco corridas**, dentro de la banda objetivo, y fijó los parámetros en el adaptador. Si el hardware de despliegue difiere, la calibración debe repetirse; la suite conserva los bordes inclusivos de la banda (`band_edges_are_inclusive`).

E.3.4 Transcripción de la demostración

La evidencia de ejecución de cara al usuario final comprende dos guiones. `scripts/demo.sh` recorre por CLI el ciclo de vida completo de la evidencia, autenticación y bitácora; su transcripción íntegra, capturada sobre OpenSSL 3.5.7, está versionada en `docs/demo-transcript.md`. `scripts/api-demo.sh` genera una PKI temporal, arranca la aplicación sobre un puerto efímero y recorre carga, persistencia cifrada, sellado, verificación, exportación y auditoría a través de HTTP; después comprueba el ZIP con `openssl` y `unzip`. Cada condición de ambos guiones es un punto de verificación real y cualquiera aborta ante un resultado inesperado. El Anexo C reproduce y explica la porción de la primera transcripción dedicada a la verificación independiente del paquete de evidencia.

E.3.5 Re-ejecución continua

Toda la evidencia anterior se regenera automáticamente en cada cambio: el flujo de integración continua del repositorio (`.github/workflows/ci.yml`) ejecuta siete jobs sobre cada push y cada pull request, y el fallo de cualquiera de ellos —salvo el job `msrv`, hoy informativo— bloquea la integración del cambio. La [tabla E.5](#) los enumera. De este modo, la matriz de este anexo no describe una fotografía única sino un invariante mantenido: las 382 pruebas en verde, los umbrales de cobertura y las fitness functions se verifican en cada modificación del código.

Tabla E.5: Jobs del flujo de integración continua que re-ejecutan la evidencia en cada cambio.

Job	Verificación
<code>format</code>	Formato canónico del código con <code>rustfmt</code> .
<code>lint</code>	<code>clippy</code> con advertencias promovidas a error (<code>-D warnings</code>).
<code>test</code>	La suite completa de este anexo: <code>cargo test --workspace</code> .
<code>msrv</code>	Compilación con la versión mínima soportada del compilador (Rust 1.78); informativo.
<code>coverage</code>	Cobertura con <code>cargo-llvm-cov</code> y <code>gate</code> por <code>crate</code> de <code>scripts/coverage-gate.sh</code> (tabla E.4).
<code>deny</code>	<code>cargo-deny</code> : avisos de seguridad de dependencias, licencias, duplicados y fuentes permitidas.
<code>binary-size</code>	Tamaño del binario de release acotado a 25 MB (fitness function FF-5).